
CozyStack

Документация v1.5

Полная offline-версия документации · cozystack.ru/docs/v1.5/

Оглавление

Введение

- Cozystack v1.5
- Введение

Начало работы

- Начало работы
 - Требования
 - 1. Установка Talos
 - 2. Установка Kubernetes
 - 3. Установка Cozystack
 - 4. Создание tenant'a
 - 5. Развертывание приложений

Изучение Cozystack

- Изучение Cozystack
 - Ключевые концепции
 - Стек платформы
 - Система tenant'ов
 - Управление ресурсами
 - Talos Linux
 - Сценарии использования
 - Public Cloud
 - Private Cloud
 - Build Your Own Platform

Развертывание Cozystack

- Развертывание Cozystack
 - Требования к оборудованию
 - Планирование ресурсов
 - 1. Установка Talos
 - boot-to-talos
 - PXE
 - ISO
 - 2. Установка Kubernetes
 - Talm
 - talos-bootstrap
 - talosctl
 - Изолированная среда
 - Troubleshooting
 - Generic Kubernetes
 - Ubuntu + Secure Boot
 - 3. Установка Cozystack
 - Как платформа
 - Собственная платформа
 - Руководства для провайдеров

- Oracle Cloud (OCI)
- Hetzner
- Servers.com
- Ansible
- Практические руководства
- Установка на один диск
- Развертывание в публичных сетях
- Включение KubeSpan
- Настройка bonding (LACP)
- Включение Hugepages

Эксплуатация

Эксплуатация

- Обновление до v1.0
- Конфигурация
- Пакет платформы
- Варианты
- Компоненты
- Лицензии
- Вебхук Lineage Controller
- Брендирование
- Телеметрия
- VPC и подсети
- Обслуживание кластера
- Обновление Cozystack
- Масштабирование кластера
- Ротация CA
- Сервисы кластера
- Объектное хранилище
- Пулы хранения
- Бакеты
- Настройка Velero
- Резервное копирование приложений
- BootBox
- EtcD
- Ingress
- SeaweedFS
- Мониторинг
- OpenID Connect
- Сервер OIDC
- Пользователи и роли
- Самоподписанные сертификаты
- Провайдеры идентификации
- GitLab
- Google
- Несколько ЦОД
- Конфигурация etcd
- Конфигурация kube-scheduler
- Топологические метки
- LINSTOR DRBD

- Сеть хранилища
- SeaweedFS в нескольких ЦОД
- Несколько площадок
- FAQ и инструкции
- ServiceAccount и API
- Генерация kubecfg
- Устранение неполадок
- etcd
- Flux CD
- Мониторинг
- Kube-OVN
- LINSTOR: контроллер
- LINSTOR: база данных
- Piraeus: custom resources
- Классы планирования

Managed Kubernetes

- Managed Kubernetes
- GPU Sharing
- Backups with Velero
- Relocate etcd replicas

Managed Applications

- Managed Applications
- Backup and Recovery
- External Apps
- ClickHouse
- FoundationDB
- Harbor Container Registry
- Kafka
- MariaDB
- MongoDB
- NATS
- OpenBAO
- PostgreSQL
- Qdrant
- RabbitMQ
- Redis
- Tenant

Виртуализация

- Виртуализация
- Виртуальная машина
- Диск виртуальной машины
- Golden Images
- Клонируемые VM
- Проброс GPU
- Резервное копирование
- Windows VM
- MikroTik RouterOS

[Миграция из Proxmox](#)
[Справочник по ресурсам](#)

Хранилище

[Хранилище](#)
[Подготовка дисков](#)
[Выделенная сеть](#)
[Настройка DRBD](#)
[Использование NFS](#)
[LINSTOR GUI](#)
[Шифрованное хранилище](#)

Сеть

[Сеть](#)
[Архитектура](#)
[VPC](#)
[VPN](#)
[Gateway API](#)
[HTTP-кеш](#)
[PROXY-protocol и hairpin](#)
[Публикация K8s API в DNS](#)
[Балансировщик TCP](#)
[Виртуальные маршрутизаторы](#)

Cozystack API

[Cozystack API](#)
[Go Types](#)
[REST API](#)
[ApplicationDefinition](#)

Прочее

[Руководство по разработке](#)
[Рoadmap](#)
[Контрибуторы](#)

Введение

Документация Cozystack v1.5

<https://cozystack.ru/docs/v1.5/>

Cozystack это свободная PaaS-платформа для построения облачных инфраструктур.

Cozystack позволяет легко трансформировать ваш парк серверов в интеллектуальную систему с помощью простого REST API для развертывания Kubernetes кластеров, виртуализации и контейнеризации, сервисов баз данных (DBaaS), виртуальных машин, балансировщиков нагрузки, HTTP-кэширования и других сервисов.

Используйте Cozystack для создания собственного облака как для продуктивной нагрузки, так и для предоставления недорогих сред разработки.

Введение в Cozystack

Что такое Cozystack и его возможности

Начало работы с Cozystack: развёртывание частного облака с нуля

Сделайте первые шаги, запустите домашнюю лабораторию и соберите POC на Cozystack.

Изучение Cozystack

Узнайте, как использовать Cozystack в роли администратора кластера и владельца tenant'a.

Руководство по развёртыванию Cozystack: от инфраструктуры до готового кластера

Узнайте, как развернуть кластер Cozystack с помощью Talos Linux и Kubernetes. В этом руководстве описаны установка, настройка и лучшие практики надежного и безопасного развёртывания Cozystack.

Руководство по настройке и управлению кластером

Настройка, мониторинг, защита и обновление кластера Cozystack.

Managed (Tenant) Kubernetes

Learn to deploy and use isolated managed Kubernetes clusters in Cozystack.

Managed Applications: Guides and Reference

Learn how to deploy, configure, access, and backup managed applications in Cozystack.

Возможности виртуализации в Cozystack

Всё о развёртывании, настройке и использовании виртуальных машин в Cozystack.

Руководства по подсистеме хранения

Практические руководства по подсистеме хранения

Сетевые возможности

Настройка сети, виртуальные маршрутизаторы, балансировщики нагрузки и другие сетевые возможности Cozystack.

Cozystack API

Cozystack API для управления сервисами и ресурсами

Cozystack Руководство по разработке

Cozystack Руководство по разработке

Cozystack Рoadmap

Cozystack Рoadmap

Контрибуторы

Люди, которые помогают развивать документацию и проект Cozystack.

Введение в Cozystack

<https://cozystack.ru/docs/v1.5/introduction/>

Что такое Cozystack

Cozystack — это фреймворк на базе Kubernetes для построения среды частного облака. Он может использоваться отдельной компанией для запуска собственного **частного облака** или поставщиком услуг для предоставления **платформы как услуги** нескольким клиентам.

Cozystack закрывает наиболее важные потребности команды разработчиков:

- **Кластеры Kubernetes** для запуска приложений в среде разработки и в продакшене
- Стандартные **управляемые приложения**: базы данных, менеджеры очередей, кэши и многое другое
- **Виртуальные машины**
- Надёжное распределённое хранилище

Платформенный стек Cozystack включает надёжные компоненты, которые обычно устанавливаются в кластеры Kubernetes по отдельности. Здесь они собраны вместе и протестированы для бесперебойной совместной работы. Платформа виртуализации также встроена и не требует дополнительного оборудования. Вместо этого виртуальные машины запускаются непосредственно внутри Kubernetes.

Ещё одна мощная функция — **система тенантов**. Она позволяет изолировать отдельных разработчиков, команды или даже целые компании в их собственных полнофункциональных пространствах — на одном и том же оборудовании.

Ключевые возможности

Многопользовательский режим, мультитенантность и встроенный SSO

Cozystack разработан для использования несколькими командами, отделами или даже компаниями. Традиционный подход с выделением каждой команде отдельного пространства имён может быть слишком ограниченным. Командам может потребоваться несколько окружений с одинаковыми именами пространств имён, или им может не хватать прав root для управления собственными моделями доступа.

Система тенантов Cozystack решает эти проблемы, позволяя пользователям разворачивать среду Kubernetes-в-Kubernetes с помощью одного приложения. Пользователи вложенных кластеров Kubernetes имеют полный доступ и контроль. Система квот обеспечивает оптимальное использование оборудования, изолируя ресурсы во избежание проблемы «шумного соседа». Пользователи платформы также могут формировать подробные отчёты об использовании ресурсов для каждого тенанта.

[Система единого входа](#) в Cozystack работает на базе Keycloak. Kubernetes API — как в корневом тенанте, так и во вложенных — поддерживает SSO из коробки.

Реплицированная система хранения

Не все компании могут позволить себе выделенное аппаратное обеспечение SAN или NAS. Cozystack включает надёжную распределённую систему хранения, позволяющую создавать отказоустойчивые реплицированные тома. Возможна даже репликация томов между несколькими центрами обработки данных.

Система виртуализации

Обычно приходится выбирать между виртуализацией и контейнеризацией. Cozystack объединяет оба подхода в единой платформе. Нет необходимости поддерживать отдельную инфраструктуру виртуализации. В Cozystack [виртуальные машины](#) запускаются непосредственно в Kubernetes и потребляют CPU, память, GPU и хранилище из общего пула ресурсов Kubernetes.

Управляемые базы данных без накладных расходов

Несмотря на высокую эффективность виртуализации Linux-на-Linux, она всё же вносит определённые накладные расходы. Cozystack избегает этого, запуская [управляемые базы данных](#) непосредственно в контейнерах на хостовом оборудовании. Можно развернуть несколько высокодоступных баз данных с выделенными IP-адресами на ограниченном оборудовании — при этом каждая имеет прямой доступ к CPU и хранилищу.

Экосистема Kubernetes

Нам неизвестен ни один другой дистрибутив Kubernetes с таким количеством встроенных инфраструктурных компонентов. (Серьёзно — пришлите нам ссылку, если найдёте!) Вместо того чтобы устанавливать компоненты и контроллеры вручную, вы просто выбираете [вариант Cozystack](#), подходящий для ваших нужд. Все компоненты предварительно настроены, проверены на совместимость и обновляются вместе с фреймворком Cozystack.

Начало работы

Начало работы с Cozystack: развёртывание частного облака с нуля

<https://cozystack.ru/docs/v1.5/getting-started/>

Это руководство поможет вам выполнить первое развёртывание кластера Cozystack. По ходу работы вы познакомитесь с ключевыми концепциями, научитесь пользоваться Cozystack через дашборд и CLI и получите рабочий proof-of-concept.

Руководство разделено на несколько шагов. Завершайте каждый шаг перед переходом к следующему:

Шаг	Описание
Требования: подготовьте инфраструктуру и инструменты	Подготовьте инфраструктуру и установите необходимые CLI-инструменты на свою машину перед началом этого руководства.
1. Установите Talos Linux	Установите дистрибутив Talos Linux, подготовленный для Cozystack, с помощью <code>boot-to-talos</code> — это, вероятно, самый простой способ установки.
2. Установите и инициализируйте кластер Kubernetes	Выполните bootstrap кластера Kubernetes с помощью Talm — инструмента управления конфигурацией Talos, созданного для Cozystack.
3. Установите и настройте Cozystack	Установите Cozystack, получите административный доступ, выполните базовую настройку и откройте дашборд Cozystack.
4. Создайте tenant для пользователей и команд	Создайте пользовательский tenant — основу RBAC в Cozystack — и получите доступ к нему через дашборд и Cozystack API.
5. Разверните управляемые приложения	Начните работать с Cozystack: разверните виртуальную машину, управляемое приложение и Kubernetes-кластер tenant'a.

Требования и набор инструментов

Подготовьте инфраструктуру и установите необходимые инструменты.

1. Установка Talos Linux

Установка Talos Linux на любую машину с помощью `cozystack/boot-to-talos`.

2. Установка и bootstrap кластера Kubernetes

Используйте Talm CLI, чтобы выполнить bootstrap кластера Kubernetes, готового к установке Cozystack.

3. Установка и настройка Cozystack

Установите Cozystack, получите административный доступ, выполните базовую настройку и включите UI-дашборд.

4. Создание пользовательского tenant'a и настройка доступа

Создайте пользовательский tenant — основу RBAC в Cozystack — и получите доступ к нему через дашборд и Cozystack API.

5. Развёртывание управляемых приложений, VM и Kubernetes-кластера tenant'a

Начните использовать Cozystack: разверните виртуальную машину, управляемое приложение и Kubernetes-кластер tenant'a.

[Cozystack Documentation](#)

Требования и набор инструментов | Cozystack

<https://cozystack.ru/docs/v1.5/getting-started/requirements/>

Набор инструментов

На вашей рабочей станции должны быть установлены следующие инструменты:

- **talosctl**, клиент командной строки для Talos Linux (используйте серию v1.13.x, соответствующую Cozystack 1.5.0).
- **kubectl**, клиент командной строки для Kubernetes.
- **Talm**, собственный менеджер конфигурации Talos Linux от Cozystack:

```
curl -sSL https://github.com/cozystack/talm/raw/refs/heads/main/hack/install.sh | sh -s
```

Требования к оборудованию

Для прохождения этого руководства вам потребуется следующая конфигурация:

Узлы кластера: три bare-metal сервера или виртуальные машины. Требования к оборудованию зависят от вашего сценария использования:

Минимальная

Ниже приведены базовые требования для запуска небольшой инсталляции. Минимальная рекомендуемая конфигурация для каждого узла:

Компонент	Требование
Хосты	3x физических хоста (или VM с host CPU passthrough)
Architecture	x86_64
CPU	8 ядер
RAM	24 ГБ
Основной диск	50 ГБ SSD (или RAW для VM)
Дополнительный диск	256 ГБ SSD (raw)

Подходит для:

- Dev/Test-сред
- Небольших демонстрационных конфигураций
- 1-2 tenants

- До 3 кластеров Kubernetes
- Небольшого числа ВМ или баз данных

Рекомендуемая

Для небольших production-сред рекомендуемая конфигурация каждого узла выглядит так:

Компонент	Требование
Хосты	3х физических хоста
Architecture	x86_64
CPU	16-32 ядра
RAM	64 ГБ
Основной диск	100 ГБ SSD или NVMe
Дополнительный диск	1-2 ТБ SSD или NVMe

Подходит для:

- Небольших и средних production-сред
- 5-10 tenants
- 5+ кластеров Kubernetes
- Десятков виртуальных машин или баз данных
- S3-совместимого хранилища

Оптимальная

Для средних и крупных production-сред оптимальная конфигурация каждого узла выглядит так:

Компонент	Требование
Хосты	6х+ физических хостов
Architecture	x86_64
CPU	32-64 ядра
RAM	128-256 ГБ
Основной диск	200 ГБ SSD или NVMe
Дополнительный диск	4-10 ТБ NVMe

Подходит для:

- Крупных production-сред
- 20+ tenants
- Десятков кластеров Kubernetes
- Сотен виртуальных машин и баз данных

-
- S3-совместимого хранилища

Хранилище:

- Основной диск: используется для Talos Linux, хранилища etcd и загруженных образов. Требуется низкая задержка.
- Дополнительный диск: используется для данных пользовательских приложений (ZFS pool).

ОС:

- Любой дистрибутив Linux, например Ubuntu.
- Есть и [другие способы установки](#), для которых на старте требуется либо любой Linux, либо вообще не требуется установленная ОС.

BIOS/UEFI Settings:

- Secure Boot.

Talos Linux ships pre-signed kernel modules and works with Secure Boot enabled. On non-Talos Ubuntu hosts, the default piraeus-operator flow compiles DRBD in-cluster; the resulting unsigned modules are rejected by kernel lockdown when Secure Boot is enforced. The simplest path is to disable Secure Boot in BIOS/UEFI; alternatively, follow [Ubuntu + Secure Boot](#) to pre-install dkms-signed DRBD on the host.

Сеть:

- Маршрутизируемый домен с FQDN. Если его нет, можно использовать [nip.io](#) с dash-нотацией.
- Узлы должны находиться в одном L2-сегменте сети.
- Anti-spoofing должен быть отключён. Это необходимо для MetalLB — балансировщика нагрузки, используемого в Cozystack.

Виртуальные машины:

- В настройках гипервизора должен быть включён CPU passthrough, а модель CPU должна быть установлена в host.
- Должна быть включена nested virtualization. Это требуется для виртуальных машин и Kubernetes-кластеров tenant'ов.

Более подробное описание требований к оборудованию для разных сценариев см. в разделе [Требования к оборудованию](#)

1. Установка Talos Linux

<https://cozystack.ru/docs/v1.5/getting-started/install-talos/>

Перед началом

Убедитесь, что у вас есть узлы (bare-metal серверы или виртуальные машины), соответствующие [требованиям к оборудованию](#).

Цели

На этом шаге вы установите Talos Linux на bare-metal серверы или виртуальные машины, на которых сейчас работает другой дистрибутив Linux.

В этом руководстве используется `boot-to-talos` — простой CLI-инструмент, созданный командой Cozystack для пользователей и команд, внедряющих Cozystack. Существуют и другие способы [установки Talos Linux для Cozystack](#), но они здесь не рассматриваются и описаны в отдельных руководствах.

Установка

1. Установите boot-to-talos

Установите `boot-to-talos` с помощью установочного скрипта:

```
curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh | sh -s
```

2. Запустите установку Talos

Запустите `boot-to-talos` и укажите значения конфигурации. Убедитесь, что используете сборку Talos от Cozystack, опубликованную в ghcr.io/cozystack/cozystack/talos. Для Cozystack v1.5.0 закрепленная версия Talos — v1.13.0. Переопределите значение по умолчанию в installer prompt:

```
$ boot-to-talos
Target disk [/dev/sda]:
Talos installer image [ghcr.io/cozystack/cozystack/talos:v1.11.6]: ghcr.io/cozystack/cozystack/talos:v1.13.0
Add networking configuration? [yes]:
Interface [eth0]:
IP address [10.0.2.15]:
Netmask [255.255.255.0]:
Gateway (or 'none') [10.0.2.2]:
Configure serial console? (or 'no') [ttyS0]:

Summary:
  Image: ghcr.io/cozystack/cozystack/talos:v1.13.0
  Disk: /dev/sda
  Extra kernel args: ip=10.0.2.15::10.0.2.2:255.255.255.0::eth0:::: console=ttyS0

WARNING: ALL DATA ON /dev/sda WILL BE ERASED!

Continue? [yes]:

2025/08/03 00:11:03 created temporary directory /tmp/installer-3221603450
2025/08/03 00:11:03 pulling image ghcr.io/cozystack/cozystack/talos:v1.13.0
2025/08/03 00:11:03 extracting image layers
2025/08/03 00:11:07 creating raw disk /tmp/installer-3221603450/image.raw (2 GiB)
2025/08/03 00:11:07 attached /tmp/installer-3221603450/image.raw to /dev/loop0
2025/08/03 00:11:07 starting Talos installer
2025/08/03 00:11:07 running Talos installer v1.13.0
2025/08/03 00:11:07 WARNING: config validation:
2025/08/03 00:11:07   use "worker" instead of "" for machine type
2025/08/03 00:11:07 created EFI (C12A7328-F81F-11D2-BA4B-00A0C93EC93B) size 104857600 bytes
2025/08/03 00:11:07 created BIOS (21686148-6449-6E6F-744E-656564454649) size 1048576 bytes
2025/08/03 00:11:07 created BOOT (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 104857600 bytes
2025/08/03 00:11:07 created META (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576 bytes
2025/08/03 00:11:07 formatting the partition "/dev/loop0p1" as "vfat" with label "EFI"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p2" as "zeroes" with label "BIOS"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p3" as "xfs" with label "BOOT"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p4" as "zeroes" with label "META"
2025/08/03 00:11:07 copying from io reader to /boot/A/vmlinuz
2025/08/03 00:11:07 copying from io reader to /boot/A/initramfs.xz
2025/08/03 00:11:08 writing /boot/grub/grub.cfg to disk
2025/08/03 00:11:08 executing: grub-install --boot-directory=/boot --removable --efi-directory=/boot/efi
2025/08/03 00:11:08 installation of v1.13.0 complete
2025/08/03 00:11:08 Talos installer finished successfully
2025/08/03 00:11:08 remounting all filesystems read-only
2025/08/03 00:11:08 copy /tmp/installer-3221603450/image.raw -> /dev/sda
2025/08/03 00:11:19 installation image copied to /dev/sda
2025/08/03 00:11:19 rebooting system
```

Следующий шаг

Продолжите руководство по Cozystack и [установите Kubernetes-кластер с помощью Talm](#).

Дополнительно:

- Прочитайте [обзор Talos Linux](#), чтобы понять, почему Talos Linux — оптимальный выбор ОС для Cozystack и что он даёт платформе.
- Узнайте больше о [\[boot-to-talos\]](#)(<https://cozystack.ru/docs/v1.5/install/talos/boot-to-talos/#about-the-application>).

-
- Загляните в github.com/cozystack/boot-to-talos и поставьте проекту звезду.
-

2. Установка и bootstrap кластера Kubernetes

<https://cozystack.ru/docs/v1.5/getting-started/install-kubernetes/>

Цели

К началу этого шага у нас уже есть [три узла с установленным Talos Linux](#).

В результате этого шага у вас будет установленный и настроенный Kubernetes-кластер, готовый к установке Cozystack. Также вы получите `kubeconfig` для этого кластера и выполните базовые проверки его состояния.

Установка Kubernetes

Установите и выполните bootstrap кластера Kubernetes с помощью [Talm](#) — декларативного CLI-инструмента управления конфигурацией с готовыми пресетами для Cozystack.

Этот фрагмент руководства сейчас перерабатывается. Здесь появятся упрощённые инструкции по установке Talm без дополнительных опций и редких пограничных случаев, которые описаны в основном руководстве по Talm.

Следующий шаг

Продолжите руководство по Cozystack и [установите Cozystack](#).

Дополнительно:

- Загляните в github.com/cozystack/talm и поставьте проекту звезду.
-

3. Установка и настройка Cozystack

<https://cozystack.ru/docs/v1.5/getting-started/install-cozystack/>

Цели

Это руководство описывает установку Cozystack как готовой к использованию платформы. Если вы хотите собрать собственную платформу, устанавливая только нужные компоненты, см. [руководство BYOP \(Build Your Own Platform\)](#).

На этом шаге мы установим Cozystack поверх [Kubernetes-кластера, подготовленного на предыдущем шаге](#).

Руководство проведёт вас через следующие этапы:

1. Установка оператора Cozystack
2. Подготовка файла конфигурации Cozystack и его применение
3. Настройка хранилища
4. Настройка сети
5. Развёртывание etcd, ingress и стека мониторинга в корневом tenant'e
6. Завершение развёртывания и вход в дашборд Cozystack

1. Установка оператора Cozystack

Установите оператор Cozystack с помощью Helm chart из OCI-реестра. Оператор управляет всеми компонентами платформы.

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system
```

Замените `X.Y.Z` на нужную версию Cozystack. Доступные версии перечислены на [странице релизов Cozystack](#).

Если установка прерывается из-за того, что `cozy-system` уже существует:

Helm отказывается забирать себе namespace, который создал не он, и выводит ошибку `invalid ownership metadata` (или `namespaces` в зависимости от версии Helm), если `cozy-system` остался после ранее прерванной установки или был создан вручную для этой цели.

Если namespace не управляется другим инструментом (Terraform, Argo CD, другим Helm release и т. д.), повторите команду с флагом `--take-ownership` (требуется Helm 3.17+), чтобы Helm смог принять его под управление:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--take-ownership
```

[!CAUTION]

Не используйте `--take-ownership`, если `cozy-system` уже принадлежит другой системе — Helm незаметно станет новым владельцем, а последующие обновления или удаление релиза Cozystack могут изменить или удалить namespace (и всё, что было принято под управление этим флагом) вопреки ожиданиям этой системы.

2. Подготовка и применение Platform Package

2.1. Подготовьте файл конфигурации

Теперь, когда оператор запущен, подготовим для него файл конфигурации. Возьмите пример ниже и сохраните его в файл `cozystack-platform.yaml`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        publishing:
          host: example.org
          apiServerEndpoint: api.example.org
          exposedServices:
            - dashboard
            - api
        networking:
          podCIDR: 10.244.0.0/16
          podGateway: 10.244.0.1
          serviceCIDR: 10.96.0.0/12
          joinCIDR: 100.64.0.0/16
```

Что нужно сделать:

1. Замените `example.org` в `publishing.host` и `publishing.apiServerEndpoint` на маршрутизируемое полное доменное имя (FQDN), которым вы управляете. Если у вас есть только публичные IP-адреса, вы можете использовать сервис nip.io с dash-нотацией.
2. Используйте для `networking.*` те же значения, что и на предыдущем шаге, где вы выполняли bootstrap Kubernetes-кластера через Talos. Чтобы узнать эти значения, вы можете проверить конфигурационные файлы Talos или выполнить команды `talosctl`. Значения из примера — это разумные значения по умолчанию, которые подходят для большинства случаев.

В этой конфигурации есть и другие значения, которые в рамках руководства менять не требуется. Тем не менее, вот что они означают:

- `metadata.name` должен быть равен `cozystack.cozystack-platform`, чтобы совпасть с `PackageSource`, созданным установщиком.
- `publishing.host` используется как основной домен для всех сервисов, создаваемых в Cozystack, например дашборда, Grafana и т. д.
- `publishing.apiServerEndpoint` — это endpoint Cluster API. Он используется для генерации kubeconfig-файлов для пользователей. Рекомендуется использовать поддомен от `publishing.host`.
- `spec.variant: isp-full` означает, что используется наиболее полный набор компонентов Cozystack. Подробнее о вариантах см. в [справочнике по вариантам Cozystack](#).
- `publishing.exposedServices` перечисляет сервисы, которые должны быть доступны пользователям — здесь это дашборд (UI) и API.
- `networking.*` — это внутренняя сетевая конфигурация базового Kubernetes-кластера:
 - `networking.podCIDR` — диапазон CIDR, из которого Kube-OVN выделяет IP-адреса pod'ам. Он не должен пересекаться ни с одной физической сетью.
 - `networking.podGateway` — адрес шлюза, который Kube-OVN назначает подсети pod'ов по умолчанию. Используйте адрес .1 сети podCIDR (например, 10.244.0.1 для 10.244.0.0/16).
 - `networking.serviceCIDR` — диапазон CIDR для сервисов ClusterIP. Он обязательно должен совпадать со значением `cluster.network.serviceSubnets`, которое вы использовали при bootstrap Kubernetes-кластера: это значение вшивается в kube-apiserver и не может быть легко изменено позже.
 - `networking.joinCIDR` — диапазон CIDR для join подсети Kube-OVN, внутренней сети, которая переносит трафик между узлами кластера и pod'ами. Значение по умолчанию 100.64.0.0/16 входит в общее адресное пространство RFC 6598 (100.64.0.0/10), зарезервированное для такого внутреннего использования. Меняйте его только если оно пересекается с вашей физической сетью. Подробнее см. в [справочнике по Kube-OVN join subnet](#).

Подробнее об этом файле конфигурации см. в [справочнике Platform Package](#).

По умолчанию Cozystack собирает анонимную статистику использования. Подробнее о том, какие данные собираются и как это отключить, см. в [документации по телеметрии](#).

2.2. Примените Platform Package

Примените файл конфигурации:

```
kubectl apply -f cozystack-platform.yaml
```

Во время установки можно следить за логами оператора:

```
kubectl logs -n cozy-system -l app.kubernetes.io/name=cozy-installer -f
```

2.3. Проверьте статус установки

Подождите немного, затем проверьте статус установки:


```
kubectl get packages.cozystack.io cozystack.cozystack-platform
```

Повторяйте проверку, пока во всех строках не появится `True`, как в этом примере:

```
kubectl get cozystack -A
```

Пример вывода:

cozy-cert-manager	cert-manager	4m1s	True	Release	reconciliation	succeeded
cozy-cert-manager	cert-manager-issuers	4m1s	True	Release	reconciliation	succeeded
cozy-cilium	cilium	4m1s	True	Release	reconciliation	succeeded
cozy-cluster-api	capi-operator	4m1s	True	Release	reconciliation	succeeded
cozy-cluster-api	capi-providers	4m1s	True	Release	reconciliation	succeeded
cozy-dashboard	dashboard	4m1s	True	Release	reconciliation	succeeded
cozy-grafana-operator	grafana-operator	4m1s	True	Release	reconciliation	succeeded
cozy-kamaji	kamaji	4m1s	True	Release	reconciliation	succeeded
cozy-kubeovn	kubeovn	4m1s	True	Release	reconciliation	succeeded
cozy-kubevirt-cdi	kubevirt-cdi	4m1s	True	Release	reconciliation	succeeded
cozy-kubevirt-cdi	kubevirt-cdi-operator	4m1s	True	Release	reconciliation	succeeded
cozy-kubevirt	kubevirt	4m1s	True	Release	reconciliation	succeeded
cozy-kubevirt	kubevirt-operator	4m1s	True	Release	reconciliation	succeeded
cozy-linstor	linstor	4m1s	True	Release	reconciliation	succeeded
cozy-linstor	piraeus-operator	4m1s	True	Release	reconciliation	succeeded
cozy-mariadb-operator	mariadb-operator	4m1s	True	Release	reconciliation	succeeded
cozy-metallb	metallb	4m1s	True	Release	reconciliation	succeeded
cozy-monitoring	monitoring	4m1s	True	Release	reconciliation	succeeded
cozy-postgres-operator	postgres-operator	4m1s	True	Release	reconciliation	succeeded
cozy-rabbitmq-operator	rabbitmq-operator	4m1s	True	Release	reconciliation	succeeded
cozy-redis-operator	redis-operator	4m1s	True	Release	reconciliation	succeeded
cozy-telepresence	telepresence	4m1s	True	Release	reconciliation	succeeded
cozy-victoria-metrics-operator	victoria-metrics-operator	4m1s	True	Release	reconciliation	succeeded
tenant-root	tenant-root	4m1s	True	Release	reconciliation	succeeded

Список компонентов в вашей установке может отличаться от приведённого выше, так как он зависит от выбранного варианта и версии Cozystack.

Когда у всех компонентов появится `READY: True`, можно переходить к настройке подсистем.

3. Настройка хранилища

Kubernetes нужен слой хранилища для предоставления persistent volumes приложениям, но собственного такого механизма у него нет. В качестве подсистемы хранения Cozystack использует [LINSTOR](#).

Далее мы получим доступ к интерфейсу LINSTOR, создадим storage pool'ы и определим storage class'ы.

3.1. Проверьте устройства хранения

Настройте alias для доступа к LINSTOR:

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

Выведите список узлов и проверьте их готовность:

```
linstor node list
```

Выведите список доступных пустых устройств:

```
linstor physical-disk list
```

3.2. Создайте storage pool'ы

Создайте storage pool'ы на основе ZFS:

```
linstor storage-pool create zfs srv1 data data
linstor storage-pool create zfs srv2 data data
linstor storage-pool create zfs srv3 data data
```

Рекомендуется установить для ZFS storage pool'ов `failmode=continue`, чтобы обработкой отказов диска занимался DRBD, а не ZFS.

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv1 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv2 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv3 -- zpool set failmode=continue data
```

Проверьте результат, выведя список storage pool'ов:

```
linstor storage-pool list
```

Пример вывода:

StoragePool	Node	Driver	PoolName	FreeCapacity	TotalCapacity	CanSnapshots
DfltDisklessStoragePool	srv1	DISKLESS				False
DfltDisklessStoragePool	srv2	DISKLESS				False
DfltDisklessStoragePool	srv3	DISKLESS				False
data	srv1	ZFS	data	96.41 GiB	99.50 GiB	True
data	srv2	ZFS	data	96.41 GiB	99.50 GiB	True
data	srv3	ZFS	data	96.41 GiB	99.50 GiB	True

3.3. Создайте storage class'ы

Наконец, можно создать несколько storage class'ов, один из которых будет классом по умолчанию.

Создайте файл с описанием storage class'ов. Ниже приведён разумный пример по умолчанию с двумя классами: `local` (по умолчанию) и `replicated`.

`storageclasses.yaml`:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: data
  linstor.csi.linbit.com/layerList: storage
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: replicated
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: data
  linstor.csi.linbit.com/autoPlace: "3"
  linstor.csi.linbit.com/layerList: drbd storage
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
  property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
  property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

Примените конфигурацию storage class'ов:

```
kubectl apply -f storageclasses.yaml
```

Убедитесь, что storage class'ы были успешно созданы:

```
kubectl get storageclasses
```

Пример вывода:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION
local (default)	linstor.csi.linbit.com	Delete	WaitForFirstConsumer	true
replicated	linstor.csi.linbit.com	Delete	Immediate	true

4. Настройка сети

Далее мы настроим способ доступа к кластеру Cozystack. На этом шаге есть два варианта в зависимости от доступной инфраструктуры:

- Для собственного bare metal или self-hosted VM выбирайте вариант с MetalLB. MetalLB — это балансировщик нагрузки по умолчанию в Cozystack.
- Для VM и выделенных серверов у облачных провайдеров выбирайте настройку через публичные IP. [Большинство облачных провайдеров не поддерживают MetalLB.](#)

Загляните в раздел [provider-specific installation](#). Возможно, там уже есть инструкции для вашего провайдера, подходящие для развёртывания production-ready кластера.

4.a Настройка MetalLB

В Cozystack используются три типа IP-адресов:

1. IP-адреса узлов: постоянные и действуют только внутри кластера.
2. Виртуальный floating IP: используется для доступа к одному из узлов кластера и также действует только внутри кластера.
3. IP-адреса внешнего доступа: используются LoadBalancer'ами для публикации сервисов за пределами кластера.

Сервисы с внешними IP могут публиковаться в двух режимах: L2 и BGP. Режим L2 проще, но требует, чтобы все узлы находились в одном L2-сегменте. Режим BGP более надёжен и масштабируем, но требует поддержки BGP со стороны сетевого оборудования.

Выберите диапазон неиспользуемых IP-адресов для сервисов; в примере используется диапазон [192.168.100.200-192.168.100.250](#). Если вы используете режим L2, эти IP должны либо принадлежать той же сети, что и узлы, либо до них должен быть настроен проброс трафика (маршрутизация).

Для режима BGP также потребуются IP-адреса BGP peer'ов и локальные и удалённые номера AS. В примере ниже используется [192.168.20.254](#) как IP peer'а и номера AS 65000 и 65001 как локальный и удалённый соответственно.

Создайте и примените файл с описанием пула адресов `metallb-ip-address-pool.yml`:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: cozystack
  namespace: cozy-metallb
spec:
  addresses:
    # используется для публикации сервисов за пределами кластера
    - 192.168.100.200-192.168.100.250
  autoAssign: true
  avoidBuggyIPs: false
```

```
kubectl apply -f metallb-ip-address-pool.yml
```

Создайте и примените ресурсы, необходимые для L2- или BGP-анонса.

L2Advertisement использует имя ресурса IPAddressPool, который мы создали на предыдущем шаге.

metallb-l2-advertisement.yml:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: cozystack
  namespace: cozy-metallb
spec:
  ipAddressPools:
  - cozystack
```

Примените изменения:

```
kubectl apply -f metallb-l2-advertisement.yml
```

Теперь, когда MetalLB настроен, включите `ingress` в `tenant-root`:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"components":{"ingres...
```

Чтобы убедиться, что всё настроено корректно, проверьте HelmRelease'ы `ingress` и `ingress-nginx-system`:

```
kubectl get hr -n tenant-root
```

Пример корректного вывода:

```
ingress          47m   True   Helm upgrade succeeded for release tenant-root/ingress.v3 with...
ingress-nginx-system 47m   True   Helm upgrade succeeded for release tenant-root/ingress-nginx-s...
```

Затем проверьте состояние сервиса `root-ingress-controller`:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развёрнут как `TYPE: LoadBalancer` и иметь корректный внешний IP:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
root-ingress-controller	LoadBalancer	10.96.16.141	192.168.100.200	80:31632/TCP, 443:30113/TCP

4.b. Настройка публичных IP узлов

Если ваш облачный провайдер не поддерживает MetalLB, вы можете опубликовать `ingress controller` через `externalIPs` напрямую.

Если публичные IP привязаны непосредственно к узлам, укажите их. Если публичные IP предоставляются облаком как виртуальные (Elastic IP), укажите IP-адреса внешних сетевых интерфейсов.

В примере используются 192.168.100.11, 192.168.100.12 и 192.168.100.13.

Сначала добавьте внешние IP в Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{"spec":{"components":{"p...
```

Затем включите `ingress` для корневого `tenant'a`:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"components":{"ingres...
```

Наконец, добавьте внешние IP-адреса в список `externalIPs` в конфигурации Ingress:

```
kubectl patch -n tenant-root ingresses.apps.cozystack.io ingress --type=merge -p '{"spec":{"externalIPs":["...
```

После этого ваш Ingress будет доступен по указанным IP-адресам. Проверьте это так:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развёрнут как `TYPE: ClusterIP` и содержать полный список внешних IP:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
root-ingress-controller	ClusterIP	10.96.91.83	192.168.100.11, 192.168.100.12, 192.168.100.13	80/TCP, 443/TCP

5. Завершение установки

5.1. Настройте сервисы корневого `tenant'a`

Включите `etcd` и `monitoring` для корневого `tenant'a`:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"components":{"monito...
```

5.2. Проверьте состояние и состав кластера

Проверьте созданные persistent volume'ы:

```
kubectl get pvc -A
```

Пример вывода:

NAME	STATUS	VOLUME	CAPACITY	...
data-etcd-0	Bound	pvc-4cbd29cc-a29f-453d-b412-451647cd04bf	10Gi	...

data-etcd-1	Bound	pvc-1579f95a-a69d-4a26-bcc2-b15ccdbede0d	10Gi	...
data-etcd-2	Bound	pvc-907009e5-88bf-4d18-91e7-b56b0dbfb97e	10Gi	...
grafana-db-1	Bound	pvc-7b3f4e23-228a-46fd-b820-d033ef4679af	10Gi	...
grafana-db-2	Bound	pvc-ac9b72a4-f40e-47e8-ad24-f50d843b55e4	10Gi	...
vmselect-cachedir-vmselect-longterm-0	Bound	pvc-622fa398-2104-459f-8744-565eee0a13f1	2Gi	...
vmselect-cachedir-vmselect-longterm-1	Bound	pvc-fc9349f5-02b2-4e25-8bef-6cbc5cc6d690	2Gi	...
vmselect-cachedir-vmselect-shortterm-0	Bound	pvc-7acc7ff6-6b9b-4676-bd1f-6867ea7165e2	2Gi	...
vmselect-cachedir-vmselect-shortterm-1	Bound	pvc-e514f12b-f1f6-40ff-9838-a6bda3580eb7	2Gi	...
vmstorage-db-vmstorage-longterm-0	Bound	pvc-e8ac7fc3-df0d-4692-aebf-9f66f72f9fef	10Gi	...
vmstorage-db-vmstorage-longterm-1	Bound	pvc-68b5ceaf-3ed1-4e5a-9568-6b95911c7c3a	10Gi	...
vmstorage-db-vmstorage-shortterm-0	Bound	pvc-cee3a2a4-5680-4880-bc2a-85c14dba9380	10Gi	...
vmstorage-db-vmstorage-shortterm-1	Bound	pvc-d55c235d-cada-4c4a-8299-e5fc3f161789	10Gi	...

Убедитесь, что все pod'ы запущены:

```
kubectl get pod -n tenant-root
```

пример вывода:

NAME	READY	STATUS	RESTARTS	AGE
etcd-0	1/1	Running	0	2m1s
etcd-1	1/1	Running	0	106s
etcd-2	1/1	Running	0	82s
grafana-db-1	1/1	Running	0	119s
grafana-db-2	1/1	Running	0	13s
grafana-deployment-74b5656d6-5dcvn	1/1	Running	0	90s
grafana-deployment-74b5656d6-q5589	1/1	Running	1 (105s ago)	111s

Получите публичный IP ingress controller:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Пример вывода:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
root-ingress-controller	LoadBalancer	10.96.16.141	192.168.100.200	80:31632/TCP, 443:30113/TCP

5.3 Доступ к дашборду Cozystack

Если вы включили `dashboard` в список `publishing.exposedServices` вашего Platform Package (как показано на шаге 2), дашборд Cozystack уже доступен.

Если в исходной конфигурации его не было, обновите Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json -p '[{"op": "add", "path": "/spec...']
```

Откройте `dashboard.example.org`, чтобы перейти в системный дашборд, где `example.org` — домен, указанный вами для `tenant-root`. Там вы увидите окно входа, ожидающее токен аутентификации.

Получите токен аутентификации для `tenant-root`:

```
kubectl get secret -n tenant-root tenant-root -o go-template='{{ printf "%s\n" .data.token | base64decode }}'
```

Войдите с этим токеном. Теперь вы можете пользоваться дашбордом как администратор.

Дальше вы сможете:

-
- Настроить OIDC и использовать его вместо токенов для аутентификации.
 - Создавать пользовательские `tenant`’ы и выдавать пользователям доступ через токены или OIDC.

5.4 Доступ к метрикам в Grafana

Используйте `grafana.example.org` для доступа к системному мониторингу, где `example.org` — домен, указанный для `tenant-root`. В этом примере `grafana.example.org` доступен по адресу 192.168.100.200.

логин: `admin`

получите пароль:

```
kubectl get secret -n tenant-root grafana-admin-password -o go-template='{{ printf "%s\n" .data.password | ...
```

Следующий шаг

Продолжите руководство по Cozystack и [создайте пользовательский tenant](#).

4. Создание пользовательского tenant'a и настройка доступа

<https://cozystack.ru/docs/v1.5/getting-started/create-tenant/>

Цели

На этом шаге вы создадите пользовательский tenant — пространство, в котором пользователи смогут развёртывать приложения и виртуальные машины. Вы также получите учётные данные tenant'a и войдёте как пользователь с доступом к этому tenant'у.

Предварительные требования

Перед началом:

- Завершите предыдущие шаги руководства, чтобы получить работающий [кластер Cozystack](#), в котором уже настроены хранилище, сеть и управляющий дашборд.
- Убедитесь, что у вас есть доступ к дашборду, как описано на [предыдущем шаге руководства](#).
- Если вы используете OIDC, пользователи и роли уже должны быть настроены. Подробности о работе со встроенным OIDC-сервером см. в [руководстве по OIDC](#).

Во время [установки Kubernetes](#) для Cozystack вы должны были получить административный файл `kubeconfig` для нового кластера. Держите его под рукой — позже он может пригодиться для диагностики. Однако для повседневной работы лучше создать отдельные пользовательские учётные данные.

Введение

Tenant'ы — это механизм изоляции в Cozystack. Они используются для разделения клиентов, команд или окружений. У каждого tenant'a есть собственный набор приложений и один или несколько вложенных Kubernetes-кластеров. Пользователи tenant'a имеют полный доступ к своим кластерам. При необходимости вы можете настроить квоты для каждого tenant'a, чтобы ограничить потребление ресурсов и избежать перерасхода.

Чтобы узнать о tenant'ах больше, прочитайте руководство [Core Concepts](#).

Создание tenant'a

Tenant'ы создаются с помощью приложения Cozystack с именем `Tenant`. После установки в Cozystack уже существует встроенный tenant `tenant-root`. Этот корневой tenant зарезервирован для администраторов платформы и должен использоваться только для создания дочерних tenant'ов. Технически установить приложения в `tenant-root` можно, но для production-окружений это не рекомендуется.

Через дашборд

1. Откройте дашборд от имени пользователя `tenant-root`.
2. Убедитесь, что текущий контекст установлен в `tenant-root`. При необходимости переключите контекст и перезагрузите страницу.
3. Перейдите на вкладку Catalog.
4. Найдите приложение Tenant и откройте его.
5. Ознакомьтесь с документацией, затем нажмите кнопку Deploy, чтобы перейти к странице параметров.
6. Заполните поле `name` tenant'a. Это единственный параметр, который нельзя изменить позже.
7. (Необязательно) Заполните доменное имя в `host`. Это доменное имя уже должно существовать. Убедитесь, что пользователь tenant'a имеет достаточный контроль над доменом для настройки DNS-записей. Если оставить поле пустым, по умолчанию будет использован домен `<name>.<cozystack-domain>`.
8. Отметьте флажки для установки системных приложений: `etcd`, `monitoring`, `ingress` и `seaweedfs`. Пользователи tenant'a не смогут устанавливать или удалять эти приложения — это могут делать только администраторы.

Параметр `etcd` требуется для вложенного Kubernetes. Выберите его до установки приложения Kubernetes в tenant'e. Отключайте его только если уверены, что tenant не будет использовать вложенный Kubernetes.

9. По умолчанию ресурсные квоты не задаются. Это означает отсутствие ограничений на использование ресурсов. При необходимости можно определить квоты, чтобы избежать их перерасхода.
10. Нажмите Deploy, чтобы установить приложение tenant'a в корневой tenant.

Через kubectl

Создайте манифест HelmRelease для tenant'a. В качестве отправной точки можно взять манифест, созданный через дашборд:

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: tenant-team1
  namespace: tenant-root
spec:
  chart:
    spec:
      chart: tenant
      reconcileStrategy: Revision
      sourceRef:
        kind: HelmRepository
        name: cozystack-apps
        namespace: cozy-public
      version: 1.9.1
  interval: 0s
  values:
    etcd: true
    host: team1.example.org
    ingress: true
    monitoring: false
    resourceQuotas: {}
    seaweedfs: false
```

Примените манифест:

```
# Используйте kubeconfig корневого tenant'a
export KUBECONFIG=./kubeconfig-tenant-root

# Примените манифест
kubectl -n tenant-root apply -f hr-tenant-team1.yaml
```

Примечание об изоляции:

Сетевые политики Cilium в Cozystack v1.0+ всегда изолируют соседние tenant'ы друг от друга — поля `isolated` больше нет ни в форме дашборда, ни в значениях `HelmRelease`. Pod'ы внутри namespace tenant'a также по умолчанию не могут обращаться к `kube-apiserver`, а при создании tenant'a с `etcd: true` ещё и к собственному `etcd`. Чтобы разрешить pod'у один из этих путей, добавьте ему метку `policy.cozystack.io/allow-to-apiserver: "true"` или `policy.cozystack.io/allow-to-etcd: "true"` соответственно. Полную таблицу и пример см. в примечаниях к обновлению: [Tenant isolated flag removed](#).

Вы можете помогать пользователям tenant'ов с установкой баз данных или вложенных Kubernetes-кластеров. Администратор может переключать контекст в дашборде и получать доступ к любому tenant'у. Пользователи tenant'a, в свою очередь, могут работать только со своим tenant'ом и его дочерними tenant'ами.

Получение kubeconfig tenant'a

Пользователям tenant'a нужен файл `kubeconfig` для доступа к своему Kubernetes-кластеру. Способ его получения зависит от того, включён ли OIDC в вашей установке Cozystack.

Если OIDC включён

Вы можете получить `kubeconfig` напрямую из дашборда, как описано в [руководстве по OIDC](#).

Если OIDC выключен

В этом случае администратору нужно получить токен `service account` из namespace `tenant'a`. Секрет с токеном имеет то же имя, что и `tenant`.

Чтобы получить токен для `tenant'a` с именем `team1`, выполните:

```
kubectl -n tenant-team1 get secret tenant-team1 -o json | jq -r '.data.token | @base64d'
```

Затем подставьте этот токен в шаблон `kubeconfig` и сохраните файл под именем `kubeconfig-tenant-<name>.yaml`.

Обязательно задайте namespace по умолчанию равным имени `tenant'a`. Многие GUI-клиенты будут показывать ошибки доступа, если namespace не указан явно.

Тот же токен можно использовать и для входа пользователя `tenant'a` в дашборд Cozystack, если OIDC отключён.

Получение kubeconfig вложенного Kubernetes

Как правило, администраторам не нужно получать `kubeconfig` для вложенных Kubernetes-кластеров. Такие кластеры устанавливаются самим пользователем `tenant'a` внутри его namespace. Пользователи `tenant'a` полностью контролируют свои вложенные Kubernetes-окружения.

Чтобы получить доступ к вложенному Kubernetes-кластеру, пользователь `tenant'a` может скачать `kubeconfig` прямо со страницы соответствующего приложения в дашборде.

5. Развертывание приложений

<https://cozystack.ru/docs/v1.5/getting-started/deploy-app/>

Содержимое этой страницы недоступно в офлайн-версии. Откройте онлайн-версию по ссылке выше.

Изучение Cozystack

Изучение Cozystack

<https://cozystack.ru/docs/v1.5/guides/>

Ключевые концепции

Познакомьтесь с ключевыми концепциями Cozystack: management cluster, tenants, variants и lifecycle PackageSource/Package.

Архитектура Cozystack и стек платформы

Узнайте о ключевых компонентах, которые обеспечивают функциональность и гибкость Cozystack

Система tenant'ов

Узнайте о tenants и о том, как Cozystack помогает управлять ресурсами и повышать безопасность.

Управление ресурсами в Cozystack

Как CPU, memory и presets работают для VM, Kubernetes-кластеров и managed workloads в Cozystack; и как перенастраивать ресурсы через UI, CLI или API.

Talos Linux в Cozystack

Узнайте, почему Cozystack использует Talos Linux как основу Kubernetes-кластеров. Познакомьтесь с преимуществами Talos Linux: надежностью, масштабируемостью и оптимизацией под Kubernetes.

Сценарии использования

Сценарии использования Cozystack.

Ключевые концепции

<https://cozystack.ru/docs/v1.5/guides/concepts/>

Cozystack — open-source, Kubernetes-native платформа, которая превращает bare-metal или виртуальную инфраструктуру в полноценное multi-tenant cloud. В основе платформы лежит несколько базовых building blocks:

- management cluster, в котором работает сама платформа;
- tenants, обеспечивающие строгую иерархическую изоляцию;
- tenant clusters, дающие пользователям собственные Kubernetes control planes;
- богатый каталог managed applications и виртуальных машин;
- variants, собирающие эти компоненты в готовый stack.

Понимание того, как эти концепции связаны друг с другом, поможет планировать, разворачивать и эксплуатировать Cozystack, независимо от того, строите ли вы внутреннюю developer platform или public cloud service.

Management Cluster

Cozystack — это система сервисов, работающих в Kubernetes-кластере, обычно развернутом поверх Talos Linux на bare metal или виртуальных машинах. Такой Kubernetes-кластер называется management cluster, чтобы подчеркнуть его роль и отличить от tenant Kubernetes clusters. Полный доступ к management cluster есть только у администраторов Cozystack.

Management cluster используется для развертывания заранее настроенных приложений: tenants, системных компонентов, managed apps, VM и tenant clusters. Пользователи Cozystack могут взаимодействовать с management cluster через dashboard и API, а также разворачивать managed applications. Однако у них нет административных прав, и они не могут разворачивать произвольные приложения непосредственно в management cluster.

Tenant

Tenant в Cozystack — основная единица изоляции и безопасности, похожая на Kubernetes namespace, но с расширенной областью действия. Каждый tenant представляет изолированную среду со своими ресурсами, сетью и RBAC (role-based access control). Некоторые cloud providers используют для похожей сущности термин “projects”.

Когда Cozystack используется для построения private cloud и внутренней development platform, tenant обычно принадлежит команде или подкоманде. В hosting business, где Cozystack является основой public cloud, tenant может принадлежать клиенту.

Подробнее: [Tenant System](#).

Tenant Cluster

Пользователи могут разворачивать отдельные Kubernetes-кластеры в своих tenants. Это не namespaces management cluster, а полноценные Kubernetes-in-Kubernetes кластеры.

Tenant clusters — это то, что многие cloud providers называют “managed Kubernetes”. Они используются как development, testing и production environments.

Подробнее: [tenant Kubernetes clusters](#).

Managed Applications

Cozystack поставляется с каталогом managed applications (services), которые можно разворачивать на платформе с минимальными усилиями. В него входят relational databases (PostgreSQL, MySQL/MariaDB), NoSQL/queues (Redis, NATS, Kafka, RabbitMQ), HTTP cache, load balancer и другие сервисы.

Tenants, tenant Kubernetes clusters и VM также являются managed applications с точки зрения Cozystack.

Подробнее: [managed applications](#).

Cozystack API

Вместо proprietary API или управления только через UI Cozystack предоставляет функциональность через [Kubernetes Custom Resources](#) и стандартный Kubernetes API, доступный через REST API, клиент `kubectl` и Cozystack dashboard.

Такой подход хорошо сочетается с role-based access control. Неадминистративные пользователи могут использовать `kubectl` для доступа к management cluster, но их kubeconfig разрешает им создавать custom resources только в своих tenants.

Подробнее: [Cozystack API](#).

Variants

Variants — это заранее определенные конфигурации Cozystack, которые задают, какие bundles и components включены. Каждый variant тестируется, версионизируется и гарантированно работает как единое целое. Они упрощают установку, снижают риск неправильной конфигурации и помогают выбрать подходящий набор функций для вашего развертывания.

Подробнее: [Variants](#).

PackageSource и Package

`PackageSource` и `Package` — две Custom Resource Definitions (CRDs), которые управляют всем lifecycle приложений в Cozystack.

- `PackageSource` (cluster-scoped) определяет, что доступно: он ссылается на Flux source (`OCIRepository` или `GitRepository`), который опрашивает внешний registry, перечисляет variants, объявляет dependencies и задает components из которых состоит каждый variant.
- `Package` (cluster-scoped) определяет, что развернуто: он выбирает variant из `PackageSource`, передает per-component configuration (values) и управляет жизненным циклом (установка, обновление, удаление) набора компонентов.

Вместе они образуют declarative pipeline: external charts проходят через Flux sources и artifact generators, превращаясь в готовые к установке HelmReleases.

OCIRepositories: Platform и Packages

Cozystack использует два ресурса `OCIRepository`, чтобы управлять update flow и гарантировать выполнение миграций.

Начальный OCIRepository (cozystack-platform)

Создается `cozystack-operator` во время bootstrap. Operator получает platform source URL (например, `oci://ghcr.io/cozystack/cozystack/cozystack-packages`) и создает `OCIRepository` с именем `cozystack-platform`. Этот repository указывает на artifact platform chart, настраивается через значения installer (`platformSourceUrl`, `platformSourceRef`) и предоставляет platform chart, который создаст migrations и вторичный `OCIRepository`.

Вторичный OCIRepository (cozystack-packages)

Создается Helm chart платформы (`packages/core/platform/templates/repository.yaml`). Он копирует spec из `cozystack-platform` и создает новый `OCIRepository` с именем `cozystack-packages`. Этот repository используется всеми `PackageSources` (`networking`, `monitoring`, `postgres-operator` и т. д.).

Порядок миграций

Схема с двумя repositories гарантирует, что system migrations выполняются до обновления любых компонентов платформы:

1. Installer Chart (helm install) — `cozystack-operator` запускается
2. Начальный `OCIRepository` (`cozystack-platform`) — создается operator'ом
3. Platform Chart из начального `OCIRepository`
 - Pre-upgrade hooks: последовательный запуск migrations
 - Обновление ConfigMap `cozystack-version`
4. Вторичный `OCIRepository` (`cozystack-packages`) — создается platform chart
5. `PackageSources` ссылаются на `cozystack-packages`
6. HelmReleases системных компонентов разворачиваются

Когда выходит новая версия платформы и кластер обновляется:

1. Начальный OCIRepository (`cozystack-platform`) предоставляет новый platform chart.
2. Во время `helm upgrade` pre-upgrade hooks из platform chart последовательно выполняют migrations (от текущей версии к целевой).
3. Каждый migration script выполняет необходимые преобразования и обновляет ConfigMap `cozystack-version`.
4. После завершения migrations platform chart создает или обновляет OCIRepository `cozystack-packages`.
5. PackageSources ссылаются на `cozystack-packages` и запускают reconciliation системных компонентов.

Это гарантирует, что migrations выполняются до обновления компонентов, а migration scripts берутся из новой версии платформы.

Reconciliation Flow

Полная цепочка reconciliation от внешнего registry до работающих Kubernetes resources:

1. Внешний Helm Registry (OCI Registry или Git Repo)
2. Flux Source (OCIRepository / GitRepository) — периодически опрашивает registry
3. PackageSource (cluster-scoped) — определяет variants, dependencies, libraries и components. References Flux Source via `sourceRef`.
4. ArtifactGenerator (namespace `cozy-system`) — собирает `ExternalArtifact` для каждого component.
5. `ExternalArtifact` — собранный Helm chart, готовый к установке.
6. Package (cluster-scoped) — выбирает variant + per-component values.
7. HelmRelease (namespace-scoped) — ссылается на `ExternalArtifact` через `chartRef`.
8. Kubernetes Resources (Pods, Services, ConfigMaps, Secrets, ...)

Имена `ExternalArtifacts` строятся по шаблону `<packagesource>-<variant>-<component>`, при этом точки заменяются дефисами, чтобы соответствовать правилам именования Kubernetes. Например, PackageSource `cozystack.keycloak` с variant `default` и component `keycloak` создает `cozystack-keycloak-default-keycloak`.

Package Dependencies

Variants в PackageSource могут объявлять `dependsOn`, чтобы отложить создание HelmRelease до готовности всех dependencies.

Если все dependencies имеют статус `Ready`, dependent Package продолжает создание своего HelmRelease. Иначе Package остается в состоянии ожидания.

Dependencies в PackageSource используются на двух уровнях:

1. Variant-level (`spec.variants.dependsOn[]`): ссылается на имена других Packages. PackageReconciler проверяет, что все dependencies готовы, прежде чем создавать HelmReleases для текущего Package. Поле `spec.ignoreDependencies` в Package может отключить эту проверку для отдельных dependencies.
2. Component-level (`spec.variants.components.install.dependsOn[]`): преобразуется в поле `spec.dependsOn[]` ресурса HelmRelease. Эти dependencies обеспечивают правильный порядок установки components внутри одного Package.

Управление namespaces и values

Когда PackageReconciler создает HelmReleases для Package, он также:

1. Создает namespaces, объявленные в полях component `Install.namespace`, и задает labels вроде `cozystack.io/system=true` и `pod-security.kubernetes.io/enforce=privileged`, где это необходимо.
2. Передает cluster-wide configuration через Secret `cozystack-values`. CozyValuesReplicator следит за этим Secret в `cozy-system` и копирует его в каждый namespace с label `cozystack.io/system=true`. Каждый HelmRelease ссылается на этот Secret через `valuesFrom`, чтобы все компоненты получали согласованную конфигурацию платформы.

Update Flow

Когда новая версия chart публикуется в registry, обновления автоматически проходят через reconciliation pipeline:

1. Публикация новой версии chart в OCI registry
2. Flux обнаруживает изменение digest (периодический polling)
3. ArtifactGenerator пересобирает ExternalArtifact
4. HelmRelease запускает `helm upgrade`
5. Новая версия развернута

Чтобы ускорить синхронизацию и не ждать следующего polling interval (Flux sources находятся в namespace `cozy-system`):

```
flux reconcile source oci <name>
```

Чтобы изменить values приложения без смены версии chart, patch'ите Package CR напрямую. Values задаются в `spec.components.<component-name>.values`:

```
kubectl patch package <name> --type merge --patch '{"spec": {"components": {"<component-name>": {"values": ...
```

Rollback Strategies

Есть три подхода к rollback Package, от наиболее до наименее рекомендуемого:

1. GitOps rollback (рекомендуется): опубликуйте предыдущую версию chart в OCI registry. Flux обнаружит изменение и запустит upgrade к предыдущей версии.
2. Emergency rollback: выполните `helm rollback` напрямую и suspend HelmRelease, чтобы Flux не применил снова более новую версию. Это обходит GitOps-flow и должно использоваться только для быстрого восстановления.

```
```bash
```

```
helm rollback <release> <revision>
```

```
flux suspend helmrelease <name>
```

```
```
```

3. Controlled rollback: сначала suspend HelmRelease, затем выполните `helm rollback`, исправьте chart в registry и resume HelmRelease.

Автоматическое восстановление FluxPlunger

FluxPlunger — компонент автоматического восстановления, который обрабатывает распространенную ошибку Helm `'has no deployed releases'`:

1. HelmRelease переходит в error state `'has no deployed releases'`
2. FluxPlunger обнаруживает ошибку
3. Находит последний release Secret
4. Suspends HelmRelease
5. Удаляет устаревший Secret
6. Записывает обработанную версию в annotation (crash recovery)
7. Resumes HelmRelease
8. Flux выполняет чистую переустановку

Если FluxPlunger аварийно завершится в середине процесса, annotation `flux-plunger.cozystack.io/last-processed-version` позволит ему корректно продолжить работу при следующей reconciliation.

Summary lifecycle operations

| Действие | Что сделать | Кто обрабатывает |
|------------------------|---|----------------------------------|
| Обновить версию chart | Опубликовать новый chart в OCI registry | Flux + ArtifactGenerator |
| Обновить values | Patch Package CR | Package controller + HelmRelease |
| Ускорить синхронизацию | <code>flux reconcile source oci <name></code> | Ручной trigger |
| GitOps rollback | Опубликовать предыдущую версию chart в registry | Flux (standard flow) |

| | | |
|-----------------------------|--|------------------------|
| Emergency rollback | <code>helm rollback + suspend HelmRelease</code> | Ручное вмешательство |
| Восстановиться после ошибки | Автоматически через FluxPlunger | FluxPlunger controller |

cozypkg CLI

`cozypkg` — command-line tool для интерактивного управления ресурсами Package и PackageSource. Он выполняет dependency resolution, выбор variant и безопасное удаление с cascade analysis, чтобы вам не приходилось вручную писать YAML manifests.

Установка

Готовые binaries для Linux, macOS и Windows (amd64 и arm64) доступны в составе каждого релиза Cozystack.

Команды

cozypkg add — установка packages

Устанавливает один или несколько packages с автоматическим dependency resolution:

```
cozypkg add <packagesource-name>
```

Для каждого package команда `cozypkg add`:

1. Находит соответствующий PackageSource в кластере.
2. Предлагает выбрать variant, если доступно несколько вариантов.
3. Разрешает все transitive dependencies (topological sort).
4. Создает Package resources в порядке dependencies-first, пропуская уже установленные packages.

cozypkg list — список packages

```
cozypkg list
```

cozypkg del — удаление packages

Безопасно удаляет packages с анализом reverse dependencies:

```
cozypkg del <package-name>
```

Перед удалением `cozypkg del` показывает, какие другие установленные packages зависят от целевого, и запрашивает подтверждение. Это предотвращает случайную поломку платформы.

cozypkg dot — визуализация dependencies

Генерирует dependency graph в формате GraphViz DOT:

```
cozyrpg dot | dot -Tpng > dependencies.png  
cozyrpg dot --installed --components # component-level graph установленных packages
```

Отсутствующие dependencies подсвечиваются красным, что помогает быстро находить неполные установки.

Как cozyrpg вписывается в lifecycle

cozyrpg работает исключительно с custom resources `Package` и `PackageSource`. Он не взаимодействует с `HelmReleases`, `ArtifactGenerators` или `Flux sources` напрямую — ими управляют controllers, описанные выше.

Вы всегда можете управлять `Package resources` через `kubectl` вместо cozyrpg. CLI лишь автоматизирует выбор variant, порядок dependencies и cascade analysis.

Архитектура Cozystack и стек платформы

<https://cozystack.ru/docs/v1.5/guides/platform-stack/>

Эта статья объясняет состав Cozystack через четыре слоя и показывает роль и ценность каждого компонента в platform stack.

Обзор

Чтобы понять состав Cozystack, удобно рассматривать его как набор подсистем, расположенных слоями от hardware до пользовательских сервисов:

[!Cozystack Architecture Layers](#)

Слой 1: ОС и hardware

Это базовый слой, который обеспечивает работу кластера на bare metal. Он состоит из Talos Linux и Kubernetes-кластера, установленного на Talos.

Talos Linux

Talos Linux — Linux-дистрибутив, созданный и оптимизированный для одной цели: запускать Kubernetes. Он обеспечивает основу надежности и безопасности в кластере Cozystack. Его использование позволяет Cozystack ограничить technology stack, повышая стабильность и безопасность.

Подробнее см. в разделе [Talos Linux](#).

Kubernetes

Kubernetes уже стал своего рода de facto standard для управления server workloads.

Одна из ключевых особенностей Kubernetes — удобный и единый API, понятный всем (все описывается YAML). Кроме того, Kubernetes использует лучшие software design patterns, обеспечивающие постоянное восстановление в любых ситуациях (reconciliation method) и эффективное масштабирование на большое количество серверов.

Это полностью решает проблему интеграции, так как существующие virtualization platforms обычно имеют устаревшие и довольно сложные APIs, которые нельзя расширять без изменения исходного кода. В результате часто приходится создавать собственные custom solutions, что требует дополнительных усилий.

Слой 2: Infrastructure Services

Второй слой содержит ключевые компоненты, отвечающие за storage, networking и virtualization. Добавление этих компонентов к базовому Kubernetes-кластеру делает его значительно функциональнее.

Flux CD

FluxCD предоставляет простой и единообразный интерфейс как для установки всех компонентов платформы, так и для управления их lifecycle. Разработчики Cozystack выбрали FluxCD как core element платформы, считая его новым industry standard для platform engineering.

KubeVirt

KubeVirt добавляет в Cozystack возможности virtualization. Он позволяет создавать virtual machines и worker nodes для tenant Kubernetes clusters.

KubeVirt — проект, начатый глобальными industry leaders с общим видением объединить Kubernetes и мир virtualization. Он расширяет возможности Kubernetes, предоставляя удобные abstractions для запуска и управления virtual machines, а также связанными сущностями: snapshots, presets, virtual volumes и другими.

Сейчас проект KubeVirt совместно развивают такие известные компании, как RedHat, NVIDIA и ARM.

DRBD и LINSTOR

DRBD и LINSTOR — основа replicated storage в Cozystack.

DRBD — самый быстрый replication block storage, работающий прямо в Linux kernel. DRBD отвечает только за репликацию данных, а для надежного хранения используются проверенные временем технологии, такие как LVM или ZFS. Kernel module DRBD включен в mainline Linux kernel и более десяти лет применяется для построения fault-tolerant systems.

DRBD управляется через LINSTOR — систему, интегрированную с Kubernetes. LINSTOR является management layer для создания virtual volumes на базе DRBD. Он позволяет управлять сотнями или тысячами virtual volumes в кластере Cozystack.

Kube-OVN

Сетевая функциональность Cozystack основана на Kube-OVN и Cilium.

OVN — свободная реализация virtual network fabric для Kubernetes и OpenStack на базе технологии Open vSwitch. С Kube-OVN вы получаете надежную и функциональную virtual network, которая обеспечивает изоляцию между tenants и предоставляет floating addresses for virtual machines.

В будущем это позволит бесшовно интегрироваться с другими clusters и customer network services.

Cilium

Использование Cilium вместе с OVN обеспечивает максимально эффективные и гибкие network policies, а также производительную services network in Kubernetes за счет offloaded Linux network stack на базе современной технологии eBPF.

Cilium — очень перспективный проект, широко используемый и поддерживаемый множеством cloud providers по всему миру.

Слой 3: Platform Services

Это компоненты, которые предоставляют пользовательскую функциональность Cozystack и его managed applications.

OpenAPI UI

OpenAPI UI предоставляет основной web interface для развертывания и управления приложениями в Cozystack. Он служит главным dashboard, через который пользователи взаимодействуют с Cozystack API в удобном интерфейсе.

Интерфейс построен поверх OpenAPI specifications Cozystack и автоматически генерирует forms и документацию для всех доступных managed applications. Пользователи могут разворачивать databases, Kubernetes clusters, virtual machines и другие services прямо через dashboard без необходимости вручную писать YAML manifests.

Dashboard также интегрируется с OIDC authentication через Keycloak, обеспечивая безопасный single sign-on доступ к платформе.

Kamaji

Cozystack использует Kamaji Control Plane для развертывания tenant Kubernetes clusters. Kamaji предоставляет простой и удобный способ запускать все необходимые Kubernetes control-plane components в containers. Worker nodes затем подключаются к этим control planes и выполняют пользовательские workloads.

Подход, разработанный проектом Kamaji, повторяет дизайн современных clouds и обеспечивает security by design: у конечных пользователей нет control plane nodes для их clusters.

Grafana

Grafana вместе с Grafana Loki и расширением OnCall предоставляет единый интерфейс для Observability. Он позволяет удобно просматривать charts, logs и управлять alerts для инфраструктуры и приложений.

Victoria Metrics

Victoria Metrics позволяет максимально эффективно собирать, хранить и обрабатывать metrics в формате Open Metrics, делая это эффективнее Prometheus в той же конфигурации.

MetalLB

MetalLB — load balancer по умолчанию для Kubernetes. С его помощью ваши services могут получать public addresses, доступные не только изнутри, но и снаружи сети вашего кластера.

HAProxy

HAProxy — продвинутый и широко известный TCP balancer. Он непрерывно проверяет доступность services и аккуратно балансирует production traffic между ними в real time.

См. справочник приложения: [TCP Balancer](#).

SeaweedFS

SeaweedFS — простая и хорошо масштабируемая distributed file system, созданная для двух основных целей: хранить миллиарды файлов и отдавать их быстрее. Она обеспечивает доступ O(1), обычно за одну операцию чтения с диска.

Kubernetes Operators

Cozystack включает набор Kubernetes operators, используемых для управления system services и managed applications.

Слой 4: Пользовательские сервисы

Cozystack поставляется с набором пользовательских приложений, заранее настроенных на надежность и resource efficiency, с включенными monitoring и observability:

- [Tenant Kubernetes clusters](#) — полнофункциональные managed Kubernetes clusters для development и production workloads.
- [Managed applications](#), например databases и queues.
- [Virtual machines](#) с поддержкой Linux и Windows OS.
- [Networking appliances](#), включая VPN, HTTP cache, TCP load balancer и virtual routers.

Managed Kubernetes

Cozystack разворачивает и управляет tenant Kubernetes clusters как самостоятельными приложениями внутри изолированной среды каждого tenant. Эти clusters полностью отделены от root management cluster и предназначены для развертывания tenant-specific или customer-developed applications.

Развертывание включает следующие компоненты:

- Kamaji Control Plane: [Kamaji](#) — open-source проект, который упрощает развертывание Kubernetes control planes как pods внутри root cluster. Каждый control plane pod включает

основные компоненты: `kube-apiserver`, `controller-manager` и `scheduler`, что обеспечивает эффективную multi-tenancy и использование ресурсов.

- Etcd Cluster: dedicated etcd cluster разворачивается с помощью [aenix-io/etcd-operator](#) от Aenix. Он предоставляет надежное и масштабируемое key-value storage для Kubernetes control plane.
- Worker Nodes: virtual machines создаются как worker nodes. Эти узлы настраиваются для подключения к tenant Kubernetes cluster, позволяя разворачивать и управлять workloads.

Такая архитектура обеспечивает изолированные, масштабируемые и эффективные Kubernetes environments для каждого tenant.

- Поддерживаемая версия: Kubernetes v1.32.4
- Operator: [aenix-io/etcd-operator](#) v0.4.2
- Справочник managed application: [Kubernetes](#)

Virtual Machines

В Cozystack virtualization features работают на базе [KubeVirt](#). Cozystack предоставляет несколько applications для virtualization functionality:

- [Virtual machine instance](#) с более расширенной конфигурацией.
- [Virtual machine disk](#) с выбором image sources.
- [VM image \(Golden Disk\)](#), который делает OS images локально доступными, ускоряя создание ВМ и экономя network traffic.

ClickHouse

ClickHouse — open source высокопроизводительная column-oriented SQL database management system (DBMS). Он используется для online analytical processing (OLAP). В платформе Cozystack для предоставления ClickHouse используется Altinity operator.

- Поддерживаемая версия: 24.9.2.42
- Kubernetes operator: [Altinity/clickhouse-operator](#) v0.25.0
- Website: [clickhouse.com](#)
- Справочник managed application: [ClickHouse](#)

Kafka

Apache Kafka — open-source distributed event streaming platform. Она предоставляет unified, high-throughput, low-latency platform для обработки real-time data feeds. Cozystack использует [Strimzi](#) для запуска Apache Kafka cluster в Kubernetes в разных deployment configurations.

- Поддерживаемая версия: Apache Kafka 3.9.0
- Kubernetes operator: [strimzi/strimzi-kafka-operator](#) v0.45.0
- Website: [kafka.apache.org](#)
- Справочник managed application: [Kafka](#)

MariaDB (MySQL fork)

MySQL — широко используемая и хорошо известная relational database. Реализация в платформе позволяет создавать replicated MariaDB cluster. Этим cluster управляет набирающий популярность mariadb-operator.

Для каждой database доступен интерфейс настройки users, их permissions, а также schedules для создания backups с помощью [Restic](#) — одного из наиболее эффективных доступных инструментов.

- Поддерживаемая версия: MariaDB 11.4.3
- Kubernetes operator: [mariadb-operator/mariadb-operator](#) v0.18.0
- Website: [mariadb.com](#)
- Справочник managed application: [MySQL](#)

NATS Messaging

NATS — open-source, простая, безопасная и высокопроизводительная messaging system. Она предоставляет data layer для cloud native applications, IoT messaging и microservices architectures.

- Поддерживаемая версия: NATS 2.10.17
- Website: [nats.io](#)
- Справочник managed application: [NATS](#)

PostgreSQL

Сегодня PostgreSQL — самая популярная relational database. Ее platform-side реализация включает self-healing replicated cluster. Управление выполняется с помощью популярного в сообществе CloudNativePG operator.

- Поддерживаемая версия: PostgreSQL 17
- Kubernetes operator: [cloudnative-pg/cloudnative-pg](#) v1.24.0
- Website: [cloudnative-pg.io](#)
- Справочник managed application: [PostgreSQL](#)

RabbitMQ

RabbitMQ — широко известный message broker. Platform-side реализация позволяет создавать failover clusters под управлением официального RabbitMQ operator.

- Поддерживаемая версия: RabbitMQ 4.1.0+ (latest stable version)
- Kubernetes operator: [rabbitmq/cluster-operator](#) v1.10.0
- Website: [rabbitmq.com](#)
- Справочник managed application: [RabbitMQ](#)

Redis

Redis — наиболее часто используемое key-value in-memory data store. Чаще всего он применяется как cache, storage для user sessions или message broker. Platform-side реализация включает replicated failover Redis cluster с Sentinel. Им управляет [spotahome/redis-operator](#).

-
- Поддерживаемая версия: Redis 6.2.6+ (based on [alpine](#))
 - Kubernetes operator: [spotahome/redis-operator](#) v1.3.0-rc1
 - Website: [redis.io](#)
 - Справочник managed application: [Redis](#)

VPN Service

VPN Service работает на базе Outline Server — продвинутого и удобного VPN solution. Внутри он известен как “Shadowbox”, что упрощает настройку и предоставление Shadowsocks servers. Он запускает Shadowsocks instances по запросу.

Протокол Shadowsocks использует symmetric encryption algorithms. Это обеспечивает быстрый доступ в интернет и затрудняет анализ и блокировку трафика через DPI (Deep Packet Inspection).

- Поддерживаемая версия: Outline Server, v1.12.3+ (stable)
- Website: [getoutline.org](#)
- Справочник managed application: [VPN](#)

HTTP Cache

HTTP caching service на базе Nginx помогает защитить приложение от перегрузки с помощью мощного Nginx. Nginx традиционно используется для построения CDNs и caching servers.

Platform-side реализация обеспечивает эффективное caching без использования clustered file system. Она также поддерживает horizontal scaling без дублирования данных на нескольких servers.

- Включенные версии: Nginx 1.25.3, HAProxy latest stable.
- Website: [nginx.org](#)
- Справочник managed application: [HTTP Cache](#)

TCP Balancer

Managed TCP Load Balancer service обеспечивает развертывание и управление load balancers. Он эффективно распределяет входящий TCP traffic между несколькими backend servers, обеспечивая high availability и optimal resource utilization.

TCP Load Balancer service работает на базе [HAProxy](#) — зрелого и надежного TCP load balancer.

- Справочник managed application: [TCP balancer](#)
- Docs: [HAProxy Documentation](#)

Tenants

Tenants в Cozystack реализованы как managed applications. Подробнее о tenants см. в разделе [Tenant System](#).

Система tenant'ов

<https://cozystack.ru/docs/v1.5/guides/tenants/>

Введение

Tenant в Cozystack — основная единица изоляции и безопасности, похожая на Kubernetes namespace, но с расширенной областью действия. Каждый tenant представляет изолированную среду со своими ресурсами, сетью и RBAC (role-based access control). Некоторые cloud providers используют для похожей сущности термин “projects”.

Администраторы и пользователи Cozystack создают tenants с помощью приложения Tenant из application catalog. Tenants можно создавать через Cozystack dashboard (UI), `kubectl` или напрямую через Cozystack API.

Вложенность tenants

Все пользовательские tenants принадлежат базовому tenant'у `root`. Tenant `root` используется только для развертывания пользовательских tenants и системных компонентов. Все пользовательские приложения разворачиваются в соответствующих tenants.

Tenants могут быть вложенными: администратор tenant'а может создавать sub-tenants как приложения из каталога Cozystack. Parent tenants могут делиться ресурсами с children и наблюдать за их приложениями. В свою очередь, children могут использовать сервисы parent tenant.

Совместное использование cluster services

В tenants могут быть развернуты `cluster services`. Cluster services — это middleware services, предоставляющие базовую функциональность tenants и user-facing applications.

В tenant `root` по умолчанию есть набор сервисов вроде `etcd`, `ingress` и `monitoring`. Tenants нижнего уровня могут запускать собственные cluster services или использовать сервисы parent tenant.

Например, пользователь Cozystack создает следующие tenants и services:

- Tenant `foo` внутри tenant `root`, со своими экземплярами `etcd` и `monitoring`.
- Tenant `bar` внутри tenant `foo`, со своим экземпляром `etcd`.
- Tenant `Kubernetes cluster` и `Postgres database` в tenant `bar`.

Всем приложениям нужны сервисы вроде `ingress` и `monitoring`. Так как у tenant `bar` этих сервисов нет, приложения будут использовать сервисы parent tenant.

Эта конфигурация будет разрешена так:

- Tenant `Kubernetes cluster` будет хранить данные в собственном `etcd` сервиса tenant `bar`.

- Все metrics будут собираться в monitoring stack parent tenant foo.
- Доступ к приложениям будет идти через общий ingress, развернутый в tenant root.

Сетевая изоляция между tenants

Каждый namespace tenant'a изолирован от sibling tenants с помощью Cilium network policies, автоматически устанавливаемых chart'ом tenant. Отключить это для отдельного tenant нельзя: предыдущее поле `isolated` было удалено в Cozystack v1.0. Pod'ы внутри namespace tenant'a также по умолчанию не могут обращаться к kube-apiserver, а также к собственному etcd tenant'a, если tenant создан с `etcd: true`. Для доступа им нужно явно включить одну из двух labels pod'a:

- `policy.cozystack.io/allow-to-apiserver: "true"` — доступ к in-cluster Kubernetes API (для operators, dashboards и т. д.).
- `policy.cozystack.io/allow-to-etcd: "true"` — доступ к собственному etcd tenant'a (применимо только если tenant создан с `etcd: true`).

См. [Tenant isolated flag removed](#) в upgrade notes, где приведен полный пример.

Настройка tenant services

Флаги tenant `etcd`, `monitoring`, `ingress` и `seaweedfs` устанавливают default конфигурацию каждого сервиса. После запуска сервиса вы можете изменить его spec: добавить storage pools, настроить resource quotas, переключить topology SeaweedFS на MultiZone и т. д., редактируя underlying application CR. Такие ручные изменения не перезаписываются при reconciliation parent Tenant.

Workflow состоит из двух шагов:

1. Включите flag в tenant (checkbox в Dashboard или `etcd: true / seaweedfs: true / ...` в `spec.values` manifest'a Tenant HelmRelease, который вы применяете через `kubectl`). Cozystack создаст соответствующий application CR со значениями по умолчанию.
2. Отредактируйте application CR на месте. Например, чтобы добавить pool в экземпляр SeaweedFS tenant-root:

```
kubectl edit -n tenant-root seaweedfses.apps.cozystack.io seaweedfs
```

Или patch'ите его non-interactively:

```
kubectl patch -n tenant-root seaweedfses.apps.cozystack.io seaweedfs \
--type=merge -p '{"spec":{"volume":{"pools":{"ssd":{"diskType":"ssd","size":"50Gi"}}}}}'
```

Та же схема применяется к каждому tenant-level application CR: `etcd`, `monitoring`, `ingress`, `seaweedfs`. См. [SeaweedFS storage pools](#) как подробный пример полного flow: включение SeaweedFS в tenant и последующая настройка созданного CR.

[!WARNING]

Не пытайтесь предварительно настроить tenant-level service, применяя его CR manifest до создания tenant — вы получите ошибку “namespace not found”. Редактирование самого ресурса **Tenant** с попыткой вложить service-specific поля (например, SeaweedFS `pools`) в спец **Tenant** тоже не сработает: tenant-level flags являются booleans, а per-service spec находится в отдельном resource. Сначала включите flag, затем редактируйте downstream CR.

Уникальные доменные имена

У каждого tenant есть собственный домен. По умолчанию, если не указано иное, он наследует домен parent tenant с префиксом из своего имени. Например, если у tenant `root` домен `example.org`, то tenant `foo` по умолчанию получает домен `foo.example.org`. Однако его можно переопределить и указать другой домен, например `example.com`.

Kubernetes-кластеры, созданные в namespace этого tenant, получают домены вида `kubernetes-cluster.foo.example.org`.

Ограничения имен tenants

Имена tenants должны быть alphanumeric. Использовать дефисы (-) в именах tenants нельзя, в отличие от других сервисов. Это ограничение нужно, чтобы сохранить согласованное именование tenants, nested tenants и сервисов, развернутых внутри них.

Например:

- Root tenant называется `root`, но внутри он представлен как `tenant-root`.
- Пользовательский tenant называется `foo`, что дает `tenant-foo`.
- Однако tenant нельзя назвать `foo-bar`, потому что разбор имен вроде `tenant-foo-bar` может быть неоднозначным.

Namespace layout tenant'ов

Каждый tenant соответствует Kubernetes workload namespace. Tenant `root` — особый случай: его namespace жестко задан как `tenant-root`. Для каждого nested tenant namespace выводится из workload namespace его parent и собственного имени по двум правилам:

- Tenant, созданный напрямую внутри `tenant-root`, получает namespace `tenant-<name>`. Prefix parent `tenant-root-` не включается.
- Tenant, созданный на любой более глубокой вложенности, получает namespace `<parent-workload-namespace>-<name>`, где имя child добавляется к полному namespace parent.

Например, начиная с `tenant-root`:

| Tenant path | Workload namespace |
|-------------|--------------------|
| root | tenant-root |

| | |
|-----------------------|-------------------------|
| root/alpha | tenant-alpha |
| root/alpha/beta | tenant-alpha-beta |
| root/alpha/beta/gamma | tenant-alpha-beta-gamma |

И Helm chart `tenant`, и aggregated API реализуют эти правила при создании нового tenant:

- helper Helm chart в [packages/apps/tenant/templates/_helpers.tpl](#) вычисляет namespace для устанавливаемого child release;
- функция `computeTenantNamespace` в [pkg/registry/apps/application/rest.go](#) публикует то же значение как `status.namespace` в Tenant CR.

Так как сами имена tenants ограничены alphanumeric-символами (см. Ограничения имен tenants выше), фрагменты namespace никогда не содержат tenant-internal дефисов.

[!IMPORTANT]

Имена Kubernetes namespaces являются RFC 1123 labels и не могут превышать 63 символа. Так как глубокие tenants накапливают всю цепочку ancestors в имени workload namespace (`tenant-alpha-beta-gamma`), длинные имена tenants в сочетании с глубокой вложенностью могут упереться в этот лимит. Планируйте иерархию заранее: короткие имена tenants на глубоких уровнях или более плоские trees, если длинные имена неизбежны. Kubernetes отклонит создание namespace, если вычисленное имя превысит 63 символа, а содержащий tenant Helm release покажет это отклонение как reconcile failure.

Определение parent и child relationships

Downstream integrations — custom dashboards, audit tooling, cost-allocation jobs, policy engines — иногда должны обходить tenant tree, чтобы показывать breadcrumbs, вычислять inherited settings или ограничивать scope запросов. Может показаться удобным выводить parent namespace, разбивая workload namespace по `-` и собирая его обратно без последнего segment. Сегодня это работает только потому, что имена tenants alphanumeric, а `-` однозначно разделяет ancestor segments; кроме того, такой способ предполагает, что текущие правила генерации namespace никогда не изменятся. Оба предположения являются implementation details, а не стабильным контрактом.

Стабильный контракт — сам custom resource `Tenant`. Cozystack хранит каждый Tenant CR в workload namespace его parent, поэтому:

- `metadata.namespace` Tenant CR равен workload namespace parent. Это надежный указатель на parent без string parsing.
- `status.namespace` Tenant CR равен собственному workload namespace tenant (там живут приложения tenant, nested tenants и `HelmRelease`).

Чтобы получить direct children tenant с workload namespace `N`, перечислите Tenant CRs, у которых `metadata.namespace == N`. Через `kubectl` это одна команда к workload namespace parent:

```
kubectl get tenants --namespace <parent-workload-namespace>
```

Ресурс `tenants` обслуживается Cozystack aggregated API (`apps.cozystack.io/v1alpha1`), поэтому `cozystack-api` должен быть запущен и доступен клиенту. Выполните `kubectl api-resources --api-group apps.cozystack.io`, чтобы убедиться, что ресурс виден в вашем kubeconfig context.

Этот подход стабилен независимо от того, является `tenant` direct child `tenant-root` или более глубоким descendant, и переживет будущие изменения namespace layout, потому что вообще от него не зависит.

Справочник

См. справочник по приложению, реализующему управление tenants:

[tenant](#)

Управление ресурсами в Cozystack

<https://cozystack.ru/docs/v1.5/guides/resource-management/>

Введение

Cozystack запускает все, включая системные компоненты и пользовательские приложения, как сервисы в Kubernetes-кластере с конечным пулом CPU и memory.

В этом руководстве описано, как пользователи могут настраивать ресурсы, доступные приложению, и как Cozystack обрабатывает такую конфигурацию.

Настройка ресурсов сервиса

Ресурсы, доступные каждому сервису (managed application, VM или tenant cluster), задаются в его configuration file. В Cozystack есть два способа указать CPU time и memory, доступные сервису:

- С помощью resource presets.
- С помощью явной resource configuration.

Использование resource presets

Cozystack предоставляет набор именованных resource presets. У каждого пользовательского сервиса, включая managed applications, tenant Kubernetes clusters и virtual machines, есть preset по умолчанию.

При развертывании сервиса preset задается в переменной конфигурации `resourcesPreset`, например:

```
## @param resourcesPreset Default sizing preset used when `resources` is omitted.
## Allowed values: none, nano, micro, small, medium, large, xlarge, 2xlarge.
resourcesPreset: "small"
```

| Название preset | CPU | memory |
|-----------------|------|--------|
| nano | 100m | 128Mi |
| micro | 250m | 256Mi |
| small | 500m | 512Mi |
| medium | 500m | 1Gi |
| large | 1 | 2Gi |
| xlarge | 2 | 4Gi |

| | | |
|---------|---|-----|
| 2xlarge | 4 | 8Gi |
|---------|---|-----|

Для CPU единица `m` означает 1/1000 полной CPU time.

Presets Cozystack определены во внутренней библиотеке [cozy-lib](#).

Явное определение ресурсов

Конфигурация сервиса может явно задавать доступные CPU и memory через переменную `resources`. В Cozystack используется простой формат resource configuration для `cpu` и `memory`:

```
## @param resources Explicit CPU and memory configuration for each ClickHouse replica.
## When left empty, the preset defined in `resourcesPreset` is applied.
resources:
  cpu: 1
  memory: 2Gi
```

Если одновременно заданы `resources` и `resourcesPreset`, используется `resources`, а `resourcesPreset` игнорируется.

Resource requests и limits

В Cozystack все запускается как Kubernetes services, а Kubernetes использует два важных механизма управления ресурсами: `requests` и `limits`. Сначала разберем, что это такое.

- Resource request определяет объем ресурса, который будет зарезервирован для сервиса и всегда предоставлен. Если ресурса недостаточно для выполнения request, сервис вообще не запустится.
- Resource limit определяет, сколько сервис может использовать из свободного пула ресурсов.

Подробное объяснение того, как `requests` и `limits` работают в Kubernetes, см. в разделе [Resource Management for Pods and Containers](#).

CPU time легко делится между несколькими сервисами с неравномерной CPU load. Поэтому распространенная практика — задавать низкие CPU requests и значительно более высокие limits. Для CPU-intensive сервисов оптимальным может быть соотношение 1:2 или 1:4. Для менее CPU-intensive сервисов даже 1:10 может дать хорошую resource efficiency и при этом быть достаточным.

Memory, наоборот, это ресурс, который после выдачи сервису обычно нельзя забрать обратно без OOM-kill сервиса. Поэтому memory requests обычно лучше задавать на уровне, который гарантирует работу сервиса.

CPU Allocation Ratio

В Cozystack есть единая configuration variable `cpuAllocationRatio`, являющаяся single source of truth. Она определяет соотношение между CPU requests и limits для всех сервисов.

CPU allocation ratio задается в Platform Package:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        # ...
      resources:
        cpuAllocationRatio: 4
```

По умолчанию `cpuAllocationRatio` равен 10. Это значит, что CPU requests будут составлять 1/10 от CPU limits. Cozystack заимствует это значение по умолчанию из [KubeVirt](#).

Как Cozystack выводит CPU requests и limits

```
## @param resources Explicit CPU and memory configuration for each ClickHouse replica.
## When left empty, the preset defined in `resourcesPreset` is applied.
resources:
  cpu: 1
  ## actual cpu limit: 1
  ## actual cpu request: (cpu / cpu-allocation-ratio)
  memory: 2Gi
```

Пример 1, настройка по умолчанию: cpu-allocation-ratio: 10

| Название preset | resources.cpu | фактический CPU request | фактический CPU limit |
|-----------------|---------------|-------------------------|-----------------------|
| nano | 100m | 10m | 100m |
| micro | 250m | 25m | 250m |
| small | 500m | 50m | 500m |
| medium | 500m | 50m | 500m |
| large | 1 | 100m | 1 |
| xlarge | 2 | 200m | 2 |
| 2xlarge | 4 | 400m | 4 |

Пример 2: cpu-allocation-ratio: 4

| Название preset | resources.cpu | фактический CPU request | фактический CPU limit |
|-----------------|---------------|-------------------------|-----------------------|
| nano | 100m | 25m | 100m |
| micro | 250m | 63m | 250m |
| small | 500m | 125m | 500m |
| medium | 500m | 125m | 500m |
| large | 1 | 250m | 1 |
| xlarge | 2 | 500m | 2 |
| 2xlarge | 4 | 1 | 4 |

Формат конфигурации до v0.31.0

До Cozystack v0.31.0 конфигурация сервисов позволяла пользователям явно задавать requests и limits. После обновления Cozystack с более ранних версий до v0.31.0 или новее такие сервисы не требуют немедленных действий.

При обновлении таких приложений пользователям нужно изменить конфигурацию на новый формат.

```
resources:
  requests:
    cpu: 250m
    memory: 512Mi
  limits:
    cpu: 1
    memory: 2Gi
```

У этого изменения было несколько причин:

1. Managed applications предполагают, что пользователю не нужно глубоко знать Kubernetes. Однако явная настройка requests/limits была “leaky abstraction”: она путала пользователей и приводила к misconfigurations.
2. Для hosting companies, запускающих public clouds на Cozystack, единое соотношение по всему cloud критически важно. Такой подход помогает обеспечить стабильный уровень сервиса и упрощает billing.
3. Пользователи, которые разворачивают собственные приложения в tenant Kubernetes clusters, по-прежнему могут задавать точные resource requests и limits.

Talos Linux в Cozystack

<https://cozystack.ru/docs/v1.5/guides/talos/>

Почему Cozystack использует Talos Linux

Talos Linux — это Linux-дистрибутив, созданный и оптимизированный для одной задачи: запуска Kubernetes. Он является основой надежности и безопасности кластера Cozystack. Такой выбор позволяет Cozystack строго ограничить технологический стек и сделать систему максимально стабильной. ([Cozystack](#))

Посмотрим, почему разработчики Cozystack выбрали Talos как основу Kubernetes-кластера и что он дает платформе. ([Cozystack](#))

Надежный и простой

Talos Linux — immutable ОС, управляемая через API. В ней нет лишних движущихся частей: традиционного пакетного менеджера, привычной файловой структуры и возможности запускать что-либо, кроме Kubernetes-контейнеров. ([Cozystack](#))

Базовый слой платформы включает актуальную версию ядра, все необходимые kernel modules, container runtime и Kubernetes-like API для взаимодействия с системой. Обновление системы выполняется полной перезаписью образа Talos на диск “как есть”. ([Cozystack](#))

Масштабируемый и воспроизводимый

Talos Linux реализует принцип infrastructure-as-code. Talos настраивается через внешний декларативный manifest, который можно хранить в Git и переиспользовать для всех операций: повторного развертывания того же кластера, добавления узлов и других задач. ([Cozystack](#))

Когда вы находите оптимальную конфигурацию или решаете эксплуатационную проблему, вы один раз применяете изменение в manifest и сразу распространяете его на любое количество узлов, что упрощает масштабирование. Все узлы автоматически сходятся к одной и той же конфигурации, что устраняет configuration drift и делает troubleshooting предсказуемым. ([Cozystack](#))

Заточен под Kubernetes

Talos содержит встроенную логику для bootstrap и сопровождения Kubernetes-кластера, снижая сложность первой установки. Он обеспечивает полный lifecycle management как операционной системы, так и Kubernetes через единый набор команд `talosctl`, включая обновления, замену узлов и disaster recovery. ([Cozystack](#))

Настроен для Cozystack

Cozystack предоставляет подготовленную сборку Talos, в которую уже включены extensions и kernel modules, необходимые для storage, networking и observability stack. Благодаря этому кластеры готовы к production-сценариям сразу после запуска. ([Cozystack](#))

Сценарии использования

<https://cozystack.ru/docs/v1.5/guides/use-cases/>

Использование Cozystack для построения public cloud

Как использовать Cozystack для построения public cloud

Использование Cozystack для построения private cloud

Как использовать Cozystack для построения private cloud

Build Your Own Platform (BYOP)

Как собрать собственную платформу с Cozystack, устанавливая только нужные компоненты

Использование Cozystack для построения public cloud

<https://cozystack.ru/docs/v1.5/guides/use-cases/public-cloud/>

Cozystack можно использовать как backend для public cloud.

Обзор

Cozystack позиционируется как framework для построения public cloud. Ключевое слово здесь — framework. В этом сценарии важно понимать, что Cozystack создан для cloud providers, а не для конечных пользователей напрямую.

Несмотря на наличие графического интерфейса, текущая модель безопасности не предполагает публичного пользовательского доступа к вашему management cluster.

Вместо этого конечные пользователи получают доступ к собственным Kubernetes-кластерам, могут заказывать LoadBalancers и дополнительные сервисы из них, но не имеют доступа к management cluster на базе Cozystack и ничего не знают о нем.

Таким образом, для интеграции с billing system достаточно научить вашу систему обращаться в management Kubernetes и размещать YAML-файл, описывающий нужный сервис. Остальную работу Cozystack выполнит за вас.

[!Cozystack for public cloud](#)

Использование Cozystack для построения private cloud

<https://cozystack.ru/docs/v1.5/guides/use-cases/private-cloud/>

Cozystack можно использовать как платформу для построения private cloud на основе Infrastructure-as-Code.

Обзор

Один из сценариев использования — self-service портал для пользователей внутри компании, где они могут заказывать нужный сервис или managed database.

Вы можете применять лучшие GitOps-практики: пользователи запускают собственные Kubernetes-кластеры и базы данных под свои задачи простым commit'ом конфигурации в ваш infrastructure Git repository.

Благодаря стандартизации подхода к разворачиванию приложений вы можете расширять возможности платформы с помощью обычных Helm charts.

[!Cozystack for private cloud](#)

Пример repository, показывающий настройку сервисов Cozystack через GitOps:

- <https://github.com/aenix-io/cozystack-gitops-example>
-

Build Your Own Platform (BYOP)

<https://cozystack.ru/docs/v1.5/guides/use-cases/kubernetes-distribution/>

Cozystack можно использовать в режиме BYOP (Build Your Own Platform): устанавливать только нужные вам компоненты из package repository Cozystack, а не разворачивать всю платформу целиком.

Обзор

Cozystack предоставляет систему управления packages, вдохновленную пакетными менеджерами Linux-дистрибутивов. Cozystack Operator управляет ресурсами `PackageSource` и `Package`, а CLI-инструмент `cozyrpk` предоставляет интерактивный интерфейс для просмотра доступных packages, разрешения зависимостей и выборочной установки.

Такой подход полезен, если:

- У вас уже есть Kubernetes-кластер, и вам нужны только отдельные компоненты.
- В кластере уже настроены сеть и хранилище.
- Вы хотите полностью контролировать, какие компоненты устанавливаются.

Variant `default` у `cozystack-platform` не устанавливает компоненты — он только регистрирует `PackageSources`. После этого вы используете `cozyrpk`, чтобы устанавливать отдельные packages: `networking`, `storage`, `ingress`, `database operators` и другие.

Пошаговое руководство см. в разделе [установка BYOP](#).

[!Build Your Own Platform with Cozystack](#)



Развертывание Cozystack

Руководство по развертыванию Cozystack: от инфраструктуры до готового кластера

<https://cozystack.ru/docs/v1.5/install/>

Руководство по Cozystack

Если вы устанавливаете Cozystack впервые, рекомендуем [пройти руководство по Cozystack](#). В нем показан самый короткий путь к созданию proof-of-concept кластера Cozystack.

Общий путь установки

Установка Cozystack на bare-metal серверы или виртуальные машины состоит из трех последовательных шагов. Для каждого шага есть несколько вариантов. Мы указываем рекомендуемый вариант, но также приводим альтернативы, чтобы процесс установки оставался гибким:

1. [Установите Talos Linux](#) на bare-metal серверы или виртуальные машины с Linux либо без операционной системы.
2. [Установите и инициализируйте кластер Kubernetes](#) поверх Talos Linux.
3. [Установите и настройте Cozystack](#) в кластере Kubernetes.

Изолированная среда

Cozystack можно установить в изолированной среде без прямого доступа к Интернету. Главное отличие такой установки — использование проху registry для образов:

1. [Установите Talos Linux](#) на bare-metal серверы или виртуальные машины с Linux либо без операционной системы.
2. [Настройте узлы Talos для изолированной среды и инициализируйте кластер Kubernetes](#).
3. [Установите и настройте Cozystack](#) в кластере Kubernetes.

Автоматизированная установка с Ansible

Для типовых развертываний Linux (Ubuntu, Debian, RHEL, Rocky, openSUSE) [коллекция Ansible](#) автоматизирует весь процесс: подготовку ОС, инициализацию кластера k3s и установку Cozystack.

Установка у конкретных провайдеров

Для облачных провайдеров есть отдельные руководства, которые охватывают все шаги: от подготовки инфраструктуры до установки и настройки Cozystack. Если это ваш случай, рекомендуем использовать руководства ниже:

- [Hetzner](#)
- [Oracle Cloud Infrastructure \(OCI\)](#)
- [Servers.com](#)

Обновление и настройка после развертывания

После развертывания кластера перейдите в раздел [Администрирование кластера](#), чтобы выполнить следующие действия:

- [Настройка OIDC](#)
- [Развертывание Cozystack в конфигурации с несколькими дата-центрами](#)
- [Обновление Cozystack](#)

Требования к оборудованию

Определите требования к оборудованию для вашего сценария использования Cozystack.

Рекомендации по планированию системных ресурсов

Сколько системных ресурсов выделять на узел в зависимости от масштаба кластера.

Установка Talos Linux на bare metal или виртуальные машины

Шаг 1: установка Talos Linux на виртуальные машины или bare metal для последующей инициализации кластера Cozystack.

Установка и настройка кластера Kubernetes

Шаг 2: установка и настройка кластера Kubernetes, готового к установке Cozystack.

Установка и настройка Cozystack

Шаг 3: установка Cozystack в Kubernetes-кластер — как готовой к использованию платформы или в режиме BYOP (Build Your Own Platform).

Развертывание кластера Cozystack в облаках и у хостинг-провайдеров

Руководства по развертыванию кластеров Cozystack у конкретных облачных и хостинг-провайдеров.

Автоматизированная установка с Ansible

Развертывание Cozystack на generic Kubernetes с помощью Ansible collection `cozystack.installer`

Руководства для особых случаев развертывания Cozystack

[Руководства для особых случаев развертывания Cozystack](#)

Требования к оборудованию

<https://cozystack.ru/docs/v1.5/install/hardware-requirements/>

Cozystack использует Talos Linux — минималистичный дистрибутив Linux, предназначенный исключительно для запуска Kubernetes. Обычно это означает, что сервер нельзя совместно использовать с сервисами, которые не запущены Cozystack. Хорошая новость в том, что какой бы сервис вам ни понадобился, Cozystack сможет запустить его корректно: безопасно, эффективно и в полностью контейнеризованной или виртуализованной среде.

Требования к оборудованию зависят от вашего сценария использования. Ниже приведены несколько распространенных вариантов развертывания; изучите их, чтобы определить, какая конфигурация лучше всего подходит под ваши задачи.

Варианты развертывания

Минимальная конфигурация

Ниже приведены базовые требования для запуска небольшой инсталляции. Минимальная рекомендуемая конфигурация для каждого узла:

| Компонент | Требование |
|---------------------|---|
| Хосты | 3х физических хоста (или VM с host CPU passthrough) |
| Architecture | x86_64 |
| CPU | 8 ядер |
| RAM | 24 ГБ |
| Основной диск | 50 ГБ SSD (или RAW для VM) |
| Дополнительный диск | 256 ГБ SSD (raw) |

Подходит для:

- Dev/Test-сред
- Небольших демонстрационных конфигураций
- 1-2 tenants
- До 3 кластеров Kubernetes
- Небольшого числа VM или баз данных

Рекомендуемая конфигурация

Для небольших production-сред рекомендуемая конфигурация каждого узла выглядит так:

| Компонент | Требование |
|---------------------|---------------------|
| Хосты | 3х физических хоста |
| Architecture | x86_64 |
| CPU | 16-32 ядра |
| RAM | 64 ГБ |
| Основной диск | 100 ГБ SSD или NVMe |
| Дополнительный диск | 1-2 ТБ SSD или NVMe |

Подходит для:

- Небольших и средних production-сред
- 5-10 tenants
- 5+ кластеров Kubernetes
- Десятков виртуальных машин или баз данных
- S3-совместимого хранилища

Оптимальная конфигурация

Для средних и крупных production-сред оптимальная конфигурация каждого узла выглядит так:

| Компонент | Требование |
|---------------------|-----------------------|
| Хосты | 6х+ физических хостов |
| Architecture | x86_64 |
| CPU | 32-64 ядра |
| RAM | 128-256 ГБ |
| Основной диск | 200 ГБ SSD или NVMe |
| Дополнительный диск | 4-10 ТБ NVMe |

Подходит для:

- Крупных production-сред
- 20+ tenants
- Десятков кластеров Kubernetes
- Сотен виртуальных машин и баз данных
- S3-совместимого хранилища

Вычислительные ресурсы:

- Три или более физических либо виртуальных сервера с архитектурой amd64/x86_64 и характеристиками, указанными в таблице выше.
- Для виртуализированных серверов должна быть включена вложенная виртуализация, а модель CPU должна быть установлена в `host` (без эмуляции).
- Для установки через PXE требуется дополнительный управляющий инстанс, подключенный к той же сети, с любой Linux-системой, способной запускать Docker-контейнер. Он также должен иметь архитектуру `x86-64-v2`; в случае виртуальной машины этого, скорее всего, можно добиться, установив модель CPU в `host`.

Хранилище:

Хранилище в кластере Cozystack используется как системой, так и пользовательскими workload'ами. Есть два варианта: выделить отдельный диск для каждой роли или выделить место под пользовательское хранилище на системном диске. Для хранилища узлов control plane критически важна низкая задержка, поэтому рекомендуются локальные SSD.

Использование двух дисков

Разделение дисков по ролям — основной и более надежный вариант.

- Основной диск: на этом диске находятся операционная система Talos Linux, необходимые системные модули ядра, базовые системные поды Cozystack, логи и базовые контейнерные образы. Также поверх него будет работать кластер etcd, поэтому следует использовать том с низкой задержкой, предпочтительно локальный SSD.

Минимальные размеры зависят от конфигурации (см. таблицу выше). Установка Talos ожидает `/dev/sda` в качестве системного диска (диски virtio обычно отображаются как `/dev/vda`).

- Дополнительный диск: предназначен для данных workload'ов, его размер можно увеличивать в зависимости от требований workload'ов. Используется для выделения томов через PersistentVolumeClaims (PVC).

Минимальные размеры зависят от конфигурации (см. таблицу выше). Путь к диску (обычно `/dev/sdb`) будет указан в конфигурации хранилища. Он не влияет на установку Talos.

Подробнее о настройке Linstor StorageClass см. в [руководстве по развертыванию Cozystack](#).

Использование одного диска

Можно использовать один диск с выделенным пространством для пользовательского хранилища. См. [Как установить Talos на машину с одним диском](#). Рекомендуется использовать локальный SSD-диск.

Сеть:

- Машинам должно быть разрешено использовать дополнительные IP-адреса, либо должен быть доступен внешний балансировщик нагрузки. Использование дополнительных IP-адресов по умолчанию отключено и в большинстве публичных облаков должно включаться явно.

-
- Могут потребоваться дополнительные публичные IP-адреса для ingress и виртуальных машин. Проверьте, поддерживает ли ваш облачный провайдер floating IP.
 - Маршрутизируемый FQDN-домен (или используйте nip.io с записью через дефис)
 - Размещение в одном L2-сегменте сети
 - Отключенный anti-spoofing (требуется для MetalLB)
 - Минимум 1 Гбит/с (для production рекомендуется 10 Гбит/с)
 - Низкая задержка между узлами кластера

Production-кластер

Для production-среды учитывайте следующее:

Вычислительные ресурсы:

- Для запуска высокодоступных приложений обязательно иметь как минимум три worker-узла. Если один из трех узлов станет недоступен из-за аппаратного сбоя или обслуживания, кластер будет работать в деградированном состоянии. Кластеры баз данных и реплицированное хранилище продолжат работать, но запуск новых экземпляров баз данных или создание реплицированных томов будет невозможным.
- Настоятельно рекомендуется, хотя и не обязательно, использовать отдельные серверы для master-узлов Kubernetes. Чистый worker-узел гораздо проще вывести из эксплуатации для обслуживания или обновления, чем узел, который одновременно является управляющим.

Сеть:

- В конфигурации с несколькими дата-центрами оптимально иметь прямые выделенные оптические каналы между ними.
- Серверы должны поддерживать out-of-band management (IPMI, iLO, iDRAC и т. д.) для удаленного мониторинга, восстановления и управления.

Распределенный кластер

С помощью Cozystack можно построить [распределенный кластер](#).

Сеть:

- Для распределенного кластера требуется быстрая и надежная сеть, и у нее обязательно должен быть низкий RTT (Round Trip Time), так как Kubernetes не рассчитан на эффективную работу в сетях с высокой задержкой.
- Дата-центры в пределах одного города обычно имеют задержку менее 1 мс, что является идеальным вариантом. Максимально допустимый RTT — 10 мс. Запуск Kubernetes или реплицированного хранилища в сети с RTT выше 20 мс настоятельно не рекомендуется. Для измерения фактического RTT можно использовать команду `ping`.
- Также рекомендуется иметь как минимум 2–3 узла на дата-центр в распределенном кластере. Это гарантирует, что кластер сможет пережить потерю одного дата-центра без серьезных нарушений работы.

-
- Если сложно поддерживать единое адресное пространство между дата-центрами, вместо внешнего VPN можно включить KubeSpan — функцию Talos Linux, которая создает full-mesh VPN между узлами на базе WireGuard.

Высокодоступные приложения

Обеспечение высокой доступности добавляет требования к базовой production-среде.

Сеть:

- Рекомендуется иметь несколько сетевых карт 10 Гбит/с или быстрее. Трафик хранилища и приложений можно разделить, назначив их разным сетевым интерфейсам.
- Ожидайте значительный объем горизонтального межузлового трафика внутри кластеров. Обычно он возникает из-за обмена данными между несколькими репликами сервисов и баз данных, развернутыми на разных узлах. Кроме того, виртуальные машины с live migration требуют реплицированных томов, что дополнительно увеличивает объем трафика.

Планирование системных ресурсов

Подробные рекомендации по выделению системных ресурсов (CPU и памяти) на узел с учетом масштаба кластера и количества tenants см. в разделе [Рекомендации по планированию системных ресурсов](#).

Рекомендации по планированию системных ресурсов | Cozystack

<https://cozystack.ru/docs/v1.5/install/resource-planning/>

Это руководство помогает спланировать выделение системных ресурсов на узел с учетом размера кластера и количества tenants. Рекомендации основаны на production-развертываниях и дают достаточно точные оценки для планирования.

Важно: указанные значения относятся только к системным компонентам. К ним нужно добавить требования tenant workload'ов (приложений, баз данных, кластеров Kubernetes, виртуальных машин и т. д.).

Быстрый старт: выделите как минимум 2 ядра CPU и 6 ГБ RAM на узел для системных компонентов. Для точного расчета требований с учетом размера кластера и количества tenants используйте таблицу или калькулятор ниже.

Примечание о выделении ресурсов: эти значения отражают ожидаемое потребление при нормальной работе, а не жесткое резервирование ресурсов. Kubernetes динамически планирует workload'ы, а системные компоненты будут потреблять примерно эти объемы, при этом оставшаяся емкость останется доступной для tenant workload'ов.

Требования к ресурсам

Требования зависят как от размера кластера (количества узлов), так и от количества tenants. Если у каждого tenant много активных сервисов (5+), ориентируйтесь на значения из следующей категории tenants.

| Размер кластера | Узлы | До 5 tenants | 6-14 tenants | 15-30 tenants | 31+ tenants |
|-----------------|------|---------------------------|----------------------------|----------------------------|----------------------------|
| Малый | 3-5 | CPU: 2 ядра
>RAM: 6 ГБ | CPU: 2 ядра
>RAM: 6 ГБ | CPU: 3 ядра
>RAM: 10 ГБ | CPU: 3 ядра
>RAM: 15 ГБ |
| Средний | 6-10 | CPU: 3 ядра
>RAM: 7 ГБ | CPU: 3 ядра
>RAM: 7 ГБ | CPU: 3 ядра
>RAM: 12 ГБ | CPU: 4 ядра
>RAM: 18 ГБ |
| Крупный | 11+ | CPU: 3 ядра
>RAM: 9 ГБ | CPU: 3 ядра
>RAM: 10 ГБ | CPU: 4 ядра
>RAM: 15 ГБ | CPU: 4 ядра
>RAM: 22 ГБ |

Советы по планированию:

- Отслеживайте фактическое потребление ресурсов и корректируйте значения при необходимости
- Закладывайте запас на рост 20-30%

-
- При высокой активности tenants рассмотрите увеличение CPU на 50-100%, а памяти — на 100-300%

Рассчитайте свои требования

Используйте калькулятор ниже, чтобы определить требования для вашей конкретной конфигурации:

Почему требования к ресурсам масштабируются

Потребление системных ресурсов растет вместе с размером кластера и количеством tenants, потому что системным компонентам нужно обрабатывать больше объектов Kubernetes для мониторинга, применять больше сетевых политик и собирать и обрабатывать больше логов.

Установка Talos Linux на bare metal или виртуальные машины

<https://cozystack.ru/docs/v1.5/install/talos/>

Первый шаг развертывания кластера Cozystack — установка Talos Linux на bare-metal серверы или виртуальные машины. Перед началом убедитесь, что виртуальные машины или bare-metal серверы уже подготовлены. Для планирования установки см. [требования к оборудованию](#).

Если вы устанавливаете Cozystack впервые, рекомендуем [начать с руководства по Cozystack](#).

Варианты установки

Есть несколько способов установить Talos на bare-metal сервер или виртуальную машину. У них разные ограничения и оптимальные сценарии использования:

- Рекомендуется: [загрузка в Talos Linux из другой Linux ОС с помощью [boot-to-talos](https://cozystack.ru/docs/v1.5/install/talos/boot-to-talos/)](<https://cozystack.ru/docs/v1.5/install/talos/boot-to-talos/>) — простой способ установки, который полностью работает из userspace и не требует внешних зависимостей, кроме образа Talos.

Выберите этот вариант, если вы впервые работаете с Talos или если у вас есть VM с предустановленной ОС от облачного провайдера.

- [Установка с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах](#) — требует дополнительной управляющей машины, но позволяет устанавливать систему сразу на несколько хостов.
- [Установка с помощью ISO-образа](#) — оптимальна для систем, которые умеют автоматизировать установку с ISO.

Следующие шаги

- После установки Talos Linux у вас будет набор узлов, готовых к следующему шагу: [установке и инициализации кластера Kubernetes](#).
- Прочитайте [обзор Talos Linux](#), чтобы узнать, почему Talos Linux является оптимальным выбором ОС для Cozystack и что он дает платформе.

Установка Talos Linux с помощью boot-to-talos

Установите Talos Linux с помощью boot-to-talos — удобного CLI-приложения, которому нужен только образ Talos.

Установка Talos Linux с помощью PXE

Как установить Talos Linux с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах.

Установка Talos Linux с помощью ISO

Как установить Talos Linux с помощью ISO.

Установка Talos Linux с помощью boot-to-talos

<https://cozystack.ru/docs/v1.5/install/talos/boot-to-talos/>

В этом руководстве описано, как установить Talos Linux на хост с любым другим дистрибутивом Linux с помощью `boot-to-talos`.

`boot-to-talos` создан командой Cozystack, чтобы помочь пользователям и командам, внедряющим Cozystack, установить Talos — самый сложный шаг процесса. Он полностью работает из `userspace` и не имеет внешних зависимостей, кроме установочного образа Talos.

Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Совместимость версий

При установке Cozystack на Talos должны совпасть три версии:

| Компонент | Откуда берется | Что должно совпадать |
|--|--|---|
| Talos на узле | флаг <code>-image</code> , переданный в <code>boot-to-talos</code> | версия Talos, поставляемая с релизом Cozystack, который вы устанавливаете |
| <code>**talosctl**</code> на вашей рабочей станции | скачивается отдельно из релизов siderolabs/talos | major.minor версии Talos, записанной на узел |
| Cozystack | флаг <code>--version</code> , переданный в <code>helm upgrade --install cozy-installer</code> | — (основа; все остальное следует за ней) |

Для Cozystack 1.5.0 закреплённая версия Talos — v1.13.0

(`[packages/core/talos/images/talos/profiles/installer.yaml]`(<https://github.com/cozystack/cozystack/blob/v1.5.0/packages/core/talos/images/talos/profiles/installer.yaml>)). Используйте

`ghcr.io/cozystack/cozystack/talos:v1.13.0` как образ для `boot-to-talos` и скачайте `talosctl v1.13.x`.

[!IMPORTANT]

`boot-to-talos v0.7.x` содержит собственный жестко заданный образ по умолчанию

(`ghcr.io/cozystack/cozystack/talos:v1.11.6` в `v0.7.1`, см.

`[cmd/boot-to-talos/main.go]`(<https://github.com/cozystack/boot-to-talos/blob/v0.7.1/cmd/boot-to-talos/main.go>)). Если в интерактивном `prompt` оставить это значение по умолчанию для кластера, на

котором вы планируете запускать Cozystack 1.5.0, в итоге вы получите узел Talos v1.11, тогда как `installer` Cozystack и шаблоны `Talm` рассчитаны на Talos v1.13 — вы столкнетесь с

несовпадением на этапе `bootstrap`. Всегда вводите образ, соответствующий целевому релизу Cozystack (или передавайте `-image` в командной строке).

Режимы

`boot-to-talos` поддерживает два режима установки:

1. `boot` – извлекает `kernel` и `initrd` из `Talos installer` и загружает их напрямую с помощью механизма `kexec`.
2. `install` – подготавливает окружение, запускает `Talos installer`, а затем перезаписывает системный диск установленным образом.

Note: Если один режим не работает в вашей системе, попробуйте другой. Разные методы могут лучше работать на разных операционных системах.

Установка

1. Установка `boot-to-talos`

- Используйте установочный скрипт:

```
curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh | sh -s
```

- Скачайте бинарный файл со [страницы релизов GitHub](#):

```
wget https://github.com/cozystack/boot-to-talos/releases/latest/download/boot-to-talos-linux-amd64.bin
```

2. Запуск установки `Talos`

Запустите `boot-to-talos` и укажите значения конфигурации. Обязательно используйте собственную сборку `Talos` от Cozystack, доступную по адресу ghcr.io/cozystack/cozystack/talos.

```
Mode:
  1. boot – extract the kernel and initrd from the Talos installer and boot them directly using the kexec
  2. install – prepare the environment, run the Talos installer, and then overwrite the system disk with th...
Mode [1]: 2
Target disk [/dev/sda]:
Talos installer image [ghcr.io/cozystack/cozystack/talos:v1.11.6]: ghcr.io/cozystack/cozystack/talos:v1.13.0
Add networking configuration? [yes]:
Interface [eth0]:
IP address [10.0.2.15]:
Netmask [255.255.255.0]:
Gateway (or 'none') [10.0.2.2]:
Configure serial console? (or 'no') [ttyS0]:

Summary:
  Image: ghcr.io/cozystack/cozystack/talos:v1.13.0
  Disk: /dev/sda
  Extra kernel args: ip=10.0.2.15::10.0.2.2:255.255.255.0::eth0:::: console=ttyS0

WARNING: ALL DATA ON /dev/sda WILL BE ERASED!

Continue? [yes]:

2025/08/03 00:11:03 created temporary directory /tmp/installer-3221603450
2025/08/03 00:11:03 pulling image ghcr.io/cozystack/cozystack/talos:v1.13.0
2025/08/03 00:11:03 extracting image layers
2025/08/03 00:11:07 creating raw disk /tmp/installer-3221603450/image.raw (2 GiB)
2025/08/03 00:11:07 attached /tmp/installer-3221603450/image.raw to /dev/loop0
2025/08/03 00:11:07 starting Talos installer
2025/08/03 00:11:07 running Talos installer v1.13.0
2025/08/03 00:11:07 WARNING: config validation:
2025/08/03 00:11:07   use "worker" instead of "" for machine type
2025/08/03 00:11:07 created EFI (C12A7328-F81F-11D2-BA4B-00A0C93EC93B) size 104857600 bytes
2025/08/03 00:11:07 created BIOS (21686148-6449-6E6F-744E-656564454649) size 1048576 bytes
2025/08/03 00:11:07 created BOOT (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576000 bytes
2025/08/03 00:11:07 created META (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576 bytes
2025/08/03 00:11:07 formatting the partition "/dev/loop0p1" as "vfat" with label "EFI"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p2" as "zeroes" with label "BIOS"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p3" as "xfs" with label "BOOT"
2025/08/03 00:11:07 formatting the partition "/dev/loop0p4" as "zeroes" with label "META"
2025/08/03 00:11:07 copying from io reader to /boot/A/vmlinuz
2025/08/03 00:11:07 copying from io reader to /boot/A/initramfs.xz
2025/08/03 00:11:08 writing /boot/grub/grub.cfg to disk
2025/08/03 00:11:08 executing: grub-install --boot-directory=/boot --removable --efi-directory=/boot/efi
2025/08/03 00:11:08 installation of v1.13.0 complete
2025/08/03 00:11:08 Talos installer finished successfully
2025/08/03 00:11:08 remounting all filesystems read-only
2025/08/03 00:11:08 copy /tmp/installer-3221603450/image.raw /dev/sda
2025/08/03 00:11:19 installation image copied to /dev/sda
2025/08/03 00:11:19 rebooting system
```

О приложениях

`boot-to-talos` — это open source проект, размещенный на github.com/cozystack/boot-to-talos. Он включает CLI, написанный на Go, и установочный скрипт на Bash. Доступны сборки для нескольких архитектур:

- `linux-amd64`
- `linux-arm64`

Как это работает

Понимать эти шаги для установки Talos Linux необязательно. Рабочий процесс зависит от выбранного режима:

Режим boot

При использовании режима boot `boot-to-talos` выполняет следующие шаги:

1. Распаковывает Talos installer в RAM

Извлекает слои из контейнера Talos installer во временный `tmpfs`. Обратите внимание, что Docker на этом шаге не нужен.

2. Извлекает kernel и initrd

Извлекает kernel (`vmlinuz`) и initial ramdisk (`initramfs.xz`) из образа Talos installer.

3. Загружает kernel через `kexec`

Использует системный вызов `kexec`, чтобы загрузить kernel Talos и `initrd` в память с переданными параметрами командной строки kernel.

4. Перезагружает в Talos

Выполняет `kexec --exec`, чтобы переключиться на kernel Talos без физической перезагрузки. После загрузки можно применить конфигурацию Talos и завершить установку.

Режим install

При использовании режима install `boot-to-talos` выполняет следующие шаги:

1. Распаковывает Talos installer в RAM

Извлекает слои из контейнера Talos installer во временный `tmpfs`. Обратите внимание, что Docker на этом шаге не нужен.

2. Собирает системный образ

Создает sparse-файл `image.raw`, подключенный через loop device, и выполняет Talos installer внутри chroot. Затем installer размечает диск, форматирует разделы и размещает GRUB и системные файлы.

3. Записывает поток на диск

Копирует `image.raw` на выбранное блочное устройство блоками по 4 MiB и выполняет `fsync` после каждой записи, чтобы данные были полностью сохранены до перезагрузки.

4. Перезагружает систему

Команда `echo b > /proc/sysrq-trigger` выполняет немедленную перезагрузку в только что установленный Talos Linux.

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Установка Talos Linux с помощью PXE

<https://cozystack.ru/docs/v1.5/install/talos/pxe/>

В этом руководстве описано, как установить Talos Linux на bare-metal серверы или виртуальные машины с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах. Этот метод требует дополнительной управляющей машины, но позволяет устанавливать систему сразу на несколько хостов.

Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Зависимости

Для установки Talos этим способом на управляющем хосте потребуются следующие зависимости:

- `docker`
- `kubectl`

Обзор инфраструктуры

[!Cozystack deployment](#)

Установка

Запустите matchbox с готовым образом Talos для Cozystack:

```
sudo docker run --name=matchbox -d --net=host ghcr.io/cozystack/cozystack/matchbox:v0.30.0 \
  -address=:8080 \
  -log-level=debug
```

Запустите DHCP-сервер:

```
sudo docker run --name=dnsmasq -d --cap-add=NET_ADMIN --net=host quay.io/poseidon/dnsmasq:v0.5.0-32-g4327d6...
-d -q -p0 \
--dhcp-range=192.168.100.3,192.168.100.199 \
--dhcp-option=option:router,192.168.100.1 \
--enable-tftp \
--tftp-root=/var/lib/tftpboot \
--dhcp-match=set:bios,option:client-arch,0 \
--dhcp-boot=tag:bios,undionly.kpxe \
--dhcp-match=set:efi32,option:client-arch,6 \
--dhcp-boot=tag:efi32,ipxe.efi \
--dhcp-match=set:efibc,option:client-arch,7 \
--dhcp-boot=tag:efibc,ipxe.efi \
--dhcp-match=set:efi64,option:client-arch,9 \
--dhcp-boot=tag:efi64,ipxe.efi \
--dhcp-userclass=set:ipxe,iPXE \
--dhcp-boot=tag:ipxe,http://192.168.100.254:8080/boot.ipxe \
--log-queries \
--log-dhcp
```

Для установки в изолированной среде добавьте NTP- и DNS-серверы:

```
--dhcp-option=option:ntp-server,10.100.1.1 \
--dhcp-option=option:dns-server,10.100.25.253,10.100.25.254 \
```

Где:

- 192.168.100.3,192.168.100.199 — диапазон для выделения IP-адресов
- 192.168.100.1 — ваш gateway
- 192.168.100.254 — адрес вашего управляющего сервера

Проверьте состояние контейнеров:

```
docker ps
```

пример вывода:

| CONTAINER ID | IMAGE | COMMAND | CREATED |
|--------------|---|------------------------|---------|
| 06115f09e689 | quay.io/poseidon/dnsmasq:v0.5.0-32-g4327d60-amd64 | "/usr/sbin/dnsmasq - " | 47 seco |
| 6bf638f0808e | ghcr.io/cozystack/cozystack/matchbox:v0.30.0 | "/matchbox -address= " | 3 minut |

Запустите серверы. Теперь они должны автоматически загрузиться с вашего PXE-сервера.

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Установка Talos Linux с помощью ISO

<https://cozystack.ru/docs/v1.5/install/talos/iso/>

В этом руководстве описано, как установить Talos Linux на bare-metal серверы или виртуальные машины. Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Установка

1. Скачайте ISO Talos Linux для Cozystack v1.5.0 со [страницы релиза](#).

```
```bash
```

```
wget https://github.com/cozystack/cozystack/releases/download/v1.5.0/metal-amd64.iso
```

```
```
```

2. Загрузите машину с подключенным ISO.
3. Нажмите F2 и заполните сетевые настройки:

[!Cozystack for private cloud](#)

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Установка и настройка кластера Kubernetes

<https://cozystack.ru/docs/v1.5/install/kubernetes/>

Второй шаг развертывания кластера Cozystack — установка и настройка кластера Kubernetes. В результате вы получите установленный и настроенный кластер Kubernetes, готовый к установке Cozystack.

Если вы устанавливаете Cozystack впервые, [начните с руководства по Cozystack](#).

Варианты установки

Talos Linux (рекомендуется)

Для production-развертываний Cozystack рекомендует использовать [Talos Linux](#) в качестве базовой операционной системы. Предварительное условие для этих методов — [установленный Talos Linux](#).

Есть несколько способов настроить узлы Talos и инициализировать кластер Kubernetes:

- Рекомендуется: [использование Talm](#) — декларативного CLI-инструмента с готовыми пресетами для Cozystack, который использует возможности Talos API.
- [Использование talos-bootstrap](#) — интерактивного скрипта для инициализации кластеров Kubernetes на Talos OS.
- [Использование talosctl](#) — специализированного CLI-инструмента для управления Talos.
- [Установка в изолированной среде](#) возможна с Talm или talosctl.

Generic Kubernetes

Cozystack также можно развернуть на других дистрибутивах Kubernetes:

- [Generic Kubernetes](#) — развертывание Cozystack на k3s, kubeadm, RKE2 или других дистрибутивах.

Если при установке возникнут проблемы, см. [раздел Troubleshooting](#).

Следующие шаги

- После установки и настройки кластера Kubernetes можно [установить и настроить Cozystack](#).

Использование Talm для инициализации кластера Cozystack

`talm` — декларативный CLI-инструмент, созданный разработчиками Cozystack и оптимизированный для развертывания Cozystack.

Рекомендуется для infrastructure-as-code и GitOps.

Использование скрипта `talos-bootstrap` для инициализации кластера Cozystack

`talos-bootstrap` — CLI для пошаговой инициализации кластера, созданный разработчиками Cozystack.

Рекомендуется для первых развертываний.

Использование `talosctl` для инициализации кластера Cozystack

`talosctl` — стандартный CLI Talos Linux: он требует больше boilerplate-кода, но дает полную гибкость настройки.

Инициализация кластера в изолированной среде

Инициализация кластера Cozystack в изолированной (air-gapped) среде с mirror'ами container registry.

Устранение неполадок при установке Kubernetes

Инструкции по решению типовых проблем, которые могут возникнуть при установке Kubernetes с помощью `talm`, `talos-bootstrap` или `talosctl`.

Развертывание Cozystack на Generic Kubernetes

Как развернуть Cozystack на k3s, kubeadm, RKE2 или других дистрибутивах Kubernetes без Talos Linux.

Ubuntu Secure Boot: DRBD prerequisites

Prepare Ubuntu LTS hosts with UEFI Secure Boot enabled for Cozystack non-Talos installation by pre-installing drbd-dkms from the LINBIT PPA.

Использование Talm для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.5/install/kubernetes/talm/>

В этом руководстве описано, как установить и настроить Kubernetes в кластере Talos Linux с помощью [Talm](#). После выполнения этого руководства у вас будет кластер Kubernetes, готовый к установке Cozystack.

Talm — Helm-подобная утилита для декларативного управления конфигурацией Talos Linux. Talm был создан Ænix, чтобы сделать конфигурации управления кластером более декларативными и настраиваемыми. Talm поставляется с готовыми пресетами для Cozystack.

Предварительные требования

К началу работы с этим руководством [Talos Linux должен быть установлен](#) на нескольких узлах, но еще не инициализирован (bootstrapped). Эти узлы должны находиться в одной подсети или иметь публичные IP-адреса.

В руководстве используется пример, где узлы кластера находятся в подсети 192.168.123.0/24 и имеют следующие IP-адреса:

- node1: private 192.168.123.11 or public 12.34.56.101.
- node2: private 192.168.123.12 or public 12.34.56.102.
- node3: private 192.168.123.13 or public 12.34.56.103.

Публичные IP-адреса необязательны. Для установки с Talm нужен только доступ к узлам: напрямую, через VPN или через SSH-туннель.

Если вы используете DHCP, вы можете не знать IP-адреса, назначенные узлам в private subnet. Узлы с установленным Talos Linux [открывают Talos API на порту 50000](#). Чтобы найти их, можно использовать `nmmap`, указав маску сети (192.168.123.0/24 в примере):

```
nmmap -Pn -n -p 50000 192.168.123.0/24 -vv | grep 'Discovered'
```

Пример вывода:

```
Discovered open port 50000/tcp on 192.168.123.11
Discovered open port 50000/tcp on 192.168.123.12
Discovered open port 50000/tcp on 192.168.123.13
```

1. Установка зависимостей

Для этого руководства нужно установить несколько инструментов:

- Talm. Чтобы установить последнюю сборку для вашей платформы, скачайте и запустите установочный скрипт:

```
curl -fsSL https://raw.githubusercontent.com/cozystack/talm/main/install.sh | sh
```

Для Talm доступны бинарные файлы для Linux, macOS и Windows, как для AMD, так и для ARM. Также можно [скачать бинарный файл с GitHub](#) или [собрать Talm из исходного кода](#).

- talosctl распространяется как brew package:

```
brew install talosctl
```

Другие варианты установки см. в [руководстве по установке talosctl](#).

2. Инициализация конфигурации кластера

Первый шаг — инициализировать шаблоны конфигурации и указать значения для шаблонизации.

2.1 Инициализация конфигурации

Начните с инициализации конфигурации для нового кластера, используя preset [cozystack](#):

```
mkdir -p cozystack-cluster  
cd cozystack-cluster  
talm init --preset cozystack --name mycluster
```

Структура проекта в основном повторяет обычный Helm chart:

- [charts](#) - каталог с общим library chart, содержащим функции для запроса информации из Talos Linux.
- [Chart.yaml](#) - файл с общей информацией о проекте; имя chart используется как имя создаваемого кластера.
- [templates](#) - каталог для описания шаблонов генерации конфигурации.
- [secrets.yaml](#) - файл с secrets вашего кластера.
- [secrets.encrypted.yaml](#), [talosconfig.encrypted](#) - encrypted counterparts produced from [talm.key](#) (commit these to git instead of the plaintext files).
- [talm.key](#) - the project-local age key used for encrypt / decrypt. Back this up; without it the encrypted files are unreadable.
- [values.yaml](#) - общий values-файл для передачи параметров в шаблоны.
- [nodes](#) - необязательный каталог для описания и хранения сгенерированной конфигурации узлов.

Available Presets

[talm](#) ships two embedded presets:

- [cozystack](#) - the production preset used by this guide.
- [talm](#) - a minimal library chart for advanced users who want to build their own preset on top of it.

Pass the preset name via `-p / --preset`.

talm init Flag Reference

Run `talm init -h` for the canonical list. Grouped by mode:

Create a new project (default mode):

- `-p, --preset <name>` - preset for file generation.
- `-N, --name <cluster-name>` - cluster name.
- `--endpoints <list>` - Talos API endpoints (comma-separated) embedded into `talosconfig.contexts.<name>.endpoints` for the `talosctl` client.
- `--cluster-endpoint <url>` - Kubernetes control-plane URL written to `values.yaml::endpoint` (e.g. `https://<vip>:6443`).
- `--image <ref>` - override the Talos installer image written to the preset's `values.yaml` (e.g. `factory.talos.dev/installer/<sha256>:<version>`).
- `--talos-version <ver>` - desired Talos contract version for backwards-compatibility templating (e.g. `v1.12`).
- `--force` - overwrite existing files without prompt.

Endpoint flags: talosctl client vs Kubernetes control plane

Two distinct concepts share the word “endpoint” in talm projects:

- `talosconfig.contexts.<name>.endpoints` - list of `host[:port]` entries the `talosctl` client uses to reach the Talos API. Populated by `--endpoints` (plural, comma-separated list).
- `values.yaml::endpoint` - single URL with scheme + host + port that the chart renders into `cluster.controlPlane.endpoint` of every node's `MachineConfig`. This is what `kubelet` and `kube-proxy` dial. Populated by `--cluster-endpoint` (singular, full URL).

When `--endpoints` is given exactly one value, `init` auto-derives `values.yaml::endpoint` as `https://<that>:6443` because the single-target case is unambiguous. Multi-endpoint inputs never auto-derive; you must provide `--cluster-endpoint` explicitly or fill `values.yaml::endpoint` later by hand.

Update an existing project to the latest bundled library chart:

- `-u, --update` - re-extract `charts/talm/` and other preset-shipped files from the talm binary. `--preset` is required.
- `--force` - auto-accept every preset-template diff (skip the interactive prompt; safe to use in CI).

Manage encrypted secrets in-place:

- `-e, --encrypt` - encrypt `secrets.yaml / talosconfig / kubeconfig` into their `.encrypted` counterparts. Requires `talm.key`.
- `-d, --decrypt` - reverse the above.

Updating to a Newer Talm Release

When a new talm version ships a newer bundled library chart, refresh your project in place:


```
cd <project-dir>
talm init --update --preset cozystack
# interactive: prompts for each preset-template diff
talm init --update --preset cozystack --force
# non-interactive: auto-accept all diffs
```

Encrypt / Decrypt Round-Trip

The encrypted copies are what you commit to git; the plaintext copies are what `talm` reads.

```
talm init --encrypt
# secrets.yaml -> secrets.encrypted.yaml; talosconfig -> talosconfig.encrypted
talm init --decrypt
# reverse — does not require --preset or --name
```

2.2. Редактирование значений конфигурации и шаблонов

Сила Talm — в шаблонизации. Есть несколько файлов с исходными значениями и шаблонами, которые можно редактировать: `Chart.yaml`, `values.yaml` и `templates/*`. Talm использует эти значения и шаблоны для генерации конфигурации Talos для всех узлов кластера.

Все часто изменяемые значения конфигурации находятся в `values.yaml`:

```
## Используется для доступа к control plane кластера
endpoint: "https://192.168.100.10:6443"

## Домен кластера Cozystack API — используется сервисами и K8s-кластерами tenant'ов для доступа к узлам
clusterDomain: cozy.local

## Floating IP — должен быть неиспользуемым IP-адресом в той же подсети, что и узлы
floatingIP: 192.168.100.10

## Исходный образ Talos: используйте последнюю доступную версию
## https://github.com/cozystack/cozystack/pkgs/container/cozystack%2Ftalos
image: "ghcr.io/cozystack/cozystack/talos:v1.13.0"

## Подсеть Pod'ов — используется для назначения IP-адресов pod'ам
podSubnets:
- 10.244.0.0/16

## Подсеть сервисов — используется для назначения IP-адресов сервисам
serviceSubnets:
- 10.96.0.0/16

## Подсеть с IP-адресами узлов
advertisedSubnets:
- 192.168.100.0/24

## Добавьте URL OIDC issuer, чтобы включить OIDC — см. комментарии ниже.
oidcIssuerUrl: ""

certSANS: []
```

На этом шаге не нужно указывать IP-адреса узлов. Вы укажете их позже, при генерации конфигураций узлов.

Extending the rendered Talos config (Talm v0.30+)

The `cozystack` preset ships curated defaults for `machine.kernel.modules`, `machine.sysctls`, `machine.kubelet.extraConfig`, and `machine.files`.

| Key | Semantics on the <code>cozystack</code> preset |
|------------------------------------|--|
| <code>extraKernelModules</code> | Appended to the built-in modules (<code>openvswitch</code> , <code>drbd</code> , <code>zfs</code> , <code>spl</code> , <code>vfio_pci</code> , <code>vfio_iommu_type1</code>). |
| <code>extraKubeletExtraArgs</code> | Merged into <code>kubelet.extraConfig</code> after the preset's <code>cpuManagerPolicy: static</code> , <code>maxPods: 512</code> . |
| <code>extraSysctls</code> | Merged into <code>machine.sysctls</code> after the preset's built-in entries. |
| <code>extraMachineFiles</code> | Appended to the preset's CRI customization and <code>lvm.conf</code> entries. |

Example `values.yaml` addition:

```
extraKernelModules:
  - name: nf_conntrack

extraKubeletExtraArgs:
  feature-gates: "NodeSwap=true"

extraSysctls:
  net.core.somaxconn: "65535"

extraMachineFiles:
  - path: /etc/example.conf
    op: create
    content: "hello = world"
```

The `cozystack` preset exposes two opinionated tunables:

| Tunable | Default | Description |
|-------------------------------------|--------------------|--|
| <code>tcpKeepaliveTuning</code> | <code>false</code> | When <code>true</code> , adds keepalive settings to <code>machine.sysctls</code> . |
| <code>etcd.quotaBackendBytes</code> | (8 GiB) | etcd backend DB size ceiling. |

2.3 Add Keycloak Configuration

By default, the cluster will be accessible only by authentication with a token. To configure Keycloak as an OIDC provider:

For Talm v0.6.6 or later: in `./templates/_helpers.tpl` replace `keycloak.example.com` with `keycloak.<your-domain.tld>`.

For Talm earlier than v0.6.6, update `./templates/_helpers.tpl`:

```
cluster:
  apiServer:
    extraArgs:
      oidc-issuer-url: "https://keycloak.example.com/realms/cozy"
      oidc-client-id: "kubernetes"
      oidc-username-claim: "preferred_username"
      oidc-groups-claim: "groups"
```

3. Генерация конфигурационных файлов узлов

Следующий шаг — создать конфигурационные файлы узлов из шаблонов. Создайте каталог `nodes` и соберите информацию с каждого узла в отдельный файл для этого узла:

```
talm template -e 192.168.123.11 --nodes 192.168.123.11 -t templates/controlplane.yaml -i > nodes/node1.yaml
talm template -e 192.168.123.12 --nodes 192.168.123.12 -t templates/controlplane.yaml -i > nodes/node2.yaml
talm template -e 192.168.123.13 --nodes 192.168.123.13 -t templates/controlplane.yaml -i > nodes/node3.yaml
```

Параметр `--insecure (-i)` нужен потому, что Talm должен получить конфигурационные данные с узлов Talos, которые еще не инициализированы.

Сгенерированные файлы содержат блок комментариев с обнаруженными сетевыми интерфейсами и дисками. Эти данные можно использовать для настройки, например, [bonding \(LACP\)](#).

4. Применение конфигурации и инициализация кластера

На этом этапе конфигурационные файлы в `nodes/*.yaml` готовы к применению на узлах.

4.1 Применение конфигурационных файлов

Используйте `talm apply`, чтобы применить конфигурационные файлы к соответствующим узлам:

```
talm apply -f nodes/node1.yaml -e 192.168.123.11 -i
talm apply -f nodes/node2.yaml -e 192.168.123.12 -i
talm apply -f nodes/node3.yaml -e 192.168.123.13 -i
```

Эта команда инициализирует узлы и настраивает аутентифицированное соединение.

Позже с `talm apply` также можно использовать опции:

- `--dry-run` - покажет diff без внесения изменений.
- `-m try` - откатит конфигурацию через 1 минуту.

4.2 Ожидание перезагрузки

Дождитесь, пока все узлы перезагрузятся. Когда узлы будут готовы, они откроют порт 50000. Для автоматизации ожидания:

```
timeout 60 sh -c 'until nc -z $TARGET_IP 50000; do sleep 1; done'
```

4.3. Инициализация Kubernetes

Инициализируйте кластер Kubernetes, выполнив `talm bootstrap` для одного из узлов control plane:

```
talm bootstrap -e 192.168.123.11
```

5. Доступ к кластеру Kubernetes

На этом этапе кластер Kubernetes готов к установке Cozystack. Взаимодействие теперь требует Kubernetes API и `kubectl`.

5.1. Получение kubeconfig

Используйте Talm, чтобы сгенерировать административный `kubeconfig`:

```
talm kubeconfig -e 192.168.123.11
```

5.2. Изменение URL Cluster API

Если вы используете публичный IP вместо floating IP, обновите endpoint в `kubeconfig`:

```
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: ...
  server: https://10.0.1.101:6443
+   server: https://12.34.56.101:6443
```

5.3. Активация kubeconfig

Настройте переменную `KUBECONFIG`:

```
export KUBECONFIG=$PWD/kubeconfig
```

5.4. Проверка доступности кластера

Проверьте доступность:

```
kubectl get ns
```

Пример вывода:

| NAME | STATUS | AGE |
|-----------------|--------|-------|
| default | Active | 7m56s |
| kube-node-lease | Active | 7m56s |
| kube-public | Active | 7m56s |
| kube-system | Active | 7m56s |

5.5. Проверка состояния узлов

Проверьте состояние узлов:

```
kubectl get nodes
```

Вывод:

| NAME | STATUS | ROLES | AGE | VERSION |
|-------|----------|---------------|-------|---------|
| node1 | NotReady | control-plane | 7m56s | v1.33.1 |
| node2 | NotReady | control-plane | 7m56s | v1.33.1 |
| node3 | NotReady | control-plane | 7m56s | v1.33.1 |

Все узлы показывают **STATUS: NotReady**. Это нормально, так как стандартный [CNI-плагин Kubernetes](#) был отключен, чтобы Cozystack мог установить собственный CNI.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes. Переходите к [Установке Cozystack](#).

Использование скрипта talos-bootstrap для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.5/install/kubernetes/talos-bootstrap/>

talos-bootstrap — интерактивный скрипт для инициализации кластеров Kubernetes на Talos OS. Он был создан разработчиками Cozystack, чтобы упростить установку Talos Linux на bare-metal узлы и сделать ее удобнее.

1. Установка зависимостей

Установите следующие зависимости:

- `talosctl`
- `dialog`
- `nmap`

Скачайте последнюю версию `talos-bootstrap` со [страницы релизов](#) или напрямую из trunk:

```
curl -fsSL -o /usr/local/bin/talos-bootstrap \
    https://github.com/cozystack/talos-bootstrap/raw/master/talos-bootstrap
chmod +x /usr/local/bin/talos-bootstrap
talos-bootstrap --help
```

2. Подготовка конфигурационных файлов

1. Начните с создания каталога конфигурации для нового кластера:

```
mkdir -p cluster1
cd cluster1
```

2. Создайте файл конфигурационного `patch` `patch.yaml` с общими настройками узлов, используя следующий пример:

```

machine:
  kubelet:
    nodeIP:
      validSubnets:
        - 192.168.100.0/24
    extraConfig:
      maxPods: 512
  sysctls:
    net.ipv4.neigh.default.gc_thresh1: "4096"
    net.ipv4.neigh.default.gc_thresh2: "8192"
    net.ipv4.neigh.default.gc_thresh3: "16384"
  kernel:
    modules:
      - name: openvswitch
      - name: drbd
      parameters:
        - usermode_helper=disabled
      - name: zfs
      - name: spl
      - name: vfio_pci
      - name: vfio_iommu_type1
  install:
    image: ghcr.io/cozystack/cozystack/talos:v1.13.0
  registries:
    mirrors:
      docker.io:
        endpoints:
          - https://mirror.gcr.io
  files:
    - content: |
        [plugins."io.containerd.grpc.v1.cri"]
          device_ownership_from_security_context = true
        [plugins."io.containerd.cri.v1.runtime"]
          device_ownership_from_security_context = true
      path: /etc/cri/conf.d/20-customization.part
      op: create
    - op: overwrite
      path: /etc/lvm/lvm.conf
      permissions: 0o644
      content: |
        backup {
          backup = 0
          archive = 0
        }
        devices {
          global_filter = [ "r|^/dev/drbd.*|", "r|^/dev/dm-.*|", "r|^/dev/zd.*|", "r|^/d"
        }
  cluster:
    network:
      cni:
        name: none
      dnsDomain: cozy.local
      podSubnets:
        - 10.244.0.0/16
      serviceSubnets:
... (1 lines truncated)

```

3. Создайте еще один файл конфигурационного patch `patch-controlplane.yaml` с настройками, которые относятся только к узлам control plane:

```

machine:
  nodeLabels:
    node.kubernetes.io/exclude-from-external-load-balancers: ""
    $patch: delete
cluster:
  allowSchedulingOnControlPlanes: true
  controllerManager:
    extraArgs:
      bind-address: 0.0.0.0
  scheduler:
    extraArgs:
      bind-address: 0.0.0.0
  apiServer:
    certSANS:
      - 127.0.0.1
  proxy:
    disabled: true
  discovery:
    enabled: false
  etcd:
    advertisedSubnets:
      - 192.168.100.0/24

```

4. Чтобы настроить Keycloak как OIDC-провайдера, добавьте следующий раздел в `patch-controlplane.yaml`, заменив `example.com` на свой домен:

```

cluster:
  apiServer:
    extraArgs:
      oidc-issuer-url: "https://keycloak.example.com/realms/cozy"
      oidc-client-id: "kubernetes"
      oidc-username-claim: "preferred_username"
      oidc-groups-claim: "groups"

```

3. Инициализация и доступ к кластеру

Когда конфигурационные файлы будут готовы, запустите `talos-bootstrap` на каждом узле кластера:

```

talos-bootstrap install
# в каталоге конфигурации кластера

```

[!] Если узлы работают во внешней сети, каждый узел нужно явно указать в аргументе:

```

talos-bootstrap install -n 1.2.3.4

```

Где `1.2.3.4` — IP-адрес удаленного узла.

`talos-bootstrap` включит bootstrap на первом настроенном узле кластера. Если нужно повторно выполнить bootstrap кластера etcd, удалите строку `BOOTSTRAP_ETCD=false` из файла `cluster.conf`.

Повторите этот шаг для остальных узлов кластера.

После завершения команды `install talos-bootstrap` сохранит конфигурацию кластера как `./kubeconfig`.

Настройте `kubectl` на использование новой конфигурации, экспортировав переменную `KUBECONFIG`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Чтобы сделать этот `kubeconfig` постоянно доступным, можно сделать его конфигурацией по умолчанию (`~/.kube/config`), использовать `kubectl config use-context` или применить другие методы. См. [документацию Kubernetes о доступе к кластеру](#).

Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get ns
```

Пример вывода:

| NAME | STATUS | AGE |
|-----------------|--------|-------|
| default | Active | 7m56s |
| kube-node-lease | Active | 7m56s |
| kube-public | Active | 7m56s |
| kube-system | Active | 7m56s |

[!] Все узлы будут отображаться как `READY: False`, и на этом этапе это нормально. Так происходит потому, что на предыдущем шаге стандартный CNI-плагин был отключен, чтобы Cozystack мог установить собственный CNI-плагин.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes, готовый к установке Cozystack. Чтобы завершить установку, следуйте руководству по разворачиванию, начиная с раздела [Установка Cozystack](#).

Использование talosctl для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.5/install/kubernetes/talosctl/>

В этом руководстве описано, как подготовить кластер Talos Linux к разворачиванию Cozystack с помощью `talosctl` — специализированного CLI-инструмента для управления Talos.

Предварительные требования

К началу работы с этим руководством Talos OS должна быть загружена с ISO на нескольких узлах, но еще не инициализирована (bootstrapped). Эти узлы должны находиться в одной подсети или иметь публичные IP-адреса.

В руководстве используется пример, где узлы кластера находятся в подсети `192.168.123.0/24` и имеют следующие IP-адреса:

- `192.168.123.11`
- `192.168.123.12`
- `192.168.123.13`

IP `192.168.123.10` — внутренний адрес, который не принадлежит ни одному из этих узлов, но создается Talos. Он используется как VIP.

[!TIP]

Если вы используете DHCP, вы можете не знать IP-адреса, назначенные узлам. Чтобы найти их, можно использовать `nmap`, указав маску сети (`192.168.123.0/24` в примере):

>

```
""bash
nmap -Pn -n -p 50000 192.168.123.0/24 -vv | grep 'Discovered'
""
```

>

Пример вывода:

>

```
""text
Discovered open port 50000/tcp on 192.168.123.11
Discovered open port 50000/tcp on 192.168.123.12
Discovered open port 50000/tcp on 192.168.123.13
""
```

1. Подготовка конфигурационных файлов

1. Начните с создания каталога конфигурации для нового кластера:

```
``bash
mkdir -p cluster1
cd cluster1
``
```

2. Сгенерируйте файл secrets. Эти secrets позже будут внедрены в конфигурацию и использованы для установления аутентифицированных соединений с узлами Talos:

```
``bash
talosctl gen secrets
``
```

3. Создайте файл конфигурационного patch `patch.yaml`:

```
``yaml
machine:
kubenet:
nodeIP:
validSubnets:
  - 192.168.123.0/24
extraConfig:
maxPods: 512
sysctls:
net.ipv4.neigh.default.gc_thresh1: "4096"
net.ipv4.neigh.default.gc_thresh2: "8192"
net.ipv4.neigh.default.gc_thresh3: "16384"
kernel:
modules:
  - name: openvswitch
  - name: drbd
parameters:
  - usermode_helper=disabled
  - name: zfs
  - name: spl
  - name: vfio_pci
  - name: vfio_iommu_type1
install:
```

```
image: ghcr.io/cozystack/cozystack/talos:v1.13.0
registries:
mirrors:
docker.io:
endpoints:
  - https://mirror.gcr.io
files:
  - content: |
[plugins]
[plugins."io.containerd.cri.v1.runtime"]
device_ownership_from_security_context = true
path: /etc/cri/conf.d/20-customization.part
op: create
  - op: overwrite
path: /etc/lvm/lvm.conf
permissions: 0o644
content: |
backup {
  backup = 0
  archive = 0
}
devices {
  global_filter = [ "r|/dev/drbd.|", "r|/dev/zvol.|", "r|/dev/zd.|", "r|/dev/mapper/pve-." ]
}
cluster:
apiServer:
extraArgs:
oidc-issuer-url: "https://keycloak.example.org/realms/cozy"
oidc-client-id: "kubernetes"
oidc-username-claim: "preferred_username"
oidc-groups-claim: "groups"
network:
cni:
name: none
dnsDomain: cozy.local
```

```
podSubnets:
  - 10.244.0.0/16
serviceSubnets:
  - 10.96.0.0/16
...

```

4. Создайте еще один файл конфигурационного patch `patch-controlplane.yaml` с настройками только для узлов control plane:

Обратите внимание, что VIP-адрес используется в `machine.network.interfaces[0].vip.ip`:

```
``yaml
machine:
nodeLabels:
node.kubernetes.io/exclude-from-external-load-balancers:
$patch: delete
network:
interfaces:
  - interface: eth0
vip:
ip: 192.168.123.10
cluster:
allowSchedulingOnControlPlanes: true
controllerManager:
extraArgs:
bind-address: 0.0.0.0
scheduler:
extraArgs:
bind-address: 0.0.0.0
apiServer:
certSANs:
  - 127.0.0.1
proxy:
disabled: true
discovery:
enabled: false
etcd:
advertisedSubnets:

```

- 192.168.123.0/24

...

2. Генерация конфигурационных файлов узлов

Когда patch-файлы будут готовы, сгенерируйте конфигурационные файлы для каждого узла. Обратите внимание, что используются три файла, созданные на предыдущем шаге: `secrets.yaml`, `patch.yaml` и `patch-controlplane.yaml`.

URL `192.168.123.10:6443` использует тот же VIP, который упоминался выше, а порт `6443` — стандартный порт Kubernetes API.

```
talosctl gen config \
  cozystack https://192.168.123.10:6443 \
  --with-secrets secrets.yaml \
  --config-patch=@patch.yaml \
  --config-patch-control-plane @patch-controlplane.yaml

export TALOSCONFIG=$PWD/talosconfig
```

`192.168.123.11`, `192.168.123.12` и `192.168.123.13` — это узлы. В этой конфигурации все узлы являются управляющими.

3. Применение конфигурации узлов

Примените конфигурацию ко всем узлам, а не только к управляющим.

```
talosctl apply -f controlplane.yaml -n 192.168.123.11 -e 192.168.123.11 -i
talosctl apply -f controlplane.yaml -n 192.168.123.12 -e 192.168.123.12 -i
talosctl apply -f controlplane.yaml -n 192.168.123.13 -e 192.168.123.13 -i
```

Также можно использовать следующие опции:

- `--dry-run` - dry run mode покажет diff с существующей конфигурацией.
- `-m try` - try mode откатит конфигурацию через 1 минуту.

3.1. Ожидание перезагрузки узлов

Дождитесь, пока все узлы перезагрузятся. Извлеките установочный носитель (например, USB-накопитель), чтобы узлы загрузились с внутреннего диска.

Готовые узлы будут открывать порт `50000`; это признак того, что узел завершил настройку Talos и перезагрузился.

Если нужно дождаться готовности узлов в скрипте, используйте такой пример:

```
timeout 60 sh -c 'until nc -nzv 192.168.123.11 50000 && \
nc -nzv 192.168.123.12 50000 && \
nc -nzv 192.168.123.13 50000; \
do sleep 1; done'
```

4. Инициализация и доступ к кластеру

Запустите `talosctl bootstrap` на одном узле control plane — этого достаточно для инициализации всего кластера:

```
talosctl bootstrap -n 192.168.123.11 -e 192.168.123.11
```

Для доступа к кластеру сгенерируйте административный `kubeconfig`:

```
talosctl kubeconfig -n 192.168.123.11 -e 192.168.123.11 kubeconfig
```

Настройте `kubectl` на использование новой конфигурации, экспортировав переменную `KUBECONFIG`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Чтобы сделать этот `kubeconfig` постоянно доступным, можно сделать его конфигурацией по умолчанию (`~/.kube/config`), использовать `kubectl config use-context` или применить другие методы. См. [документацию Kubernetes о доступе к кластеру](#).

Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get ns
```

Пример вывода:

| NAME | STATUS | AGE |
|-----------------|--------|-------|
| default | Active | 7m56s |
| kube-node-lease | Active | 7m56s |
| kube-public | Active | 7m56s |
| kube-system | Active | 7m56s |

[!] Все узлы будут отображаться как `READY: False`, и на этом этапе это нормально. Так происходит потому, что на предыдущем шаге стандартный CNI-плагин был отключен, чтобы Cozystack мог установить собственный CNI-плагин.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes, готовый к установке Cozystack. Чтобы завершить установку, следуйте руководству по разворачиванию, начиная с раздела [Установка Cozystack](#).

Инициализация кластера в изолированной среде

<https://cozystack.ru/docs/v1.5/install/kubernetes/air-gapped/>

Введение

В этом руководстве описаны шаги для инициализации кластера Cozystack в изолированной среде.

Установка в `air-gapped` среде означает, что кластер не имеет прямого доступа к Интернету. Все необходимые ресурсы, такие как образы и `metadata`, должны быть доступны в частной сети.

Настройка узлов Talos

Для установки с `Talm` достаточно один раз внести все указанные изменения в `/templates/_helpers.tpl`, а затем собрать фактические конфигурационные файлы узлов с помощью `talm template`.

Для установки с `talosctl` изменения нужно внести в `patch.yaml` и `patch-controlplane.yaml`.

1. Настройка NTP-серверов

Точная синхронизация времени критически важна для кластера. В конфигурации Talos machine укажите локальные NTP-серверы, доступные внутри вашей частной сети:

```
machine:
  time:
    servers:
      # example values
      - 192.168.0.4
      - 10.10.0.5
```

Убедитесь, что эти NTP-серверы доступны с первого узла Talos.

2. Настройка mirror'ов Container Registry

Поскольку кластер не может обращаться к публичным container registries, он должен использовать их локальные mirror'ы. Создание таких mirror'ов выходит за рамки этого руководства.

Обновите конфигурацию machine следующим образом, указав IP-адреса и порты локальных mirror'ов для каждого registry:

```
machine:
  registries:
    mirrors:
      docker.io:
        endpoints:
          - http://10.0.0.1:8082
      ghcr.io:
        endpoints:
          - http://10.0.0.1:8083
      gcr.io:
        endpoints:
          - http://10.0.0.1:8084
      registry.k8s.io:
        endpoints:
          - http://10.0.0.1:8085
      quay.io:
        endpoints:
          - http://10.0.0.1:8086
      cr.fluentbit.io:
        endpoints:
          - http://10.0.0.1:8087
      docker-registry3.mariadb.com:
        endpoints:
          - http://10.0.0.1:8088
    config:
      "10.0.0.1:8082":
        tls:
          insecureSkipVerify: true
        auth:
          username: myuser
          password: mypass
```

Значения `config.[0].auth.*` приведены как пример; используйте реальные учетные данные. Убедитесь, что ваши локальные registry proxies mirror'ят все необходимые образы для компонентов Talos и Kubernetes.

3. Добавление CA-сертификата

Чтобы использовать частный Certificate Authority, добавьте его сертификат на узлы.

```
# talm: nodes=["10.10.10.10"], endpoints=["10.10.10.10"], templates=["templates/controlplane.yaml"]
# THIS FILE IS AUTOGENERATED. PREFER TEMPLATE EDITS OVER MANUAL ONES.
machine:
# ...
# ...
  discovery:
    enabled: false
  etcd:
    advertisedSubnets:
      - 10.4.100.10/24
    allowSchedulingOnControlPlanes: true
---
apiVersion: v1alpha1
kind: TrustedRootsConfig
name: my-enterprise-ca
certificates:
  |
  -----BEGIN CERTIFICATE-----
  ...
  -----END CERTIFICATE-----
```

4. Применение изменений

После внесения описанных выше изменений можно применить конфигурацию и инициализировать кластер:

Использование Talm

Пересоберите конфигурационные файлы узлов и примените их к каждому узлу:

```
talm template -e <ip> -n <ip> -t templates/controlplane.yaml -i > nodes/node1.yaml
talm apply -f nodes/node1.yaml
# повторите для каждого узла
```

Затем инициализируйте кластер обычным способом:

```
talm bootstrap -f nodes/node1.yaml
```

Подробнее см. в [руководстве по настройке Talm](#).

Использование talosctl

Примените конфигурацию к каждому узлу:

```
talosctl apply-config --insecure -n <node-ip> -f nodes/node1.yaml
```

Затем инициализируйте кластер с помощью одного из узлов:

```
talosctl bootstrap -n <ip> -e <ip>
```

Подробнее см. в [руководстве по настройке talosctl](#).

5. Настройка mirror'ов Container Registry для Tenant Kubernetes

Tenant-кластеры Kubernetes в Cozystack используют [Kamaji](#) для control plane. Компоненты control plane работают как pods на узлах management-кластера, поэтому они автоматически используют registry mirrors, настроенные для Talos на [шаге 2](#).

Однако tenant worker nodes работают как отдельные виртуальные машины со своим экземпляром containerd. Для этих worker-узлов нужна отдельная конфигурация registry mirror.

Чтобы выполнить эту настройку, сначала нужно развернуть кластер Cozystack версии v0.32.0 или новее (или обновить существующий кластер до этой версии). Проверьте текущую версию кластера командой:

```
kubectl get deploy -n cozy-system cozystack -o yaml | grep installer
```

Вариант А: настройка через platform package

Platform package может автоматически сгенерировать secret `patch-containerd` из раздела `registries` в platform values.

Добавьте раздел `registries` в `cozystack-platform.yaml`:

```

apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        # ... существующие publishing, networking и т. д. ...
        registries:
          mirrors:
            docker.io:
              endpoints:
                - http://10.0.0.1:8082
            ghcr.io:
              endpoints:
                - http://10.0.0.1:8083
            gcr.io:
              endpoints:
                - http://10.0.0.1:8084
            registry.k8s.io:
              endpoints:
                - http://10.0.0.1:8085
            quay.io:
              endpoints:
                - http://10.0.0.1:8086
            cr.fluentbit.io:
              endpoints:
                - http://10.0.0.1:8087
            docker-registry3.mariadb.com:
              endpoints:
                - http://10.0.0.1:8088
          config:
            "10.0.0.1:8082":
              tls:
                insecureSkipVerify: true
              auth:
                username: myuser
                password: mypass

```

Затем примените его:

```
kubectl apply -f cozystack-platform.yaml
```

Это создаст secret `patch-containerd` в namespace `cozy-system`, который автоматически копируется в каждый tenant-кластер Kubernetes.

Альтернативно: применить patch к существующему platform package

Вариант В: создание secret вручную

Также можно напрямую создать [Kubernetes Secret](#) с именем `patch-containerd`:

```

apiVersion: v1
kind: Secret
metadata:
  name: patch-containerd
  namespace: cozy-system
type: Opaque
stringData:
  docker.io.toml: |
    server = "https://registry-1.docker.io"
    [host."http://10.0.0.1:8082"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  ghcr.io.toml: |
    server = "https://ghcr.io"
    [host."http://10.0.0.1:8083"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  gcr.io.toml: |
    server = "https://gcr.io"
    [host."http://10.0.0.1:8084"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  registry.k8s.io.toml: |
    server = "https://registry.k8s.io"
    [host."http://10.0.0.1:8085"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  quay.io.toml: |
    server = "https://quay.io"
    [host."http://10.0.0.1:8086"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  cr.fluentbit.io.toml: |
    server = "https://cr.fluentbit.io"
    [host."http://10.0.0.1:8087"]
      capabilities = ["pull", "resolve"]
      skip_verify = true
  docker-registry3.mariadb.com.toml: |
    server = "https://docker-registry3.mariadb.com"
    [host."http://10.0.0.1:8088"]
      capabilities = ["pull", "resolve"]
      skip_verify = true

```

Если registry mirrors требуют аутентификации, добавьте пользовательский header `Authorization` с учетными данными в Base64:

```

server = "https://registry-1.docker.io"
[host."http://10.0.0.1:8082"]
  capabilities = ["pull", "resolve"]
  skip_verify = true
[host."http://10.0.0.1:8082".header]
  Authorization = "Basic bXl1c2Vy0m15cGFzcw=="

```

Чтобы сгенерировать значение в Base64, выполните:

```
echo -n 'myuser:mypass' | base64
```

Для динамической или token-based аутентификации (например, Docker Hub) используйте [Kubernetes image pull secrets](#) вместо учетных данных в открытом виде.

Как это работает

Secret `patch-containerd` из namespace `cozy-system` автоматически копируется в namespace каждого tenant-кластера Kubernetes во время развертывания. Данные этого secret монтируются в worker node VM как конфигурационные файлы registry для containerd в `/etc/containerd/certs.d/<registry>/hosts.toml`.

Конфигурация для отдельного кластера

Можно настроить registry mirrors для конкретного tenant-кластера Kubernetes вместо использования глобального secret `patch-containerd`:

- Tenant-кластер должен быть развернут с Kubernetes package версии 0.23.1 или новее, которая доступна начиная с Cozystack 0.32.1.
- Перед развертыванием tenant-кластера создайте Kubernetes Secret с именем `kubernetes-<cluster-name>-patch-containerd` в namespace tenant-кластера, используя тот же формат, что и в примерах выше.

Важно: если существуют и глобальный secret `patch-containerd`, и secret для отдельного кластера, глобальный secret имеет приоритет, а per-cluster secret игнорируется. Конфигурация для отдельного кластера работает только в том случае, если глобального secret `patch-containerd` в namespace `cozy-system` нет.

Подробнее о значениях конфигурации registry см. в [руководстве по настройке CRI Plugin](#).

Устранение неполадок при установке Kubernetes

<https://cozystack.ru/docs/v1.5/install/kubernetes/troubleshooting/>

На этой странице приведены инструкции по решению типовых проблем, которые могут возникнуть при установке Kubernetes с помощью `tal`, `talos-bootstrap` или `talosctl`.

No Talos nodes in maintenance mode found!

Если скрипт `talos-bootstrap` не обнаруживает узлы, выполните следующие шаги для диагностики и устранения проблемы:

1. Проверьте сетевой сегмент

Убедитесь, что скрипт запускается в том же сетевом сегменте, что и узлы. Это критически важно, чтобы скрипт мог взаимодействовать с узлами.

2. Используйте Nmap для обнаружения узлов

Проверьте, может ли `nmap` обнаружить узел, выполнив следующую команду:

```
``bash
nmap -Pn -n -p 50000 192.168.0.0/24
``
```

Эта команда сканирует сеть на наличие узлов, которые слушают порт `50000`. В выводе должны быть перечислены все узлы в сетевом сегменте, слушающие этот порт; это означает, что они доступны.

3. Проверьте подключение talosctl

Затем убедитесь, что `talosctl` может подключиться к конкретному узлу, особенно если узел находится в maintenance mode:

```
``bash
talosctl -e ${node} -n ${node} get machinestatus -i
``
```

Ошибка вида ниже обычно означает, что локальный бинарный файл `talosctl` устарел:

```
``text
rpc error: code = Unimplemented desc = unknown service resource.ResourceService
``
```

Обновление `talosctl` до последней версии должно решить проблему.

4. Запустите talos-bootstrap в debug mode

Если предыдущие шаги не помогли, запустите `talos-bootstrap` в debug mode, чтобы получить больше диагностической информации.

Выполните скрипт с опцией `-x`, чтобы включить debug mode:

```
```bash
bash -x talos-bootstrap
```
```

Обратите внимание на последнюю команду, показанную перед ошибкой; часто она указывает на команду, которая завершилась неудачно, и помогает в дальнейшей диагностике.

Исправление ext-lldpd на узлах Talos

Ожидание runtime service в Talos может привести к тому, что узел останется в состоянии booting в консоли Talos. Если вы хотите использовать lldpd, можно применить patch к узлам. Продолжайте, если у вас есть подключение через `talosctl`.

```
cat > lldpd.patch.yaml <<EOF
apiVersion: v1alpha1
kind: ExtensionServiceConfig
name: lldpd
configFiles:
  - content: |
      configure lldp status disabled
      mountPath: /usr/local/etc/lldp/lldpd.conf
EOF
```

Чтобы применить patch к конкретному узлу, выполните:

```
talosctl patch mc -p @lldpd.patch.yaml -n <node> -e <node>
```

Проверьте, на каких узлах установлен lldpd:

```
node_net='192.168.100.0/24'
nmap -Pn -n -T4 -p50000 --open -oG - $node_net | awk '/50000\open/ { system("talosctl get extensions -n \"$...
```

Если хотите применить patch ко всем узлам:

```
nmap -Pn -n -T4 -p50000 --open -oG - $node_net | awk '/50000\open/ {print "talosctl patch mc -p @lldpd.pat...
```

Проверьте состояние в консоли Talos:

```
talosctl dashboard -n $(nmap -Pn -n -T4 -p50000 --open -oG - $node_net | awk '/50000\open/ {print $2}') | p...
```

Generic Kubernetes

<https://cozystack.ru/docs/v1.5/install/kubernetes/generic/>

Содержимое этой страницы недоступно в офлайн-версии. Откройте онлайн-версию по ссылке выше.

Ubuntu Secure Boot: DRBD prerequisites

<https://cozystack.ru/docs/v1.5/install/kubernetes/ubuntu-secure-boot/>

This page covers a single prerequisite for installing Cozystack on Ubuntu hosts where UEFI Secure Boot is enabled. If your nodes are Talos Linux, or Ubuntu with Secure Boot disabled, you can skip this page — the default `piraeus-operator` flow handles those cases.

Why this is needed

Cozystack uses `LINSTOR` for replicated block storage, which depends on the DRBD 9.x kernel module. The `piraeus-operator` that ships with Cozystack defaults to compiling DRBD from source on each node at runtime and loading the module via `insmod`.

On hosts where UEFI Secure Boot is enabled, the kernel's lockdown subsystem rejects unsigned modules at load time:

```
insmod: ERROR: could not insert module ./drbd.ko: Key was rejected by service
```

(A preceding `chcon: can't apply partial context to unlabeled file './drbd.ko'` line may also appear when the loader image sets `LB_SELINUX_AS` — that warning is benign and unrelated to the load failure.)

The compiled `.ko` files are not signed against any key the host trusts, so `init_module()` fails. The `linstor` satellite Pod never reaches `Ready` and Cozystack storage stays broken.

This is common on bare-metal Ubuntu installs and on cloud SKUs that ship with Secure Boot enabled. Stock cloud-VM images on AWS, GCP, and Azure typically ship without Secure Boot enforcement and the default `piraeus-operator` flow works as-is.

This guide covers Ubuntu LTS only. Debian 12 has the same DRBD-unsigned-module problem under Secure Boot but LINBIT does not publish a Debian PPA — those operators must use LINBIT's customer-portal apt mirror or build and sign `drbd-dkms` manually. RHEL/SUSE flows (LINBIT-managed RPM repos with pre-signed kmods) are out of scope here.

Recommended fix: pre-install drbd-dkms on the host

Install the LINBIT-published `drbd-dkms` package on each node before deploying Cozystack, then add a `usermode_helper=disabled` `modprobe.d` drop-in and mask the host-side `drbd.service`. Minimal Ubuntu cloud images do not ship `add-apt-repository`; install `software-properties-common` first if it is missing. The full flow on each node:

1. Add LINBIT PPA, install `drbd-dkms`. The package goes through Ubuntu's standard `shim-signed` + `dkms` pipeline:
 - On first `dkms` install with no existing MOK present, Ubuntu's `shim-signed` `postinst` generates a per-host signing keypair at `/var/lib/shim-signed/mok/MOK.priv` and

`/var/lib/shim-signed/mok/MOK.der` (referenced from `/etc/dkms/framework.conf` as `mok_signing_key / mok_certificate`).

- dkms compiles the module against the running kernel and signs the freshly built `.ko` against that key.

- During `apt-get install`, `debconf` prompts the operator for an enrollment password; the matching public key is queued for MOK enrollment via `mokutil --import`.

- If the install runs non-interactively (`DEBIAN_FRONTEND=noninteractive`), the prompt is bypassed and the operator must run `mokutil --import /var/lib/shim-signed/mok/MOK.der` manually after the install.

2. Write `/etc/modprobe.d/cozystack-drbd.conf` with options `drbd usermode_helper=disabled`.

piraeus-operator's `daemonset` sets `LB_FAIL_IF_USERMODE_HELPER_NOT_DISABLED=yes` on the loader, so the loader fails on a host-loaded module without that param (see [LINBIT/drbd entry.sh line 332](#) and the env wiring in piraeus-operator's [satellite daemonset](#)).

3. `systemctl mask drbd.service`. `drbd-utils` lands on the host transitively as a hard apt dependency of `drbd-dkms` and ships a `drbd.service` unit that would race piraeus-operator's satellite container if accidentally enabled.

4. Write `/etc/modules-load.d/cozystack-drbd.conf` containing `drbd` so `systemd-modules-load.service` loads the module on every boot. Without this file the host depends on whatever (if any) `modules-load.d` entry the `drbd-utils` package ships, which has varied across releases — making it explicit removes the ambiguity and matches the path the companion Ansible playbook writes.

5. Reboot each node and confirm MOK enrollment at the shim console (Enroll MOK -> View key -> Continue -> enter the password set during step 1's `debconf` prompt or via `mokutil --import`). One reboot per node is required for the operator to walk through shim's MOK Manager. This step cannot be automated — it is the design of UEFI Secure Boot.

6. After enrollment, the kernel trusts the signing key and the dkms-built DRBD module loads with the right param.

Once the host has DRBD 9.x loaded with `usermode_helper=disabled`, piraeus-operator's loader detects the host-loaded module on Pod startup and exits cleanly without attempting its own compile and `insmod`. No operator-side configuration is required.

MOK enrollment is interactive and per-node. The operator must access the console (physical, IPMI, or KVM) of each node on the next reboot and complete the MOK Manager flow.

For the manual procedure on each node, see below. Automation is being added to [cozystack/ansible-cozystack \(PR #39\)](#); v1.5 ships only the manual path because the automation has not yet landed in a tagged collection release.

Manual path

For operators who do not use Ansible, the equivalent steps on each Ubuntu LTS node are:

```
# 1. Install add-apt-repository (not present on minimal cloud images).
sudo apt-get update
sudo apt-get install --yes software-properties-common

# 2. Add LINBIT PPA and install drbd-dkms. During the install, debconf
#   prompts for a one-time MOK enrollment password – choose any
#   password you can re-enter at the shim console after reboot.
sudo add-apt-repository --yes ppa:linbit/linbit-drbd9-stack
sudo apt-get install --yes drbd-dkms

# 3. Configure the required module parameter.
sudo tee /etc/modprobe.d/cozystack-drbd.conf <<'EOF'
options drbd usermode_helper=disabled
EOF

# 4. Mask the host drbd.service so it cannot race piraeus-operator.
sudo systemctl mask drbd.service

# 5. Persist the module load on boot. The drbd-utils package's own
#   /lib/modules-load.d/ entry has varied across releases; an explicit
#   /etc/ entry overrides anything in /lib/ and removes the ambiguity.
echo drbd | sudo tee /etc/modules-load.d/cozystack-drbd.conf

# 6. Reboot. At the shim MOK Manager menu, choose:
#   Enroll MOK -> View key -> Continue -> Yes -> enter the password
#   you set at the debconf prompt during step 2.
sudo reboot
```

After the reboot, verify the module loaded with the right param:

```
cat /sys/module/drbd/parameters/usermode_helper
# expected: disabled
```

Then deploy Cozystack as usual. The piraeus-operator loader will detect the host-loaded DRBD and exit cleanly.

What happens at deploy time

When piraeus-operator's satellite Pod starts on a node, its `drbd-module-loader` initContainer runs [LINBIT/drbd's entry.sh](#). Lines 328–339 short-circuit the compile + insmod path when DRBD is already loaded on the host:

```
if grep -q '^drbd ' /proc/modules; then
    echo "DRBD module is already loaded"
    [[ $LB_FAIL_IF_USERMODE_HELPER_NOT_DISABLED == yes ]] \
        && ! grep -qw disabled /sys/module/drbd/parameters/usermode_helper \
        && die "...
    drbd_matches_min_version "$LB_DRBD_MIN_LOADED_VERSION" || die "...
    exit 0
fi
```

Two preconditions are checked:

1. The module must be loaded with `usermode_helper=disabled`. The `modprobe.d` drop-in above takes care of this.

2. The loaded version must satisfy `LB_DRBD_MIN_LOADED_VERSION`. `piraeus-operator`'s daemonset sets it to 9. LINBIT's `drbd-dkms` ships DRBD 9.x, so this is automatic.

Both conditions met -> `exit 0` -> satellite Pod proceeds. No operator-side change to `piraeus-operator` or `LinstorSatelliteConfiguration` is needed.

Alternatives

- Talos Linux ([guide](#)) ships pre-signed DRBD modules in its system extensions and has none of these problems. Recommended for new deployments where you do not need a custom Linux distro.
- Disable Secure Boot in the host's UEFI firmware. The default in-cluster compile path then works without modification. Operationally undesirable for fleets where Secure Boot is part of the security baseline, but a valid escape hatch.
- Build and sign `drbd-dkms` manually against your own enterprise CA. This is what production environments with their own MOK / shim signing infrastructure already do; it is out of scope for this guide.

Troubleshooting

- ****modprobe drbd returns Key was rejected by service after dkms install**** — the dkms-generated MOK key is queued but not yet enrolled. Reboot the node and confirm enrollment at the shim console (Enroll MOK -> View key -> Continue -> dkms password). Re-run `modprobe`.
 - ****cat /sys/module/drbd/parameters/usermode_helper is not disabled**** — the `modprobe.d` drop-in is missing or the module was loaded before it was written. `sudo rmmod drbd && sudo modprobe drbd` after the drop-in is in place, or reboot.
 - ****piraeus-operator loader Pod logs Could not load DRBD kernel modules even after host install**** — check `LB_DRBD_MIN_LOADED_VERSION` via `kubectl --namespace cozy-linstor describe pod --selector app.kubernetes.io/component=linstor-satellite` and `cat /proc/drbd` on the host. The host module must be at least the loader's required minimum (9 per `piraeus-operator`'s daemonset). LINBIT PPA `drbd-dkms` is 9.x so this is rarely the issue.
 - Ubuntu 26.04+ or interim releases (Oracular 24.10, Plucky 25.04) — LINBIT's PPA publishes `drbd-dkms` only for the LTS series LINBIT keeps current. Check [the PPA detail page](#) for the current series list. On unsupported releases the PPA add fails on a 404 Release file. Build and sign `drbd-dkms` manually, downgrade to a supported LTS, or wait for LINBIT to publish for your release.
-

Установка и настройка Cozystack

<https://cozystack.ru/docs/v1.5/install/cozystack/>

Третий шаг развертывания кластера Cozystack — установка Cozystack в ранее [установленный и настроенный кластер Kubernetes](#). Предварительное условие для этого шага — установленный кластер Kubernetes.

Cozystack можно установить в двух режимах, в зависимости от того, насколько детально вы хотите управлять устанавливаемыми компонентами:

Как платформа

Установите Cozystack как готовую к использованию платформу, где все компоненты управляются автоматически. Вы выбираете [variant](#) (например, `isp-full`), а Cozystack устанавливает и настраивает все необходимые компоненты — сеть, хранилище, мониторинг, dashboard, operators и managed applications.

Это рекомендуемый подход для большинства пользователей, которым нужна полностью функциональная платформа из коробки.

[Установить Cozystack как платформу](#)

Build Your Own Platform (BYOP)

Используйте Cozystack, чтобы собрать собственную платформу, устанавливая только нужные компоненты. Вы устанавливаете operator с variant `default`, который предоставляет только package registry (PackageSources). Затем с помощью CLI-инструмента `cozypkg` выборочно устанавливаете отдельные packages — `networking`, `storage`, `ingress`, `operators` и любые другие компоненты, доступные в репозитории Cozystack.

Этот подход подходит, если у вас уже есть существующий кластер Kubernetes с частью инфраструктуры или если вам нужны только отдельные компоненты из экосистемы Cozystack.

[Собрать собственную платформу с Cozystack](#)

Установка Cozystack как платформы

Установите Cozystack как готовую к использованию платформу, где все компоненты управляются автоматически.

Собственная платформа (BYOP)

Соберите собственную платформу с Cozystack, устанавливая только нужные компоненты с помощью CLI-инструмента `cozyrkg`.

Установка Cozystack как платформы

<https://cozystack.ru/docs/v1.5/install/cozystack/platform/>

Третий шаг разворачивания кластера Cozystack — установка Cozystack в кластер Kubernetes, ранее установленный и настроенный на узлах Talos Linux. Предварительное условие для этого шага — [установленный кластер Kubernetes](#).

Если вы устанавливаете Cozystack впервые, рекомендуем начать с [руководства по Cozystack](#).

Чтобы спланировать production-ready установку, следуйте руководству ниже. По структуре оно повторяет tutorial, но содержит гораздо больше деталей и объясняет разные варианты установки.

1. Установка Cozystack Operator

Установите Cozystack operator с помощью Helm из OCI registry:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace
```

Замените X.Y.Z на нужную версию Cozystack. Доступные версии можно найти на [странице релизов Cozystack](#).

Эта команда устанавливает operator, CRD и создает ресурс PackageSource.

Установка на ОС без Talos

По умолчанию Cozystack operator настроен на использование функции [KubePrism](#) из Talos Linux, которая позволяет обращаться к Kubernetes API через локальный адрес на узле.

Если вы устанавливаете Cozystack не на Talos Linux, укажите variant operator во время установки:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--set cozystackOperator.variant=generic \
--set cozystack.apiServerHost=<YOUR_API_SERVER_IP> \
--set cozystack.apiServerPort=6443
```

Замените <YOUR_API_SERVER_IP> на внутренний IP-адрес вашего Kubernetes API server (только IP, без протокола и порта).

Полное руководство по разворачиванию Cozystack на generic-дистрибутивах Kubernetes см. в разделе [Развертывание Cozystack на Generic Kubernetes](#).

2. Описание и применение Platform Package

После запуска operator следующий шаг — описать Platform Package и применить его. Platform Package — это ресурс Package, который определяет [variant Cozystack](#), [настройки компонентов](#), ключевые сетевые настройки, опубликованные сервисы и другие параметры.

Конфигурацию Cozystack можно обновлять после установки. Однако некоторые значения, показанные ниже, обязательны для установки.

Ниже минимальный пример `cozystack-platform.yaml`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        publishing:
          host: example.org
          apiServerEndpoint: api.example.org
          exposedServices:
            - dashboard
            - api
        networking:
          podCIDR: "10.244.0.0/16"
          podGateway: "10.244.0.1"
          serviceCIDR: "10.96.0.0/16"
          joinCIDR: "100.64.0.0/16"
```

Имя Package должно быть `cozystack.cozystack-platform`, чтобы совпадать с PackageSource, созданным installer. Проверить доступные PackageSources можно командой `kubectl get packagesource`.

2.1. Выбор variant

Состав Cozystack определяется variant. Variant `isp-full` — самый полный: он охватывает все уровни от оборудования до managed applications. Выбирайте его, если планируете строить публичное или приватное облако.

Если вы разворачиваете Cozystack в предоставленном Kubernetes-кластере или хотите развернуть только часть компонентов, выберите другой variant. См. [обзор и сравнение variants](#).

2.2. Тонкая настройка компонентов

Можно добавить дополнительные компоненты или убрать компоненты, включенные по умолчанию. См. [справочник components](#).

Если вы разворачиваете платформу на ВМ или выделенных серверах облачного провайдера, скорее всего, нужно изменить конфигурацию некоторых компонентов. См. раздел [установка у](#)

конкретных провайдеров. В нем может быть полное руководство для вашего провайдера, которое можно использовать для развертывания production-ready кластера.

2.3. Описание сетевой конфигурации

Замените `example.org` в `publishing.host` и `publishing.apiServerEndpoint` на маршрутизируемое fully-qualified domain name (FQDN), которым вы управляете. Если у вас есть только IP-адреса, можно использовать сервис nip.io с записью через дефис.

В следующем разделе приведены разумные значения по умолчанию. Проверьте, что они совпадают с настройками узлов Talos, которые вы использовали на предыдущих шагах. Если вы использовали Talos для установки Kubernetes, значения должны совпадать.

```
networking:
  podCIDR: "10.244.0.0/16"
  podGateway: "10.244.0.1"
  serviceCIDR: "10.96.0.0/16"
  joinCIDR: "100.64.0.0/16"
```

Cozystack по умолчанию собирает анонимную статистику использования. Подробнее о том, какие данные собираются и как отказаться от сбора, см. в [документации по Telemetry](#).

2.4. Применение Platform Package

Когда конфигурационный файл будет готов, примените его:

```
kubectl apply -f cozystack-platform.yaml
```

Во время установки можно отслеживать логи operator:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

Подождите некоторое время, затем проверьте состояние установки:

```
kubectl get hr -A
```

Дождитесь, пока все releases перейдут в состояние `Ready`:

| | | | | | | |
|--------------------------------|---------------------------|------|------|---------|----------------|-----------|
| cozy-cert-manager | cert-manager | 4m1s | True | Release | reconciliation | succeeded |
| cozy-cert-manager | cert-manager-issuers | 4m1s | True | Release | reconciliation | succeeded |
| cozy-cilium | cilium | 4m1s | True | Release | reconciliation | succeeded |
| cozy-cluster-api | capi-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-cluster-api | capi-providers | 4m1s | True | Release | reconciliation | succeeded |
| cozy-dashboard | dashboard | 4m1s | True | Release | reconciliation | succeeded |
| cozy-grafana-operator | grafana-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kamaji | kamaji | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kubeovn | kubeovn | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kubevirt-cdi | kubevirt-cdi | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kubevirt-cdi | kubevirt-cdi-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kubevirt | kubevirt | 4m1s | True | Release | reconciliation | succeeded |
| cozy-kubevirt | kubevirt-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-linstor | linstor | 4m1s | True | Release | reconciliation | succeeded |
| cozy-linstor | piraeus-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-mariadb-operator | mariadb-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-metallb | metallb | 4m1s | True | Release | reconciliation | succeeded |
| cozy-monitoring | monitoring | 4m1s | True | Release | reconciliation | succeeded |
| cozy-postgres-operator | postgres-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-rabbitmq-operator | rabbitmq-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-redis-operator | redis-operator | 4m1s | True | Release | reconciliation | succeeded |
| cozy-telepresence | telepresence | 4m1s | True | Release | reconciliation | succeeded |
| cozy-victoria-metrics-operator | victoria-metrics-operator | 4m1s | True | Release | reconciliation | succeeded |
| tenant-root | tenant-root | 4m1s | True | Release | reconciliation | succeeded |

Разделение узлов Control Plane и Worker

Обычно Cozystack требует как минимум три worker-узла для запуска workload'ов в HA mode. В компонентах Cozystack нет tolerations, которые позволили бы им запускаться на узлах control plane.

Однако для тестирования часто используется всего три узла. Либо у вас могут быть только крупные аппаратные узлы, и вы хотите использовать их одновременно для control-plane и worker workload'ов. В этом случае нужно удалить control-plane taint с узлов.

Пример удаления control-plane taint с узлов:

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

3. Настройка хранилища

Kubernetes нужна подсистема хранения, чтобы предоставлять persistent volumes приложениям, но собственной такой подсистемы в Kubernetes нет. Cozystack предоставляет [LINSTOR](#) как подсистему хранения.

На следующих шагах мы получим доступ к интерфейсу LINSTOR, создадим storage pools и определим storage classes.

3.1. Проверка устройств хранения

Настройте alias для доступа к LINSTOR:

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

Выведите список узлов и проверьте их готовность:

```
linstor node list
```

Пример вывода показывает имена узлов и состояние:

```
+-----+
| Node | NodeType | Addresses | State |
+-----+
srv1	SATELLITE	100.64.0.1:3366 (PLAIN)	Online
srv2	SATELLITE	100.64.0.2:3366 (PLAIN)	Online
srv3	SATELLITE	100.64.0.3:3366 (PLAIN)	Online
+-----+
```

Выведите список доступных пустых устройств:

```
linstor physical-storage list
```

Пример вывода показывает те же имена узлов:

```
+-----+
| Size | Rotational | Node | DeviceName | PhysicalStoragePool |
+-----+
100.00 GiB	False	srv1	/dev/nvme0n1
100.00 GiB	False	srv2	/dev/nvme0n1
100.00 GiB	False	srv3	/dev/nvme0n1
+-----+
```

3.2. Создание Storage Pools

Создайте storage pools с помощью ZFS или LVM. Также можно восстановить ранее созданные storage pools после reset узла.

```
linstor physical-storage create-device-pool --pool-name data ZFS srv1 /dev/nvme0n1
linstor physical-storage create-device-pool --pool-name data ZFS srv2 /dev/nvme0n1
linstor physical-storage create-device-pool --pool-name data ZFS srv3 /dev/nvme0n1
```

Рекомендуется выставить `failmode=continue` для ZFS storage pools чтобы позволить DRBD обрабатывать ошибки диска вместо ZFS.

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv1 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv2 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv3 -- zpool set failmode=continue data
```

Проверьте результат, выведя список storage pools:

```
linstor storage-pool list
```

Пример вывода:

```
+-----+
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | CanSnapshots |
+-----+
DfltDisklessStorPool	srv1	DISKLESS				False
DfltDisklessStorPool	srv2	DISKLESS				False
DfltDisklessStorPool	srv3	DISKLESS				False
data	srv1	ZFS	data	96.41 GiB	99.50 GiB	True
data	srv2	ZFS	data	96.41 GiB	99.50 GiB	True
data	srv3	ZFS	data	96.41 GiB	99.50 GiB	True
+-----+
```

3.3. Создание Storage Classes

Создайте storage classes, один из которых должен быть default class.

Создайте файл с определениями storage classes. Ниже приведен разумный пример по умолчанию с двумя классами: `local` (default) и `replicated`.

storageclasses.yaml:

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: data
  linstor.csi.linbit.com/layerList: storage
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: replicated
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: data
  linstor.csi.linbit.com/autoPlace: "3"
  linstor.csi.linbit.com/layerList: drbd storage
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
  property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
  property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

Примените конфигурацию storage class:

```
kubectl apply -f storageclasses.yaml
```

Проверьте, что storage classes успешно созданы:

```
kubectl get storageclasses
```

Пример вывода:

| NAME | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE | ALLOWVOLUMEEXPANSION | AGE |
|-----------------|------------------------|---------------|----------------------|----------------------|-----|
| local (default) | linstor.csi.linbit.com | Delete | WaitForFirstConsumer | true | 1m |
| replicated | linstor.csi.linbit.com | Delete | Immediate | true | 1m |

4. Настройка сети

Далее настроим доступ к кластеру Cozystack. На этом шаге есть два варианта в зависимости от доступной инфраструктуры:

- Для собственного bare metal или self-hosted VM выберите вариант MetalLB. MetalLB — load balancer Cozystack по умолчанию.
- Для VM и выделенных серверов у облачных провайдеров выберите настройку public IP. **Большинство облачных провайдеров не поддерживают MetalLB.**

См. раздел [установка у конкретных провайдеров](#). Там могут быть инструкции для вашего провайдера, которые можно использовать для развертывания production-ready кластера.

4.a Настройка MetalLB

Cozystack использует три типа IP-адресов:

1. Node IPs: постоянные адреса, действительные только внутри кластера.
2. Virtual floating IP: используется для доступа к одному из узлов кластера и действителен только внутри кластера.
3. External access IPs: используются LoadBalancers для публикации сервисов за пределами кластера.

Сервисы с external IP можно публиковать в двух режимах: L2 и BGP. Режим L2 прост, но требует, чтобы все IP находились в одной L2-сети. BGP более надежен и масштабируем, но требует поддержки со стороны сетевого оборудования.

Выберите диапазон неиспользуемых IP для сервисов; здесь используется диапазон `192.168.100.200-192.168.100.250`. Если используется режим L2, эти IP должны быть либо из той же сети, что и узлы, либо к ним должны быть прописаны статические маршруты.

Для режима BGP также потребуются IP-адреса BGP peers и локальный и удаленный AS numbers. Здесь мы используем `192.168.20.254` как peer IP, а AS numbers 65000 и 65001 — как local и remote.

Создайте и примените файл с описанием address pool.

metallb-ip-address-pool.yml:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: cozystack
  namespace: cozy-metallb
spec:
  addresses:
    # используется для публикации сервисов за пределами кластера
    - 192.168.100.200-192.168.100.250
  autoAssign: true
  avoidBuggyIPs: false
```

```
kubectl apply -f metallb-ip-address-pool.yml
```

Создайте и примените ресурсы, необходимые для L2 или BGP advertisement. L2Advertisement использует имя ресурса IPAddressPool, который мы создали на предыдущем шаге.

metallb-l2-advertisement.yml:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: cozystack
  namespace: cozy-metallb
spec:
  ipAddressPools:
    - cozystack
```

Примените изменения:

```
kubectl apply -f metallb-l2-advertisement.yml
```

После настройки MetalLB включите ingress в tenant-root:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"ingress":true}}'
```

Чтобы подтвердить успешную настройку, проверьте HelmReleases `ingress` и `ingress-nginx-system`:

```
kubectl get hr -n tenant-root
```

Пример корректного вывода:

| | | | |
|----------------------|-----|------|---|
| ingress | 47m | True | Helm upgrade succeeded for release tenant-root/ingress.v3 with... |
| ingress-nginx-system | 47m | True | Helm upgrade succeeded for release tenant-root/ingress-nginx-s... |

Затем проверьте состояние сервиса `root-ingress-controller`:


```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развернут как **TYPE: LoadBalancer** и иметь корректный external IP:

| NAME | TYPE | CLUSTER-IP | EXTERNAL-IP | PORT(S) | AGE |
|-------------------------|--------------|--------------|-----------------|----------------------------|-----|
| root-ingress-controller | LoadBalancer | 10.96.16.141 | 192.168.100.200 | 80:31632/TCP,443:30113/TCP | 1m |

4.b. Настройка Node Public IP

Если ваш облачный провайдер не поддерживает MetalLB, ingress controller можно опубликовать через external IPs узлов.

Если public IP подключены напрямую к узлам, укажите их. Если public IP предоставляются через 1:1 NAT, укажите внутренние адреса внешних сетевых интерфейсов.

Здесь используются 192.168.100.11, 192.168.100.12 и 192.168.100.13.

Сначала примените patch к Platform Package, указав IP для публикации:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{"spec":{"components":{"p...
```

Затем включите ingress для root tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"ingress":true}}'
```

После этого Ingress будет доступен на указанных IP. Проверьте это следующим образом:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развернут как **TYPE: ClusterIP** и иметь полный диапазон external IP:

| NAME | TYPE | CLUSTER-IP | EXTERNAL-IP | PORT(S) | ... |
|-------------------------|-----------|-------------|--|-----------------|-----|
| root-ingress-controller | ClusterIP | 10.96.91.83 | 192.168.100.11,192.168.100.12,192.168.100.13 | 80/TCP,443/T... | |

5. Завершение установки

5.1. Настройка сервисов Root Tenant

Включите `etcd` и `monitoring` для root tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"etcd":true,"monitori...
```

5.2. Проверка состояния и состава кластера

Проверьте созданные persistent volumes:

```
kubectl get pvc -A
```

Пример вывода:

| NAME | STATUS | VOLUME | CAPACITY | ACC... |
|--|--------|--|----------|--------|
| data-etcd-0 | Bound | pvc-4cbd29cc-a29f-453d-b412-451647cd04bf | 10Gi | RWO... |
| data-etcd-1 | Bound | pvc-1579f95a-a69d-4a26-bcc2-b15ccdbede0d | 10Gi | RWO... |
| data-etcd-2 | Bound | pvc-907009e5-88bf-4d18-91e7-b56b0dbfb97e | 10Gi | RWO... |
| grafana-db-1 | Bound | pvc-7b3f4e23-228a-46fd-b820-d033ef4679af | 10Gi | RWO... |
| grafana-db-2 | Bound | pvc-ac9b72a4-f40e-47e8-ad24-f50d843b55e4 | 10Gi | RWO... |
| vmselect-cachedir-vmselect-longterm-0 | Bound | pvc-622fa398-2104-459f-8744-565eee0a13f1 | 2Gi | RWO... |
| vmselect-cachedir-vmselect-longterm-1 | Bound | pvc-fc9349f5-02b2-4e25-8bef-6cbc5cc6d690 | 2Gi | RWO... |
| vmselect-cachedir-vmselect-shortterm-0 | Bound | pvc-7acc7ff6-6b9b-4676-bd1f-6867ea7165e2 | 2Gi | RWO... |
| vmselect-cachedir-vmselect-shortterm-1 | Bound | pvc-e514f12b-f1f6-40ff-9838-a6bda3580eb7 | 2Gi | RWO... |
| vmstorage-db-vmstorage-longterm-0 | Bound | pvc-e8ac7fc3-df0d-4692-aebf-9f66f72f9fef | 10Gi | RWO... |
| vmstorage-db-vmstorage-longterm-1 | Bound | pvc-68b5ceaf-3ed1-4e5a-9568-6b95911c7c3a | 10Gi | RWO... |
| vmstorage-db-vmstorage-shortterm-0 | Bound | pvc-cee3a2a4-5680-4880-bc2a-85c14dba9380 | 10Gi | RWO... |
| vmstorage-db-vmstorage-shortterm-1 | Bound | pvc-d55c235d-cada-4c4a-8299-e5fc3f161789 | 10Gi | RWO... |

Проверьте, что все pods запущены:

```
kubectl get pods -A
```

Теперь можно получить public IP ingress controller:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Пример вывода:

| NAME | TYPE | CLUSTER-IP | EXTERNAL-IP | PORT(S) | AGE |
|-------------------------|--------------|--------------|-----------------|----------------------------|-----|
| root-ingress-controller | LoadBalancer | 10.96.16.141 | 192.168.100.200 | 80:31632/TCP,443:30113/TCP | 1m |

5.3 Доступ к Cozystack Dashboard

Если вы включили `dashboard` в `publishing.exposedServices` вашего Platform Package, Cozystack Dashboard уже должен быть доступен.

Если исходный Package не включал его, примените patch к Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json -p '[{"op": "add", "path": "/spec...}
```

Откройте dashboard.example.org, чтобы получить доступ к системному dashboard, где `example.org` — ваш домен, указанный для `tenant-root`. Там вы увидите окно входа, которое ожидает authentication token.

Получите authentication token для `tenant-root`:

```
kubectl get secret -n tenant-root tenant-root -o go-template='{{ printf "%s\n" (index .data "password" | ba...
```

Войдите с помощью token. Теперь dashboard доступен вам как администратору.

Далее вы сможете:

- Настроить OIDC для аутентификации вместо tokens.
- Создавать user tenants и предоставлять пользователям доступ к ним через tokens или OIDC.

5.4 Доступ к метрикам в Grafana

Используйте `grafana.example.org` для доступа к системному мониторингу, где `example.org` — ваш домен, указанный для `tenant-root`. В этом примере `grafana.example.org` находится по адресу `192.168.100.200`.

login: `admin`

запросите password:

```
kubectl get secret -n tenant-root grafana-admin-password -o go-template='{{ printf "%s\n" (index .data "pas...
```

Следующие шаги

- [Настройте OIDC.](#)
- [Создайте user tenant.](#)

Собственная платформа (BYOP)

<https://cozystack.ru/docs/v1.5/install/cozystack/kubernetes-distribution/>

Обзор

Cozystack можно использовать в режиме BYOP (Build Your Own Platform) — примерно так же, как дистрибутивы Linux позволяют устанавливать только нужные пакеты. Вместо развертывания полной платформы со всеми компонентами вы выборочно устанавливаете из package repository Cozystack только то, что вам нужно.

Такой подход полезен, когда:

- У вас уже есть кластер Kubernetes, и вам нужны только отдельные компоненты (например, Postgres operator или monitoring).
- В вашем кластере уже настроены networking (CNI) и storage, и вы не хотите, чтобы Cozystack управлял ими.
- Вам нужен полный контроль над тем, какие компоненты установлены и как они настроены.

Рабочий процесс опирается на два ресурса Kubernetes, управляемых Cozystack Operator:

- PackageSource — описывает package repository и доступные variants для каждого package.
- Package — объявляет, что конкретный package должен быть установлен в выбранном variant, опционально с пользовательскими values.

CLI-инструмент `cozypkg` предоставляет удобный интерфейс для работы с этими ресурсами: просмотра доступных packages, разрешения зависимостей и интерактивной установки packages.

1. Установка Cozystack Operator

Установите Cozystack operator с помощью Helm из OCI registry:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
  --version X.Y.Z \
  --namespace cozy-system \
  --create-namespace
```

Замените `X.Y.Z` на нужную версию Cozystack. Доступные версии можно найти на [странице релизов Cozystack](#).

Если установка выполняется на Kubernetes-дистрибутив без Talos (k3s, kubeadm, RKE2 и т. д.), укажите variant operator:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--set cozystackOperator.variant=generic \
--set cozystack.apiServerHost=<YOUR_API_SERVER_IP> \
--set cozystack.apiServerPort=6443
```

Operator устанавливает FluxCD (в all-in-one mode, работающем без CNI) и создает начальный PackageSource `cozystack.cozystack-platform`.

На этом этапе существует только один PackageSource:

```
kubectl get packagesource
```

| NAME | VARIANTS | READY | STATUS |
|------------------------------|------------------------------|-------|--------|
| cozystack.cozystack-platform | default,isp-full,isp-full... | True | ... |

2. Установка cozyrpg

Установите CLI-инструмент `cozyrpg` с помощью Homebrew:

```
brew tap cozystack/tap
brew install cozyrpg
```

Готовые бинарные файлы для других платформ доступны на [странице релизов GitHub](#).

3. Установка Platform Package

Первый шаг — установить package `cozystack-platform` с variant `default`. Этот variant не устанавливает компоненты — он только регистрирует PackageSources для всех packages, доступных в репозитории Cozystack.

```
cozyrpg add cozystack.cozystack-platform
```

Инструмент предложит выбрать variant. Выберите `default`:

```
PackageSource: cozystack.cozystack-platform
Available variants:
1. default
2. isp-full
3. isp-full-generic
4. isp-hosted
Select variant (1-4): 1
```

После установки platform package станут доступны все остальные PackageSources:

```
cozypkg list
```

| NAME | VARIANTS | READY | STATUS |
|------------------------------|------------------------------|-------|--------|
| cozystack.cert-manager | default | True | ... |
| cozystack.cozystack-platform | default,isp-full,isp-full... | True | ... |
| cozystack.ingress-nginx | default | True | ... |
| cozystack.linstor | default | True | ... |
| cozystack.metallb | default | True | ... |
| cozystack.monitoring | default | True | ... |
| cozystack.networking | noop,cilium,cilium-kilo,... | True | ... |
| cozystack.postgres-operator | default | True | ... |
| ... | | | |

4. Установка packages

Используйте `cozypkg add`, чтобы установить любой доступный package. Инструмент автоматически разрешает зависимости и предлагает выбрать variant для каждого package, который нужно установить.

```
cozypkg add <package-name>
```

Например, при установке package, зависящего от `networking`, `cozypkg` обнаружит зависимость, покажет уже установленные packages и предложит выбрать variant для каждой отсутствующей зависимости.

Сетевые variants

Package `cozystack.networking` имеет несколько variants для разных окружений:

| Variant | Описание |
|------------------------|--|
| noop | Ничего не устанавливает. Используйте, если <code>networking</code> уже настроен в кластере (например, существующий CNI). |
| cilium | Cilium CNI для кластеров Talos Linux. |
| cilium-generic | Cilium CNI для generic-дистрибутивов Kubernetes (k3s, kubeadm, RKE2). |
| kubeovn-cilium | Cilium + KubeOVN для Talos Linux. Требуется для полноценной виртуализации (live migration). |
| kubeovn-cilium-generic | Cilium + KubeOVN для generic-дистрибутивов Kubernetes. |
| cilium-kilo | Cilium + Kilo для cluster mesh на базе WireGuard. |

Если в вашем кластере уже настроен CNI-плагин, выберите `noop`. Поскольку `networking` является зависимостью большинства других `packages`, `variant noop` удовлетворяет зависимость, ничего не устанавливая.

Просмотр установленных `packages`

Чтобы увидеть, какие `packages` сейчас установлены и какие у них `variants`:

```
cozypkg list --installed
```

| NAME | VARIANT | READY | STATUS |
|------------------------------|---------|-------|--------|
| cozystack.cozystack-platform | default | True | ... |
| cozystack.networking | noop | True | ... |
| cozystack.cert-manager | default | True | ... |

5. Переопределение `values` компонентов

Каждый `package` состоит из одного или нескольких компонентов (Helm charts). `Values` для конкретных компонентов можно переопределить, напрямую отредактировав ресурс `Package`.

Спец `Package` поддерживает `map components`, где можно указать `values` для каждого компонента:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.metallb
spec:
  variant: default
  components:
    metallb:
      values:
        metallb:
          frrk8s:
            enabled: true
```

Примените ресурс:

```
kubectl apply -f metallb-package.yaml
```

Доступные `values` для компонента смотрите в соответствующем `values.yaml` в [репозитории Cozystack](#).

Также можно включать или отключать отдельные компоненты внутри `package`:

```
spec:
  components:
    some-component:
      enabled: false
```

6. Удаление packages

Чтобы удалить установленный package:

```
cozypkg del <package-name>
```

Инструмент проверяет обратные зависимости: если другие установленные packages зависят от удаляемого, он перечислит их и запросит подтверждение перед удалением всех затронутых packages.

Следующие шаги

- Узнайте о [variants Cozystack](#) и о том, как они определяют состав packages.
 - Подробности о переопределении параметров компонентов см. в [справочнике Components](#).
 - Для установки полной платформы см. [руководство по установке Platform](#).
-

Развертывание кластера Cozystack в облаках и у хостинг-провайдеров

<https://cozystack.ru/docs/v1.5/install/providers/>

В этом разделе собраны руководства по развертыванию кластеров Cozystack у конкретных облачных и хостинг-провайдеров. Они объясняют все шаги и детали процесса развертывания, включая:

- Особенности инфраструктуры и сети провайдера
- Установку Talos Linux способом, подходящим для провайдера
- Настройку кластера Kubernetes, включая специфичные для провайдера параметры сети, хранилища и других ресурсов
- Установку Cozystack в Kubernetes-кластер с компонентами и конфигурациями, специфичными для провайдера

Как установить Cozystack в Oracle Cloud Infrastructure

Как установить Cozystack в Oracle Cloud Infrastructure

Как установить Cozystack в Hetzner

Как установить Cozystack в Hetzner

Установка Cozystack в Servers.com

Установка Cozystack в инфраструктуре Servers.com.

Как установить Cozystack в Oracle Cloud Infrastructure

<https://cozystack.ru/docs/v1.5/install/providers/oracle-cloud/>

Введение

В этом руководстве описано, как установить Talos в Oracle Cloud Infrastructure и развернуть кластер Kubernetes, готовый для Cozystack. После выполнения руководства можно перейти к [установке самого Cozystack](#).

[!IMPORTANT]

Это руководство создано для поддержки развертывания development-кластеров командой Cozystack. Если при прохождении руководства возникнут проблемы, создайте [issue](#) в [cozystack/website](#) или поделитесь опытом в [сообществе Cozystack](#).

1. Загрузка образа Talos в Oracle Cloud

Первый шаг — сделать установочный образ Talos Linux доступным в Oracle Cloud как custom image.

1. Скачайте архив образа Talos Linux для Cozystack v1.5.0 со [страницы релиза](#) и распакуйте его:

```
""bash
wget https://github.com/cozystack/cozystack/releases/download/v1.5.0/metal-amd64.raw.xz
xz -d metal-amd64.raw.xz
""
```

В результате вы получите файл `metal-amd64.raw`, который затем можно загрузить в OCI.

2. Следуйте документации OCI, чтобы [загрузить образ в bucket в OCI Object Storage](#).
3. Затем следуйте документации, чтобы [импортировать этот образ как custom image](#).

Используйте следующие настройки:

- Image type: QCOW2
- Launch mode: Paravirtualized mode

4. Затем получите [OCID](#) образа и сохраните его для следующих шагов.

2. Создание инфраструктуры

Цель этого шага — подготовить инфраструктуру согласно [требованиям к кластеру Cozystack](#).

Это можно сделать вручную через Oracle Cloud dashboard или с помощью Terraform.

2.1 Подготовка конфигурации Terraform

Если вы используете Terraform, первый шаг — подготовить конфигурацию.

См. [полный пример конфигурации Terraform](#) для развертывания нескольких узлов Talos в Oracle Cloud Infrastructure.

[!NOTE]

Ниже приведен сокращенный пример конфигурации Terraform, создающей три виртуальные машины со следующими private IP-адресами:

* 192.168.1.11

* 192.168.1.12

* 192.168.1.13

Эти VM также будут иметь VLAN-интерфейс с подсетью 192.168.100.0/24, используемой для внутреннего взаимодействия кластера.

Обратите внимание на часть, которая ссылается на OCID образа Talos с предыдущего шага:

```
source_details {
  source_type = "image"
  source_id   = var.talos_image_id
}
```

Полный пример конфигурации:

```

terraform {
  backend "s3" {}
  required_providers {
    oci = {
      source = "oracle/oci"
      version = ">= 4.80.0"
    }
  }
}

resource "oci_core_vcn" "cozy_dev1" {
  display_name = "cozy-dev1"
  cidr_blocks = ["192.168.1.0/24"]
  compartment_id = var.compartment_id
}

resource "oci_core_internet_gateway" "cozy_dev1" {
  display_name = "cozy-dev1-igw"
  compartment_id = var.compartment_id
  vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_subnet" "test_subnet" {
  display_name = "cozy-dev1-subnet"
  cidr_block = "192.168.1.0/24"
  compartment_id = var.compartment_id
  vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_network_security_group" "cozy_dev1_allow_all" {
  display_name = "cozy-dev1-allow-all"
  compartment_id = var.compartment_id
  vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_network_security_group_security_rule" "allow_all_ingress" {
  network_security_group_id = oci_core_network_security_group.cozy_dev1_allow_all.id
  direction = "INGRESS"
  protocol = "all"
  source = "0.0.0.0/0"
  source_type = "CIDR_BLOCK"
}

resource "oci_core_network_security_group_security_rule" "allow_all_egress" {
  network_security_group_id = oci_core_network_security_group.cozy_dev1_allow_all.id
  direction = "EGRESS"
  protocol = "all"
  destination = "0.0.0.0/0"
  destination_type = "CIDR_BLOCK"
}

resource "oci_core_vlan" "cozy_dev1_vlan" {
  display_name = "cozy-dev1-vlan"
  compartment_id = var.compartment_id
  vcn_id = oci_core_vcn.cozy_dev1.id
... (82 lines truncated)

```

2.2 Применение конфигурации

Когда конфигурация будет готова, аутентифицируйтесь в OCI и примените ее с Terraform:

```
oci session authenticate --region us-ashburn-1 --profile-name=cozy
terraform init
terraform apply -var="compartment_id=..." -var="availability_domain=..." -var="talos_image_id=..."
```

В результате этих команд виртуальные машины будут развернуты и настроены.

Сохраните public IP-адреса, назначенные ВМ, для следующего шага. В этом примере адреса такие:

- 1.2.3.4
- 1.2.3.5
- 1.2.3.6

3. Настройка Talos и инициализация кластера Kubernetes

Следующий шаг — применить конфигурации и установить Talos Linux. Это можно сделать несколькими способами.

В этом руководстве используется [Talm](#) — CLI-инструмент для декларативного управления Talos Linux. В Talm есть шаблоны конфигурации, специально подготовленные для Cozystack.

Если Talm не установлен, [скачайте последний binary](#) для вашей ОС и архитектуры. Сделайте его исполняемым и сохраните в `/usr/local/bin/talm`:

```
# выберите нужную архитектуру из release artifacts
curl -L https://github.com/cozystack/talm/releases/download/v0.3.4/talm-linux-amd64 -o /usr/local/bin/talm
chmod +x /usr/local/bin/talm
```

3.1 Подготовка конфигурации Talm

1. Создайте каталог для конфигурационных файлов нового кластера:

```
```bash
mkdir cozy-dev1
cd cozy-dev1
```
```

2. Инициализируйте конфигурацию Talm для Cozystack:

```
```bash
talm init --repo https://github.com/cozystack/talm.git --branch main
```
```

3. Сгенерируйте шаблон конфигурации для каждого узла, указав IP-адрес узла:

```
```bash
Используйте public IP узла, назначенный OCI
talm template \
--nodes 1.2.3.4 \
--endpoints 1.2.3.4 \
```

---

```
--template templates/controlplane.yaml \
```

```
--insecure
```

```
'''
```

Повторите то же для каждого узла, используя его public IP:

```
```bash
```

```
talm template --nodes 1.2.3.5 --endpoints 1.2.3.5 --template templates/controlplane.yaml --insecure
```

```
talm template --nodes 1.2.3.6 --endpoints 1.2.3.6 --template templates/controlplane.yaml --insecure
```

```
'''
```

> [!TIP]

> Использование `templates/controlplane.yaml` означает, что узел будет работать одновременно как control plane и worker. Три совмещенных узла — минимально рекомендуемая конфигурация для отказоустойчивого кластера Cozystack.

> Параметр `--insecure (-i)` нужен потому, что Talm должен получить конфигурационные данные с узла, который еще не инициализирован. После инициализации и выполнения `talm apply` . этот параметр больше не потребуется.

> Public IP узла должен быть указан и в параметре `--nodes (-n)`, и в `--endpoints (-e)`. Подробнее о конфигурации узлов Talos и endpoints см. в [документации Talos](#).

4. При необходимости отредактируйте конфигурационный файл узла.

- Обновите `hostname` на нужное имя.

- Добавьте конфигурацию private interface в раздел `machine.network.interfaces` и перенесите `vip` в эту конфигурацию. Эта часть конфигурации не генерируется автоматически, поэтому значения нужно заполнить вручную:

- `interface`: берется из “Discovered interfaces” путем сопоставления параметров private interface.

- `addresses`: используйте адрес, указанный для Layer 2 (L2).

Пример:

```
```yaml
```

```
machine:
```

```
network:
```

```
interfaces:
```

- interface: eth0

```
addresses:
```

- 1.2.3.4/29

```
routes:
```

- network: 0.0.0.0/0

```
gateway: 1.2.3.1
```

- interface: eth1

```
addresses:
```

---

- 192.168.100.11/24

`vip:`

`ip: 192.168.100.10`

...

После этих шагов конфигурационные файлы узлов готовы к применению.

## 3.2 Инициализация Talos и запуск кластера Kubernetes

Следующий этап — инициализировать узлы Talos и выполнить bootstrap кластера Kubernetes.

1. Выполните `talm apply` для всех узлов, чтобы применить конфигурации:

```
```bash
```

```
talm apply . --insecure
```

```
```
```

Узлы перезагрузятся, и Talos будет установлен на диск. Параметр `--insecure (-i)` нужен при первом запуске `talm apply` на каждом узле.

2. Выполните `talm bootstrap` на первом узле кластера. Например:

```
```bash
```

```
talm bootstrap -n 1.2.3.4 -e 1.2.3.4
```

```
```
```

3. Получите `kubeconfig` с любого узла control plane с помощью Talos. В этом примере все три узла являются control-plane узлами:

```
```bash
```

```
talm kubeconfig .
```

```
```
```

4. Отредактируйте `kubeconfig`, указав IP-адрес server как один из узлов control plane, например:

```
```yaml
```

```
server: https://1.2.3.4:6443
```

```
```
```

5. Экспортируйте переменную `KUBECONFIG`, чтобы использовать `kubeconfig`, и проверьте подключение к кластеру:

```
```bash
```

```
export KUBECONFIG=${PWD}/kubeconfig
```

```
kubectl get nodes
```

```
```
```

Вы должны увидеть, что узлы доступны и находятся в состоянии `NotReady`, что ожидаемо на этом этапе:

---

```
```text
```

```
NAME STATUS ROLES AGE VERSION
```

```
cozy-dev1-node0 NotReady master 2m v1.24.4
```

```
cozy-dev1-node1 NotReady master 2m v1.24.4
```

```
cozy-dev1-node2 NotReady master 2m v1.24.4
```

```
```
```

Теперь у вас есть кластер Kubernetes, подготовленный к установке Cozystack. Чтобы завершить установку, перейдите к следующему шагу: [Установка Cozystack](#).

---



# Как установить Cozystack в Hetzner

<https://cozystack.ru/docs/v1.5/install/providers/hetzner/>

Это руководство поможет установить Cozystack на выделенные серверы [Hetzner](#). Процесс состоит из нескольких шагов: подготовки инфраструктуры, установки Talos Linux, настройки cloud-init и инициализации кластера.

## Подготовка инфраструктуры и сети

Установка в Hetzner включает общие [требования к оборудованию](#) с несколькими дополнениями.

### Варианты сети

Есть два варианта сетевой связности между узлами Cozystack в кластере:

- Создание подсети с помощью vSwitch. Этот вариант рекомендуется для production-сред. Для этого варианта выделенные серверы должны быть развернуты через [Hetzner robot](#). Также Hetzner требует использовать собственный load balancer RobotLB вместо стандартного для Cozystack MetalLB. Cozystack включает RobotLB как optional component начиная с релиза v0.35.0.
- Использование только public IP выделенных серверов. Этот вариант подходит для proof-of-concept установки, но не рекомендуется для production.

### Настройка подсети с vSwitch

Выполните следующие шаги, чтобы подготовить серверы к установке Cozystack:

1. Выполните сетевые настройки в Hetzner (только для варианта vSwitch subnet).
2. Выполните шаги из [раздела Prerequisites](#) в README RobotLB:
  - Создайте [vSwitch](#).
  - Используйте его, чтобы назначить IP вашим выделенным серверам в Hetzner.
  - Создайте подсеть, чтобы [подключить выделенные серверы](#).
3. Обратите внимание, что RobotLB не нужно развертывать вручную. Вместо этого вы настроите Cozystack на установку RobotLB как optional component на шаге "Установка Cozystack" в этом руководстве.

### Отключение Secure Boot

Убедитесь, что Secure Boot отключен. The Talos installation procedure used in this guide (rescue-mode installimage writing a standard EFI/MBR layout) requires Secure Boot disabled. Talos does support UEFI Secure Boot via its UKI / SecureBoot installation flow, but that path is not covered here. If your server is configured to use Secure Boot, disable it in BIOS before continuing — otherwise it will block the server from booting after Talos installation.

Проверьте это следующей командой:

```
mokutil --sb-state
SecureBoot disabled
Platform is in Setup Mode
```

В остальной части руководства будем считать, что используется следующая сетевая конфигурация:

- Облачная сеть Hetzner — 10.0.0.0/16, имя network-1.
- vSwitch subnet с выделенными серверами — 10.0.1.0/24
- VLAN ID vSwitch — 4000
- Есть три выделенных сервера со следующими public и private IP:
  - node1, public IP 12.34.56.101, IP в vSwitch subnet 10.0.1.101
  - node2, public IP 12.34.56.102, IP в vSwitch subnet 10.0.1.102
  - node3, public IP 12.34.56.103, IP в vSwitch subnet 10.0.1.103

## 1. Установка Talos Linux

Первый этап развёртывания Cozystack — установка Talos Linux на выделенные серверы. Talos — дистрибутив Linux, созданный для максимально безопасного и эффективного запуска Kubernetes. Чтобы узнать, почему Cozystack использует Talos как основу кластера, прочитайте [Talos Linux в Cozystack](#).

### 1.1 Установка boot-to-talos в Rescue Mode

Talos будет загружен из rescue system Hetzner с помощью утилиты [boot-to-talos](#). Позже, при применении конфигурации Talm, Talos будет установлен на диск. Выполните эти шаги на каждом выделенном сервере:

1. Переведите сервер в rescue mode и войдите на него по SSH.
2. Определите диск, который позже будет использоваться для Talos (например, /dev/nvme0n1).
3. Скачайте и установите [boot-to-talos](#):

```
curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh | bash
```

После этого бинарный файл `boot-to-talos` должен быть доступен в PATH:

```
boot-to-talos --version
```

### 1.2. Установка Talos Linux с boot-to-talos

Запустите installer:

```
boot-to-talos
```

При запросе:

1. Выберите режим 1. `boot`.
2. Подтвердите или измените образ Talos installer. Значение по умолчанию указывает на образ Talos от Cozystack.
3. Укажите сетевые настройки (имя интерфейса, IP-адрес, netmask, gateway), соответствующие подготовленной конфигурации.
4. При необходимости настройте serial console, если используете ее для удаленного доступа.

Утилита скачает образ Talos installer, извлечет kernel и initramfs и загрузит узел в Talos Linux (сначала в maintenance mode).

### 1.3. Загрузка в Talos Linux

После завершения `boot-to-talos` сервер автоматически перезагрузится в Talos Linux в maintenance mode. Повторите ту же процедуру для всех выделенных серверов в кластере. Когда все узлы загрузятся в Talos Linux, можно переходить к следующему этапу.

## 2. Установка кластера Kubernetes

Теперь, когда Talos загружен в maintenance mode, он должен получить конфигурацию и настроить кластер. В Cozystack есть [несколько вариантов](#) создания и применения конфигурации Talos. В этом руководстве используется [Talm](#) — собственный инструмент Cozystack для управления конфигурацией Talos.

Эта часть руководства основана на общем [руководстве по Talm](#), но содержит инструкции и примеры, специфичные для Hetzner.

### 2.1. Подготовка конфигурации узлов с Talm

Если Talm еще не установлен, начните с установки последней версии для вашей ОС:

```
curl -sSL https://github.com/cozystack/talm/raw/main/hack/install.sh | bash
```

Создайте каталог для конфигурации кластера и инициализируйте в нем проект Talm. Обратите внимание, что в Talm есть встроенный preset для Cozystack, который используется через `--preset cozystack`:

```
mkdir hetzner-cluster
cd hetzner-cluster
talm init --preset cozystack
```

Теперь в каталоге `hetzner-cluster` создан набор файлов. Подробнее о роли каждого файла см. в [руководстве по Talm](#).

Отредактируйте `values.yaml`, изменив следующие значения:

- В списке `advertisedSubnets` должна быть vSwitch subnet.

- `endpoint` и `floatingIP` должны использовать неназначенный IP из этой подсети. Этот IP будет использоваться для доступа к кластеру через `talosctl` и `kubectl`.
- `podSubnets` и `serviceSubnets` должны использовать другие подсети из облачной сети Hetzner, которые не пересекаются друг с другом и с `vSwitch subnet`.

```
endpoint: 10.0.1.100
clusterDomain: cozy.local
floatingIP указывает на primary etcd node
floatingIP: 10.0.1.100
image: ghcr.io/cozystack/talos:v1.7.5
podSubnets:
- 10.244.0.0/16
serviceSubnets:
- 10.96.0.0/16
advertisedSubnets:
vSwitch subnet
- 10.0.1.0/24
oidcIssuerUrl: ""
certSANS: []
```

Создайте конфигурационные файлы узлов из шаблонов и `values`:

```
talm apply-templates
```

В этом руководстве предполагается, что у вас только три выделенных сервера, поэтому все они должны быть узлами control plane. Используйте `templates/controlplane.yaml` для создания их конфигурации. Если у вас больше узлов, используйте `templates/worker.yaml` для создания worker configs:

```
talm gen-config --name node1 --ip 12.34.56.101 --template templates/controlplane.yaml
talm gen-config --name node2 --ip 12.34.56.102 --template templates/controlplane.yaml
talm gen-config --name node3 --ip 12.34.56.103 --template templates/controlplane.yaml
```

Отредактируйте конфигурационный файл каждого узла, добавив VLAN configuration. Используйте следующий diff как пример и учитывайте, что для каждого узла должен использоваться его IP в `vSwitch subnet`:

```
diff example for node1 configuration
machine:
 network:
 interfaces:
 - interface: eth0
 vlans:
 - vlanId: 4000
 addresses:
 - 10.0.1.101/24
```

## 2.2. Применение конфигурации узлов

Когда конфигурационные файлы будут готовы, примените конфигурацию к каждому узлу:

```
talosctl apply-config -n 12.34.56.101 -f node1.yaml -i
talosctl apply-config -n 12.34.56.102 -f node2.yaml -i
talosctl apply-config -n 12.34.56.103 -f node3.yaml -i
```

Эта команда инициализирует узлы и настраивает аутентифицированное соединение, поэтому дальше `-i` (`--insecure`) не потребуется. Если команда выполнена успешно, она вернет IP узла:

```
$ talosctl apply-config -n 12.34.56.101 -f node1.yaml -i
12.34.56.101
```

Дождитесь, пока все узлы перезагрузятся, и переходите к следующему шагу. Когда узлы будут готовы, они начнут отвечать по порту `50000`; это признак того, что узел завершил настройку Talos и перезагрузился.

Если нужно автоматизировать проверку готовности узлов, используйте такой пример:

```
timeout 60 sh -c 'until talosctl -n 12.34.56.101 version; do sleep 1; done'
```

Инициализируйте кластер Kubernetes с одного из узлов control plane:

```
talosctl bootstrap -n 12.34.56.101
```

Сгенерируйте административный `kubeconfig` для доступа к кластеру через тот же узел control plane:

```
talosctl kubeconfig -n 12.34.56.101 .
```

Измените URL server в `kubeconfig` на public IP узла control plane или на floating IP (если настроен доступ к нему):

```
apiVersion: v1
clusters:
- cluster:
 server: https://12.34.56.101:6443
```

Затем настройте переменную `KUBECONFIG` или другие инструменты, чтобы сделать эту конфигурацию доступной клиенту `kubectl`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get nodes
```

Пример вывода:

| NAME  | STATUS | ROLES         | AGE | VERSION |
|-------|--------|---------------|-----|---------|
| node1 | Ready  | control-plane | 5m  | v1.30.2 |
| node2 | Ready  | control-plane | 5m  | v1.30.2 |
| node3 | Ready  | control-plane | 5m  | v1.30.2 |

На этом этапе у вас есть выделенные серверы с Talos Linux и развернутый на них кластер Kubernetes. У вас также есть административный `kubeconfig`, который будет использоваться для доступа к кластеру через `kubectl` и установки Cozystack.

## 3. Установка Cozystack

Финальный этап развертывания кластера Cozystack в Hetzner — установка Cozystack в подготовленный кластер Kubernetes.

### 3.1. Запуск Cozystack Installer

Установите Cozystack operator:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
 --version 1.5.0 \
 --namespace cozy-system \
 --create-namespace
```

В примере installer закреплен на Cozystack v1.5.0. Для более нового patch-релиза в той же minor-серии v1.5 проверьте актуальную версию на [странице релизов](#).

Создайте файл Platform Package, `cozystack-platform.yaml`. Обратите внимание, что этот файл повторно использует подсети для pods и services, которые использовались в `values.yaml` перед созданием конфигурации Talos с Talm. Также обратите внимание, как стандартный load balancer `cozystack.metallb` заменяется на `cozystack.hetzner-robotlb` с помощью списков `disabledPackages` и `enabledPackages`.

Замените `example.org` на маршрутизируемое fully-qualified domain name (FQDN), которое вы будете использовать для платформы. Если у вас нет собственного домена, можно использовать сервис [nip.io](#) с записью через дефис.

```

apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full
 components:
 platform:
 values:
 bundles:
 disabledPackages:
 - cozystack.metallb
 enabledPackages:
 - cozystack.hetzner-robotlb
 publishing:
 host: example.org
 apiServerEndpoint: api.example.org
 exposedServices:
 - dashboard
 - api
 networking:
 ## podSubnets из конфигурации узлов
 podCIDR: 10.244.0.0/16
 podGateway: 10.244.0.1
 ## serviceSubnets из конфигурации узлов
 serviceCIDR: 10.96.0.0/16

```

Примените Platform Package:

```
kubectl apply -f cozystack-platform.yaml
```

Operator запустит установку, которая займет некоторое время. При необходимости можно отслеживать логи installer:

```
kubectl logs -n cozy-system -l app.kubernetes.io/name=cozy-installer -f
```

Проверьте состояние установки:

```
kubectl get packages
```

Когда установка завершится, все сервисы перейдут в состояние **READY: True**:

| NAME              | RELEASE-NAME         | VERSION | READY | STATUS                           |
|-------------------|----------------------|---------|-------|----------------------------------|
| cozy-cert-manager | cert-manager         | 4m1s    | True  | Release reconciliation succeeded |
| cozy-cert-manager | cert-manager-issuers | 4m1s    | True  | Release reconciliation succeeded |
| cozy-cilium       | cilium               | 4m1s    | True  | Release reconciliation succeeded |
| ...               |                      |         |       |                                  |

## 3.2 Создание Load Balancer с RobotLB

Hetzner требует использовать собственный RobotLB вместо стандартного MetalLB в Cozystack. RobotLB управляет load balancer в инфраструктуре Hetzner через API.

1. Создайте Hetzner API token для RobotLB. Перейдите в консоль Hetzner, откройте Security и создайте token с разрешениями [Read](#) и [Write](#).
2. Передайте token в RobotLB, чтобы создать load balancer в Hetzner. Используйте Hetzner API token, чтобы создать Kubernetes secret в Cozystack.

Если используется private network (vSwitch), укажите имя сети:

```
kubectl create secret generic -n cozy-system hetzner-robot \
 --from-literal=ROBOTLB_HTTP_TOKEN=<your-token> \
 --from-literal=ROBOTLB_DEFAULT_NETWORK=network-1
```

Если используются только public IP (без vSwitch), не указывайте `ROBOTLB_DEFAULT_NETWORK`:

```
kubectl create secret generic -n cozy-system hetzner-robot \
 --from-literal=ROBOTLB_HTTP_TOKEN=<your-token>
```

После получения token сервис RobotLB в Cozystack создаст load balancer в Hetzner.

### 3.3 Настройка хранилища с LINSTOR

Настройка LINSTOR в Hetzner не отличается от других инфраструктурных конфигураций. Следуйте [руководству по настройке хранилища](#) из tutorial по Cozystack.

### 3.4. Запуск сервисов в Root Tenant

Настройте базовые сервисы (`etcd`, `monitoring` и `ingress`) в root tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"bundles":{"enabledPa...
```

## Примечания и troubleshooting

[!] Если возникают проблемы с загрузкой Talos Linux на узле, это может быть связано с параметрами serial console в GRUB. По умолчанию Hetzner добавляет `console=tty1 console=ttyS0`. Попробуйте перезагрузиться в rescue mode и удалить эти параметры из конфигурации GRUB на третьем разделе системного диска (обычно `$DISK1`).



# Установка Cozystack в Servers.com

<https://cozystack.ru/docs/v1.5/install/providers/servers-com/>

## Перед установкой

### 1. Сеть

1. Настройте L2 Network

1. Перейдите в Networks > L2 Segment и нажмите Add Segment.

!L2 Segments

!L2 Segments

!L2 Segments

Сначала выберите Private, выберите регион, добавьте серверы, задайте имя и сохраните.

2. Установите тип Native. Сделайте то же самое для Public.

!Type

### 2. Доступ

1. Создайте SSH-ключи для доступа к серверу.

2. Перейдите в Identity and Access > SSH and Keys.

!SSH

3. Создайте новые ключи или добавьте свои.

!SSH

!SSH

## Настройка ОС

### 1. Операционная система и доступ

[!] В rescue mode доступна только публичная сеть; частная L2 network недоступна. Для установки Talos используйте обычную ОС (например, Ubuntu), а не rescue mode.

1. В панели управления Servers.com установите Ubuntu на сервер (например, через Dedicated Servers > Server Details > OS install) и убедитесь, что выбран ваш SSH-ключ.

2. После завершения установки подключитесь по SSH, используя внешний IP сервера (Details > Public IP).

!Public IP

---

## 2. Установка Talos с boot-to-talos

Talos будет загружен из установленной Ubuntu с помощью утилиты `[boot-to-talos]`(<https://github.com/cozystack/boot-to-talos>). Позже, при применении конфигурации Talm, Talos будет установлен на диск. Выполните эти шаги на каждом сервере.

1. Проверьте информацию о блочных устройствах, чтобы найти диск, который позже будет использоваться для Talos (например, `/dev/sda`).

```
lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 259:4 0 476.9G 0 disk
sdb 259:0 0 476.9G 0 disk
```

2. Скачайте и установите `boot-to-talos`:

```
curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh | sudo sh
```

После установки проверьте, что бинарный файл доступен:

```
boot-to-talos -h
```

3. Запустите installer:

```
sudo boot-to-talos
```

При запросе:

- Выберите режим 1. `boot`.
- Подтвердите или измените образ Talos installer (стандартный образ Cozystack подходит).
- Укажите сетевые настройки, соответствующие публичному интерфейсу (`bond0`) и default gateway.

Утилита скачает образ Talos installer и загрузит узел в Talos Linux (с помощью механизма kexes), не изменяя диски.

Для полностью автоматизированной установки можно использовать non-interactive mode:

```
sudo boot-to-talos -yes
```

## 3. Загрузка в Talos

После завершения `boot-to-talos` сервер автоматически перезагрузится в Talos Linux в maintenance mode. Повторите ту же процедуру для всех серверов, затем переходите к их настройке с Talm.

---

## Конфигурация Talos

Используйте `Talm`, чтобы применить config и установить Talos Linux на диск.

1. Скачайте последний binary Talm и сохраните его в `/usr/local/bin/talm`

---

2. Сделайте его исполняемым:

```
chmod +x /usr/local/bin/talm
```

## Установка с Talm

1. Создайте каталог для нового кластера:

```
mkdir -p cozystack-cluster
cd cozystack-cluster
```

2. Выполните следующую команду, чтобы инициализировать Talm для Cozystack:

```
talm init --preset cozystack --name mycluster
```

После инициализации сгенерируйте шаблон конфигурации командой:

```
talm -n 1.2.3.4 -e 1.2.3.4 template -t templates/controlplane.yaml -i > nodes/nodeN.yaml
```

3. При необходимости отредактируйте конфигурационный файл узла:

1. Обновите `hostname` на нужное имя.

```
``yaml
machine:
network:
hostname: node1
...
```

2. Обновите `nameservers`, указав публичные серверы, потому что внутренний DNS `servers.com` недоступен из частной сети.

```
``yaml
machine:
network:
nameservers:
 - 8.8.8.8
 - 1.1.1.1
...
```

3. Добавьте конфигурацию `private interface` и перенесите `vip` в этот раздел. Этот раздел не генерируется автоматически:

- `interface` - берется из "Discovered interfaces" путем сопоставления MAC-адреса `private interface`, указанного в письме провайдера. (Из двух интерфейсов выберите тот, у которого есть `uplink`).
- `addresses` - используйте адрес, указанный для Layer 2 (L2).

```
``yaml
```

---

```
machine:
network:
interfaces:
 - interface: bond0
addresses:
 - 1.2.3.4/29
routes:
 - network: 0.0.0.0/0
gateway: 1.2.3.1
bond:
interfaces:
 - enp1s0f1
 - enp3s0f1
mode: 802.3ad
xmitHashPolicy: layer3+4
lacpRate: slow
miimon: 100
 - interface: bond1
addresses:
 - 192.168.102.11/23
bond:
interfaces:
 - enp1s0f0
 - enp3s0f0
mode: 802.3ad
xmitHashPolicy: layer3+4
lacpRate: slow
miimon: 100
vip:
ip: 192.168.102.10
'''
```

#### Шаги выполнения:

1. Выполните `taln apply -f nodeN.yml` для всех узлов, чтобы применить конфигурации. Узлы будут перезагружены, и Talos будет установлен на диск.
2. Убедитесь, что Talos установлен на диск, выполнив `taln get systemdisk -f nodeN.yml` для каждого узла. Вывод должен быть похож на:

---

| NODE    | NAMESPACE | TYPE       | ID          | VERSION | DISK |
|---------|-----------|------------|-------------|---------|------|
| 1.2.3.4 | runtime   | SystemDisk | system-disk | 1       | sda  |

Если вывод пустой, значит Talos все еще работает из RAM и еще не установлен на диск.

3. Выполните bootstrap-команду для первого узла кластера, например:

```
talctl bootstrap -f nodes/node1.yml
```

4. Получите `kubeconfig` с первого узла, например:

```
talctl kubeconfig -f nodes/node1.yml
```

5. Отредактируйте `kubeconfig`, указав IP-адрес одного из узлов control plane, например:

```
server: https://1.2.3.4:6443
```

6. Экспортируйте переменную для использования `kubeconfig` и проверьте подключение к Kubernetes:

```
export KUBECONFIG=${PWD}/kubeconfig
kubectl get nodes
```

Теперь для продолжения установки следуйте руководству Get Started, начиная с раздела **Установка Cozystack**.

---

# Автоматизированная установка с Ansible

<https://cozystack.ru/docs/v1.5/install/ansible/>

Ansible collection [cozystack.installer](#) автоматизирует весь pipeline разворачивания: подготовку ОС, bootstrap кластера k3s и установку Cozystack. Она подходит для разворачивания Cozystack на bare-metal серверах или VM со стандартным дистрибутивом Linux.

## Когда использовать Ansible

Рассмотрите этот подход, если:

- Вам нужно полностью автоматизированное, повторяемое разворачивание от чистой ОС до работающего Cozystack
- Вы разворачиваете платформу на generic Linux (Ubuntu, Debian, RHEL, Rocky, openSUSE), а не на Talos Linux
- Вы хотите управлять несколькими узлами через один inventory-файл

Шаги ручной установки без Ansible см. в руководстве [Generic Kubernetes](#).

## Предварительные требования

### Управляющая машина

- Python  $\geq 3.9$
- Ansible  $\geq 2.15$

### Целевые узлы

- Операционная система: Ubuntu/Debian, RHEL 8+/CentOS Stream 8+/Rocky/Alma или openSUSE/SLE
- Архитектура: amd64 или arm64
- SSH-доступ с passwordless sudo
- Требования к CPU, RAM и дискам см. в [требованиях к оборудованию](#)

## Установка

### 1. Установка Ansible collection

```
ansible-galaxy collection install git+https://github.com/cozystack/ansible-cozystack.git
```

Установите необходимые dependency collections. Файл `requirements.yml` не включен в packaged collection, поэтому скачайте его из репозитория:

```
curl --silent --location --output /tmp/requirements.yml \
 https://raw.githubusercontent.com/cozystack/ansible-cozystack/main/requirements.yml
ansible-galaxy collection install --requirements-file /tmp/requirements.yml
```

Это установит следующие зависимости:

- `ansible.posix`, `community.general`, `kubernetes.core` — из Ansible Galaxy
- `k3s.orchestration` — collection для развертывания k3s, устанавливается из Git

## 2. Создание inventory

Создайте файл `inventory.yml`. Внутренний (private) IP каждого узла должен использоваться как host key, потому что KubeOVN проверяет IP хостов через `NODE_IPS`. Public IP, если он отличается, указывается в `ansible_host`.

```
cluster:
 children:
 server:
 hosts:
 10.0.0.10:
 ansible_host: 203.0.113.10
 agent:
 hosts:
 10.0.0.11:
 ansible_host: 203.0.113.11
 10.0.0.12:
 ansible_host: 203.0.113.12
 vars:
 ansible_port: 22
 ansible_user: ubuntu

настройки k3s — доступные версии см. на https://github.com/k3s-io/k3s/releases
k3s_version: v1.35.0+k3s3
token: # ЗАМЕНИТЕ на надежный случайный secret
api_endpoint:
cluster_context: my-cluster

настройки Cozystack
cozystack_api_server_host:
cozystack_root_host:
cozystack_platform_variant:
cozystack_k3s_extra_args:
```

Замените `token` на надежный случайный secret. Этот token используется для подключения узлов k3s и дает полный доступ к кластеру. Сгенерируйте его командой `openssl rand -hex 32`.

Всегда явно закрепляйте `cozystack_chart_version`. Collection поставляется с версией по умолчанию, которая может не совпадать с релизом, который вы хотите развернуть. Укажите ее в inventory, чтобы избежать неожиданных обновлений:

```
cozystack_chart_version: "1.5.0"
```

---

Доступные версии см. в [релизах Cozystack](#).

### 3. Создание playbook

Создайте файл `site.yml`, который последовательно запускает подготовку ОС, развертывание k3s и установку Cozystack.

Репозиторий collection содержит примеры prepare playbooks для каждого поддерживаемого семейства ОС в каталоге [examples/](#). Скопируйте подходящий для вашей целевой ОС файл в каталог проекта, затем подключите его как локальный playbook.

Скопируйте `prepare-ubuntu.yml` из [examples/ubuntu/](#), затем создайте `site.yml`:

```
- name: Prepare nodes
 ansible.builtin.import_playbook: prepare-ubuntu.yml

- name: Deploy k3s cluster
 ansible.builtin.import_playbook: k3s.orchestration.site

- name: Install Cozystack
 ansible.builtin.import_playbook: cozystack.installer.site
```

### 4. Запуск playbook

```
ansible-playbook --inventory inventory.yml site.yml
```

Playbook автоматически выполняет следующие шаги:

1. Prepare nodes — устанавливает необходимые пакеты (`nfs-common`, `open-iscsi`, `multipath-tools`), настраивает `sysctl`, включает storage services
2. Deploy k3s — выполняет bootstrap кластера k3s с настройками, совместимыми с Cozystack (отключает встроенные Traefik, ServiceLB, kube-proxy, Flannel; задает `cluster-domain=cozy.local`)
3. Install Cozystack — устанавливает Helm и plugin helm-diff (используется для идиоматичных обновлений), развертывает chart `cozy-installer`, ожидает operator и CRD, затем создает Platform Package

## Справочник конфигурации

---

### Основные переменные

| Переменная                             | По умолчанию  | Описание                                       |
|----------------------------------------|---------------|------------------------------------------------|
| <code>cozystack_api_server_host</code> | (обязательно) | Внутренний IP узла control plane.              |
| <code>cozystack_chart_version</code>   | 1.5.0         | Версия Helm chart Cozystack. Закрепляйте явно. |



|                            |                  |                                                                                                                            |
|----------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------|
| cozystack_platform_variant | isp-full-generic | Platform variant: <code>default</code> , <code>isp-full</code> , <code>isp-hosted</code> , <code>isp-full-generic</code> . |
| cozystack_root_host        | ""               | Домен для сервисов Cozystack. Оставьте пустым, чтобы пропустить publishing configuration.                                  |

## Сеть

| Переменная                | По умолчанию  | Описание                                  |
|---------------------------|---------------|-------------------------------------------|
| cozystack_pod_cidr        | 10.42.0.0/16  | Диапазон Pod CIDR.                        |
| cozystack_pod_gateway     | 10.42.0.1     | Gateway pod-сети.                         |
| cozystack_svc_cidr        | 10.43.0.0/16  | Диапазон Service CIDR.                    |
| cozystack_join_cidr       | 100.64.0.0/16 | Join CIDR для межузлового взаимодействия. |
| cozystack_api_server_port | 6443          | Порт Kubernetes API server.               |

## Расширенные настройки

| Переменная                        | По умолчанию                                     | Описание                                                                            |
|-----------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------|
| cozystack_chart_ref               | oci://ghcr.io/cozystack/cozystack/cozy-installer | OCI reference для Helm chart.                                                       |
| cozystack_operator_variant        | generic                                          | Operator variant: <code>generic</code> , <code>talos</code> , <code>hosted</code> . |
| cozystack_namespace               | cozy-system                                      | Namespace для Cozystack                                                             |
| cozystack_release_name            | cozy-installer                                   |                                                                                     |
| cozystack_release_namespace       | kube-system                                      |                                                                                     |
| cozystack_kubeconfig              | /etc/rancher/k3s/k3s.yaml                        |                                                                                     |
| cozystack_create_platform_package | true                                             |                                                                                     |
| cozystack_helm_version            | 3.17.3                                           |                                                                                     |
| cozystack_helm_binary             | /usr/local/bin/helm                              |                                                                                     |
| cozystack_helm_diff_version       | 3.12.5                                           |                                                                                     |
| cozystack_operator_wait_timeout   | 300                                              |                                                                                     |

---

## Переменные prepare playbook

Примерные prepare playbooks (скопированные из каталога `examples/`) поддерживают дополнительные переменные:

| Переменная                            | По умолчанию       | Описание                                                                                                       |
|---------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------|
| <code>cozystack_flush_iptables</code> | <code>false</code> |                                                                                                                |
| <code>cozystack_k3s_extra_args</code> |                    | Дополнительные аргументы для k3s server (например, <code>--tls-san=&lt;PUBLIC_IP&gt;</code> для узлов за NAT). |

## Проверка

---

После завершения playbook проверьте развертывание с первого server-узла:

```
Проверить operator
kubectl get deployment cozystack-operator --namespace cozy-system

Проверить Platform Package
kubectl get packages.cozystack.io cozystack.cozystack-platform

Проверить все pods
kubectl get pods --all-namespaces
```

## Идемпотентность

---

Playbook идемпотентен: повторный запуск не будет повторно применять ресурсы, которые не изменились. Cozystack operator использует Helm для управления ресурсами, и Ansible collection использует `helm-diff`, чтобы видеть изменения перед их применением, аналогично `kubectl diff`.

---

---

## Руководства для особых случаев развертывания Cozystack

<https://cozystack.ru/docs/v1.5/install/how-to/>

### Как установить Talos на машину с одним диском

---

Как установить Talos на машину с одним диском, выделив место на системном диске под пользовательское хранилище

### Развертывание Kubernetes в публичной сети

---

### Как включить KubeSpan

---

Как включить KubeSpan.

### Как настроить network bonding (LACP)

---

Как настроить network bonding LACP (802.3ad) для агрегации каналов и резервирования

### Как включить Hugerpages

---

Как включить Hugerpages

---

# Как установить Talos на машину с одним диском

<https://cozystack.ru/docs/v1.5/install/how-to/single-disk/>

Стандартная конфигурация Talos предполагает, что у каждого узла есть основной и дополнительный диски, используемые соответственно для системного и пользовательского хранилища. Однако можно использовать один диск, выделив на нем место для пользовательского хранилища.

Эту конфигурацию нужно применить при первом `talosctl apply` или `talm apply` — том, который выполняется с флагом `-i` (`--insecure`). Применение изменений после инициализации не даст эффекта.

Для `talosctl` добавьте следующие строки в `patch.yaml`:

```

apiVersion: v1alpha1
kind: VolumeConfig
name: EPHEMERAL
provisioning:
 minSize: 70GiB

apiVersion: v1alpha1
kind: UserVolumeConfig
name: data-storage
provisioning:
 diskSelector:
 match:
 disk.transport == 'nvme'
 minSize: 400GiB
```

Для `talm` добавьте те же строки в конец конфигурационного файла первого узла, например `nodes/node1.yaml`.

Подробнее см. в документации Talos: <https://www.talos.dev/v1.13/talos-guides/configuration/disk-management/>.

После применения конфигурации очистите раздел `data-storage`:

```
kubectl -n kube-system debug -it --profile sysadmin --image=alpine node/node1

apk add util-linux

umount /dev/nvme0n1p6 # Раздел, выделенный под пользовательское хранилище
rm -rf /host/var/mnt/data-storage
wipefs -a /dev/nvme0n1p6
exit
```

После настройки хранилища добавьте новый раздел в LINSTOR:

```
linstor ps cdp zfs node1 nvme0n1p6 --pool-name data --storage-pool data1
```

Проверьте результат:

```
linstor sp l
```

Вывод будет похож на этот пример:

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | CanSnapshots | State | ...
|=====+=====+=====+=====+=====+=====+=====+=====+
| DfltDisklessStorPool | node1 | DISKLESS | | | | False | 0k | ...
| data | node1 | ZFS | data | 351.46 GiB | 476 GiB | True | 0k | ...
| data1 | node1 | ZFS | data | 378.93 GiB | 412 GiB | True | 0k | ...
+-----+-----+-----+-----+-----+-----+-----+-----+
```

---

## Развертывание Kubernetes в публичной сети

<https://cozystack.ru/docs/v1.5/install/how-to/public-ip/>

Кластер Kubernetes для Cozystack можно развернуть, используя только публичные сети:

- И управляющие, и worker-узлы имеют публичные IP-адреса.
- Worker-узлы подключаются к управляющим узлам через публичный Интернет, без частной внутренней сети или VPN.

Такая конфигурация не рекомендуется для production, но может использоваться для исследований и тестирования, когда ограничения хостинга не позволяют создать частную сеть.

Чтобы включить такую конфигурацию при развертывании с `talosctl`, добавьте следующие данные в конфигурационные файлы узлов:

```
cluster:
 controlPlane:
 endpoint: https://<MANAGEMENT_NODE_IP>:6443
```

Для `taln` добавьте те же строки в конец конфигурационного файла первого узла, например `nodes/node1.yaml`.

---

---

## Как включить KubeSpan

<https://cozystack.ru/docs/v1.5/install/how-to/kubespans/>

Talos Linux предоставляет full-mesh сеть WireGuard для вашего кластера ([Cozystack Documentation](#)).

Чтобы включить эту функциональность, настройте [KubeSpan](#) и [Cluster Discovery](#) в конфигурации Talos Linux ([Cozystack Documentation](#)):

```
machine:
 network:
 kubespans:
 enabled: true
 cluster:
 discovery:
 enabled: true
```

Поскольку KubeSpan инкапсулирует трафик в туннель WireGuard, для Kube-OVN также нужно настроить более низкое значение MTU ([Cozystack Documentation](#)).

Для этого добавьте следующее в компонент `networking` вашего Platform Package ([Cozystack Documentation](#)):

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 # ...
 components:
 networking:
 values:
 kube-ovn:
 mtu: 1222
```

# Как настроить network bonding (LACP)

<https://cozystack.ru/docs/v1.5/install/how-to/bonding/>

Network bonding позволяет объединить несколько физических сетевых интерфейсов в один логический интерфейс. Это увеличивает пропускную способность и обеспечивает резервирование каналов.

LACP (Link Aggregation Control Protocol, IEEE 802.3ad) — самый распространенный режим bonding, который динамически согласует агрегацию каналов с сетевым коммутатором.

## [!IMPORTANT]

LACP требует настройки и на сервере, и на сетевом коммутаторе. Убедитесь, что на коммутаторе настроен соответствующий LACP port-channel для портов сервера.

## Определение сетевых интерфейсов

После выполнения `talm template` сгенерированный конфигурационный файл узла будет содержать блок комментариев с обнаруженными сетевыми интерфейсами:

```
machine:
 network:
 # -- Discovered interfaces:
 # eno1:
 # hardwareAddr: aa:bb:cc:dd:ee:f0
 # busPath: 0000:02:00.0
 # driver: tg3
 # vendor: Broadcom Inc. and subsidiaries
 # product: NetXtreme BCM5719 Gigabit Ethernet PCIe
 # eno2:
 # hardwareAddr: aa:bb:cc:dd:ee:f1
 # busPath: 0000:02:00.1
 # driver: tg3
 # vendor: Broadcom Inc. and subsidiaries
 # product: NetXtreme BCM5719 Gigabit Ethernet PCIe
 # eth0:
 # hardwareAddr: aa:bb:cc:dd:ee:f2
 # busPath: 0000:04:00.0
 # driver: bnx2x
 # vendor: Broadcom Inc. and subsidiaries
 # product: NetXtreme II BCM57810 10 Gigabit Ethernet
 # eth1:
 # hardwareAddr: aa:bb:cc:dd:ee:f3
 # busPath: 0000:04:00.1
 # driver: bnx2x
 # vendor: Broadcom Inc. and subsidiaries
 # product: NetXtreme II BCM57810 10 Gigabit Ethernet
```

Выберите интерфейсы, которые хотите объединить в bond. Обычно это порты одной скорости, подключенные к одному коммутатору или стеку коммутаторов. Запишите значения `busPath` — они понадобятся далее.



## Настройка bonding

Отредактируйте сгенерированный конфигурационный файл узла (например, `nodes/node1.yaml`) и замените стандартный раздел `machine.network.interfaces` на конфигурацию `bond`:

```
machine:
 network:
 interfaces:
 - interface: bond0
 dhcp: false
 bond:
 mode: 802.3ad
 adSelect: bandwidth
 miimon: 100
 updelay: 200
 downdelay: 200
 minLinks: 1
 xmitHashPolicy: encap3+4
 deviceSelectors:
 - busPath: "0000:04:00.0"
 - busPath: "0000:04:00.1"
 addresses:
 - 192.168.100.11/24
 routes:
 - network: 0.0.0.0/0
 gateway: 192.168.100.1
```

## Описание параметров bond

| Параметр  | Значение  | Описание                                                                   |
|-----------|-----------|----------------------------------------------------------------------------|
| mode      | 802.3ad   | LACP — динамическая агрегация каналов с согласованием на коммутаторе       |
| adSelect  | bandwidth | Выбирает активный агрегатор по наибольшей суммарной пропускной способности |
| miimon    | 100       | Интервал мониторинга канала в миллисекундах                                |
| updelay   | 200       | Задержка (ms) перед переводом восстановленного канала в активное состояние |
| downdelay | 200       | Задержка (ms) перед объявлением отказавшего канала отключенным             |

|                |          |                                                                             |
|----------------|----------|-----------------------------------------------------------------------------|
| minLinks       | 1        | Минимальное количество активных каналов, при котором bond остается поднятым |
| xmitHashPolicy | encap3+4 | Хеширование по IP и TCP/UDP-порту для распределения нагрузки между линками  |

## Выбор интерфейсов

Рекомендуемый способ выбора участников bond — по пути PCI-шины с помощью `deviceSelectors`. Это надежнее, чем имена интерфейсов, которые могут меняться между перезагрузками:

```
bond:
 deviceSelectors:
 - busPath: "0000:04:00.0"
 - busPath: "0000:04:00.1"
```

Также можно выбирать по имени интерфейса:

```
bond:
 interfaces:
 - eth0
 - eth1
```

Или по аппаратному адресу:

```
bond:
 deviceSelectors:
 - hardwareAddr: "aa:bb:cc:dd:ee:f2"
 - hardwareAddr: "aa:bb:cc:dd:ee:f3"
```

## VLAN поверх bond

Поверх bond можно создавать VLAN-интерфейсы. Это удобно для разделения трафика (например, management, storage, tenant-сетей):

```
machine:
 network:
 interfaces:
 - interface: bond0
 dhcp: false
 bond:
 mode: 802.3ad
 adSelect: bandwidth
 miimon: 100
 updelay: 200
 downdelay: 200
 minLinks: 1
 xmitHashPolicy: encap3+4
 deviceSelectors:
 - busPath: "0000:04:00.0"
 - busPath: "0000:04:00.1"
 addresses:
 - 192.168.100.11/24
 routes:
 - network: 0.0.0.0/0
 gateway: 192.168.100.1
 vlans:
 - vlanId: 100
 addresses:
 - 10.0.0.11/24
```

## Floating IP (VIP) с bonding

Для узлов control plane разместите раздел `vip` на интерфейсе (или VLAN), который используется для API endpoint кластера:

```
machine:
 network:
 interfaces:
 - interface: bond0
 dhcp: false
 bond:
 mode: 802.3ad
 adSelect: bandwidth
 miimon: 100
 updelay: 200
 downdelay: 200
 minLinks: 1
 xmitHashPolicy: encap3+4
 deviceSelectors:
 - busPath: "0000:04:00.0"
 - busPath: "0000:04:00.1"
 addresses:
 - 192.168.100.11/24
 routes:
 - network: 0.0.0.0/0
 gateway: 192.168.100.1
 vip:
 ip: 192.168.100.10
```

Убедитесь, что floating IP совпадает с адресом, настроенным в `values.yaml`.

---

## Применение конфигурации

---

После редактирования всех файлов узлов примените конфигурацию обычным способом:

```
talm apply -f nodes/node1.yaml
talm apply -f nodes/node2.yaml
talm apply -f nodes/node3.yaml
```

### [!NOTE]

Флаг `-i` (`--insecure`) нужен только при первом применении, когда узлы находятся в `maintenance mode`. Для уже инициализированных узлов не указывайте этот флаг: `talm apply -f nodes/node1.yaml`.

# Как включить Hugepages

<https://cozystack.ru/docs/v1.5/install/how-to/hugepages/>

Hugepages для Cozystack можно включить как во время первоначальной установки, так и в любой момент после нее. Применение этой конфигурации после установки потребует полной перезагрузки узла.

Подробнее см. в документации ядра Linux: [HugeTLB Pages](#).

## Использование Talm

Требуется Talm v0.16.0 или новее.

1. Добавьте следующие строки в `values.yaml`:

```
...
certSANS: []
nr_hugepages: 3000
```

`vm.nr_hugepages` — это количество страниц на 2Mi.

2. Примените конфигурацию:

```
talm apply -f nodes/node0.yaml
```

3. Затем перезагрузите узлы:

```
talm -f nodes/node0.yaml reboot
```

## Использование talosctl

1. Добавьте следующие строки в шаблон узла:

```
machine:
 sysctls:
 vm.nr_hugepages: "3000"
```

`vm.nr_hugepages` — это количество страниц на 2Mi.

2. Примените конфигурацию:

```
talosctl apply -f nodetemplate.yaml -n 192.168.123.11 -e 192.168.123.11
```

3. Перезагрузите узлы:

```
talosctl reboot -n 192.168.123.11 -e 192.168.123.11
```





---

# Эксплуатация

## Руководство по настройке и управлению кластером

<https://cozystack.ru/docs/v1.5/operations/>

Настройка, мониторинг, защита и обновление кластера Cozystack.

### Обновление с v0.41 до v1.0

---

Пошаговое руководство по обновлению Cozystack с v0.41.x до v1.0

### Конфигурация кластера Cozystack

---

Как настраивать кластер Cozystack, включая варианты, компоненты и другие ключевые параметры

### VPC и подсети

---

Как использовать VPC и подсети

### Руководства по обслуживанию кластера

---

Руководства по регулярным операциям с кластером: добавлению и удалению узлов, обновлению Talos и другим задачам.

### Справочник cluster services

---

Системные middleware packages, которые разворачиваются в tenants и предоставляют основную функциональность пользовательским приложениям.

### Использование OpenID Connect с Cozystack

---

OIDC в Cozystack



---

## Руководства для нескольких дата-центров

---

Как запускать Cozystack в растянутом Kubernetes-кластере

## Multi-Location кластеры

---

Расширение Cozystack management clusters на несколько locations с помощью Kilo WireGuard mesh, cloud autoscaling и local cloud controller manager.

## Частые вопросы и практические руководства

---

База знаний с частыми вопросами и расширенными настройками

## Руководство по устранению неполадок Cozystack

---

Начальные шаги для проверки состояния кластера и поиска проблем.

## Scheduling Classes

---

Ограничение tenant workload конкретными узлами или failure domain с помощью ресурсов SchedulingClass и планировщика Cozystack.

---

# Обновление с v0.41 до v1.0

<https://cozystack.ru/docs/v1.5/operations/upgrades/>

## Обзор

Версия 1.0 вносит крупное изменение в control plane Cozystack: теперь он полностью модульный и состоит из независимых пакетов, которыми управляет новый `cozystack-operator`.

Ключевые изменения:

- Старый `installer deployment` заменен на `cozystack-operator`.
- Конфигурация больше не хранится в `ConfigMap`: теперь она задается `custom resource Package`.
- `Assets server` заменен одним OCI image.
- Добавлены новые CRD: `Package` и `PackageSource`.

Нижележащими сущностями по-прежнему остаются Helm releases, поэтому во время обновления workload не пересоздаются и не затрагиваются.

## Критические изменения совместимости

В этом разделе перечислены все пользовательские breaking changes, появившиеся в v1.0. Большинство изменений обрабатывается автоматически миграциями платформы, которые запускаются во время обновления. Перед обновлением просмотрите этот список, чтобы понять влияние на ваши рабочие нагрузки.

Пользователи FerretDB: приложение FerretDB удалено без автоматической миграции. Нужно сделать резервную копию данных до обновления. См. [FerretDB удален](#) ниже.

## MySQL переименован в MariaDB

Приложение `mysql` переименовано в `mariadb`, чтобы точнее отражать используемый движок базы данных.

Все Kubernetes resources переименовываются автоматически во время обновления:

| Ресурс             | До                                            | После                                           |
|--------------------|-----------------------------------------------|-------------------------------------------------|
| Application Kind   | MySQL                                         | MariaDB                                         |
| HelmRelease prefix | mysql-                                        | mariadb-                                        |
| Имена Service      | mysql- <code>&lt;name&gt;</code> -primary     | mariadb- <code>&lt;name&gt;</code> -primary     |
| Имена Secret       | mysql- <code>&lt;name&gt;</code> -credentials | mariadb- <code>&lt;name&gt;</code> -credentials |

|           |                                           |                                             |
|-----------|-------------------------------------------|---------------------------------------------|
| Имена PVC | <code>storage-mysql-&lt;name&gt;-*</code> | <code>storage-mariadb-&lt;name&gt;-*</code> |
|-----------|-------------------------------------------|---------------------------------------------|

Если ваши приложения подключаются к MySQL services по Kubernetes DNS name, например `mysql-mysdb-primary.<namespace>.svc`, после миграции нужно обновить строку подключения и использовать новый префикс `mariadb-`.

## FerretDB удален

Приложение FerretDB полностью удалено из платформы. Автоматической миграции нет.

Если у вас есть запущенные инстансы FerretDB, необходимо сделать резервную копию всех данных до обновления. После обновления FerretDB больше не будет доступен как managed application.

## Virtual Machine разделен на VM Disk и VM Instance

Монолитное приложение `virtual-machine` заменено двумя отдельными приложениями:

- `vm-disk` управляет образами дисков виртуальных машин.
- `vm-instance` управляет инстансами виртуальных машин и ссылается на диски, созданные `vm-disk`.

Миграция выполняется автоматически и сохраняет:

- Данные дисков: PersistentVolumes сохраняются и перепривязываются.
- IP- и MAC-адреса Kube-OVN.
- LoadBalancer IP для VM, опубликованных наружу.

Кроме того, `boolean`-поле `running` заменено на `runStrategy`:

| До                          | После                            |
|-----------------------------|----------------------------------|
| <code>running: true</code>  | <code>runStrategy: Always</code> |
| <code>running: false</code> | <code>runStrategy: Halted</code> |

Поле `runStrategy` также принимает значения `Manual`, `RerunOnFailure` и `Once`.

## Monitoring переведен на новую схему deployment

Стек мониторинга был реструктурирован. HelmRelease с именем `monitoring` в каждом `tenant namespace` мигрируется в новый release с именем `monitoring-system`.

Миграция выполняется автоматически: все компоненты мониторинга (VictoriaMetrics, Grafana, Alerta, VMAlert) сохраняют свои данные и конфигурацию.

## Формат VPC subnets изменен с map на array

---

Поле `subnets` в конфигурации VPC (VirtualPrivateCloud) изменено с `map` на `array`.

До:

```
subnets:
 my-subnet:
 cidr: 10.0.0.0/24
```

После:

```
subnets:
 - name: my-subnet
 cidr: 10.0.0.0/24
```

Для существующих VPC resources миграция выполняется автоматически.

## Конфигурация MongoDB users и databases унифицирована

Формат конфигурации пользователей MongoDB был реструктурирован. Users и databases теперь задаются в более структурированном виде.

До:

```
users:
 myuser:
 db: mydb
 roles:
 - name: readWrite
 db: mydb
```

После:

```
users:
 myuser: {}
databases:
 mydb:
 roles:
 admin:
 - myuser
```

Для существующих инстансов MongoDB миграция выполняется автоматически.

## Флаг `tenant isolated` удален

Поле `isolated` удалено из конфигурации Tenant. Сетевая изоляция теперь всегда применяется для каждого tenant через Cilium Network Policies. Если вы ранее использовали `isolated: false`, чтобы разрешить неограниченный трафик между tenant, теперь это больше невозможно.

Workload внутри tenant namespace по-прежнему должны обращаться к нескольким control-plane targets: API server, `etcd` tenant и т. д. Tenant chart предоставляет набор Cilium network policies, которые открывают эти пути по принципу opt-in через pod labels. Если pod внутри tenant namespace не может обратиться к одному из этих targets, добавьте соответствующую метку:

| Target                                                                                                                      | Pod label                                               |
|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Kubernetes API server                                                                                                       | <code>policy.cozystack.io/allow-to-apiserver: ""</code> |
| Services собственного <code>etcd</code> cluster tenant (применимо только если tenant был создан с <code>etcd: true</code> ) | <code>policy.cozystack.io/allow-to-etcd: ""</code>      |

Политика `allow-to-apiserver`, устанавливаемая tenant chart, сопоставляет трафик со встроенной сущностью Cilium `kube-apiserver`, которую Cilium разрешает в реальные endpoints API server. Вам не нужно знать Service CIDR или адрес сервиса `kubernetes`: метки на pod достаточно.

Пример: разрешить pod из Deployment обращаться к `kube-apiserver`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-operator
spec:
 selector:
 matchLabels:
 app: my-operator
 template:
 metadata:
 labels:
 app: my-operator
 policy.cozystack.io/allow-to-apiserver: ""
 spec:
 containers:
 - name: my-operator
 image: example.com/my-operator:v1.0.0
```

Без метки трафик к `kube-apiserver` блокируется политикой `allow-to-apiserver` `CiliumNetworkPolicy`, которую tenant chart устанавливает в каждый tenant namespace. Тот же шаблон применяется к `allow-to-etcd`.

## Внутренние изменения архитектуры

Следующие внутренние изменения не влияют напрямую на рабочие нагрузки приложений, но важны для скриптов автоматизации или пользовательских инструментов, которые взаимодействуют с внутренними компонентами Cozystack:

- Flux AIO теперь устанавливается и управляется `cozystack-operator`, а не отдельным компонентом.
- CRD `CozystackResourceDefinition` переименован в `ApplicationDefinition`.
- Компоненты legacy installer (`cozystack` Deployment и `cozystack-assets` StatefulSet) удалены.

- Namespace и HelmRelease tenant-root теперь управляются Helm через release cozystack-basics.

## Предварительные требования

### 1. Установите необходимые инструменты

Для миграции нужны следующие инструменты:

- `kubectl` и `jq` - стандартные инструменты администрирования кластера.
- `helm` - нужен для установки нового operator.
- `cozyrpg` - новый CLI для управления ресурсами Package и PackageSource. Скачайте его со [страницы релизов Cozystack](#).
- `cozyhr` - необязательный инструмент для управления значениями HelmRelease. Скачайте его из [репозитория cozyhr](#).

### 2. Проверьте kubectl context

Убедитесь, что текущий kubectl context указывает на кластер, который вы обновляете:

```
kubectl cluster-info
```

### 3. Обновитесь до последней версии v0.41.x

Перед миграцией на v1.0 убедитесь, что используете самый свежий patch release v0.41.

Проверьте текущую версию:

```
kubectl get configmap -n cozy-system cozystack -o jsonpath='{.metadata.labels.cozystack\.io/version}'
```

Если используется более старая версия, сначала обновитесь до последней v0.41.x по [стандартной процедуре обновления](#).

### 4. Проверьте состояние кластера

Перед обновлением убедитесь, что все HelmRelease находятся в healthy-состоянии:

```
kubectl get hr -A | grep -v "True"
```

Если какие-либо releases не находятся в состоянии Ready, устраните эти проблемы до продолжения.

## Шаги обновления

---

## Шаг 1. Защитите критичные ресурсы

---

Добавьте аннотации к namespace `cozy-system` и ConfigMap `cozystack-version`, чтобы Helm не удалил их при обновлении `installer release`:

```
kubectl annotate namespace cozy-system helm.sh/resource-policy=keep --overwrite
kubectl annotate configmap -n cozy-system cozystack-version helm.sh/resource-policy=keep --overwrite
```

Этот шаг обязателен. Без этих аннотаций обновление Helm `installer release` может удалить namespace `cozy-system` и все ресурсы внутри него.

## Шаг 2. Установите Cozystack Operator

---

Установите новый operator с помощью Helm из OCI registry. Команда развернет `cozystack-operator`, установит две новые CRD (`Package` и `PackageSource`) и создаст ресурс `PackageSource` для платформы.

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
 --version <TARGET_VERSION> \
 --namespace cozy-system \
 --create-namespace \
 --take-ownership
```

Замените `<TARGET_VERSION>` на нужную версию release, например `1.0.0`.

Проверьте, что operator запущен:

```
kubectl get pods -n cozy-system -l app=cozystack-operator
```

## Шаг 3. Сгенерируйте Platform Package

---

Скрипт миграции читает существующие ConfigMap (`cozystack`, `cozystack-branding`, `cozystack-scheduling`) из namespace `cozy-system` и преобразует их в ресурс `Package` с новой структурой `values`.

Скачайте и запустите скрипт миграции из репозитория Cozystack:

```
curl -fsSL https://raw.githubusercontent.com/cozystack/cozystack/main/hack/migrate-to-version-1.0.sh | bash
```

Скрипт:

1. Прочитает конфигурацию из существующих ConfigMap.
2. Преобразует старые имена bundle (`paas-*`) в новые имена variant (`isp-*`).
3. Сгенерирует ресурс `Package` и покажет его для проверки.
4. Запросит подтверждение перед применением.

Также можно скачать скрипт и запустить его локально, чтобы просмотреть перед выполнением:

```
curl -fsSL -o migrate-to-version-1.0.sh https://raw.githubusercontent.com/cozystack/cozystack/main/hack/mig...
```

## Шаг 4. Наблюдайте за миграцией

Как только Platform Package применен, operator запускает процесс миграции. Миграции удаляют старые Helm releases и создают новые с обновленной структурой.

Следите за статусами HelmRelease:

```
watch kubectl get hr -A
```

Дождитесь, пока все releases покажут `READY: True`.

## Шаг 5. Удалите старые ConfigMap

После проверки, что все компоненты healthy, удалите старые ConfigMap, которые больше не используются:

```
kubectl delete configmap -n cozy-system cozystack cozystack-branding cozystack-scheduling
```

## Шаг 6. Проверьте миграцию

Проверьте, что Platform Package прошел reconcile:

```
kubectl get packages.cozystack.io cozystack-platform
```

Запустите полную проверку состояния кластера:

```
kubectl get hr -A | grep -v "True"
kubectl get pods -n cozy-system
```

Если какие-либо HelmRelease не находятся в состоянии Ready, проверьте логи operator для подробностей.

## Устранение неполадок

### Operator не запускается

Если pod operator находится в CrashLoopBackOff, проверьте логи:

```
kubectl logs -n cozy-system deploy/cozystack-operator --previous
```



---

## HelmRelease зависли после миграции

---

Во время миграции некоторые HelmRelease могут временно показывать ошибки, пока operator выполняет reconcile. Подождите несколько минут и проверьте снова. Если проблемы сохраняются, обратитесь к [чеклисту диагностики](#).

---

---

# Конфигурация кластера Cozystack

<https://cozystack.ru/docs/v1.5/operations/configuration/>

В этом разделе документации описана конфигурация кластера Cozystack.

## Справочник Platform Package

---

Справочник по Cozystack Platform Package, который задает ключевые параметры конфигурации для установки и эксплуатации Cozystack.

## Варианты Cozystack: обзор и сравнение

---

Справочник по вариантам Cozystack: состав, конфигурация и сравнение.

## Справочник компонентов Cozystack

---

Полный справочник по компонентам Cozystack.

## Лицензии

---

Лицензии open-source компонентов, поставляемых с Cozystack.

## Lineage Controller Webhook

---

Что делает lineage-controller-webhook, как он разворачивается и какой параметр важно знать.

## White Labeling

---

Настройка брендинга в Cozystack Dashboard и на страницах аутентификации Keycloak, включая custom Keycloak themes.

## Telemetry

---

Телеметрия Cozystack

---

# Справочник Platform Package

<https://cozystack.ru/docs/v1.5/operations/configuration/platform-package/>

На этой странице описана роль Cozystack Platform Package и приведен полный справочник по его values.

Основная конфигурация Cozystack задается custom resource Package. Этот Package включает вариант Cozystack, настройки компонентов, ключевые сетевые параметры, опубликованные сервисы и другие опции.

## Пример

Ниже пример конфигурации для установки Cozystack с вариантом `isp-full`, root host `example.org`, а также опубликованными и доступными пользователям Cozystack Dashboard и API:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full
 components:
 platform:
 values:
 publishing:
 host: "example.org"
 apiServerEndpoint: "https://api.example.org:443"
 exposedServices:
 - dashboard
 - api
 networking:
 podCIDR: "10.244.0.0/16"
 podGateway: "10.244.0.1"
 serviceCIDR: "10.96.0.0/16"
 joinCIDR: "100.64.0.0/16"
```

## Reference

### Package-level fields

| Field                     | Description                                                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>spec.variant</code> | Variant to use for installation (e.g., <code>isp-full</code> , <code>isp-full-generic</code> , <code>isp-hosted</code> , <code>distro-full</code> ). |

### Platform values (`spec.components.platform.values.*`)

## Publishing

| Value                                                            | Default                                                        | Description                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>publishing.host</code>                                     | <code>"example.org"</code>                                     | The main domain for services created under Cozystack, such as the dashboard, Grafana, Keycloak, etc.                                                                                                                                                                                                                                                                  |
| <code>publishing.apiServerEndpoint</code>                        | <code>" "</code>                                               | Used for generating kubeconfig files for your users. It is recommended to use a routable FQDN or IP address instead of localhost or internal-only addresses.                                                                                                                                                                                                          |
| <code>publishing.exposedServices</code>                          | <code>[api, dashboard, vm-exportproxy, cdi-uploadproxy]</code> | List of services to expose. Possible values: <code>api</code> , <code>dashboard</code> , <code>cdi-uploadproxy</code> , <code>vm-exportproxy</code> .                                                                                                                                                                                                                 |
| <code>publishing.ingressName</code>                              |                                                                |                                                                                                                                                                                                                                                                                                                                                                       |
| <code>publishing.externalIPs</code>                              | <code>[]</code>                                                |                                                                                                                                                                                                                                                                                                                                                                       |
| <code>publishing.exposure</code>                                 |                                                                | Exposure mode for the ingress-nginx Service. Possible values: <code>externalIPs</code> , <code>loadBalancer</code> . The default writes <code>Service.spec.externalIPs</code> from <code>publishing.externalIPs</code> ; <code>loadBalancer</code> switches to <code>Service.type: LoadBalancer</code> and a <code>CiliumLoadBalancerIPPool</code> over the same IPs. |
| <code>publishing.certificates.solver</code>                      |                                                                | ACME challenge solver type for default letsencrypt issuer. Possible values: <code>http01</code> , <code>dns01</code> .                                                                                                                                                                                                                                                |
| <code>publishing.certificates.issuerName</code>                  |                                                                | <code>ClusterIssuer</code> name for TLS certificates used in system Helm releases.                                                                                                                                                                                                                                                                                    |
| <code>publishing.certificates.dns01.provider</code>              |                                                                | DNS-01 provider when <code>solver=dns01</code> . Possible values: <code>cloudflare</code> , <code>route53</code> , <code>digitalocean</code> , <code>rfc2136</code> .                                                                                                                                                                                                 |
| <code>publishing.certificates.dns01.cloudflare.secretName</code> |                                                                | Secret name holding a Cloudflare API token.                                                                                                                                                                                                                                                                                                                           |

|                                                                    |       |                                                                                                                                       |
|--------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>publishing.certificates.dns01.cloudflare.secretKey</code>    |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.route53.region</code>          |       | AWS region of the Route53 hosted zone.                                                                                                |
| <code>publishing.certificates.dns01.route53.accessKeyID</code>     |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.route53.secretName</code>      |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.route53.secretKey</code>       |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.digitalocean.secretName</code> |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.digitalocean.secretKey</code>  |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.fc2136.nameserver</code>       |       | host:port of the authoritative nameserver.                                                                                            |
| <code>publishing.certificates.dns01.fc2136.tsigKeyName</code>      |       | TSIG key name.                                                                                                                        |
| <code>publishing.certificates.dns01.fc2136.tsigAlgorithm</code>    |       |                                                                                                                                       |
| <code>publishing.certificates.dns01.fc2136.secretName</code>       |       | Secret name holding the TSIG key material.                                                                                            |
| <code>publishing.certificates.dns01.fc2136.secretKey</code>        |       |                                                                                                                                       |
| <code>publishing.proxyProtocol</code>                              | false | Enables PROXY-protocol on the host ingress-nginx and auto-deploys <a href="#">ouroboros</a> to fix the resulting hairpin-NAT problem. |
| <code>publishing.proxyProtocolAcknowledgeUnclean</code>            | false | Acknowledgement gate for the <a href="#">helm.sh/resource-policy</a> : keep asymmetry on the host disable path.                       |

## Networking

| Value                                 | Default | Description |
|---------------------------------------|---------|-------------|
| <code>networking.clusterDomain</code> |         |             |

|                                              |  |                                                                                                                               |
|----------------------------------------------|--|-------------------------------------------------------------------------------------------------------------------------------|
| <code>networking.podCIDR</code>              |  |                                                                                                                               |
| <code>networking.podGateway</code>           |  |                                                                                                                               |
| <code>networking.serviceCIDR</code>          |  |                                                                                                                               |
| <code>networking.joinCIDR</code>             |  | The <code>join</code> subnet for network communication between the Node and Pod. See <a href="#">kube-ovn documentation</a> . |
| <code>networking.kubeovn.MASTER_NODES</code> |  | Comma-separated list of KubeOVN master node IPs.                                                                              |

## Bundles

| Value                                 | Default            | Description                                                                                                        |
|---------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>bundles.system.enabled</code>   | <code>false</code> | Enable the system bundle. Managed by the operator based on <code>spec.variant</code> .                             |
| <code>bundles.system.variant</code>   |                    | System bundle variant. Options: <code>isp-full</code> , <code>isp-full-generic</code> , <code>isp-hosted</code> .  |
| <code>bundles.iaas.enabled</code>     | <code>false</code> | Enable the IaaS bundle.                                                                                            |
| <code>bundles.paas.enabled</code>     | <code>false</code> | Enable the PaaS bundle.                                                                                            |
| <code>bundles.naas.enabled</code>     | <code>false</code> | Enable the NaaS bundle.                                                                                            |
| <code>bundles.enabledPackages</code>  | <code>[]</code>    | List of optional bundle components to include. See <a href="#">“How to enable and disable bundle components”</a> . |
| <code>bundles.disabledPackages</code> | <code>[]</code>    | List of bundle components to exclude.                                                                              |

## Authentication

| Value                                               | Default            | Description                                       |
|-----------------------------------------------------|--------------------|---------------------------------------------------|
| <code>authentication.oidc.enabled</code>            | <code>false</code> | Enable <a href="#">OIDC</a> feature in Cozystack. |
| <code>authentication.oidc.insecureSkipVerify</code> | <code>false</code> |                                                   |

|                                                              |  |                                                           |
|--------------------------------------------------------------|--|-----------------------------------------------------------|
| <code>authentication.oidc.keycloakExternalRedirectUri</code> |  |                                                           |
| <code>authentication.oidc.keycloakInternalUrl</code>         |  | Internal URL for backend-to-backend requests to Keycloak. |

## Gateway

Platform-wide Gateway API integration. The actual per-tenant Gateway is materialised only for tenants with `tenant.spec.gateway: true`. See the [Gateway API guide](#) for the full architecture.

| Value                                   | Default            | Description                                                                                                                       |
|-----------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>gateway.enabled</code>            | <code>false</code> | Enable Gateway API support across the platform.                                                                                   |
| <code>gateway.attachedNamespaces</code> |                    | Namespaces whose <a href="#">HTTPRoute</a> / <a href="#">TLSRoute</a> resources should publish through the owning tenant Gateway. |

Default `gateway.attachedNamespaces`:

```
gateway:
 attachedNamespaces:
 - cozy-cert-manager
 - cozy-dashboard
 - cozy-keycloak
 - cozy-system
 - cozy-harbor
 - cozy-bucket
 - cozy-kubevirt
 - cozy-kubevirt-cdi
 - cozy-monitoring
 - cozy-linstor-gui
 - default
```

## Scheduling

| Value                                                      | Default | Description |
|------------------------------------------------------------|---------|-------------|
| <code>scheduling.globalAppTopologySpreadConstraints</code> |         |             |

## Branding

| Value | Default | Description |
|-------|---------|-------------|
|-------|---------|-------------|

|          |    |                                                                                  |
|----------|----|----------------------------------------------------------------------------------|
| branding | {} | UI branding configuration object. See the <a href="#">White Labeling guide</a> . |
|----------|----|----------------------------------------------------------------------------------|

## Registries

Container registry mirrors configuration. Allows routing image pulls through local mirrors.

| Value              | Default | Description                                    |
|--------------------|---------|------------------------------------------------|
| registries.mirrors | {}      | Map of registry hostnames to mirror endpoints. |
| registries.config  | {}      |                                                |

Example:

```
registries:
 mirrors:
 docker.io:
 endpoints:
 - http://10.0.0.1:8082
 ghcr.io:
 endpoints:
 - http://10.0.0.1:8083
 config:
 ":":
 tls:
 insecureSkipVerify: true
```

## Ресурсы

| Value                                     | Default | Description                                                                                                                     |
|-------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------|
| resources.cpuAllocationRatio              | 10      | CPU allocation ratio: на 1 vCPU запрашивается 1/cpuAllocationRatio CPU. Подробности см. в <a href="#">Resource Management</a> . |
| resources.memoryAllocationRatio           | 1       | Memory allocation ratio: на единицу настроенной памяти запрашивается 1/memoryAllocationRatio памяти.                            |
| resources.ephemeralStorageAllocationRatio | 40      | Ephemeral storage allocation ratio.                                                                                             |

## Внутренние поля

Этими полями автоматически управляет оператор Cozystack, их не следует изменять вручную.



---

| Value                    | Default | Description |
|--------------------------|---------|-------------|
| sourceRef.kind           |         |             |
| sourceRef.name           |         |             |
| sourceRef.namespace      |         |             |
| sourceRef.path           |         |             |
| migrations.enabled       | false   |             |
| migrations.image         |         |             |
| migrations.targetVersion |         |             |

---

# Варианты Cozystack: обзор и сравнение

<https://cozystack.ru/docs/v1.5/operations/configuration/variants/>

## Введение

Variants - это предопределенные конфигурации Cozystack, определяющие, какие bundles и компоненты включены. Каждый вариант тестируется, версионизируется и гарантированно работает как единое целое. Они упрощают установку, снижают риск неправильной конфигурации и помогают выбрать подходящий набор возможностей для вашего развертывания.

Это руководство предназначено для infrastructure engineers, DevOps-команд и platform architects, которые планируют разворачивать Cozystack в разных окружениях. В нем объясняется, как варианты Cozystack помогают адаптировать установку под конкретные потребности: от полнофункциональной platform-as-a-service до полного ручного контроля над установленными packages.

## Обзор вариантов

| Компонент                | default | isp-full          | isp-full-generic  | isp-hosted |
|--------------------------|---------|-------------------|-------------------|------------|
| Managed Kubernetes       |         | +                 | +                 |            |
| Managed Applications     |         | +                 | +                 | +          |
| Virtual Machines         |         | +                 | +                 |            |
| Cozystack Dashboard (UI) |         | +                 | +                 | +          |
| Cozystack API            |         | +                 | +                 | +          |
| Kubernetes Operators     |         | +                 | +                 | +          |
| Monitoring subsystem     |         | +                 | +                 | +          |
| Подсистема хранения      |         | LINSTOR           | LINSTOR           |            |
| Сетевая подсистема       |         | Kube-OVN + Cilium | Kube-OVN + Cilium |            |
| Подсистема виртуализации |         | KubeVirt          | KubeVirt          |            |

|                            |  |             |  |  |
|----------------------------|--|-------------|--|--|
| Подсистема OS и Kubernetes |  | Talos Linux |  |  |
|----------------------------|--|-------------|--|--|

## Выбор подходящего варианта

Variants объединяют bundles из разных слоев под конкретные задачи. Одни рассчитаны на полноценные platform-сценарии, другие - на workloads в cloud-hosted окружениях или полностью ручное управление packages.

### default

default - минимальный вариант, который предоставляет только набор PackageSources (ссылки на package registry). Bundles и компоненты заранее не настроены: все packages управляются вручную через [cozypkg](#). Используйте этот вариант, когда нужен полный контроль над тем, какие packages установлены и настроены. Именно этот вариант используется в workflow [Build Your Own Platform \(BYOP\)](#).

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: default
```

### isp-full

isp-full - полнофункциональный вариант PaaS и IaaS, предназначенный для установки на Talos Linux. Он включает все bundles и предоставляет полный набор компонентов Cozystack, создавая полноценный PaaS-опыт. Некоторые компоненты верхних слоев опциональны и могут быть исключены при установке.

isp-full предназначен для установки на bare-metal servers или VM.

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full
 components:
 platform:
 values:
 networking:
 podCIDR: "10.244.0.0/16"
 podGateway: "10.244.0.1"
 serviceCIDR: "10.96.0.0/16"
 joinCIDR: "100.64.0.0/16"
 publishing:
 host: "example.org"
 apiServerEndpoint: "https://192.168.100.10:6443"
 exposedServices:
 - api
 - dashboard
 - cdi-uploadproxy
 - vm-exportproxy
```

## isp-full-generic

isp-full-generic предоставляет такой же полнофункциональный PaaS и IaaS, как isp-full, но предназначен для обычных дистрибутивов Kubernetes, таких как k3s, kubeadm или RKE2. Используйте этот вариант, если нужен полный набор возможностей Cozystack без обязательного Talos Linux.

Подробные инструкции по установке см. в [руководстве Generic Kubernetes](#).

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full-generic
 components:
 platform:
 values:
 networking:
 podCIDR: "10.244.0.0/16"
 podGateway: "10.244.0.1"
 serviceCIDR: "10.96.0.0/16"
 joinCIDR: "100.64.0.0/16"
 publishing:
 host: "example.org"
 apiServerEndpoint: "https://192.168.100.10:6443"
 exposedServices:
 - api
 - dashboard
 - cdi-uploadproxy
 - vm-exportproxy
```

---

## isp-hosted

---

Cozystack можно установить как platform-as-a-service (PaaS) поверх существующего managed Kubernetes-кластера, обычно созданного cloud provider. Для этого сценария предназначен вариант `isp-hosted`. Его можно использовать с `kind` и любыми cloud-based Kubernetes-кластерами.

`isp-hosted` включает PaaS и NaaS bundles, предоставляя Cozystack API и UI, managed applications и tenant Kubernetes clusters. Он не включает CNI plugins, виртуализацию или хранилище.

Kubernetes-кластер, используемый для развертывания Cozystack, должен соответствовать следующим требованиям:

- Listening address некоторых компонентов Kubernetes должен быть изменен с `localhost` на маршрутизируемый адрес.
- Kubernetes API server должен быть доступен на `localhost`.

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-hosted
 components:
 platform:
 values:
 publishing:
 host: "example.org"
 apiServerEndpoint: "https://192.168.100.10:6443"
 exposedServices:
 - api
 - dashboard
```

## Подробнее

---

Полный список параметров конфигурации для каждого варианта см. в [справочнике по конфигурации](#).

Полный список компонентов и инструкции по их включению и отключению см. в [справочнике компонентов](#).

Чтобы развернуть выбранный вариант, следуйте [руководству по установке Cozystack](#) или [руководствам для конкретных провайдеров](#). Если вы устанавливаете Cozystack впервые, лучше использовать вариант `isp-full` и пройти [tutorial Cozystack](#).

---

# Справочник компонентов Cozystack

<https://cozystack.ru/docs/v1.5/operations/configuration/components/>

## Переопределение параметров компонентов

Иногда нужно переопределить отдельные параметры компонентов. Для этого измените соответствующий ресурс Package и укажите значения в секции `spec.components`. Структура values соответствует файлу [values.yaml](#) соответствующего системного chart в репозитории Cozystack.

Например, если нужно включить режим FRR-K8s для MetalLB, посмотрите его [values.yaml](#), чтобы понять доступные параметры, а затем измените Package `cozystack.metallb`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.metallb
 namespace: cozy-system
spec:
 variant: default
 components:
 metallb:
 values:
 metallb:
 frrk8s:
 enabled: true
```

## Включение и отключение компонентов

В bundles есть опциональные компоненты, которые нужно явно включать в установку. Обычные компоненты bundle, наоборот, можно отключить и исключить из установки, если они не нужны.

Используйте `bundles.enabledPackages` и `bundles.disabledPackages` в values Platform Package. Каждая запись в этих списках - это полное имя Package, такое же, как в выводе `kubectl get package`. Все platform packages находятся под префиксом `cozystack.`, например `cozystack.metallb`, `cozystack.hetzner-robotlb`, `cozystack.nfs-driver`. Перед редактированием Platform Package выполните `kubectl get package`, чтобы увидеть точные имена, доступные в вашем кластере.

Например, при [установке Cozystack в Hetzner](#) нужно заменить load balancer по умолчанию, MetalLB, на RobotLB, специально сделанный для Hetzner:

---

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full
 components:
 platform:
 values:
 bundles:
 disabledPackages:
 - cozystack.metallb
 enabledPackages:
 - cozystack.hetzner-robotlb
остальная конфигурация
```

Компоненты нужно отключать до установки Cozystack. Применение обновленной конфигурации с `disabledPackages` не удалит компоненты, которые уже установлены. Чтобы удалить уже установленные компоненты, вручную удалите Helm release следующей командой:

```
kubectl delete hr -n <namespace> <component>
```

# Лицензии

<https://cozystack.ru/docs/v1.5/operations/configuration/licenses/>

На этой странице перечислены open-source компоненты, поставляемые с Cozystack, сгруппированные по роли в платформе. Charts, CRDs, controllers и application APIs, поддерживаемые Cozystack, лицензированы под Apache-2.0 и не перечисляются ниже по отдельности. Для каждого upstream-компонента карточка содержит ссылку на upstream-файл лицензии.

Этот справочник вручную сверяется с текущим набором компонентов `next`. Container images могут включать дополнительные пакеты операционной системы и библиотечные зависимости со своими лицензиями.

Зафиксированные upstream-версии managed runtimes (PostgreSQL, MariaDB, Kafka и другие) могут меняться между minor-релизами Cozystack: проверяйте версию Cozystack, которую вы используете.

## Операционная система и Kubernetes runtime

- **Talos Linux**: Immutable Linux-дистрибутив, созданный для узлов Kubernetes. (MPL-2.0)
- **Kubernetes**: Ядро оркестрации контейнеров, используемое и для management-кластера, и для tenant-кластеров. (Apache-2.0)
- **Kamaji**: Hosted control planes для tenant Kubernetes-кластеров. (Apache-2.0)
- **Cluster API**: Декларативное создание tenant Kubernetes-кластеров: core, operator и Kamaji/KubeVirt providers. (Apache-2.0)
- **KubeVirt**: Виртуальные машины как Kubernetes-native workloads: core, CDI, CSI иinstancetypeypes. (Apache-2.0)

## Networking

- **Cilium**: CNI на базе eBPF для pod networking и NetworkPolicy. (Apache-2.0)
- **Kube-OVN**: Виртуальная сеть на базе OVN, используемая для VPC и floating IP. (Apache-2.0)
- **Multus CNI**: Несколько сетевых интерфейсов на pod. (Apache-2.0)
- **MetalLB**: Bare-metal load balancer для Kubernetes Services. (Apache-2.0)
- **ingress-nginx**: HTTP ingress controller. (Apache-2.0)
- **Gateway API CRDs**: Стандартные определения Kubernetes Gateway API. (Apache-2.0)
- **CoreDNS**: Cluster DNS server. (Apache-2.0)
- **ExternalDNS**: Синхронизация Kubernetes-ресурсов с внешними DNS providers. (Apache-2.0)
- **Kilo**: Mesh-сеть между географически распределенными узлами. (Apache-2.0)
- **Hetzner RobotLB**: Интеграция load balancer для dedicated hardware Hetzner. (MIT)



---

## Хранилище

---

- [LINSTOR / Piraeus](#): Реплицированное блочное хранилище на базе DRBD: LINSTOR server, CSI, scheduler extender, GUI, Piraeus operator. (GPL-3.0; Apache-2.0)
- [SeaweedFS](#): Распределенное object storage, лежащее в основе managed Bucket service. (Apache-2.0)
- [CSI Driver NFS](#): NFS CSI driver. (Apache-2.0)
- [CSI Snapshot Controller](#): External snapshotter и VolumeSnapshot CRDs. (Apache-2.0)
- [Velero](#): Резервное копирование кластера и persistent volumes. (Apache-2.0)
- [Container Object Storage Interface](#): COSI controller для managed object storage. (Apache-2.0)
- [S3 Manager](#): Web UI для S3-compatible buckets. (Apache-2.0)

## Наблюдаемость

---

- [VictoriaMetrics Operator](#): Хранение, прием и Prometheus-compatible query layer для метрик. (Apache-2.0)
- [Grafana Operator](#): Управляет экземплярами Grafana, dashboards и datasources. (Apache-2.0)
- [Fluent Bit](#): Log forwarder, работающий на каждом узле. (Apache-2.0)
- [kube-state-metrics](#): Экспортирует состояние объектов Kubernetes как метрики. (Apache-2.0)
- [node-exporter](#): Системные и аппаратные метрики с каждого узла. (Apache-2.0)
- [Prometheus Operator CRDs](#): CRDs для Prometheus-style monitoring resources, используемые VictoriaMetrics. (Apache-2.0)
- [Metrics Server](#): Метрики Kubelet для HPA и `kubectl top`. (Apache-2.0)
- [Goldpinger](#): Проверки pod-to-pod connectivity в кластере. (Apache-2.0)

## Autoscaling и управление ресурсами

---

- [Vertical Pod Autoscaler](#): Вертикальный подбор ресурсов для pods (chart: MIT). (Apache-2.0)
- [Cluster Autoscaler](#): Горизонтальное масштабирование node pools. (Apache-2.0)
- [Stakater Reloader](#): Перезапускает pods при изменении их ConfigMaps или Secrets. (Apache-2.0)

## GPU и ускорители

---

- [NVIDIA GPU Operator](#): Lifecycle драйвера, container runtime и device-plugin для NVIDIA GPUs. (Apache-2.0)
- [HAMi](#): GPU sharing и fractional GPU scheduling. (Apache-2.0)

## GitOps и автоматизация платформы

---

- [Flux](#): GitOps engine. ControlPlane Flux Operator и instance chart лицензированы под AGPL-3.0; upstream Flux v2 — Apache-2.0. (Apache-2.0; AGPL-3.0)

- 
- **Aenix etcd Operator**: Управляет etcd-кластерами, используемыми tenant Kamaji control planes. (Apache-2.0)
  - **cert-manager**: Автоматический выпуск и ротация TLS-сертификатов. (Apache-2.0)
  - **External Secrets Operator**: Синхронизирует secrets из внешнего KMS в Kubernetes. (Apache-2.0)
  - **SAP ClusterSecret Operator**: Реплицирует secrets между namespaces. (Apache-2.0)
  - **Tinkerbelle Smee**: iPXE / DHCP boot server для bare-metal provisioning. (Apache-2.0)
  - **Telepresence**: Локальная разработка против удаленного кластера (Traffic Manager). (Apache-2.0)

## Identity, registry и admin UI

---

- **Keycloak**: OIDC provider для platform и tenant SSO; разворачивается через KubeRocketCI Keycloak Operator. (Apache-2.0)
- **Harbor**: OCI registry для container images и Helm charts. (Apache-2.0)

## Managed database runtimes

---

- **PostgreSQL**: Управляется через CloudNativePG operator (Apache-2.0). (PostgreSQL License)
- **MariaDB Server**: Управляется через mariadb-operator (MIT). (GPL-2.0)
- **MongoDB (Percona Server)**: Управляется через Percona Operator for MongoDB (Apache-2.0). (SSPL-1.0)
- **ClickHouse**: Server и Keeper, управляются через Altinity ClickHouse Operator (Apache-2.0). (Apache-2.0)
- **OpenSearch**: Управляется через opensearch-k8s-operator (Apache-2.0). (Apache-2.0)
- **Qdrant**: Vector database, разворачивается через upstream Qdrant Helm chart. (Apache-2.0)
- **FoundationDB**: Управляется через FoundationDB Kubernetes Operator (Apache-2.0). (Apache-2.0)
- **Redis**: Управляется через Spotahome Redis Operator (Apache-2.0). Cozystack поддерживает Redis 7.4 и Redis 8. (RSALv2 or SSPLv1 (7.x) / AGPLv3 (8.x))

## Managed messaging и caching runtimes

---

- **Apache Kafka**: Управляется через Strimzi Kafka Operator (Apache-2.0). (Apache-2.0)
- **NATS**: Lightweight messaging server, разворачивается через upstream NATS Helm chart. (Apache-2.0)
- **RabbitMQ**: Управляется через RabbitMQ Cluster Operator (MPL-2.0). (MPL-2.0; Apache-2.0 for some files)
- **OpenBao**: Fork HashiCorp Vault для управления secrets, разворачивается через upstream OpenBao Helm chart. (MPL-2.0)

## Managed networking services

---

- **NGINX**: Используется managed HTTP Cache service. (BSD-2-Clause)

- 
- [HAProxy](#): Используется managed TCP Balancer и HTTP Cache services. (GPL-2.0 with exceptions)
  - [IP2Location modules](#): GeoIP modules, включенные в HTTP Cache (IP2Location и IP2Proxy). (MIT)
  - [Outline Server \(Shadowsocks\)](#): Основа managed VPN service. (Apache-2.0)
-

# Lineage Controller Webhook

<https://cozystack.ru/docs/v1.5/operations/configuration/lineage-controller-webhook/>

lineage controller webhook — это mutating admission webhook, поставляемый в составе Package `cozystack.cozystack-engine`. При каждом CREATE и UPDATE tenant-ресурсов `Pod`, `Secret`, `Service`, `PersistentVolumeClaim`, `Ingress` или `WorkloadMonitor` он поднимается по графу владения и записывает идентичность владеющего Cozystack Application в labels ресурса (`apps.cozystack.io/application.{group,kind,name}`). Dashboard Cozystack, aggregated API server и механизм `SchedulingClass` опираются на эти labels.

Webhook зарегистрирован с `failurePolicy: Fail`, поэтому kube-apiserver должен иметь доступ к рабочему pod webhook, чтобы tenant CREATE/UPDATE-запросы проходили успешно.

## Схема разворачивания по умолчанию

Chart разворачивает один Deployment, построенный по образцу `cozystack-api`:

- 2 replicas (переопределяется через `replicas`).
- **Мягкий nodeAffinity** с предпочтением `node-role.kubernetes.io/control-plane` (Exists совпадает и с пустым значением Talos, и с "true" в k3s/kubeadm). Это именно мягкое предпочтение: pods попадают на control-plane-узел, когда он доступен, иначе на любой worker. Для managed Kubernetes (EKS / AKS / GKE), tenant-кластеров Cozy-in-Cozy и любых кластеров, где control-plane-узлы не видны, переопределение не нужно: webhook просто планируется в другом месте.
- **Разрешающие tolerations** (`{operator: Exists}`), чтобы control-plane-узел с taint `NoSchedule` принимал pod, когда мягкий affinity может быть выполнен.
- **Мягкий podAntiAffinity** по `kubernetes.io/hostname`, чтобы реплики по возможности распределялись по разным узлам.
- **PodDisruptionBudget** с `maxUnavailable: 1`. При `replicas: 2+` он ограничивает disruption одним pod; при `replicas: 1` это полезный no-op.
- **Service spec.trafficDistribution: PreferClose**, чтобы apiserver предпочитал endpoint webhook на своем узле, если он существует, и прозрачно переключался на удаленный endpoint иначе. Требуется Kubernetes  $\geq 1.31$ ; более старые кластеры незаметно возвращаются к стандартному распределению по всему кластеру, что тоже безопасно, но без предпочтения локальности.

Такая схема работает как есть во всех дистрибутивах Kubernetes, которые поддерживает Cozystack. В нормальной эксплуатации ничего переопределять не требуется.

## Увеличение числа реплик

---

Если нужно больше двух реплик, например чтобы держать по одному webhook pod рядом с каждым apiserver на control plane из пяти узлов, переопределите значение `replicas` через Package `cozystack.cozystack-engine` так же, как для любого другого компонента (см. [Компоненты](#)):

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-engine
 namespace: cozy-system
spec:
 variant: default
 components:
 lineage-controller-webhook:
 values:
 lineageControllerWebhook:
 replicas: 5
```

## localK8sAPIEndpoint.enabled deprecated

Chart все еще предоставляет `localK8sAPIEndpoint.enabled`. При значении `true` он добавляет `KUBERNETES_SERVICE_HOST=status.hostIP` и `KUBERNETES_SERVICE_PORT=6443`, чтобы webhook обращался к apiserver на своем узле. Изначально это было добавлено, чтобы избежать задержки на пути webhook-to-apiserver. Сейчас значение по умолчанию — `false`, а сам параметр планируется удалить после того, как причина с задержкой будет устранена внутри webhook.

## Важно

Не включайте `localK8sAPIEndpoint.enabled` со стандартными values chart. Добавленный `status.hostIP` корректен только когда pod работает на узле с kube-apiserver, а мягкий control-plane affinity в chart этого не гарантирует. Если флаг включен, а pod запланирован не на control-plane-узел, controller уйдет в crash loop, пытаясь подключиться к IP, где нет apiserver. В сочетании с `failurePolicy: Fail` это означает отказ tenant CREATE/UPDATE-операций.

## Проверка развертывания

```
kubectl -n cozy-system get deploy lineage-controller-webhook
kubectl -n cozy-system get pods -l app=lineage-controller-webhook
kubectl -n cozy-system get svc lineage-controller-webhook -o yaml | grep trafficDistribution
```

Быстрая end-to-end-проверка, которая вызывает webhook через apiserver:

```
kubectl create ns lineage-webhook-test
kubectl -n lineage-webhook-test create service clusterip probe \
 --clusterip=None --dry-run=server
kubectl delete ns lineage-webhook-test
```

---

Dry-run CREATE проходит через mutating admission webhook; если webhook недоступен, команда завершается ошибкой `failed calling webhook "lineage.cozystack.io"`.

---

# White Labeling

<https://cozystack.ru/docs/v1.5/operations/configuration/white-labeling/>

White labeling позволяет заменить стандартный брендинг Cozystack собственными логотипами и текстами в Dashboard UI и на страницах аутентификации Keycloak.

## Обзор

Брендинг настраивается через поле `branding` в Platform Package (список `spec.components.platform.values.branding`). Конфигурация автоматически распространяется на:

- Dashboard: логотип, заголовок страницы, текст footer, favicon и идентификатор tenant
- Keycloak: display name realm на страницах аутентификации

## Конфигурация

Измените Platform Package, чтобы добавить или обновить секцию `branding`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.cozystack-platform
spec:
 variant: isp-full # используйте свой вариант
 components:
 platform:
 values:
 branding:
 # Брендинг Dashboard
 titleText: "My Company Dashboard"
 footerText: "My Company Platform"
 tenantText: "Production v1.0"
 logoText: ""
 logoSvg: "<base64-encoded SVG>"
 iconSvg: "<base64-encoded SVG>"
 # Брендинг Keycloak
 brandName: "My Company"
 brandHtmlName: "<div style='font-weight:bold; '>My Company</div>"
```

Примените изменения:

```
kubectl apply --server-side --filename platform-package.yaml
```

## Поля конфигурации

### Поля Dashboard

| Поле       | По умолчанию               | Описание                                                                   |
|------------|----------------------------|----------------------------------------------------------------------------|
| titleText  | Cozystack Dashboard        | Заголовок вкладки браузера и текст header в Dashboard.                     |
| footerText | Cozystack                  | Текст, отображаемый в footer Dashboard.                                    |
| tenantText | Строка версии platform     | Версия или идентификатор tenant, отображаемый в Dashboard.                 |
| logoText   | "" (пусто)                 | Альтернативный текстовый логотип. Используется, если SVG-логотип не задан. |
| logoSvg    | Логотип Cozystack (base64) | SVG-логотип в base64, отображаемый в header Dashboard.                     |
| iconSvg    | Иконка Cozystack (base64)  | SVG-иконка в base64, используемая как browser favicon.                     |

## Поля Keycloak

| Поле          | По умолчанию | Описание                                                                                                                             |
|---------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------|
| brandName     | Не задано    | Plain text имя realm, отображаемое во вкладке браузера Keycloak.                                                                     |
| brandHtmlName | Не задано    | HTML-форматированное имя realm, отображаемое на страницах входа Keycloak. Поддерживает inline HTML/CSS для стилизованного брендинга. |

## Подготовка SVG-логотипов

### SVG-переменные с учетом темы

Dashboard поддерживает template variables в SVG, которые адаптируются к светлой и темной темам:

- `{token.colorText}` - во время выполнения заменяется на цвет текста текущей темы



Note: Синтаксис `{token.colorText}` не является валидным XML. Значение атрибута намеренно не заключено в кавычки, потому что Dashboard выполняет raw string substitution в исходном SVG перед render: заменяет `{token.colorText}` на фактическое значение цвета. Это означает, что SVG-файлы с такими placeholders нельзя напрямую открыть в браузере или проверить XML parser. Это ожидаемое поведение, соответствующее upstream-реализации Dashboard.

Пример SVG с theme-aware переменной:

```
<svg width="24" height="24" viewBox="0 0 24 24" fill="none" xmlns="http://www.w3.org/2000/svg">
 <path d="M12 2L2 7L12 12L22 7L12 2Z" fill={token.colorText}/>
 <text x="0" y="20" fill={token.colorText}>Logo</text>
</svg>
```

## Конвертация SVG в Base64

Закодируйте SVG-файлы в строки base64:

```
base64 < logo.svg | tr -d '\n'
```

## Пример workflow

```
Закодировать логотипы
LOGO_B64=$(base64 < logo.svg | tr -d '\n')
ICON_B64=$(base64 < icon.svg | tr -d '\n')

Изменить Platform Package
kubectl patch packages.cozystack.io cozystack.cozystack-platform \
 --type merge --server-side \
 --patch \
 "{
 \"spec\": {
 \"components\": {
 \"platform\": {
 \"values\": {
 \"branding\": {
 \"logoSvg\": \"\${LOGO_B64}\",
 \"iconSvg\": \"\${ICON_B64}\"
 }
 }
 }
 }
 }
 }"
```

## Проверка

После применения изменений проверьте, что брендинг настроен корректно:

1. Проверьте Platform Package:

```
``bash
```

```
kubectl get packages.cozystack.io cozystack.cozystack-platform \
```

```
--output jsonpath='{.spec.components.platform.values.branding}' | jq .
```

...

2. Dashboard: откройте URL Dashboard и проверьте логотип, заголовок, footer и favicon.

3. Keycloak: откройте страницу входа Keycloak и проверьте display name realm.

Note: Чтобы увидеть обновленный брендинг, может потребоваться hard refresh (Ctrl+Shift+R / Cmd+Shift+R) или очистка кеша браузера.

## Custom Keycloak themes

Для более глубокой визуальной настройки страниц аутентификации Keycloak (login, registration, account management) можно подключать custom themes, собранные как container images.

### Контракт theme image

Theme image должен содержать файлы темы в директории `/themes/`. Структура директорий должна соответствовать стандартному [формату themes Keycloak](#):

```
/themes/
 my-brand/
 login/
 theme.properties
 resources/
 css/
```

При запуске `pod init containers` копируют файлы из каждого theme image в директорию Keycloak `/opt/keycloak/themes/`. Встроенные themes Keycloak, упакованные в JAR-файлы, не затрагиваются.

Если несколько theme images содержат файлы по одному и тому же path, более поздние записи в списке переопределяют предыдущие.

## Конфигурация

Custom themes настраиваются на системном компоненте Keycloak. Измените Package `cozystack.keycloak`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.keycloak
 namespace: cozy-system
spec:
 variant: default
 components:
 keycloak:
 values:
 themes:
 - name: my-brand
 image: registry.example.com/my-keycloak-theme:v1.0
```

---

Примените изменения:

```
kubectl apply --server-side --filename keycloak-package.yaml
```

## Поля theme

Поле	Обязательно	Описание
name	Да	Идентификатор theme. Используется как имя init container после приведения к формату DNS-1123.
image	Да	Container image, содержащий файлы theme в /themes/.

## Private registries

Если theme images хранятся в private registry, добавьте `imagePullSecrets`:

```
keycloak:
 values:
 themes:
 - name: my-brand
 image: private-registry.example.com/my-keycloak-theme:v1.0
 imagePullSecrets:
 - name: my-registry-secret
```

Указанный Secret должен существовать в namespace `cozy-keycloak`.

## Активация custom theme

После развертывания theme image активируйте его в Keycloak:

1. Откройте admin console Keycloak.
2. Перейдите в Realm Settings > Themes.
3. Выберите custom theme в выпадающем списке для нужного типа темы: login, account, email или admin.
4. Сохраните изменения.

## Миграция с v0

В Cozystack v0 брендинг настраивался через отдельный ConfigMap `cozystack-branding` в namespace `cozy-system`. В v1 этот ConfigMap больше не используется.

[Скрипт миграции](#) автоматически преобразует старые значения ConfigMap в поле `branding` Platform Package.

---

Если раньше вы использовали подход с ConfigMap, ручная миграция не нужна: процесс обновления выполнит ее автоматически.

---

# Telemetry

<https://cozystack.ru/docs/v1.5/operations/configuration/telemetry/>

В этом документе описана телеметрия в проекте Cozystack: зачем собираются данные, какие именно данные собираются, как они обрабатываются и как отказаться от участия.

## Зачем мы собираем телеметрию

Cozystack как open source проект развивается благодаря обратной связи сообщества и пониманию реального использования. Данные телеметрии позволяют maintainers понимать, как Cozystack применяется в реальных сценариях. Эти данные помогают принимать решения о приоритете функций, стратегиях тестирования, исправлениях ошибок и общем развитии проекта. Без телеметрии решения приходилось бы принимать на основе предположений или ограниченной обратной связи, что могло бы замедлить улучшения или привести к функциям, которые не соответствуют потребностям пользователей. Телеметрия помогает направлять разработку реальными паттернами использования и требованиями сообщества, делая платформу более надежной и ориентированной на пользователей.

## Что и как мы собираем

Cozystack стремится соблюдать [LF Telemetry Data Policy](#), обеспечивая ответственную практику сбора данных с уважением к приватности пользователей и прозрачности.

Мы собираем не персональные сведения о пользователях, а неперсональные метрики использования компонентов Cozystack. В частности, собирается информация об инфраструктуре кластера (узлы, хранилище, сеть), установленных packages и экземплярах приложений. Эти данные помогают понимать распространенные конфигурации и тенденции использования в разных установках.

Телеметрию собирают два компонента:

- cozystack-operator — собирает метрики уровня кластера (узлы, хранилище, packages)
- cozystack-controller — собирает метрики уровня приложений (развернутые экземпляры приложений)

Чтобы подробно увидеть, какие данные собираются, можно изучить реализацию телеметрии:

- [Telemetry Client](#)
- [Telemetry Server](#)

## Пример telemetry payload

Ниже показан типичный telemetry payload в Cozystack.

---

От cozystack-operator (инфраструктура кластера):

```
cozy_cluster_info{cozystack_version="v1.0.0",kubernetes_version="v1.31.4"} 1
cozy_nodes_count{os="linux (Talos (v1.8.4))",kernel="6.6.64-talos"} 3
cozy_cluster_capacity{resource="cpu"} 168
cozy_cluster_capacity{resource="memory"} 811020009472
cozy_cluster_capacity{resource="nvidia.com/TU104GL_TESLA_T4"} 3
cozy_loadbalancers_count 1
cozy_pvs_count{driver="linstor.csi.linbit.com",size="5Gi"} 7
cozy_pvs_count{driver="linstor.csi.linbit.com",size="10Gi"} 6
cozy_package_info{name="cozystack.core",variant="default"} 1
cozy_package_info{name="cozystack.storage",variant="linstor"} 1
cozy_package_info{name="cozystack.monitoring",variant="default"} 1
```

От cozystack-controller (экземпляры приложений):

```
cozy_application_count{kind="Tenant"} 2
cozy_application_count{kind="Postgres"} 5
cozy_application_count{kind="Redis"} 3
cozy_application_count{kind="Kubernetes"} 2
cozy_application_count{kind="VirtualMachine"} 0
```

Данные собираются компонентами, работающими внутри Cozystack. Они периодически собирают и отправляют статистику использования в наш защищенный backend. Система телеметрии обеспечивает анонимизацию, агрегацию и безопасное хранение данных, а доступ к ним строго контролируется для защиты приватности пользователей.

## Отказ от телеметрии

Мы уважаем вашу приватность и выбор в отношении телеметрии. Если вы не хотите участвовать в сборе telemetry data, Cozystack предоставляет простой способ отказаться.

Отключение:

Чтобы отключить отправку телеметрии, обновите Helm release оператора Cozystack с флагом `disableTelemetry`:

```
helm upgrade cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
 --namespace cozy-system \
 --version X.Y.Z \
 --set cozystack0operator.disableTelemetry=true
```

Замените `X.Y.Z` на установленную у вас версию Cozystack.

**\*\*Do not use `--reuse-values`\*\*** when upgrading the Cozystack operator. The Helm chart values contain hardcoded references to the platform OCI repository. Reusing old values would result in the operator pointing to old package versions.

>

If you have custom values (e.g., `disableTelemetry`), pass them explicitly with `--set`.

---

Эта команда обновляет оператор и отключает сбор телеметрии. Если позже нужно снова включить телеметрию, выполните ту же команду с `disableTelemetry=false`.

## Заключение

---

Телеметрия в Cozystack нужна, чтобы развитие проекта опиралось на данные, отвечало потребностям сообщества и обеспечивало постоянное улучшение. Ваше участие или отказ от него помогает формировать будущее Cozystack и делать платформу более эффективной и удобной для всех.

---

---

# VPC и подсети

<https://cozystack.ru/docs/v1.5/operations/vpc/>

## Подключение workloads к подсетям

---

После создания VPC с набором подсетей к ним можно подключать рабочую нагрузку. Сейчас подключение к подсетям поддерживают только виртуальные машины.

### 1. Получите ID подсетей

Сначала нужно определить ID подсетей. ID автоматически формируются из имен ресурсов, указанных при создании. Проверьте их на вкладке Details вашей VPC.

[!Подсети VPC](#)

### 2. Укажите ID подсетей при создании ресурса

При создании VM укажите ID подсетей в разделе Subnets настроек ресурса.

[!Подсети VM](#)

Каждая подсеть будет представлена как дополнительный сетевой интерфейс. Для некоторых гостевых операционных систем нужно добавить конфигурацию сетевого интерфейса в user-data виртуальной машины. Также можно поднять дополнительные интерфейсы вручную и получить IP-адреса через DHCP.

---



---

# Руководства по обслуживанию кластера

<https://cozystack.ru/docs/v1.5/operations/cluster/>

Руководства по регулярным операциям с кластером: добавлению и удалению узлов, обновлению Talos и другим задачам.

## Обновление Cozystack и проверки после обновления

---

Обновление системных компонентов Cozystack.

## Масштабирование кластера: добавление и удаление узлов

---

Добавление и удаление узлов в кластере Cozystack.

## Как ротировать Certificate Authority

---

Как ротировать Certificate Authority

---

# Обновление Cozystack и проверки после обновления | Cozystack

<https://cozystack.ru/docs/v1.5/operations/cluster/upgrade/>

## О версиях Cozystack

Cozystack использует поэтапный процесс выпуска релизов, чтобы сохранять стабильность и гибкость во время разработки.

Существует три типа релизов:

- Alpha, Beta и Release Candidate (RC) — предварительные версии, например `v0.42.0-alpha.1` или `v0.42.0-rc.1`, которые используются для финального тестирования и проверки.
- Стабильные релизы — обычные версии, например `v0.42.0`, с полным набором функций и тщательным тестированием. Такие версии обычно добавляют новые возможности, обновляют зависимости и могут содержать изменения API.
- Патч-релизы — обновления только с исправлениями ошибок, например `v0.42.1`, которые выпускаются после стабильного релиза из отдельной release-ветки.

В production-средах настоятельно рекомендуется устанавливать только стабильные релизы и патч-релизы.

Полный список релизов доступен на странице [Releases](#) в GitHub.

Подробнее о процессе выпуска Cozystack см. в документе [Cozystack Release Workflow](#).

## Обновление Cozystack

### 1. Проверьте состояние кластера

Перед обновлением проверьте текущее состояние кластера Cozystack по шагам из раздела:

- [Чек-лист диагностики](#)

Убедитесь, что Platform Package находится в рабочем состоянии и содержит ожидаемую конфигурацию:

```
kubectl get packages.cozystack.io cozystack.cozystack-platform -o yaml
```

### 2. Защитите критически важные ресурсы

Перед обновлением добавьте аннотацию `helm.sh/resource-policy=keep` к namespace `cozy-system` и ConfigMap `cozystack-version`, чтобы Helm не удалил их во время обновления:

```
kubectl annotate namespace cozy-system helm.sh/resource-policy=keep --overwrite
kubectl annotate configmap -n cozy-system cozystack-version helm.sh/resource-policy=keep --overwrite
```

Этот шаг обязателен. Без этих аннотаций удаление или обновление Helm-релиза установщика может удалить namespace `cozy-system` и все ресурсы внутри него.

### 3. Обновите оператор Cozystack

Обновите Helm-релиз оператора Cozystack до целевой версии:

Do not use `--reuse-values` when upgrading the Cozystack operator. The Helm chart values contain hardcoded references to the platform OCI repository. Reusing old values would result in the operator pointing to old package versions.

>

If you have custom values (e.g., `disableTelemetry`), pass them explicitly with `--set`.

```
helm upgrade cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
 --version X.Y.Z \
 --namespace cozy-system
```

Логи оператора можно посмотреть так:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

### 4. Проверьте состояние кластера после обновления

```
kubectl get pods -n cozy-system
kubectl get hr -A | grep -v "True"
```

Если статус pod указывает на сбой, проверьте логи:

```
kubectl logs -n cozy-system deploy/cozystack-operator --previous
```

Чтобы убедиться, что все работает как ожидается, повторите шаги из раздела:

- [Чек-лист диагностики](#)

# Масштабирование кластера: добавление и удаление узлов

<https://cozystack.ru/docs/v1.5/operations/cluster/scaling/>

## Как добавить узел в кластер Cozystack

Узел добавляется почти так же, как при обычной установке Cozystack.

1. Установите Talos на узел, используя кастомный образ Talos для Cozystack.
2. Сгенерируйте конфигурацию для нового узла по руководству [Talm](#) или [talosctl](#).

Например, конфигурация узла control plane:

```
talm template -e 192.168.123.20 -n 192.168.123.20 -t templates/controlplane.yaml -i > nodes/nodeN.yaml
```

а для worker-узла:

```
talm template -e 192.168.123.20 -n 192.168.123.20 -t templates/worker.yaml -i > nodes/nodeN.yaml
```

3. Примените сгенерированную конфигурацию к узлу по руководству [Talm](#) или [talosctl](#).

Например:

```
talm apply -f nodes/nodeN.yaml -i
```

4. Дождитесь, пока узел перезагрузится и самостоятельно подключится к кластеру. Запускать bootstrap вручную или устанавливать на него Cozystack не нужно: все будет выполнено автоматически.

Результат можно проверить командой `kubectl get nodes`.

## Как удалить узел из кластера Cozystack

Когда узел кластера выходит из строя, Cozystack автоматически поддерживает отказоустойчивость, пересоздавая реплицированные PVC и workloads на других узлах. Однако иногда для устранения проблем узел нужно удалить:

- PV локального хранилища могут остаться привязанными к отказавшему узлу, что вызывает проблемы с новыми pod. Такие ресурсы нужно очистить вручную.
- Отказавший узел будет по-прежнему существовать в кластере, что может привести к несогласованности состояния кластера и повлиять на планирование pod.

### Шаг 1: удалите узел из кластера

Выполните следующую команду, чтобы удалить отказавший узел. Замените `mynode` на фактическое имя узла:

```
kubectl delete node mynode
```

Если отказавший узел был узлом control plane, также удалите его etcd member из кластера etcd:

```
talm -f nodes/node1.yaml etcd member list
```

Пример вывода:

NODE	ID	HOSTNAME	PEER URLS	CLIENT URLS	STATUS
37.27.60.28	2ba6e48b8cf1a0c1	node1	https://192.168.100.11:2380	https://192.168.100.11:2379	HEALTHY
37.27.60.28	b82e2194fb76ee42	node2	https://192.168.100.12:2380	https://192.168.100.12:2379	HEALTHY
37.27.60.28	f24f4de3d01e5e88	node3	https://192.168.100.13:2380	https://192.168.100.13:2379	HEALTHY

Затем удалите соответствующий member. Замените ID на ID отказавшего узла:

```
talm -f nodes/node1.yaml etcd remove-member f24f4de3d01e5e88
```

## Шаг 2: удалите PVC и pod, привязанные к отказавшему узлу

Ниже несколько команд, которые помогут очистить отказавший узел:

- Удалите PVC, привязанные к отказавшему узлу:

(Замените `mynode` на имя отказавшего узла)

```
kubectl get pv -o json | jq -r '.items[] | select(.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExp...
```

- Удалите pod, зависшие в состоянии `Pending` во всех namespaces:

```
kubectl get pod -A | awk '/Pending/ {print "kubectl delete pod -n " $1 " " $2}' | sh -x
```

## Шаг 3: проверьте состояние ресурсов

После очистки проверьте проблемы с ресурсами с помощью `linstor advise`:

---

```
linstor advise resource
```

Resource	Issue	Possible f...
pvc-02b0c0a1-e0b6-4e98-9384-60ff24f3b3b6	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-06e3b406-23f0-4f10-8b03-84063c1b2a12	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-a0b8aeaf-076e-4bd9-93ed-c4db09c04d0b	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-a523ebeb-c3b6-468d-abe5-f6afbbf31081	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-cf7e87b5-3e6d-4034-903d-4625830fb5b4	Resource expected to have 1 replicas, got only 0.	linstor rd...
pvc-d344bc83-97fd-4489-bbe7-5399eea57165	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-d39345a9-5446-4c64-a5ba-957ff7c7a31f	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-db6d4236-93bd-4268-9dcc-0ed275b17067	Resource expected to have 1 replicas, got only 0.	linstor rd...
pvc-ebb412c3-083c-4eee-93dc-70917ea6d87e	Resource expected to have 1 replicas, got only 0.	linstor rd...
pvc-f107aacb-78d7-4ac6-97f8-8ed529a9c292	Resource expected to have 3 replicas, got only 2.	linstor rd...
pvc-f347d71a-b646-45e5-a717-f0a745061beb	Resource expected to have 1 replicas, got only 0.	linstor rd...
pvc-f6e96c83-6144-4510-b0ab-61936db52391	Resource expected to have 3 replicas, got only 2.	linstor rd...

Run the `linstor rd ap` commands suggested in the “Possible fix” column to restore the desired replica count.

# Как ротировать Certificate Authority

<https://cozystack.ru/docs/v1.5/operations/cluster/rotate-ca/>

Talos создает корневые центры сертификации со сроком действия 10 лет, и все сертификаты Talos API и Kubernetes API выпускаются этими корневыми СА. В обычной эксплуатации почти никогда не требуется ротировать корневой СА-сертификат и ключ для Talos API и Kubernetes API.

Ротация корневого СА нужна только в следующих случаях:

- есть подозрение, что private key был скомпрометирован;
- нужно отозвать доступ к кластеру для утекшего `talosconfig` или `kubeconfig`;
- прошло 10 лет.

## Ротация СА для Talos API

Чтобы ротировать Talos СА для management-кластера, используйте следующую команду.

Сначала запустите ее в dry-run-режиме, чтобы предварительно посмотреть изменения:

```
talml -f nodes/node.yaml rotate-ca --talos=true --kubernetes=false
```

Затем выполните реальную ротацию:

```
talml -f nodes/node.yaml rotate-ca --talos=true --kubernetes=false --dry-run=false
```

После завершения ротации скачайте новый `talosconfig` из secrets.

## Ротация СА для management-кластера Kubernetes

Чтобы ротировать Kubernetes СА для management-кластера, используйте следующую команду.

Сначала запустите ее в dry-run-режиме, чтобы предварительно посмотреть изменения:

```
talml -f nodes/node.yaml rotate-ca --talos=false --kubernetes=true
```

Затем выполните реальную ротацию:

```
talml -f nodes/node.yaml rotate-ca --talos=false --kubernetes=true --dry-run=false
```

## Ротация СА для tenant-кластера Kubernetes

См.: <https://kamaji.clastix.io/guides/certs-lifecycle/>

---

```
export NAME=k8s-cluster-name
export NAMESPACE=k8s-cluster-namespace

kubectl -n ${NAMESPACE} delete secret ${NAME}-ca
kubectl -n ${NAMESPACE} delete secret ${NAME}-sa-certificate

kubectl -n ${NAMESPACE} delete secret ${NAME}-api-server-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-api-server-kubelet-client-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-datastore-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-front-proxy-client-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-konnectivity-certificate

kubectl -n ${NAMESPACE} delete secret ${NAME}-admin-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-controller-manager-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-konnectivity-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-scheduler-kubeconfig

kubectl delete po -l app.kubernetes.io/name=kamaji -n cozy-kamaji
kubectl delete po -l app=${NAME}-kcsi-driver
```

Дождитесь перезапуска pod `virt-launcher-kubernetes-*`. После этого скачайте новый сертификат Kubernetes.

---



# Справочник cluster services

<https://cozystack.ru/docs/v1.5/operations/services/>

## Мониторинг

Система мониторинга в Cozystack предоставляет полноценную наблюдаемость для ресурсов системного уровня и уровня tenant. Она работает на двух основных уровнях: общекластерный мониторинг инфраструктурных компонентов и tenant-specific мониторинг пользовательских приложений и сервисов.

### Обзор архитектуры

- Системный уровень: мониторит основные компоненты Cozystack, Kubernetes-кластеры и базовую инфраструктуру.
- Уровень tenant: предоставляет изолированные monitoring stacks для каждого tenant, позволяя им мониторить свои приложения без взаимного влияния.

### Ключевые компоненты

- VMAgent: собирает метрики из разных источников и отправляет их в VictoriaMetrics.
- VMCluster: кластер VictoriaMetrics для хранения и запросов метрик.
- Grafana: инструмент визуализации и dashboards для метрик и логов.
- Alerta: система alerting для уведомлений на основе thresholds метрик.

### Потоки данных

Метрики идут от exporters, например node-exporters и kube-state-metrics, в VMAgent, который затем записывает их в VMCluster. Grafana запрашивает данные из VMCluster для визуализации, а Alerta обрабатывает alerts из VMCluster или других источников.

Подробную конфигурацию см. в [справочнике Monitoring Hub](#).

Cozystack включает ряд cluster services. Они разворачиваются через настройки tenant, а не через каталог приложений.

Каждый tenant может иметь собственную копию cluster service или использовать сервисы родительского tenant. Подробнее о механизме sharing services см. в [Tenant System](#).

## Object Storage

---

S3-compatible object storage в Cozystack на базе SeaweedFS и COSI

## Конфигурация Velero Backup

---

Настройка backup storage, strategies и BackupClasses для cluster backups (для cluster administrators).

## Конфигурация backup для managed applications

---

Настройка strategies и BackupClasses для logical data backups managed databases (Postgres, MariaDB, ClickHouse).

## Справочник сервиса BootBox

---

## Справочник сервиса Etcd

---

## Справочник Ingress-NGINX Controller

---

## Справочник сервиса SeaweedFS

---

## Справочник Monitoring Hub

---

---

# Object Storage

<https://cozystack.ru/docs/v1.5/operations/services/object-storage/>

Cozystack предоставляет S3-compatible object storage на базе [SeaweedFS](#) и [SeaweedFS COSI Driver](#).

## Как это работает

Object storage в Cozystack состоит из нескольких уровней:

1. SeaweedFS работает как cluster service в namespace tenant и предоставляет S3 backend.
2. Storage pools (опционально) разделяют volume servers по типу диска (SSD, HDD, NVMe), каждый pool создает собственный набор COSI resources.
3. BucketClasses определяют, как создаются buckets. Каждый pool создает стандартный BucketClass и BucketClass с суффиксом `-lock` для object locking.
4. BucketAccessClasses определяют credential policies: read-write и read-only.
5. Buckets - пользовательские ресурсы, которые запрашивают хранилище из BucketClass и создают credentials для каждого пользователя через BucketAccess.

```
SeaweedFS Cluster
 Storage Pool "ssd" (diskType: ssd)
 BucketClass: tenant-seaweedfs-ssd
 BucketClass: tenant-seaweedfs-ssd-lock
 BucketAccessClass: tenant-seaweedfs-ssd (readwrite)
 BucketAccessClass: tenant-seaweedfs-ssd-readonly (readonly)
 Storage Pool "hdd" (diskType: hdd)
 BucketClass: tenant-seaweedfs-hdd
 ...
```

## Руководства

- [Storage Pools](#) – настройка SeaweedFS storage pools для tiered storage
- [Buckets](#) – создание buckets и управление user credentials

## Справочник

- [SeaweedFS Service Reference](#) – autogenerated parameter table для SeaweedFS package
- [SeaweedFS Multi-DC Configuration](#) – развертывание SeaweedFS в нескольких дата-центрах

## Storage Pools

Настройка SeaweedFS storage пулов для tiered object storage

---

## Buckets и пользователи

---

Создание S3 buckets и управление учетными данными

---

# Storage Pools

<https://cozystack.ru/docs/v1.5/operations/services/object-storage/storage-pools/>

Storage пулы позволяют разделять SeaweedFS volume servers по типу диска. Каждый пул создает отдельный Volume StatefulSet с tag SeaweedFS `diskType` и соответствующий набор COSI resources (BucketClasses и BucketAccessClasses), на которые могут ссылаться buckets.

## Когда использовать пулы

Используйте storage пулы, когда в кластере доступны разные уровни хранения и требуется управлять тем, какой из них будет использоваться для бакета. Например, у вас могут быть быстрые NVMe-диски для часто запрашиваемых данных и большие HDD-диски для архивного хранения.

Если все volume servers используют одно и то же хранилище, пулы не нужны: достаточно BucketClass по умолчанию.

## Включение SeaweedFS для tenant

Перед настройкой пулов включите SeaweedFS для tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{"spec":{"seaweedfs": true}}'
```

Дождитесь готовности SeaweedFS HelmRelease:

```
kubectl -n tenant-root get hr seaweedfs
```

Ожидаемый вывод:

NAME	AGE	READY	STATUS
seaweedfs	2m	True	Helm upgrade succeeded for release tenant-root/seaweedfs.v1 with chart seaweedfs-...

## Конфигурация pool

Когда SeaweedFS запущен, пропатчите его HelmRelease, чтобы добавить storage пулы:

```
kubectl patch -n tenant-root helmreleases.helm.toolkit.fluxcd.io seaweedfs --type=merge -p '{"spec":{"value..."
```

Эквивалентный полный ресурс выглядит так:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
 name: seaweedfs
 namespace: tenant-example
spec:
 host: s3.example.com
 topology: Simple
 volume:
 replicas: 2
 size: 100Gi
 pools:
 ssd:
 diskType: ssd
 size: 50Gi
 storageClass: local-nvme
 hdd:
 diskType: hdd
 size: 500Gi
 storageClass: local-hdd
 replicas: 3
```

## Параметры pool

Параметр	Обязательный	Описание
diskType	Да	Tag типа диска SeaweedFS: lowercase alphanumeric, например <code>ssd</code> , <code>hdd</code> , <code>nvme</code>
replicas	Нет	Количество реплик volume server. По умолчанию <code>volume.replicas</code>
size	Нет	Размер PVC на реплику. По умолчанию <code>volume.size</code>
storageClass	Нет	Kubernetes StorageClass для PVC. По умолчанию <code>volume.storageClass</code>
resources	Нет	Явные CPU/memory limits. По умолчанию <code>volume.resources</code>
resourcesPreset	Нет	Sizing preset, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code>

## Правила именования

Имена пулов должны быть валидными DNS labels: строчные буквы, цифры и дефисы. Следующие суффиксы зарезервированы и не должны использоваться как имена пулов:

- Имена, заканчивающиеся на `-lock` (зарезервировано для object-lock BucketClasses)
- Имена, заканчивающиеся на `-readonly` (зарезервировано для read-only BucketAccessClasses)

## COSI resources, создаваемые для каждого pool

Каждый pool автоматически создает четыре COSI resources:

Ресурс	Шаблон имени	Назначение
BucketClass	{namespace}-{pool}	Стандартное создание bucket
BucketClass	{namespace}-{pool}-lock	Создание bucket с включенным object locking
BucketAccessClass	{namespace}-{pool}	Read-write credentials
BucketAccessClass	{namespace}-{pool}-readonly	Read-only credentials

Например, pool с именем `ssd` в namespace `tenant-example` создает:

- BucketClass `tenant-example-ssd`
- BucketClass `tenant-example-ssd-lock`
- BucketAccessClass `tenant-example-ssd`
- BucketAccessClass `tenant-example-ssd-readonly`

Note: Набор COSI resources по умолчанию без пула всегда создается с использованием только имени namespace, например `tenant-example`, `tenant-example-lock`. Он соответствует volume servers, работающим с настройкой верхнего уровня `volume.diskType`.

## MultiZone topology с пулами

В многозонной топологии пулы определяются отдельно для каждой зоны в `volume.zones[zone].pools`.

## Параметры зоны

Каждая зона, помимо пула, принимает следующие параметры:

Параметр	Обязательный	Описание
replicas	Нет	Количество реплик volume server в этой зоне. По умолчанию <code>volume.replicas</code>
size	Нет	Размер PVC на реплику. По умолчанию <code>volume.size</code>

storageClass	Нет	Kubernetes StorageClass для PVC. По умолчанию <code>volume.storageClass</code>
dataCenter	Нет	Имя data center в SeaweedFS. По умолчанию имя ключа <code>zone</code> , например <code>zone dc1</code> получает <code>dataCenter: dc1</code>
nodeSelector	Нет	YAML nodeSelector для планирования pods сервера <code>volume</code> . По умолчанию <code>topology.kubernetes.io/zone: &lt;zoneName&gt;</code>
pools	Нет	сопоставление пулов хранения для этой зоны. Структура такая же, как у <code>volume.pools</code>

## Пример

Пропатчите SeaweedFS HelmRelease, чтобы добавить пулы для каждой зоны:

```
kubectl patch -n tenant-example helmreleases.helm.toolkit.fluxcd.io seaweedfs --type=merge -p '{"spec":{"va...
```

В этом примере `zone dc1` автоматически получает `dataCenter: dc1` и `nodeSelector: {topology.kubernetes.io/zone: dc1}`.

Чтобы переопределить значения по умолчанию, задайте их явно:

```
volume:
 zones:
 us-east-1a:
 dataCenter: us-east
 nodeSelector:
 topology.kubernetes.io/zone: us-east-1a
 node.kubernetes.io/instance-type: storage-optimized
 pools:
 ssd:
 diskType: ssd
 size: 50Gi
```

Эквивалентный полный ресурс с явными настройками `zones`:



```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
 name: seaweedfs
 namespace: tenant-example
spec:
 host: s3.example.com
 topology: MultiZone
 volume:
 replicas: 2
 size: 100Gi
 zones:
 dc1:
 pools:
 ssd:
 diskType: ssd
 size: 50Gi
 hdd:
 diskType: hdd
 size: 500Gi
 dc2:
 pools:
 ssd:
 diskType: ssd
 size: 50Gi
 hdd:
 diskType: hdd
 size: 500Gi
```

Каждая комбинация зона+пул создает собственный Volume StatefulSet. В этом примере это четыре StatefulSets: `seaweedfs-volume-dc1-ssd`, `seaweedfs-volume-dc1-hdd`, `seaweedfs-volume-dc2-ssd` и `seaweedfs-volume-dc2-hdd`.

COSI resources дедуплицируются между зонами: если `dc1`, и `dc2` определяют `pool ssd` с одинаковым `diskType`, создается только один набор ресурсов `BucketClass/BucketAccessClass`.

`volume.pools` верхнего уровня не разрешен в `MultiZone topology`. Вместо этого определяйте пулы внутри каждой зоны.

## Проверка

После развертывания SeaweedFS с пулами проверьте ресурсы:

```
Проверьте, что volume server StatefulSets созданы для каждого pool
kubectl get statefulset -n tenant-example -l app.kubernetes.io/name=seaweedfs

Проверьте BucketClasses
kubectl get bucketclass

Проверьте BucketAccessClasses
kubectl get bucketaccessclass
```

Вы должны увидеть ресурсы `BucketClass` и `BucketAccessClass` для каждого имени `pool`.

---

## Связанная документация

---

- [Buckets](#) – создание buckets, использующих конкретный storage pool
  - [SeaweedFS Service Reference](#) – полный справочник параметров
  - [SeaweedFS Multi-DC Configuration](#) – руководство по multi-DC развертыванию
-

# Buckets и пользователи

<https://cozystack.ru/docs/v1.5/operations/services/object-storage/buckets/>

Приложение Bucket создает S3 bucket через COSI и подготавливает учетные данные для каждого пользователя в виде Kubernetes Secrets.

## Создание Bucket

Минимальный bucket использует BucketClass по умолчанию и не создает пользователей:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: my-bucket
 namespace: tenant-example
spec: {}
```

Это создает BucketClaim в BucketClass по умолчанию (`tenant-example`). Чтобы bucket был полезен, добавьте хотя бы одного пользователя (см. [Пользователи](#) ниже).

## Выбор Storage Pool

Если экземпляр SeaweedFS определяет [storage pools](#), используйте поле `storagePool`, чтобы выбрать конкретный pool:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: my-bucket
 namespace: tenant-example
spec:
 storagePool: ssd
```

Это создает BucketClaim в BucketClass `tenant-example-ssd`.

Когда `storagePool` пустой (значение по умолчанию), bucket использует BucketClass по умолчанию.

## Object Locking

Чтобы создать bucket с включенной блокировкой объектов, задайте `locking: true`:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: my-bucket
 namespace: tenant-example
spec:
 storagePool: ssd
 locking: true
```

Это создает BucketClaim в BucketClass с суффиксом `-lock`, например `tenant-example-ssd-lock`. BucketClasses с включенным блокировкой используют политику удаления `Retain` и настраивают блокировку объектов в режиме COMPLIANCE с периодом хранения по умолчанию.

Note: Object locking нельзя включить или отключить после создания bucket. Если нужно изменить эту настройку, создайте новый bucket.

## Пользователи

Map `users` определяет именованных S3-пользователей для bucket. Каждая запись создает ресурс COSI BucketAccess и соответствующий Kubernetes Secret.

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: my-bucket
 namespace: tenant-example
spec:
 storagePool: ssd
 users:
 admin: {}
 reader:
 readonly: true
```

Это создает двух пользователей:

Имя пользователя	Роль	BucketAccessClass	Секрет
admin	read-write	tenant-example-ssd	my-bucket-admin
reader	read-only	tenant-example-ssd-readonly	my-bucket-reader

## Параметры пользователя

Параметр	Тип	По умолчанию	Описание
readonly	bool	false	При true создает учетные данные из BucketAccessClass с суффиксом <code>-readonly</code>

## Доступ к учетным данным

Каждый пользователь получает Kubernetes Secret с именем `{bucket-name}-{username}` в том же namespace. Secret содержит S3 учетные данные, созданные COSI driver:

```
kubectrl get secret my-bucket-admin -n tenant-example -o yaml
```

Secret содержит поля, необходимые для настройки S3 client: endpoint, access key, secret key. Точный набор полей зависит от реализации COSI driver.

## Ротация учетных данных

Учетные данные пользователя bucket (access key и secret key) генерируются один раз при первом создании пользователя и не могут быть обновлены на месте. Чтобы ротировать учетные данные пользователя, удалите пользователя из map `users` и примените изменения, затем добавьте пользователя обратно и примените изменения еще раз:

```
Шаг 1: удалите пользователя, чтобы удалить существующие учетные данные
spec:
 users: {}
```

```
Шаг 2: добавьте пользователя обратно, чтобы создать новый набор учетных данных
spec:
 users:
 admin: {}
```

Warning: Все приложения, использующие старые учетные данные, потеряют доступ между шагом 1 и шагом 2. После завершения шага 2 обновите приложения, указав новые учетные данные из Secret.

## Логика выбора BucketClass

Имя BucketClass составляется из трех частей:

storagePool	locking	Результирующий BucketClass
(empty)	false	tenant-example
(empty)	true	tenant-example-lock
ssd	false	tenant-example-ssd
ssd	true	tenant-example-ssd-lock

---

Аналогично составляется BucketAccessClass.

## Полный пример

---

Разверните bucket в pool `ssd` с одним admin-пользователем и одним read-only пользователем:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: media-assets
 namespace: tenant-example
spec:
 storagePool: ssd
 locking: false
 users:
 app:
 readonly: false
 backup-reader:
 readonly: true
```

После создания bucket получите учетные данные:

```
Read-write учетные данные для пользователя "app"
kubectl get secret media-assets-app -n tenant-example \
 -o jsonpath='{.data}' | jq 'map_values(@base64d)'

Read-only учетные данные для пользователя "backup-reader"
kubectl get secret media-assets-backup-reader -n tenant-example \
 -o jsonpath='{.data}' | jq 'map_values(@base64d)'
```

## Связанная документация

---

- [Storage Pools](#) – настройка tiered storage для выбора pool
  - [SeaweedFS Service Reference](#) – полный справочник параметров
-

# Конфигурация Velero Backup

<https://cozystack.ru/docs/v1.5/operations/services/velero-backup-configuration/>

Это руководство предназначено для cluster administrators, которые настраивают backup-инфраструктуру в Cozystack: S3 storage, Velero locations, backup strategies и BackupClasses. Затем tenant users используют существующие BackupClasses для создания BackupJobs и Plans.

Эта страница описывает backups, выполняемые через Velero, которые объединяют HelmRelease приложения, CRs и PVC snapshots. Такая модель используется для VMInstance / VMDisk. Для data-only backups managed databases (Postres, MariaDB, ClickHouse, FoundationDB), выполняемых нативным механизмом соответствующего оператора, см. [Managed Application Backup Configuration](#).

## Предварительные требования

- Административный доступ к Cozystack (management) cluster.
- S3-compatible storage: если вы хотите хранить backups в Cozy, включите SeaweedFS и создайте Bucket или используйте внешний S3 service.
- Включите отключенный по умолчанию компонент `cozystack.velero` в `bundles.enabledPackages Platform Package`. Для tenant clusters задайте `spec.addons.velero.enabled` равным `true` в ресурсе Kubernetes.

## 1. Настройте storage credentials и конфигурацию

Создайте следующие ресурсы в management cluster в namespace `cozy-velero`, чтобы Velero мог хранить backups и volume snapshots.

### 1.1 Создайте secret с S3 credentials

```
apiVersion: v1
kind: Secret
metadata:
 name: s3-credentials
 namespace: cozy-velero
type: Opaque
stringData:
 cloud: |
 [default]
 aws_access_key_id=<KEY>
 aws_secret_access_key=<SECRET KEY>

 services = seaweed-s3
 [services seaweed-s3]
 s3 =
 endpoint_url = https://s3.tenant-name.cozystack.example.com
```

## 1.2 Настройте BackupStorageLocation

Этот ресурс определяет, где Velero хранит backups (S3 bucket).

```
apiVersion: velero.io/v1
kind: BackupStorageLocation
metadata:
 name: default
 namespace: cozy-velero
spec:
 provider: aws
 objectStorage:
 bucket: <BUCKET_NAME>
 config:
 checksumAlgorithm: ''
 profile: "default"
 s3ForcePathStyle: "true"
 s3Url: https://s3.tenant-name.cozystack.example.com
 credential:
 name: s3-credentials
 key: cloud
```

`BUCKET_NAME` можно найти командой:

```
kubectl get bucketclaim -A -o custom-columns=NAME:.metadata.name,NAMESPACE:.metadata.namespace,BUCKET_NAME:...
```

См. [BackupStorageLocation](#) в документации Velero.

Проверьте, что ресурс успешно создан:

```
k get BackupStorageLocation -n cozy-velero
```

Вывод должен быть похож на:

NAME	PHASE	LAST VALIDATED	AGE	DEFAULT
default	Available	5s	3d9h	true

## 1.3 Настройте VolumeSnapshotLocation

Этот ресурс определяет конфигурацию volume snapshots.

```
apiVersion: velero.io/v1
kind: VolumeSnapshotLocation
metadata:
 name: default
 namespace: cozy-velero
spec:
 provider: aws
 credential:
 name: s3-credentials
 key: cloud
 config:
 region: "us-west-2"
 profile: "default"
```



---

См. [VolumeSnapshotLocation](#) в документации Velero.

## 2. Определите backup strategy

---

Strategy описывает template [Velero Backup](#). Это переиспользуемый template, на который ссылаются BackupClasses.

В strategy задаются:

- Scope: namespaces и resources, например tenant namespace или resources по label.
- Volume handling: делать ли snapshot volumes и использовать ли `snapshotMoveData`.
- Retention: backup TTL по умолчанию.

Проверьте CRD group, version и kind в вашем кластере:

```
kubectl get crd | grep -i backup
kubectl explain <strategy-kind> --recursive
```

Пример strategy для VMInstance (включает все VM resources и подключенные volumes):

```

apiVersion: strategy.backups.cozystack.io/v1alpha1
kind: Velero
metadata:
 name: vminstance-strategy
spec:
 template:
 restoreSpec:
 existingResourcePolicy: update
 includedNamespaces:
 - '{{ .Application.metadata.namespace }}'
 orLabelSelectors:
 - matchLabels:
 app.kubernetes.io/instance: 'vm-instance-{{ .Application.metadata.name }}'
 - matchLabels:
 apps.cozystack.io/application.kind: '{{ .Application.kind }}'
 apps.cozystack.io/application.name: '{{ .Application.metadata.name }}'
 includedResources:
 - helmreleases.helm.toolkit.fluxcd.io
 - virtualmachines.kubevirt.io
 - virtualmachineinstances.kubevirt.io
 - pods
 - persistentvolumeclaims
 - configmaps
 - secrets
 - controllerrevisions.apps
 includeClusterResources: false
 excludedResources:
 - datavolumes.cdi.kubevirt.io
 spec:
 includedNamespaces:
 - '{{ .Application.metadata.namespace }}'
 orLabelSelectors:
 - matchLabels:
 app.kubernetes.io/instance: 'vm-instance-{{ .Application.metadata.name }}'
 - matchLabels:
 apps.cozystack.io/application.kind: '{{ .Application.kind }}'
 apps.cozystack.io/application.name: '{{ .Application.metadata.name }}'
 includedResources:
 - helmreleases.helm.toolkit.fluxcd.io
 - virtualmachines.kubevirt.io
 - virtualmachineinstances.kubevirt.io
 - pods
 - datavolumes.cdi.kubevirt.io
 - persistentvolumeclaims
 - configmaps
 - secrets
 - controllerrevisions.apps
 includeClusterResources: false
 storageLocation: '{{ .Parameters.backupStorageLocationName }}'
 volumeSnapshotLocations:
 - '{{ .Parameters.backupStorageLocationName }}'
 snapshotVolumes: true
 snapshotMoveData: true
 ttl: 720h0m0s
 itemOperationTimeout: 24h0m0s

```

Пример strategy для VMDisk (только disk и его volume):

```

apiVersion: strategy.backups.cozystack.io/v1alpha1
kind: Velero
metadata:
 name: vmdisk-strategy
spec:
 template:
 restoreSpec:
 existingResourcePolicy: update
 includedNamespaces:
 - '{{ .Application.metadata.namespace }}'
 orLabelSelectors:
 - matchLabels:
 app.kubernetes.io/instance: 'vm-disk-{{ .Application.metadata.name }}'
 - matchLabels:
 apps.cozystack.io/application.kind: '{{ .Application.kind }}'
 apps.cozystack.io/application.name: '{{ .Application.metadata.name }}'
 includedResources:
 - helmreleases.helm.toolkit.fluxcd.io
 - persistentvolumeclaims
 - configmaps
 includeClusterResources: false
 spec:
 includedNamespaces:
 - '{{ .Application.metadata.namespace }}'
 orLabelSelectors:
 - matchLabels:
 app.kubernetes.io/instance: 'vm-disk-{{ .Application.metadata.name }}'
 - matchLabels:
 apps.cozystack.io/application.kind: '{{ .Application.kind }}'
 apps.cozystack.io/application.name: '{{ .Application.metadata.name }}'
 includedResources:
 - helmreleases.helm.toolkit.fluxcd.io
 - persistentvolumeclaims
 - configmaps
 includeClusterResources: false
 storageLocation: '{{ .Parameters.backupStorageLocationName }}'
 volumeSnapshotLocations:
 - '{{ .Parameters.backupStorageLocationName }}'
 snapshotVolumes: true
 snapshotMoveData: true
 ttl: 720h0m0s
 itemOperationTimeout: 24h0m0s

```

Template variables (`{{ .Application.* }}` и `{{ .Parameters.* }}`) берутся из ApplicationRef в BackupJob/Plan и параметров, заданных в BackupClass.

Не забудьте применить strategy в management cluster:

```
kubectl apply -f velero-backup-strategy.yaml
```

### 3. Создайте BackupClass

BackupClass связывает strategy с приложениями; в нем можно задать параметры.

Проверьте CRD BackupClass в кластере:

```
kubectl get backupclasses
kubectl explain backupclasses.spec --recursive
```

```
apiVersion: backups.cozystack.io/v1alpha1
kind: BackupClass
metadata:
 name: velero
spec:
 strategies:
 - strategyRef:
 apiGroup: strategy.backups.cozystack.io
 kind: Velero
 name: vminstance-strategy
 application:
 kind: VMInstance
 apiGroup: apps.cozystack.io
 parameters:
 backupStorageLocationName: default
 - strategyRef:
 apiGroup: strategy.backups.cozystack.io
 kind: Velero
 name: vmdisk-strategy
 application:
 kind: VMDisk
 apiGroup: apps.cozystack.io
 parameters:
 backupStorageLocationName: default
```

Примените и выведите список:

```
kubectl apply -f backupclass.yaml
kubectl get backupclasses
```

## 4. Как пользователи запускают backups

Когда strategies и BackupClasses настроены, tenant users могут запускать backups, не меняя Velero или storage configuration:

- One-off backup: создайте [BackupJob](#), который ссылается на BackupClass.
- Scheduled backups: создайте [Plan](#) с cron schedule и ссылкой на BackupClass.

Прямое использование Velero CRDs ([Backup](#), [Schedule](#), [Restore](#)) остается доступным для advanced или recovery scenarios:

```
kubectl get backup.velero.io -n cozy-velero
kubectl get schedule.velero.io -n cozy-velero
kubectl get restores.velero.io -n cozy-velero
```

Если установлен [Velero CLI](#), можно также выполнить:

---

```
velero -n cozy-velero backup get
velero -n cozy-velero schedule get
velero -n cozy-velero restore get
```

Чтобы посмотреть Velero logs, используйте команду:

```
kubectl logs -n cozy-velero -l app.kubernetes.io/name=velero --tail=100
```

## 5. Восстановление из backup

---

Когда strategies и BackupClasses настроены, tenant users могут восстанавливаться из backup с помощью RestoreJob. Инструкции по restore для in-place восстановления VMInstance и VMDisk см. в руководстве [Backup and Recovery](#).

---

# Конфигурация backup для managed applications

<https://cozystack.ru/docs/v1.5/operations/services/managed-app-backup-configuration/>

Это руководство предназначено для cluster administrators, которые настраивают backup strategies для database applications, управляемых Cozystack: Postgres, MariaDB и ClickHouse. После настройки strategies и ресурсов BackupClass tenants запускают backups и restores, создавая ресурсы [BackupJob](#), [Plan](#) и [RestoreJob](#), без дополнительных действий администратора.

Эта страница описывает data-only backups, выполняемые нативным backup-механизмом каждого оператора (CloudNativePG barman, dumps mariadb-operator, Altinity clickhouse-backup). apps.cozystack.io/\* CR, его HelmRelease, chart values и Secrets, управляемые оператором, не попадают в эти strategies.

>

Для backups, которые объединяют Helm release + CRs + PVC snapshots (используются VMInstance / VMDisk), см. [Velero Backup Configuration](#).

## Предварительные требования

- Административный доступ к Cozystack (management) cluster.
- Компоненты backup-controller и backupstrategy-controller установлены и работают.
- S3-compatible storage доступен из management cluster: либо внутрикластерный SeaweedFS, созданный через приложение Bucket, либо любой внешний S3 endpoint.
- Для каждого application Kind, который нужно backup-ить, развернут соответствующий upstream operator.

## Как работает strategy для managed application

Последовательность для каждого BackupJob:

1. Tenant создает BackupJob (или Plan, который создает его по cron), ссылающийся на BackupClass и приложение apps.cozystack.io/<Kind>.
2. Core backup controller разрешает BackupClass и сопоставляет application Kind с driver-specific strategy strategy.backups.cozystack.io/<Kind>.
3. Driver рендерит template strategy относительно live application object (.Application) и параметров BackupClass (.Parameters), затем создает operator-native backup CR (Backup для mariadb, HTTP-вызов in-pod sidecar для ClickHouse, barman-driven snapshot в cnpg.io для Postgres).
4. При успехе driver создает артефакт Cozystack Backup в том же namespace; позже ресурсы RestoreJob ссылаются на этот артефакт.

BackupClass является cluster-scoped: один экземпляр покрывает все tenant namespaces.

---

Tenant users не могут просматривать ресурсы `BackupClass` через свой `kubeconfig` (cluster-scoped resources недоступны через tenant `RoleBinding`). После создания `BackupClass` сообщите его имя tenants вне Kubernetes: в `platform handbook`, в ticket на onboarding приложения или во внутреннем Slack-канале. Tenants указывают это имя дословно в `BackupJob.spec.backupClassName`.

## Настройка по драйверам

---

Strategies ниже написаны для внутрикластерного приложения `SeaweedFS Bucket`. Если используется external S3 storage, удалите секции `endpointCA` / TLS и укажите endpoint вашего provider.

## Postgres (CNPG strategy)

---

CNPG driver делегирует работу нативному barman backup CloudNativePG. Каждый `BackupJob` - это barman snapshot, отправляемый в S3; `RestoreJob` пересоздает `cnpg.io/Cluster` из архива.

Создайте strategy:

```
apiVersion: strategy.backups.cozystack.io/v1alpha1
kind: CNPG
metadata:
 name: postgres-data-cnpg-strategy
spec:
 template:
 serverName: "{{ .Application.metadata.name }}"
 barmanObjectStore:
 destinationPath: "s3://REPLACE_WITH_COSI_BUCKET_NAME/{{ .Application.metadata.name }}"
 endpointURL: "https://REPLACE_WITH_S3_ENDPOINT"
 retentionPolicy: "30d"
 endpointCA:
 secretRef:
 name: "{{ .Application.metadata.name }}-cnpg-backup-ca"
 key: "ca.crt"
 s3Credentials:
 secretRef:
 name: "{{ .Application.metadata.name }}-cnpg-backup-creds"
 data:
 compression: gzip
 wal:
 compression: gzip
```

Свяжите application Kind:

```

apiVersion: backups.cozystack.io/v1alpha1
kind: BackupClass
metadata:
 name: postgres-data-backup
spec:
 strategies:
 - application:
 apiGroup: apps.cozystack.io
 kind: Postgres
 strategyRef:
 apiGroup: strategy.backups.cozystack.io
 kind: CNPG
 name: postgres-data-cnpg-strategy

```

Per-application Secrets, которые tenant должен создать в application namespace:

Secret	Keys	Назначение
<app>-cnpg-backup-creds	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY	S3 credentials, используемые barman
<app>-cnpg-backup-ca (только для self-signed endpoints)	ca.crt	CA bundle, которому доверяет barman client

Удалите блок `endpointCA` из strategy, если S3 endpoint использует publicly-trusted certificate.

## MariaDB

MariaDB driver делегирует работу [mariadb-operator](#). Backups создаются как CRs `k8s.mariadb.com/v1alpha1 Backup` (logical `mariadb-dump`); restores создаются как CRs `Restore`, которые через `mariadb-import` возвращают dump в live database.

Создайте strategy:



```

apiVersion: strategy.backups.cozystack.io/v1alpha1
kind: MariaDB
metadata:
 name: mariadb-data-strategy
spec:
 template:
 storage:
 s3:
 bucket: "REPLACE_WITH_COSI_BUCKET_NAME"
 endpoint: "REPLACE_WITH_S3_ENDPOINT"
 prefix: "{{ .Application.metadata.name }}/"
 accessKeyIdSecretKeyRef:
 name: "{{ .Application.metadata.name }}-mariadb-backup-creds"
 key: AWS_ACCESS_KEY_ID
 secretAccessKeySecretKeyRef:
 name: "{{ .Application.metadata.name }}-mariadb-backup-creds"
 key: AWS_SECRET_ACCESS_KEY
 tls:
 enabled: true
 caSecretKeyRef:
 name: "{{ .Application.metadata.name }}-mariadb-backup-ca"
 key: ca.crt
 compression: gzip

```

`endpoint` указывается как path-style без scheme (например, `seaweedfs-s3.<seaweedfs-namespace>.svc:8333` для внутрикластерного SeaweedFS по умолчанию; подставьте namespace, где SeaweedFS развернут в вашем кластере). Удалите блок `tls`, если endpoint использует publicly-trusted certificate.

Свяжите application Kind:

```

apiVersion: backups.cozystack.io/v1alpha1
kind: BackupClass
metadata:
 name: mariadb-data-backup
spec:
 strategies:
 - application:
 apiGroup: apps.cozystack.io
 kind: MariaDB
 strategyRef:
 apiGroup: strategy.backups.cozystack.io
 kind: MariaDB
 name: mariadb-data-strategy

```

Per-application Secrets, которые tenant должен создать в application namespace:

Secret	Keys	Назначение
<code>&lt;app&gt;-mariadb-backup-creds</code>	<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code>	S3 credentials, используемые mariadb-operator
<code>&lt;app&gt;-mariadb-backup-ca</code> (только для self-signed endpoints)	<code>ca.crt</code>	CA bundle для TLS verification

Блок `backup.*` уровня `chart` в `apps.cozystack.io/MariaDB` (legacy путь `mariadb-dump + restic`) deprecated в пользу этого BackupClass flow. У существующих tenants с `backup.enabled=true` legacy resources продолжают рендериться без изменений.

## ClickHouse (Altinity strategy)

Altinity driver не template-ит backup CR. Он рендерит небольшой `PodTemplateSpec`, который запускает `curl + jq` против in-pod HTTP API `clickhouse-backup` (port 7171), предоставляемого sidecar внутри каждого Pod `chi-*`.

Altinity strategy требует `**backup.enabled=true**` на каждом экземпляре ClickHouse application. Именно этот флаг создает in-pod sidecar и Secret `clickhouse-<release>-backup-api-auth`, через который аутентифицируется strategy. В отличие от MariaDB, блок `backup.*` уровня `chart` в ClickHouse не deprecated; BackupClass flow использует тот же sidecar.

Создайте strategy. `template` - это `PodTemplateSpec`, который управляет sidecar; полный reference template (с shell script, который делает `POST create_remote / restore_remote` и опрашивает action log) см. в [examples/backups/clickhouse/01-create-strategy.sh](#) в репозитории cozystack.

```
apiVersion: strategy.backups.cozystack.io/v1alpha1
kind: Altinity
metadata:
 name: clickhouse-data-altinity-strategy
spec:
 template:
 spec:
 restartPolicy: Never
 containers:
 - name: ch-backup-client
 image: alpine:3.19
 env:
 - name: API_USERNAME
 valueFrom:
 secretKeyRef:
 name: clickhouse-{{ .Release.Name }}-backup-api-auth
 key: username
 - name: API_PASSWORD
 valueFrom:
 secretKeyRef:
 name: clickhouse-{{ .Release.Name }}-backup-api-auth
 key: password
 command: ["/bin/sh", "-c"]
 args:
 # Полный script см. в examples/backups/clickhouse/01-create-strategy.sh:
 # ветвится по .Mode (backup|restore), делает POST /backup/create_remote
 # или /backup/restore_remote/<name>, затем опрашивает /backup/actions
 # до terminal status.
 - |
 # ... (см. пример в репозитории)
```

Свяжите application Kind. Параметры не требуются: strategy template обращается к sidecar через детерминированное имя сервиса, а sidecar берет S3 credentials из Secret `<release>-backup-s3`, созданного chart.

```
apiVersion: backups.cozystack.io/v1alpha1
kind: BackupClass
metadata:
 name: clickhouse-data-backup
spec:
 strategies:
 - application:
 apiGroup: apps.cozystack.io
 kind: ClickHouse
 strategyRef:
 apiGroup: strategy.backups.cozystack.io
 kind: Altinity
 name: clickhouse-data-altinity-strategy
```

## Применение и проверка

Примените manifests strategy и BackupClass:

```
kubectl apply -f <strategy>.yaml
kubectl apply -f <backupclass>.yaml
```

Выведите ресурсы:

```
kubectl get cnpgs.strategy.backups.cozystack.io
kubectl get mariadbs.strategy.backups.cozystack.io
kubectl get altinities.strategy.backups.cozystack.io
kubectl get backupclasses
```

У каждой strategy не должно быть error conditions; каждый BackupClass должен показывать заданные вами strategy entries.

## Onboarding tenant

Tenant users не могут создавать объекты Secret в стандартном RBAC Cozystack и не могут читать credential Secrets, созданные Bucket. Прежде чем tenant сможет запустить первый BackupJob, administrator должен подготовить per-tenant storage и per-application credential Secrets, которые ожидает driver.

Ниже мы используем tenant-user как namespace tenant и my-postgres / my-mariadb / my-clickhouse как имя приложения; замените их на свои значения.

## Подготовьте storage Bucket

Если у tenant нет external S3 coordinates, создайте внутрикластерный Bucket в его namespace:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: db-backups
 namespace: tenant-user
spec:
 users:
 backup:
 readonly: false
```

```
kubectl -n tenant-user wait hr/bucket-db-backups --for=condition=ready --timeout=2m
```

Bucket controller создает в namespace Secret `bucket-<name>-backup` с JSON blob `BucketInfo`; S3 endpoint, имя bucket и access keys берутся оттуда.

## Прочитайте bucket credentials

Выполните это один раз на shell session. Каждый per-driver блок ниже переиспользует `$ACCESS_KEY`, `$SECRET_KEY` и `/tmp/bucket.json`:

```
kubectl -n tenant-user get secret bucket-db-backups-backup -o jsonpath='{.data.BucketInfo}' | base64 -d > /...
ACCESS_KEY=$(jq -r .spec.secretS3.accessKeyID /tmp/bucket.json)
SECRET_KEY=$(jq -r .spec.secretS3.accessSecretKey /tmp/bucket.json)
```

## Создайте per-application credential Secrets

Каждый driver ожидает per-application credential Secrets в application namespace; strategy template обращается к ним по детерминированному имени (например, `{{ .Application.metadata.name }}`-...).

### Postgres (CNPG)

Запишите credentials в keys, которые ожидает barman client CNPG:

```
kubectl -n tenant-user create secret generic my-postgres-cnpg-backup-creds \
 --from-literal=AWS_ACCESS_KEY_ID=$ACCESS_KEY \
 --from-literal=AWS_SECRET_ACCESS_KEY=$SECRET_KEY
```

Если S3 endpoint использует self-signed certificate (поведение SeaweedFS по умолчанию), также создайте CA secret:

```
kubectl -n tenant-user create secret generic my-postgres-cnpg-backup-ca \
 --from-file=ca.crt=/path/to/ca.crt
```

### MariaDB

```
kubectl -n tenant-user create secret generic my-mariadb-mariadb-backup-creds \
--from-literal=AWS_ACCESS_KEY_ID=$ACCESS_KEY \
--from-literal=AWS_SECRET_ACCESS_KEY=$SECRET_KEY
```

Для self-signed endpoints аналогично добавьте my-mariadb-mariadb-backup-ca с ca.crt.

## ClickHouse

Для BackupClass flow дополнительный Secret не нужен. Altinity strategy напрямую читает S3 credentials из Secret <release>-backup-s3, созданного chart. Убедитесь, что backup.enabled: true задан на каждом экземпляре ClickHouse application, который tenant хочет backup-ить, и что блок backup.\* в application values содержит координаты bucket (см. [справочник приложения ClickHouse](#)).

## Передача tenants

Tenants запускают backups и restores по именам BackupClass, созданным выше, с помощью ресурсов BackupJob, Plan и RestoreJob. Дайте им руководство [Application Backup and Recovery](#); для работы с существующим BackupClass им не нужны admin permissions. Перед тем как отправить их к руководству:

1. Сообщите доступные имена BackupClass (tenants не могут вывести их список, потому что cluster-scoped resources недоступны через tenant RoleBinding).
2. Убедитесь, что для каждого managed application, который tenant хочет backup-ить, в его namespace уже существует per-application credential Secret, описанный в [Tenant onboarding](#).

## Tenant escalation: диагностика на стороне drivers

Если BackupJob или RestoreJob tenant завершается с phase: Failed, а status.message не указывает точную причину, tenant не может самостоятельно посмотреть operator-native CRs: его RBAC исключает cnpg.io, k8s.mariadb.com и subresource pods/log. Выполните эти команды от его имени, используя имя BackupJob, которое он вам передаст:

```
Postgres (CloudNativePG)
kubectl -n tenant-user get backups.cnpg.io -l backups.cozystack.io/owned-by.BackupJobName=<job-name> -o yaml

MariaDB
kubectl -n tenant-user get backups.k8s.mariadb.com,restores.k8s.mariadb.com -l backups.cozystack.io/owned-b...

ClickHouse - strategy работает как one-shot Pod, который обращается к in-pod sidecar
kubectl -n tenant-user logs -l backups.cozystack.io/owned-by.BackupJobName=<job-name>
```

Для удаления архивов ClickHouse tenant не может напрямую обратиться к HTTP API in-pod sidecar clickhouse-backup; по его запросу выполните exec в ClickHouse pod и вызовите DELETE /backup/<name>/remote на локальном sidecar (credentials находятся в Secret clickhouse-<release>-backup-api-auth, созданном chart).

# Справочник сервиса BootBox

<https://cozystack.ru/docs/v1.5/operations/services/bootbox/>

## Параметры

### Общие параметры

Имя	Описание	Тип	Значение
whitelistHTTP	Защищает HTTP, включая whitelist клиентских сетей.	bool	true
whitelist	Список клиентских сетей.	[]string	[]
machines	Конфигурация экземпляров физических машин.	[]object	[]
machines[i].hostname	Hostname.	string	""
machines[i].arch	Архитектура.	string	""
machines[i].ip	Конфигурация IP-адреса.	object	{}
machines[i].ip.address	IP-адрес.	string	""
machines[i].ip.gateway	IP gateway.	string	""
machines[i].ip.netmask	Netmask.	string	""
machines[i].leaseTime	Lease time.	int	0
machines[i].mac	MAC-адреса.	[]string	[]
machines[i].nameServers	Name servers.	[]string	[]
machines[i].timeServers	Time servers.	[]string	[]
machines[i].uefi	UEFI.	bool	false

Согласно [документации Cozystack](#), сервис BootBox предоставляет параметры для конфигурации белых списков HTTP и параметров физических машин.

# Справочник сервиса Etcd

<https://cozystack.ru/docs/v1.5/operations/services/etcd/>

## Backups

DEPRECATED: the `backup.*` values block (`backup.enabled`, `backup.schedule`, `backup.destinationPath`, ...) is superseded by the Cozystack BackupClass flow driven by the Etcd strategy in `strategy.backups.cozystack.io/v1alpha1`. New tenants should use a BackupJob / RestoreJob against an Etcd strategy + BackupClass instead — see `examples/backups/etcd/` for the end-to-end demo and `internal/backupcontroller/etcdstrategy_controller.go` for the driver. Existing tenants with `backup.enabled=true` continue to render the legacy EtcdBackupSchedule + S3 credentials Secret unchanged — the two flows coexist, the chart's scheduled backups have NOT stopped working, and the `backup.*`-conditional rendered output is identical to prior releases.

## Upgrade note: serverTrustedCASecret is now set unconditionally

This release adds `security.tls.serverTrustedCASecret: etcd-ca-tls` to the rendered EtcdCluster. The field is required so that the etcd-operator's `backup_agent` / `restore_agent` (v0.4.4+) trust the server cert when they connect from their own pod via `etcdctl` — without it the agent falls back to the system trust store and the backup/restore handshake fails with “certificate signed by unknown authority”. The chart reuses the existing `etcd-ca-tls` Secret that already issues every client/server cert today, so the field points at material that has been present in the namespace since the cluster was first deployed. On a Helm upgrade the etcd-operator observes the new field and starts enforcing server-cert verification on its own client connections; it does NOT roll the etcd member pods (the etcd container's TLS configuration was already authoritative). Tenants who rely on the backup-agent today will continue to need this field set, so leaving it on by default keeps the chart consistent with the documented backup path.

When `backup.enabled` is set to `true`, the chart renders an EtcdBackupSchedule (`etcd.aenix.io/v1alpha1`) and an S3 credentials Secret. The etcd-operator (v0.4.3+) reconciles the schedule into a CronJob that periodically snapshots the cluster to S3. This release bumps the bundled `packages/system/etcd-operator` chart from v0.4.3 to v0.4.5 (so the new `EtcdBackup.status.snapshot` field — added in v0.4.4 — and the `restore-agent` path fixes — added in v0.4.5 — are available to the strategy driver). The legacy `backup.enabled=true` path is unchanged by the bump and continues to function on v0.4.5 exactly as it did on v0.4.3.

Enabling backup requires the following fields to be explicitly set (defaults are empty strings so that missing values fail fast at template render time): `backup.s3AccessKey`, `backup.s3SecretKey`, `backup.destinationPath` (must start with `s3://` and have no `//` segments), and `backup.endpointURL`. S3 credentials passed through plain values end up in the HelmRelease manifest — for production deployments prefer an external secret management tool (ESO, Sealed Secrets, etc.) over committing the keys to Git.

---

Restore and ad-hoc backup: the primary supported path is the Cozystack BackupClass / BackupJob / RestoreJob flow described above and demonstrated end-to-end under [examples/backups/etcd/](#). The driver suspends this chart's `HelRelease` for the duration of an in-place restore, deletes the live `EtcdCluster`, and re-creates it with `spec.bootstrap.restore.source.s3` populated from the Backup artefact's coordinates. The upstream `EtcdCluster.spec.bootstrap` field and the one-shot `EtcdBackup` CR (v0.4.4) are NOT exposed through this chart's values themselves; tenants who need to bypass the BackupClass flow (e.g. an out-of-band recovery using a snapshot that has no Backup artefact) can hand-apply the corresponding custom resource manifest as an escape hatch.

## Parameters

---

### Common parameters

---

Name	Description	Type	Value
<code>size</code>	Persistent Volume size.	<code>quantity</code>	<code>4Gi</code>
<code>storageClass</code>	StorageClass used to store the data.	<code>string</code>	<code>""</code>
<code>replicas</code>	Number of etcd replicas.	<code>int</code>	<code>3</code>
<code>resources</code>	Resource configuration for etcd.	<code>object</code>	<code>{}</code>
<code>resources.cpu</code>	Number of CPU cores allocated.	<code>quantity</code>	<code>1000m</code>
<code>resources.memory</code>	Amount of memory allocated.	<code>quantity</code>	<code>512Mi</code>

### Backup parameters

---

Name	Description	Type	Value
------	-------------	------	-------



<code>backup</code>	DEPRECATED: Backup configuration. The chart still renders the legacy <code>EtcdBackupSchedule</code> when <code>backup.enabled=true</code> , but new tenants should drive backups through a <code>BackupClass</code> bound to the <code>Etcd</code> strategy ( <code>strategy.backups.cozystack.io/v1alpha1</code> ).	<code>object</code>	<code>{}</code>
<code>backup.enabled</code>	DEPRECATED: Enable scheduled S3 backups. Use a <code>BackupJob</code> against an <code>Etcd</code> strategy instead.	<code>bool</code>	<code>false</code>
<code>backup.schedule</code>	DEPRECATED: Cron schedule for automated backups. Use a <code>backups.cozystack.io/Plan</code> instead.	<code>string</code>	<code>0 2 * * *</code>
<code>backup.destinationPath</code>	DEPRECATED: Destination path for backups (e.g. <code>s3://bucket/path/</code> ).	<code>string</code>	<code>""</code>
<code>backup.endpointURL</code>	DEPRECATED: S3 endpoint URL for uploads.	<code>string</code>	<code>""</code>
<code>backup.region</code>	DEPRECATED: S3 region.	<code>string</code>	<code>""</code>
<code>backup.forcePathStyle</code>	DEPRECATED: Use path-style S3 URLs (required for MinIO and most S3-compatible providers).	<code>bool</code>	<code>true</code>
<code>backup.s3AccessKey</code>	DEPRECATED: Access key for S3 authentication.	<code>string</code>	<code>""</code>
<code>backup.s3SecretKey</code>	DEPRECATED: Secret key for S3 authentication.	<code>string</code>	<code>""</code>

---

<code>backup.successfulJobsHistoryLimit</code>	DEPRECATED: Number of successful backup jobs to retain.	<code>int</code>	3
<code>backup.failedJobsHistoryLimit</code>	DEPRECATED: Number of failed backup jobs to retain.	<code>int</code>	1

---

# Справочник Ingress-NGINX Controller

<https://cozystack.ru/docs/v1.5/operations/services/ingress/>

## Parameters

### Common parameters

Name	Description	Type	Value
replicas	Number of ingress-nginx replicas.	int	2
whitelist	List of client networks.	[]string	[]
cloudflareProxy	Restoring original visitor IPs when Cloudflare proxied is enabled.	bool	false

<code>proxyProtocol</code>	Enable PROXY-protocol (use-proxy-protocol) on a tenant Ingress application. The upstream L4 load balancer in front of ingress-nginx MUST already inject PROXY-protocol v1 headers, otherwise all traffic to this ingress breaks — including in-cluster and direct-Service (hairpin) access to its public hostnames. Mutually exclusive with <code>cloudflareProxy</code> . For the host (external-facing) ingress use the platform-level <code>publishing.proxyProtocol</code> switch instead; the per-app value is rejected there because it would bypass the ouroboros hairpin handling and the disable-hazard guard.	<code>bool</code>	<code>false</code>
<code>resources</code>	Explicit CPU and memory configuration for each ingress-nginx replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied.	<code>object</code>	<code>{}</code>
<code>resources.cpu</code>	CPU available to each replica.	<code>quantity</code>	<code>" "</code>
<code>resources.memory</code>	Memory (RAM) available to each replica.	<code>quantity</code>	<code>" "</code>
<code>resourcesPreset</code>	Default sizing preset used when <code>resources</code> is omitted.	<code>string</code>	<code>t1.micro</code>

# Справочник сервиса SeaweedFS

<https://cozystack.ru/docs/v1.5/operations/services/seaweedfs/>

## Параметры

### Общие параметры

Имя	Описание	Тип	Значение
host	Hostname для внешнего доступа к SeaweedFS (по умолчанию subdomain s3 для host tenant).	string	""
topology	Топология кластера SeaweedFS.	string	Simple
replicationFactor	Replication factor: количество реплик для каждого volume в кластере SeaweedFS.	int	2

### Конфигурация компонентов SeaweedFS

Имя	Описание	Тип	Значение
db	Конфигурация database.	object	{}
db.replicas	Количество реплик database.	int	2
db.size	Размер Persistent Volume.	quantity	10Gi
db.storageClass	StorageClass, используемый для хранения данных.	string	
db.resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из resourcesPreset.	object	{}

db.resources.cpu		quantity	
db.resources.memory		quantity	
db.resourcesPreset	Sizing preset по умолчанию, используется, когда <code>resources</code> не задан.	string	t1.small
master		object	{}
master.replicas		int	3
master.resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
master.resources.cpu		quantity	
master.resources.memory		quantity	
master.resourcesPreset	Sizing preset по умолчанию, используется, когда <code>resources</code> не задан.	string	t1.small
filer		object	{}
filer.replicas		int	2
filer.resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
filer.resources.cpu		quantity	
filer.resources.memory		quantity	
filer.resourcesPreset	Sizing preset по умолчанию, используется, когда <code>resources</code> не задан.	string	t1.small
filer.grpcHost		string	
filer.grpcPort		int	443
filer.whitelist		[]string	[]

volume		object	{}
volume.replicas		int	2
volume.size		quantity	10Gi
volume.storageClass		string	
volume.diskType		string	
volume.resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
volume.resources.cpu		quantity	
volume.resources.memory		quantity	
volume.resourcesPreset	Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан.	string	t1.small
volume.zones		map[string]object	{}
volume.zones[name].replicas		int	0
volume.zones[name].size		quantity	
volume.zones[name].dataCenter		string	
volume.zones[name].nodeSelector	YAML nodeSelector для этой zone (по умолчанию: <code>topology.kubernetes.io/zone: &lt;zone_name&gt;</code> ).	string	
volume.zones[name].storageClass		string	
volume.zones[name].pools		map[string]object	{}
volume.zones[name].pools[name].diskType		string	
volume.zones[name].pools[name].replicas		int	0

volume.zones[name].pools[name].size		quantity	
volume.zones[name].pools[name].storageClass		string	
volume.zones[name].pools[name].resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
volume.zones[name].pools[name].resources.cpu		quantity	
volume.zones[name].pools[name].resources.memory		quantity	
volume.zones[name].pools[name].resourcesPreset	Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code> .	string	{}
volume.pools		map[string]object	{}
volume.pools[name].diskType		string	
volume.pools[name].replicas		int	0
volume.pools[name].size		quantity	
volume.pools[name].storageClass		string	
volume.pools[name].resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
volume.pools[name].resources.cpu		quantity	
volume.pools[name].resources.memory		quantity	



volume.pools[name].resourcesPreset	Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code> .	string	{}
s3		object	{}
s3.replicas		int	2
s3.resources	Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> .	object	{}
s3.resources.cpu		quantity	
s3.resources.memory		quantity	
s3.resourcesPreset	Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан.	string	t1.small

# Справочник Monitoring Hub

<https://cozystack.ru/docs/v1.5/operations/services/monitoring/>

## Архитектура потоков данных

[!Архитектура потоков данных](#)

### Описание компонентов

- VMAgent: легкий agent, который собирает metrics из разных источников и отправляет их в VictoriaMetrics.
- VMCluster: кластер VictoriaMetrics, который хранит и обрабатывает time-series data для эффективных queries.
- Grafana: open-source платформа для monitoring и observability с настраиваемыми dashboards.
- Alerta: alerting system, которая обрабатывает и управляет alerts из monitoring systems.
- Fluent Bit: быстрый и легкий log processor и forwarder.
- VLogs: VictoriaLogs, высокопроизводительная система управления logs для хранения и запросов.

## Архитектура визуализации

[!Архитектура визуализации](#)

### Описание компонентов визуализации

- VictoriaMetrics VMCluster: основное хранилище metrics и query engine, предоставляющий данные Grafana.
- VLogs: система VictoriaLogs для интеграции log data в visualizations.
- External Prometheus: дополнительные sources metrics, которые можно интегрировать.
- Custom Application Metrics: пользовательские metrics из приложений.
- Grafana: платформа визуализации, которая отображает dashboards.
- Pre-built Dashboards: стандартные dashboards для типовых monitoring views.
- Custom Dashboards: созданные пользователями dashboards с разными типами panels.
- Visualization: итоговый вывод с metrics вроде CPU, memory и network usage.

## Архитектура мониторинга

[!Архитектура мониторинга](#)

---

## Описание компонентов архитектуры мониторинга

---

- Kubernetes Cluster: основная платформа, где работают workloads и доступны metrics endpoints.
- Applications: пользовательские приложения, предоставляющие custom metrics.
- Infrastructure: базовое оборудование и системные metrics.
- VMAgent: собирает metrics из разных sources и отправляет их в VictoriaMetrics.
- VictoriaMetrics VMCluster: хранит и обрабатывает time-series metrics data.
- Grafana: предоставляет visualization и dashboarding.
- Alerta: управляет alerting и notifications.
- Dashboards & Visualizations: user interfaces для monitoring data.
- Alerts & Notifications: система уведомления operators о проблемах.

## Поток alerting

---

!Поток alerting

## Описание компонентов потока alerting

---

- Metrics Collection: сбор metrics из sources через VMAgent.
- VictoriaMetrics VMCluster: хранение и querying metrics data.
- Alert Rules Evaluation: проверка metrics по заранее заданным thresholds.
- Generate Alert: создание alert instances при выполнении условий.
- Alerta Alert Manager: обработка и управление alerts.
- Grouping & Deduplication: организация alerts для устранения duplicates.
- Routing & Notification: направление alerts в нужные channels.
- Email/SMS/Slack/etc.: конечные способы доставки notifications.

## Архитектура логирования

---

!Архитектура логирования

## Описание компонентов архитектуры логирования

---

- Application Logs: logs, генерируемые пользовательскими приложениями.
- Kubernetes Container Logs: logs контейнеров, работающих в Kubernetes.
- System Logs: infrastructure и system-level logs.
- Fluent Bit: легкий log processor, который собирает и пересылает logs.
- VictoriaLogs VLogs: высокопроизводительная система хранения и querying logs.
- Grafana Log Panels: интеграция для визуализации logs в Grafana dashboards.
- Log Visualization & Search: interfaces для просмотра и поиска log data.

- 
- Log Queries & Analysis: tools для querying и анализа log information.
- 

## Параметры мониторинга

---

Настройка и управление параметрами мониторинга в Cozystack.

## Настройка мониторинга

---

Руководство по установке и настройке мониторинга в Cozystack

## Monitoring Dashboards

---

Как визуализировать метрики и создавать custom dashboards в Grafana для мониторинга кластеров и приложений Cozystack.

## Логи мониторинга

---

Как собирать, хранить, искать и анализировать логи в Cozystack с помощью Fluent Bit и VictoriaLogs для полноценной наблюдаемости.

## Сбор custom metrics

---

Подключение собственных Prometheus exporters к tenant monitoring stack Cozystack с помощью VMServiceScrape и VMPodScrape.

## Alerting в мониторинге

---

Настройка и управление alerts в системе мониторинга Cozystack с помощью Alerta и Alertmanager.

---

---

# Использование OpenID Connect с Cozystack

<https://cozystack.ru/docs/v1.5/operations/oidc/>

## Включение OIDC Server

---

Как включить OIDC Server

## Создание пользователей и назначение ролей

---

Как создавать пользователей и назначать им роли

## Self-Signed Certificates

---

Как настроить OIDC с self-signed certificates

## Identity providers

---

Управление identity providers.

---

# Включение OIDC Server

[https://cozystack.ru/docs/v1.5/operations/oidc/enable\\_oidc/](https://cozystack.ru/docs/v1.5/operations/oidc/enable_oidc/)

## Предварительные требования

1. Конфигурация OIDC API server должен быть настроен на использование OIDC. Если используется Talos Linux, machine configuration должна включать следующие параметры:

```
cluster:
 apiServer:
 extraArgs:
 oidc-issuer-url: "https://keycloak.example.org/realms/cozy"
 oidc-client-id: "kubernetes"
 oidc-username-claim: "preferred_username"
 oidc-groups-claim: "groups"
```

Для Talm Добавьте в `values.yaml` в репозитории `talm`:

```
oidcIssuerUrl: "https://keycloak.<YOUR_ROOT_DOMAIN>/realms/cozy"
```

2. Доступность домена Убедитесь, что домен `keycloak.example.org` доступен из кластера и резолвится в root ingress controller.
3. Конфигурация storage Storage должен быть корректно настроен.

## Конфигурация

Если все предварительные требования выполнены, можно переходить к настройке.

## Шаг 1: включите OIDC в Cozystack

Пропатчите Platform Package, чтобы включить OIDC. Это также автоматически опубликует сервис Keycloak:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
 "spec": {
 "components": {
 "platform": {
 "values": {
 "authentication": {
 "oidc": {
 "enabled": true
 }
 }
 }
 }
 }
 }
}'
```

Если нужно добавить дополнительные redirect URLs для dashboard client (например, при доступе к dashboard через port-forwarding), пропатчите Platform Package. Несколько redirect URLs разделяются запятыми.

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
 "spec": {
 "components": {
 "platform": {
 "values": {
 "authentication": {
 "oidc": {
 "keycloakExtraRedirectUri": "http://127.0.0.1:8080/oauth2/callback/*,http://localhost:8080/oa..."
 }
 }
 }
 }
 }
 }
}'
```

Опционально: если нужно, чтобы dashboard обращался к Keycloak через внутреннюю сеть кластера вместо external ingress, задайте `keycloakInternalUrl`. Это полезно в окружениях с self-signed certificates или ограниченным внешним доступом. Подробности см. в [Self-Signed Certificates](#).

В течение минуты CozyStack выполнит reconcile и создаст три новых ресурса `HelmRelease`:

```
kubectl get hr -n cozy-keycloak
cozy-keycloak keycloak 26s Unknown Running 'install' action with...
cozy-keycloak keycloak-configure 26s False dependency 'cozy-keycloak/key...'
cozy-keycloak keycloak-operator 26s False dependency 'cozy-keycloak/key...'
```

## Шаг 2: дождитесь завершения установки

Дождитесь, пока все ресурсы успешно установятся и перейдут в состояние `Ready`:

NAME	AGE	READY	STATUS
keycloak	2m19s	True	Release reconciliation succeeded
keycloak-configure	2m19s	True	Release reconciliation succeeded
keycloak-operator	2m19s	True	Release reconciliation succeeded

Выполните reconcile tenants:

```
kubectl annotate -n tenant-root hr/tenant-root reconcile.fluxcd.io/forceAt=$(date +"%Y-%m-%dT%H:%M:%SZ") --...
```

## Шаг 3: откройте Keycloak

Теперь Keycloak доступен по адресу <https://keycloak.example.org> (замените `example.org` на домен вашей инфраструктуры).

Чтобы получить credentials Keycloak для пользователя `admin` по умолчанию, выполните:

```
kubectl get secret -o yaml -n cozy-keycloak keycloak-credentials -o go-template='{{ printf "%s\n" (index .data
```

1. Переключите realm на `cozy`.
2. Создайте пользователя в realm `cozy`. Создание пользователя в realm `cozy` описано в [документации Keycloak](#).
3. После создания пользователя перейдите к его details в Keycloak admin console и включите toggle "Verified email". Это нужно для корректной работы OIDC authentication.
4. Добавьте пользователя в группе `cozystack-cluster-admin`.
5. Теперь вы сможете войти в dashboard с OIDC credentials.

Если dashboard все еще запрашивает token вместо login/password, вручную выполните reconcile:

```
kubectl annotate -n cozy-dashboard hr/dashboard reconcile.fluxcd.io/forceAt=$(date +"%Y-%m-%dT%H:%M:%SZ") --...
```

## Шаг 4: получите kubeconfig

Чтобы получить доступ к кластеру через Dashboard, скачайте kubeconfig: выберите развернутый tenant и скопируйте secret из resource map.

Этот kubeconfig будет автоматически настроен на использование OIDC authentication и namespace, выделенного tenant.

Установите [kubelogin](#), необходимый для использования kubeconfig с OIDC.



---

```
Homebrew (macOS and Linux)
brew install int128/kubelogin/kubelogin

Krew (macOS, Linux, Windows and ARM)
kubectl krew install oidc-login

Chocolatey (Windows)
choco install kubelogin
```

---

# Создание пользователей и назначение ролей

[https://cozystack.ru/docs/v1.5/operations/oidc/users\\_and\\_roles/](https://cozystack.ru/docs/v1.5/operations/oidc/users_and_roles/)

Создание пользователей и назначение им ролей.

## Обзор

При создании tenant в Cozy (начиная с версии 1.6.0) roles, RoleBindings и группы Keycloak автоматически создаются в Kubernetes cluster.

Создание пользователя описано в документации: [Keycloak Admin Console Documentation](#)

## Назначение роли пользователю для tenant

1. Откройте Keycloak: Чтобы получить login credentials, проверьте secret следующей командой:

```
```bash
kubectl get secret keycloak-credentials -n cozy-keycloak -o yaml
```
```

Адрес Keycloak: Адрес Keycloak будет соответствовать значению `publishing.host`, указанному в Platform Package. Например, если Package содержит:

```
spec:
 components:
 platform:
 values:
 publishing:
 host: "infra.example.org"
```

Тогда Keycloak будет доступен по адресу: `keycloak.infra.example.org`

Если вы планируете интеграцию с внешними сервисами как clients или как IdPs, адрес Keycloak должен быть публично доступен и достижим для этих сервисов.

## Настройка ролей для каждого tenant в Cozy

### На уровне кластера

- **cozystack-cluster-admin**
  - Разрешено все.
- **cozystack-cluster-admin**
  - Разрешено все в api group ""

- 
- Разрешено все для helmreleases в helm.toolkit.fluxcd.io и apps.cozystack.io

## На уровне tenant

- **tenant-abc-view**
    - Read-only доступ к ресурсам из нашего API.
    - Возможность просматривать logs.
  - **tenant-abc-use**
    - Все предыдущие permissions.
    - VNC access для virtual machines.
  - **tenant-abc-admin**
    - Все предыдущие permissions.
    - Возможность удалять pods, а также все permissions из tenant-abc-use.
    - Возможность создавать, обновлять и удалять ресурсы из нашего API, кроме tenant, monitoring, etcd, ingress.
  - **tenant-abc-super-admin**
    - Все предыдущие permissions.
    - Возможность создавать, обновлять и удалять tenant, monitoring, etcd и ingress.
-

# Self-Signed Certificates

<https://cozystack.ru/docs/v1.5/operations/oidc/self-signed-certificates/>

В этом руководстве описано, как настроить Kubernetes API server для OIDC authentication с Keycloak при использовании self-signed certificates. По умолчанию Cozystack выпускает certificates через LetsEncrypt, но в некоторых окружениях, например air-gapped или private enterprise networks, может использоваться custom CA.

## Предварительные требования

- Cozystack cluster с включенным OIDC (см. [Enable OIDC Server](#))
- Control plane nodes на Talos Linux
- `talosctl`, настроенный для вашего кластера
- Установленный `kubelogin`

## Шаг 1: получите сертификат Keycloak

Получите certificate из ingress controller:

```
echo | openssl s_client -connect <KEYCLOAK_INGRESS_IP>:443 \
 -servername keycloak.example.org 2>/dev/null | openssl x509
```

Замените `<KEYCLOAK_INGRESS_IP>` на IP-адрес ingress controller, а `keycloak.example.org` - на фактический домен Keycloak.

Сохраните вывод (certificate между `-----BEGIN CERTIFICATE-----` и `-----END CERTIFICATE-----`) для следующего шага.

## Шаг 2: настройте Talos control plane nodes

Для каждого control plane node добавьте следующее в machine configuration:

```

machine:
 network:
 extraHostEntries:
 - ip: <KEYCLOAK_INGRESS_IP>
 aliases:
 - keycloak.example.org
 files:
 - content: |
 -----BEGIN CERTIFICATE-----
 <YOUR_CERTIFICATE_CONTENT>
 -----END CERTIFICATE-----
 permissions: 00644
 path: /var/oidc-ca.crt
 op: create
 cluster:
 apiServer:
 extraArgs:
 oidc-issuer-url: https://keycloak.example.org/realms/cozy
 oidc-client-id: kubernetes
 oidc-username-claim: preferred_username
 oidc-groups-claim: groups
 oidc-ca-file: /etc/kubernetes/oidc/ca.crt
 extraVolumes:
 - hostPath: /var/oidc-ca.crt
 mountPath: /etc/kubernetes/oidc/ca.crt

```

Примените конфигурацию к каждому control plane node:

```
talosctl apply-config -n <NODE_IP> -f nodes/<node>.yaml
```

Конфигурация `extraHostEntries` гарантирует, что домен Keycloak корректно резолвится внутри кластера, что важно при использовании internal ingress IPs.

## Опционально: настройте internal Keycloak URL для Dashboard

По умолчанию oauth2-proxy Cozystack Dashboard подключается к Keycloak через external ingress URL. В окружениях с self-signed certificates или ограниченным внешним доступом можно настроить dashboard на использование внутреннего cluster service Keycloak для backend requests (token exchange, JWKS validation, userinfo, logout), сохранив browser redirects на внешний URL.

Пропатчите Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
 "spec": {
 "components": {
 "platform": {
 "values": {
 "authentication": {
 "oidc": {
 "keycloakInternalUrl": "http://keycloak-http.cozy-keycloak.svc:8080/realms/cozy"
 }
 }
 }
 }
 }
 }
}'
```

Это влияет только на oauth2-proxy dashboard (pod-to-pod communication). Kubernetes API server все равно требует `extraHostEntries` для доступа к Keycloak, потому что `kube-apiserver` использует host-level DNS и не может резолвить cluster service names.

### Шаг 3: настройте kubelogin

Установите kubelogin, если он еще не установлен:

```
Homebrew (macOS and Linux)
brew install int128/kubelogin/kubelogin

Krew (macOS, Linux, Windows and ARM)
kubectl krew install oidc-login

Chocolatey (Windows)
choco install kubelogin
```

Save the CA certificate from Step 1 to a file on your local machine:

```
Save the certificate to a file (e.g., ~/.kube/oidc-ca.pem)
cat > ~/.kube/oidc-ca.pem <<EOF
-----BEGIN CERTIFICATE-----
<YOUR_CERTIFICATE_CONTENT>
-----END CERTIFICATE-----
EOF
```

Set up OIDC login (this will open a browser for authentication):

```
kubectl oidc-login setup \
 --oidc-issuer-url=https://keycloak.example.org/realms/cozy \
 --oidc-client-id=kubernetes \
 --certificate-authority=~/.kube/oidc-ca.pem
```

Configure kubectl credentials:

```
kubectl config set-credentials oidc \
 --exec-api-version=client.authentication.k8s.io/v1 \
 --exec-interactive-mode=IfAvailable \
 --exec-command=kubectl \
 --exec-arg=oidc-login \
 --exec-arg=get-token \
 --exec-arg=--oidc-issuer-url=https://keycloak.example.org/realms/cozy \
 --exec-arg=--oidc-client-id=kubernetes \
 --exec-arg=--certificate-authority=$HOME/.kube/oidc-ca.pem
```

Переключитесь на OIDC user и проверьте:

```
kubectl config set-context --current --user=oidc
kubectl get nodes
```

Если CA вашей организации уже установлен в system trust store (часто встречается в enterprise environments), флаг `--certificate-authority` можно полностью опустить: `kubelogin` автоматически использует system CA bundle.

Избегайте `--insecure-skip-tls-verify`. Если вы не можете установить CA certificate на свою машину или передать его через `--certificate-authority`, можно временно использовать `--insecure-skip-tls-verify`, но это отключает TLS verification и не рекомендуется для production.

## Устранение неполадок

### Проверьте OIDC logs API Server

```
kubectl logs -n kube-system -l component=kube-apiserver --tail=50 | grep oidc
```

### Проверьте, что OIDC flags применены

```
kubectl get pods -n kube-system -l component=kube-apiserver \
 -o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep oidc
```

### Типичные проблемы

- Certificate not found: убедитесь, что path certificate file в `extraVolumes` совпадает с path, указанным в `oidc-ca-file`.
- Domain resolution fails: проверьте, что `extraHostEntries` корректно настроен на всех control plane nodes.
- Authentication fails: проверьте, что пользователь существует в Keycloak и состоит в нужных groups (см. [Users and Roles](#)).

---

## Identity providers

[https://cozystack.ru/docs/v1.5/operations/oidc/identity\\_providers/](https://cozystack.ru/docs/v1.5/operations/oidc/identity_providers/)

### Как настроить GitLab как Identity Provider

---

Как настроить GitLab как Identity Provider

### Как настроить Google как Identity Provider

---

Как настроить Google как Identity Provider

---



# Как настроить GitLab как Identity Provider

[https://cozystack.ru/docs/v1.5/operations/oidc/identity\\_providers/gitlab/](https://cozystack.ru/docs/v1.5/operations/oidc/identity_providers/gitlab/)

GitLab можно использовать как identity provider для Keycloak.

## Обзор

### Создайте Application в GitLab

- Откройте [https://gitlab.com/groups/<YOUR\\_GROUP>/-/settings/applications](https://gitlab.com/groups/<YOUR_GROUP>/-/settings/applications)
- Нажмите Add new application
- Name: cozy, Redirect URI: <https://keycloak.<root-host>/realms/cozy/broker/gitlab/endpoint>
- Включите Confidential, api, read\_api, read\_user, openid, profile, email
- Скопируйте и сохраните Secret

### Настройте Keycloak Identity Provider

Создайте ресурс KeycloakRealmIdentityProvider со следующей конфигурацией:

```
apiVersion: v1.edp.epam.com/v1
kind: KeycloakRealmIdentityProvider
metadata:
 name: gitlab
spec:
 realmRef:
 name: keycloakrealm-cozy
 kind: ClusterKeycloakRealm
 alias: gitlab
 authenticateByDefault: false
 enabled: true
 providerId: "gitlab"
 config:
 clientId: "YOUR GITLAB APP ID"
 clientSecret: "YOUR GITLAB APP SECRET"
 syncMode: "IMPORT"
 mappers:
 - name: "username"
 identityProviderMapper: "oidc-username-idp-mapper"
 identityProviderAlias: "gitlab"
 config:
 target: "LOCAL"
 syncMode: "INHERIT"
 template: "${ALIAS}---${CLAIM.preferred_username}"
```

# Как настроить Google как Identity Provider

[https://cozystack.ru/docs/v1.5/operations/oidc/identity\\_providers/google/](https://cozystack.ru/docs/v1.5/operations/oidc/identity_providers/google/)

## Настройте Google

- Перейти в [Google Console](#), войдите в консоль с Google account, после чего откроется Google Developer Console. После входа используйте выпадающее меню вверху слева, чтобы создать новый project.
- Нажмите “New Project”, чтобы продолжить.
- Введите имя project и выберите Organisation, если у вас несколько organisations. Затем нажмите “Create”.
- После создания project появится pop-up с предложением настроить consent screen. Если он не появился, перейдите в Dashboard и откройте “Explore and enable APIs”. Затем нажмите “Credentials” > “Configure Consent Screen” и переходите к следующему шагу.
- Нажмите “External”, так как нужно разрешить вход в приложение любому Google account, затем нажмите “Create”.
- После этого вы будете перенаправлены на страницы, где нужно настроить несколько параметров:
  - Application type: Public
  - Application name: Your application name
  - Authorised domains: Your application top-level domain name
  - Application Homepage link: Your application homepage
  - Application Privacy Policy link: Your application privacy policy link
- Теперь перейдите к Create Credentials в navbar и нажмите “OAuth Client ID”.
- Выберите Application type “Web application” и задайте имя приложения. Затем добавьте ссылку из вкладки Keycloak в “Authorized Redirect URIs” и нажмите “Create”. Ссылка должна выглядеть примерно так:

```
https://YOUR_KEYCLOAK_DOMAIN/auth/realms/cozy/broker/google/endpoint
```

- После завершения появится pop-up с информацией, нужной на следующем шаге. Вам понадобятся “Client ID” и “Client secret”, поэтому сохраните их в безопасном месте.

## Настройте Keycloak Identity Provider

Создайте ресурс `KeycloakRealmIdentityProvider` со следующей конфигурацией:

---

```
apiVersion: v1.edp.epam.com/v1
kind: KeycloakRealmIdentityProvider
metadata:
 name: google
spec:
 realmRef:
 name: keycloakrealm-cozy
 kind: ClusterKeycloakRealm
 alias: google
 authenticateByDefault: false
 enabled: true
 providerId: "google"
 config:
 clientId: "YOUR GOOGLE APP ID"
 clientSecret: "YOUR GOOGLE APP SECRET"
 syncMode: "IMPORT"
 mappers:
 - name: "username"
 identityProviderMapper: "oidc-username-idp-mapper"
 identityProviderAlias: "google"
 config:
 target: "LOCAL"
 syncMode: "INHERIT"
 template: "${ALIAS}---${CLAIM.email}"
```

---

---

# Руководства для нескольких дата-центров

<https://cozystack.ru/docs/v1.5/operations/stretched/>

Эти руководства показывают, как настроить кластер для запуска Cozystack на узлах, расположенных в разных дата-центрах.

---

## Настройка таймаутов etcd

---

Параметры, необходимые для работы etcd в растянутом кластере

## Конфигурация kube-scheduler

---

Конфигурация kube-scheduler

## Метки узлов

---

Добавьте метки на узлы, чтобы workloads планировались корректно

## Конфигурация LINSTOR DRBD

---

Параметры, необходимые для работы LINSTOR в растянутом кластере

## Настройка выделенной сети для распределенного хранилища с LINSTOR

---

Настройте LINSTOR так, чтобы он предпочитал локальные каналы для хранилища и переходил на междатацентровые соединения при необходимости

## Конфигурация SeaweedFS Multi-DC

---

Как развернуть SeaweedFS в нескольких дата-центрах

---

# Настройка таймаутов etcd

<https://cozystack.ru/docs/v1.5/operations/stretched/etcd/>

## Возможные проблемы

etcd — надежное key-value хранилище для критически важных данных, где Kubernetes API server хранит свое состояние. Значения не считаются записанными, пока кворум узлов не подтвердит запись данных. Кроме того, etcd постоянно проверяет, что в кластере есть лидер и каждый узел знает, какой именно. Обычно кластер etcd запускается на узлах в одном дата-центре, где задержка мала, а таймауты по умолчанию настроены на максимально быстрое сообщение об ошибках. В растянутом кластере, где узлы находятся в разных дата-центрах, RTT значительно выше, и стандартные таймауты могут вызывать ложные срабатывания, как при разделении сети. В этом случае данные остаются согласованными, но etcd перейдет в readonly-режим, чтобы предотвратить split-brain. Если это происходит часто, многие другие компоненты Kubernetes начинают падать с неинформативными сообщениями об ошибках.

## Конфигурация

Чтобы избежать частых перевыборов лидера, настройте в конфигурации etcd следующие параметры:

- `heartbeat-interval` — интервал между heartbeat
- `election-timeout` — время ожидания heartbeat перед запуском выборов

Пример аргументов командной строки для etcd:

```
--election-timeout=10000 --heartbeat-interval=1000
```

## Пример для Talos

Если вы управляете узлами Talos через `tal`, добавьте параметры в секцию `cluster.etcd` шаблона:

```
cluster:
 etcd:
 extraArgs:
 heartbeat-interval: "1000"
 election-timeout: "10000"
```

# Конфигурация kube-scheduler

<https://cozystack.ru/docs/v1.5/operations/stretched/kubeschedulerconfiguration/>

## Добавьте метки на узлы

```
kubectl label node <nodename> topology.kubernetes.io/zone=A
```

## Создайте глобальные topology spread constraints для приложений

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: cozystack-scheduling
 namespace: cozy-system
data:
 globalAppTopologySpreadConstraints: |
 topologySpreadConstraints:
 - maxSkew: 1
 topologyKey: topology.kubernetes.io/zone
 whenUnsatisfiable: DoNotSchedule
```

## Настройте PodTopologySpread

См.: [Kubernetes Documentation](#)

Для установки через Talm добавьте в `templates/_helpers.tpl`:

```
cluster:
...
scheduler:
 config:
 apiVersion: kubescheduler.config.k8s.io/v1beta3
 kind: KubeSchedulerConfiguration
 profiles:
 - schedulerName: default-scheduler
 pluginConfig:
 - name: PodTopologySpread
 args:
 defaultConstraints:
 - maxSkew: 1
 topologyKey: topology.kubernetes.io/zone
 whenUnsatisfiable: ScheduleAnyway
 defaultingType: List
```

Примените изменения:

---

```
talm template -f nodes/node1.yaml -I
talm apply -f nodes/node1.yaml
```

---

# Метки узлов

<https://cozystack.ru/docs/v1.5/operations/stretched/labels/>

## Как работают топологические метки

При запуске Kubernetes-кластера в нескольких дата-центрах важно понимать, какие workloads нужно размещать рядом друг с другом, например базу данных и backend-приложение, а какие — распределять по разным площадкам, например несколько реплик frontend, реплики базы данных или реплики томов. Первый шаг к этому — добавить метки на узлы Kubernetes. В публичных облаках обычно используются термины `zone` и `region`. В Kubernetes наиболее распространенный способ обозначить географическое расположение — использовать метки `topology.kubernetes.io/zone` и `topology.kubernetes.io/region` или только метку `zone` (Cozystack).

## Пример для Talos

В других дистрибутивах Kubernetes метки обычно задаются вручную, но в Talos топологию можно описать как код.

Пример конфигурации `talm`:

`values.yaml`:

```
nodes:
 node1:
 labels:
 topology.kubernetes.io/zone: "us-west-1"
 topology.kubernetes.io/region: "us-west"
 node2:
 labels:
 topology.kubernetes.io/zone: "us-east-1"
 topology.kubernetes.io/region: "us-east"
```

`_helpers.tpl`:

```
{{- $nodeLabels := .Values.nodes | dig (include "talm.discovered.hostname" .) "labels" nil }}
machine:
 {{- with $nodeLabels }}
 nodeLabels:
 {{- toYaml . | nindent 4 }}
 {{- end }}
```



# Конфигурация LINSTOR DRBD

<https://cozystack.ru/docs/v1.5/operations/stretched/drbd-tuning/>

## Введение

В этом руководстве описана конфигурация, необходимая для использования хранилища LINSTOR в растянутом (распределенном) кластере Cozystack.

DRBD (Distributed Replicated Block Device) - система репликации блочных устройств на уровне ядра, работающая по сети. Сервер LINSTOR управляет томами DRBD, включая их создание, удаление и оркестрацию между узлами.

## Сложности при использовании DRBD

DRBD считает данные записанными только после того, как они достигли кворума узлов. Но поскольку для конечного пользователя DRBD выглядит как блочное устройство, при нехватке узлов для кворума он должен вернуть ошибку в пределах заданного таймаута.

Проблема в том, что таймауты по умолчанию рассчитаны на локальные сети с высокой пропускной способностью и низкой задержкой. При обмене между дата-центрами подтверждение от удаленного узла может приходить долго из-за сетевой перегрузки. Это похоже на поведение etcd в растянутых условиях, где стандартные таймауты могут приводить к ложным отказам кворума.

Если хотя бы одно устройство DRBD сообщает о потере кворума, Piraeus HA controller выполнит fencing узла, чтобы предотвратить сбой других workloads. Это может привести к workloads, которые невозможно запланировать, и даже к шторму ребалансировки.

## Конфигурация

Самый эффективный подход - задать глобальные параметры соединения для кластера LINSTOR командой `linstor controller drbd-options`. Она сразу применяет настройки ко всем существующим ресурсам DRBD без индивидуальной настройки или перезапуска:

```
Также применяется к существующим ресурсам DRBD
linstor controller drbd-options --connect-int 15 --ping-int 15 --ping-timeout 20 --drbd-timeout 120
```

Эти значения рассчитаны на междатацентровые окружения, где задержка выше, чем в обычной локальной сети.

| Параметр | Значение | Значение по умолчанию | Рекомендуемое значение |
|----------|----------|-----------------------|------------------------|
|----------|----------|-----------------------|------------------------|

|                             |                                                                                                   |    |     |
|-----------------------------|---------------------------------------------------------------------------------------------------|----|-----|
| <code>--connect-int</code>  | Интервал между попытками TCP-соединения, в секундах.                                              | 10 | 15  |
| <code>--ping-int</code>     | Интервал между кеерalive ping, в секундах.                                                        | 10 | 15  |
| <code>--ping-timeout</code> | Время ожидания ответа на ping перед тем, как реер считается недоступным, в десятых долях секунды. | 5  | 20  |
| <code>--drbd-timeout</code> | Максимальное время ожидания сетевого ответа до срабатывания таймаута, в десятых долях секунды.    | 60 | 120 |

Корректировка этих параметров помогает избежать лишнего fencing и прерывания workloads в растянутых кластерах.

Также обратите внимание на руководство по [общей настройке DRBD](#).

# Настройка выделенной сети для распределенного хранилища с LINSTOR

<https://cozystack.ru/docs/v1.5/operations/stretched/linstor-dedicated-network/>

## Введение

В этом руководстве описано, как повысить надежность и производительность хранилища в распределенных кластерах Cozystack.

В гиперконвергентных кластерах для трафика хранилища обычно выделяют отдельную сеть. Однако не всегда возможно выделить отдельные storage-каналы между дата-центрами.

Если для хранилища нет выделенных междатацентровых каналов, есть два варианта:

- изолировать storage-узлы в каждом дата-центре;
- передавать storage-трафик через существующие uplink вместе с другими workloads.

В этом руководстве показано, как настроить LINSTOR на использование выделенной сети для storage-трафика внутри каждого дата-центра и при необходимости переключаться на общие каналы между дата-центрами.

## Предварительные требования

Это руководство опирается на руководство [Выделенная сеть для LINSTOR](#) и добавляет методы и шаблоны конфигурации, характерные для окружений с несколькими дата-центрами. Чтобы применять описанные здесь шаблоны, важно понимать, как работают интерфейсы узлов и пути соединений. Сначала ознакомьтесь с предыдущим руководством: в нем эти концепции описаны подробно.

Чтобы применять разные настройки соединений в зависимости от расположения узла, добавьте на узлы соответствующие метки. Инструкции см. в руководстве [Топологические метки узлов](#). В этом руководстве для различения дата-центров используется метка `topology.kubernetes.io/zone`.

## Конфигурация соединений

В этом примере есть три дата-центра: `dc1`, `dc2` и `dc3`. Дата-центры соединены прямыми оптическими линиями, а интерфейс называется `region10g`. Узлы в дата-центрах `dc1` и `dc2` имеют отдельный сетевой коммутатор и сетевые интерфейсы `san` только для storage-трафика. В дата-центре `dc3` нет выделенного сетевого коммутатора для хранилища, и весь трафик между узлами в `dc3` маршрутизируется через сеть по умолчанию. Для подключения к другим дата-центрам используется VPN-сервер, подключенный к оптическим линиям. На узлах в `dc3` есть VLAN-интерфейс с IP из подсети `region10g`.

---

Рассмотрим сценарий с тремя дата-центрами: `dc1`, `dc2` и `dc3`.

- Дата-центры соединены прямыми оптическими линиями, которые доступны узлам как интерфейс `region10g`.
- Узлы в `dc1` и `dc2` подключены к выделенному сетевому коммутатору для хранилища и используют отдельный интерфейс `san` только для storage-трафика.
- В дата-центре `dc3` нет выделенной storage-сети. Весь трафик внутри `dc3` идет через сеть по умолчанию.

Для такой схемы оптимальная маршрутизация storage-трафика выглядит так:

- Узлы в `dc1` и `dc2` используют интерфейс `san` для локальной репликации хранилища.
- Узлы в `dc3` используют сетевой интерфейс по умолчанию для локальной репликации хранилища, избегая лишних переходов через VPN-сервер.
- Междатацентровый storage-трафик идет через интерфейс `region10g`.

Ниже custom resource definition (CRD) LINSTOR, который реализует эту логику:

```

apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
 name: intra-datacenter
spec:
 selector:
 - matchLabels:
 - key: topology.kubernetes.io/zone
 op: Same
 - key: topology.kubernetes.io/zone
 op: In
 values:
 # Только в этих дата-центрах есть интерфейс `san`
 - dc1
 - dc2
 paths:
 - name: intra-datacenter
 interface: san

Узлы в `dc3` используют сеть по умолчанию.
Для репликации внутри этого дата-центра отдельное соединение не требуется.

apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
 name: cross-datacenter
spec:
 selector:
 - matchLabels:
 - key: topology.kubernetes.io/zone
 op: NotSame
 paths:
 - name: cross-datacenter
 interface: region10g
```

После применения этой конфигурации можно проверить получившиеся пути соединений:

---

```
LINSTOR ==> node-connection list
```

| Node A | Node B | Properties                        |
|--------|--------|-----------------------------------|
| node01 | node02 | last-applied=["Paths/intra-dat... |
| node01 | node03 | last-applied=["Paths/cross-dat... |
| node01 | node04 | last-applied=["Paths/cross-dat... |
| ....   |        |                                   |

Пары узлов внутри дата-центра `dc3` не будут иметь настроенного `custom node connection` и будут использовать интерфейс по умолчанию.

---

# Конфигурация SeaweedFS Multi-DC

<https://cozystack.ru/docs/v1.5/operations/stretched/seaweedfs-multidc/>

В этом руководстве описано, как развернуть SeaweedFS в нескольких дата-центрах ("multi-DC"). Multi-zone конфигурация SeaweedFS доступна начиная с Cozystack v0.34.0.

## Конфигурация SeaweedFS Multi-DC

Чтобы растянуть SeaweedFS на несколько DC, создайте новый кластер в режиме multi-DC.

По умолчанию SeaweedFS работает в одном дата-центре (DC), и уже запущенное single-DC-развертывание нельзя переключить в multi-DC-режим. Если нужно изменить топологию, удалите текущий экземпляр SeaweedFS и создайте новый с нужным режимом.

Удобный порядок действий:

1. Разверните tenant с `seaweedfs: false`.
2. Создайте новый экземпляр SeaweedFS в namespace tenant, используя нужную топологию.
3. Обновите tenant, установив `seaweedfs: true`.

### 1. Создайте tenant без SeaweedFS

Поле `isolated`, которое в ранних релизах Cozystack было доступно в объекте Tenant, удалено в v1.0. Текущая модель сетевой изоляции описана в [upgrade notes: флаг isolated у Tenant удален](#).

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
 name: dev
 namespace: tenant-root
spec:
 etcd: false
 host: ""
 ingress: false
 monitoring: false
 seaweedfs: false
```

### 2. Разверните SeaweedFS в режиме Multi-Zone

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
 name: seaweedfs
 namespace: tenant-dev
spec:
 host: dev.s3.example.org
 replicas: 4
 size: 100Gi
 topology: MultiZone
 zones:
 dc1: {}
 dc2: {}
 dc3: {}
```

В этом примере SeaweedFS запускает 12 volume server: 4 реплики на 3 зоны.

Параметр `zones` перечисляет каждый дата-центр. Любая настройка, пропущенная внутри зоны, наследует значения верхнего уровня: `replicas`, `size` и другие.

### 3. Включите SeaweedFS в tenant

Когда экземпляр SeaweedFS будет готов, включите его в tenant. Примените следующее обновленное описание ресурса:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
 name: dev
 namespace: tenant-root
spec:
 etcd: false
 host: ""
 ingress: false
 monitoring: false
 seaweedfs: true
```

Теперь создание Bucket доступно для всего дерева этого tenant.

## Использование удаленного экземпляра SeaweedFS

Можно опубликовать одно развертывание SeaweedFS для других кластеров Cozystack и разрешить им подключаться в режиме Client.

### 1. Экспортируйте SeaweedFS

Задайте gRPC endpoint filer и список сетей, которым разрешено подключение:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
 name: seaweedfs
 namespace: tenant-dev
spec:
 host: dev.s3.example.org
 replicas: 4
 size: 100Gi
 topology: MultiZone
 zones:
 dc1: {}
 dc2: {}
 dc3: {}
 filer:
 grpcHost: filer-dev.s3.example.org
 whitelist:
 - 0.0.0.0/0 # открыть для всех сетей (настройте для production)
```

## 2. Настройте удаленный SeaweedFS

Подключите удаленный экземпляр SeaweedFS из другого кластера:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
 name: seaweedfs
spec:
 host: dev.s3.example.org
 topology: Client
 filer:
 grpcHost: filer-dev.s3.example.org
```

## 3. Включите доступ к удаленному SeaweedFS

Чтобы локальный кластер мог аутентифицироваться в удаленном SeaweedFS filer, экспортируйте secret `seaweedfs-client-cert` из удаленного кластера:

```
kubectl get secret seaweedfs-client-cert -n tenant-dev -o yaml \
> seaweedfs-client-cert.yaml
```

Откройте `seaweedfs-client-cert.yaml` и удалите поля `namespace`, `labels` и `annotations`. Они относятся к удаленному кластеру и не должны применяться локально. После очистки manifest должен выглядеть примерно так:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/tls
metadata:
 name: seaweedfs-client-cert
data:
 tls.crt: ...
 tls.key: ...
```



---

Примените secret в целевой namespace локального кластера:

```
kubectl apply -f seaweedfs-client-cert.yaml -n tenant-root
```

Примечание: В режиме Client кластер не создает volume server, а просто переиспользует удаленный экземпляр SeaweedFS для всех операций с bucket.

---

---

# Multi-Location кластеры

<https://cozystack.ru/docs/v1.5/operations/multi-location/>

В этом разделе описано расширение Cozystack management cluster на несколько физических locations (on-premises + cloud, multi-cloud и т. д.) с помощью WireGuard mesh networking.

Настройка состоит из трех компонентов:

- **Networking Mesh** – Kilo WireGuard mesh с Cilium IPIP encapsulation
- **Local CCM** – cloud controller manager для определения IP узлов и lifecycle
- **Cluster Autoscaling** – автоматическое создание узлов в cloud providers

---

## Networking Mesh

Настройка Kilo WireGuard mesh с Cilium для связности multi-location cluster.

---

## Local Cloud Controller Manager

Определение IP узлов и lifecycle management для multi-location clusters.

---

## Cluster Autoscaling

Автоматическое масштабирование узлов Cozystack management clusters с помощью Kubernetes Cluster Autoscaler.

---

# Частые вопросы и практические руководства

<https://cozystack.ru/docs/v1.5/operations/faq/>

## Диагностика

Рекомендации по диагностике доступны в [шпаргалке по troubleshooting](#).

## Развертывание Cozystack

- Как включить KubeSpan

Развертывание Cozystack, [как включить KubeSpan](#)

- Как включить HugePages

Развертывание Cozystack, [как включить HugePages](#).

- Что делать, если cloud-провайдер не поддерживает MetalLB

Большинство cloud-провайдеров не поддерживают MetalLB. Вместо него можно опубликовать основной ingress-контроллер через external IP. Для развертывания в Hetzner используйте отдельное [руководство по установке в Hetzner](#). Для других провайдеров используйте раздел [Public IP Setup](#) в руководстве по установке Cozystack.

## Эксплуатация

- Как включить доступ к dashboard через ingress-controller

Обновите приложение `ingress` и включите в нем параметр `dashboard: true`. Dashboard станет доступен по адресу: `https://dashboard.<your_domain>`

- Как настраивать Cozystack с помощью FluxCD или ArgoCD

В этом [reference-репозитории](#) показано, как настраивать сервисы Cozystack через GitOps:

- <https://github.com/aenix-io/cozystack-gitops-example>

- Как сгенерировать kubeconfig для пользователей tenant

Перенесено в раздел [Как сгенерировать kubeconfig для пользователей tenant](#).

- Как использовать токены ServiceAccount для доступа к API

См. [Токены ServiceAccount для доступа к API](#).

- Как ротировать Certificate Authority

Перенесено в раздел обслуживания кластера: [Как ротировать Certificate Authority](#).

## Bundles

- Как переопределять параметры отдельных компонентов

---

Перенесено в конфигурацию кластера: [Справочник компонентов](#).

- Как отключить отдельные компоненты из bundle

Перенесено в конфигурацию кластера: [Справочник компонентов](#).

---

## Токены ServiceAccount для доступа к API

---

Как получить и использовать токены ServiceAccount в Cozystack.

## Как сгенерировать kubeconfig для пользователей tenant

---

Руководство по генерации kubeconfig-файла для пользователей tenant в Cozystack.

---

# Токены ServiceAccount для доступа к API

<https://cozystack.ru/docs/v1.5/operations/faq/serviceaccount-api-access/>

## Предварительные требования

Перед началом убедитесь, что:

- Tenant уже существует в Cozystack. Если он еще не создан, см. [Создание пользовательского tenant](#).
- У вас есть доступ к namespace tenant - через OIDC-учетные данные или административный kubeconfig.
- `kubectl` установлен и настроен.
- (Опционально) установлен `jq`.

## Получение токена ServiceAccount

У каждого tenant в Cozystack есть Secret с токеном ServiceAccount. Secret имеет то же имя, что и tenant, и находится в namespace этого tenant.

### Dashboard

1. Войдите в Dashboard пользователем, у которого есть доступ к tenant.
2. При необходимости переключите контекст на нужный tenant.
3. В левой боковой панели перейдите на страницу Administration -> Info и откройте вкладку Secrets.
4. Найдите secret с именем `tenant-<name>` (например, `tenant-team1`), где Key равен token.
5. Нажмите значок глаза, чтобы показать поле Value, затем нажмите на показанные данные. Текст будет автоматически скопирован в буфер обмена.

### kubectl

Получите токен для tenant с именем `<name>`:

```
kubectl -n tenant-<name> get tenantsecret tenant-<name> -o json | jq -r '.data.token | @base64d'
```

Чтобы сохранить токен в переменную для последующих команд:

```
export TOKEN=$(kubectl -n tenant-<name> get tenantsecret tenant-<name> -o json | jq -r '.data.token | @base64d')
```

## Использование токена для доступа к API

---

Получив токен, вы можете [сгенерировать kubeconfig](#) для доступа через kubectl или использовать токен напрямую через curl, как показано ниже.

Безопасность токена

>

Токены ServiceAccount в Cozystack по умолчанию не имеют срока действия. Обращайтесь с ними так же аккуратно, как с паролями.

## Проверка подключения

---

Сначала убедитесь, что текущий контекст kubectl указывает на нужный кластер Cozystack:

```
kubectl config current-context
kubectl cluster-info
```

Затем получите адрес API-сервера:

```
export API_SERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
```

После этого извлеките CA-сертификат из secret tenant:

```
kubectl -n tenant-<name> get secret tenant-<name> -o jsonpath='{.data.ca\.crt}' | base64 -d > ca.crt
```

Теперь проверьте подключение:

```
curl --cacert ca.crt -H "Authorization: Bearer ${TOKEN}" ${API_SERVER}/api
```

После проверки файл ca.crt можно удалить.

---

# Как сгенерировать kubeconfig для пользователей tenant

<https://cozystack.ru/docs/v1.5/operations/faq/generate-kubeconfig/>

Чтобы сгенерировать kubeconfig для пользователей tenant, используйте следующий скрипт ([Cozystack Documentation](#)). В результате вы получите файл tenant-kubeconfig, который можно передать пользователю ([Cozystack Documentation](#)).

```
SERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
kubectl get secret tenant-root -n tenant-root -o go-template='
apiVersion: v1
kind: Config
clusters:
- name: tenant-root
 cluster:
 server: "'$SERVER'"
 certificate-authority-data: {{ index .data "ca.crt" }}
contexts:
- name: tenant-root
 context:
 cluster: tenant-root
 namespace: {{ index .data "namespace" | base64decode }}
 user: tenant-root
current-context: tenant-root
users:
- name: tenant-root
 user:
 token: {{ index .data "token" | base64decode }}
' \
> tenant-root.kubeconfig
```

# Руководство по устранению неполадок Cozystack

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/>

В этом руководстве приведены начальные шаги для проверки состояния кластера и поиска проблем. Внизу страницы находятся ссылки на руководства по устранению неполадок для разных компонентов Cozystack и аспектов эксплуатации кластера.

## Чеклист диагностики

Используйте следующие команды, чтобы проверить состояние кластера.

```
=== Flux CD ===
не должно быть сломанных HelmRelease
kubectl get hr -A | grep -v True

=== Kubernetes ===
не должно быть узлов не в состоянии Ready
kubectl get node

=== LINSTOR ===
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -ti -- linstor'

узлы LINSTOR находятся online
linstor node list

storage-pool LINSTOR находятся в состоянии Ok
linstor storage-pool list

нет сломанных ресурсов
linstor resource list --faulty

=== Kube-OVN ===
alias ovn-appctl='kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl'

Проверить Northbound database
ovn-appctl -t /var/run/ovn/ovnnb_dbctl cluster/status OVN_Northbound

Проверить Southbound database
ovn-appctl -t /var/run/ovn/ovnsb_dbctl cluster/status OVN_Southbound

убедитесь, что:
1. Количество Servers совпадает с количеством control-plane узлов
2. IP-адреса корректны
3. Нет дубликатов (например, двух servers с одним IP)

вывести список control-plane узлов
kubectl get node -o wide -l node-role.kubernetes.io/control-plane

Проверить, что вы не упираетесь в namespace quotas и есть ресурсы для обновления
kubectl get resourcequota --all-namespaces
```

Дополнительно можно проверить, есть ли в кластере pod не в состоянии Running:



```
kubectl get pod -A | grep -v 'Running\|Completed'
```

## Получение базовой информации

Логи оператора Cozystack можно посмотреть командой:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

Все компоненты платформы устанавливаются через Flux CD HelmRelease.

Получить все установленные HelmRelease можно так:

```
kubectl get hr -A
```

| NAMESPACE                      | NAME                      | AGE  | READY | STATUS                 |
|--------------------------------|---------------------------|------|-------|------------------------|
| cozy-cert-manager              | cert-manager              | 4m1s | True  | Release reconciliation |
| cozy-cert-manager              | cert-manager-issuers      | 4m1s | True  | Release reconciliation |
| cozy-cilium                    | cilium                    | 4m1s | True  | Release reconciliation |
| cozy-cluster-api               | capi-operator             | 4m1s | True  | Release reconciliation |
| cozy-cluster-api               | capi-providers            | 4m1s | True  | Release reconciliation |
| cozy-dashboard                 | dashboard                 | 4m1s | True  | Release reconciliation |
| cozy-fluxcd                    | cozy-fluxcd               | 4m1s | True  | Release reconciliation |
| cozy-grafana-operator          | grafana-operator          | 4m1s | True  | Release reconciliation |
| cozy-kamaji                    | kamaji                    | 4m1s | True  | Release reconciliation |
| cozy-kubeovn                   | kubeovn                   | 4m1s | True  | Release reconciliation |
| cozy-kubevirt-cdi              | kubevirt-cdi              | 4m1s | True  | Release reconciliation |
| cozy-kubevirt-cdi              | kubevirt-cdi-operator     | 4m1s | True  | Release reconciliation |
| cozy-kubevirt                  | kubevirt                  | 4m1s | True  | Release reconciliation |
| cozy-kubevirt                  | kubevirt-operator         | 4m1s | True  | Release reconciliation |
| cozy-linstor                   | linstor                   | 4m1s | True  | Release reconciliation |
| cozy-linstor                   | piraeus-operator          | 4m1s | True  | Release reconciliation |
| cozy-mariadb-operator          | mariadb-operator          | 4m1s | True  | Release reconciliation |
| cozy-metallb                   | metallb                   | 4m1s | True  | Release reconciliation |
| cozy-monitoring                | monitoring                | 4m1s | True  | Release reconciliation |
| cozy-postgres-operator         | postgres-operator         | 4m1s | True  | Release reconciliation |
| cozy-rabbitmq-operator         | rabbitmq-operator         | 4m1s | True  | Release reconciliation |
| cozy-redis-operator            | redis-operator            | 4m1s | True  | Release reconciliation |
| cozy-telepresence              | telepresence              | 4m1s | True  | Release reconciliation |
| cozy-victoria-metrics-operator | victoria-metrics-operator | 4m1s | True  | Release reconciliation |
| tenant-root                    | tenant-root               | 4m1s | True  | Release reconciliation |

В нормальном состоянии все они должны быть Ready и иметь статус Release reconciliation succeeded.

## Packages застряли в DependenciesNotReady

Если некоторые packages показывают статус DependenciesNotReady:

```
$ kubectl get pkg -A | grep -v True
```

| NAME                             | VARIANT | READY | STATUS                         |
|----------------------------------|---------|-------|--------------------------------|
| cozystack.cozystack-basics       | default | False | One or more dependencies are n |
| cozystack.tenant-application     | default | False | One or more dependencies are n |
| cozystack.monitoring-application | default | False | One or more dependencies are n |

---

Обычно это означает, что package в цепочке зависимостей отсутствует или отключен. Для диагностики:

1. Найдите первопричину: проверьте логи оператора на сообщения "dependency not found":

```
kubectl logs -n cozy-system deploy/cozystack-operator | grep "dependency not found"
```

Команда покажет, какой зависимости не хватает, например:

```
dependency not found, marking as not ready package=cozystack.monitoring-application dependency=co
```

2. Проверьте, не отключили ли вы обязательный package: некоторые packages зависят от других packages. Если отключить package, например `cozystack.postgres-operator`, от которого зависят другие packages, будет заблокирована вся цепочка зависимостей.

3. Исправьте проблему: либо снова включите отключенный package, либо, если вы намеренно хотите оставить его отключенным, добавьте его в `ignoreDependencies` у затронутого package:

```
kubectl edit pkg cozystack.monitoring-application
```

```
spec:
 ignoreDependencies:
 - cozystack.postgres-operator
```

## Специализированные руководства

---

### Bootstrap кластера

См. [устранение неполадок установки Kubernetes](#).

### Обслуживание кластера

### Удаление отказавшего узла из кластера

См. [Обслуживание кластера](#) & [Масштабирование кластера](#).

### Flux CD

[Устранение неполадок Flux CD](#).

### Kube-OVN

[Устранение неполадок Kube-OVN](#).

---

## Устранение неполадок etcd

---

Как устранять проблемы и ошибки etcd.

## **Устранение неполадок Flux CD**

Как устранять ошибки Flux CD.

## **Устранение неполадок мониторинга**

Руководство по диагностике и устранению проблем с компонентами мониторинга в Cozystack

## **Устранение неполадок Kube-OVN**

Как устранять сбои Kube-OVN, вызванные поврежденной базой данных OVN.

## **Устранение crash loop у LINSTOR controller**

Как устранять проблемы LINSTOR controller.

## **Устранение LINSTOR CrashLoopBackOff из-за поврежденной базы данных**

Как устранять LINSTOR CrashLoopBackOff, связанный с поврежденной базой данных.

## **Устранение неполадок Piraeus custom resources**

Как устранять проблемы с зависшими Piraeus custom resources.

---

---

# Устранение неполадок etcd

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/etcd/>

## Как очистить состояние etcd

---

Чтобы удалить state etcd с узла, используйте `talm` или `talosctl` со следующими командами:

### Talm

Замените `nodeN` именем отказавшего узла, например `node0.yaml`:

```
talm reset -f nodes/nodeN.yaml --system-labels-to-wipe=EPHEMERAL --graceful=false --reboot
```

### talosctl

```
talosctl reset --system-labels-to-wipe=EPHEMERAL --graceful=false --reboot
```

[!] Эта команда удалит state с указанного узла. Используйте ее с осторожностью.

---

# Устранение неполадок Flux CD

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/flux-cd/>

## Диагностика ошибки `install retries exhausted`

Иногда можно столкнуться с такой ошибкой:

```
kubectl get hr -A -n cozy-dashboard dashboard
NAMESPACE NAME AGE READY STATUS
cozy-dashboard dashboard 15m False install retries exhausted
```

Попробуйте разобраться через events:

```
kubectl describe hr -n cozy-dashboard dashboard
```

Если указано Events: `<none>`, приостановите и возобновите release:

```
kubectl patch hr -n cozy-dashboard dashboard -p '{"spec": {"suspend": true}}' --type=merge
kubectl patch hr -n cozy-dashboard dashboard -p '{"spec": {"suspend": null}}' --type=merge
```

Затем снова проверьте events:

```
kubectl describe hr -n cozy-dashboard dashboard
```

# Устранение неполадок мониторинга

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/monitoring-troubleshooting/>

В этом руководстве приведены шаги диагностики типичных проблем с компонентами мониторинга в Cozystack, включая сбор метрик, alerting, визуализацию и сбор логов.

## Диагностика отсутствующих метрик

Если метрики не появляются в Grafana или VictoriaMetrics, выполните следующие шаги:

### Проверьте состояние VMAgent

Убедитесь, что VMAgent запущен и собирает метрики:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=vmagent
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=vmagent --tail=50
```

### Проверьте targets

Проверьте, может ли VMAgent выполнять scrape targets:

```
kubectl exec -n cozy-monitoring -c vmagent deploy/vmagent -- curl -s http://localhost:8429/targets | jq .
```

Найдите targets с health: "up". Если targets недоступны, проверьте сетевую связность и права RBAC.

### Лимиты ресурсов

Если VMAgent ограничен ресурсами, увеличьте лимиты в конфигурации monitoring:

```
vmagent:
 resources:
 limits:
 cpu: 500m
 memory: 1Gi
 requests:
 cpu: 100m
 memory: 256Mi
```

### Требования безопасности

Убедитесь, что TLS включен для безопасного сбора метрик:

- Проверьте сертификаты в конфигурации VMAgent.

- Проверьте, что роли RBAC позволяют VMAgent обращаться к нужным endpoints.

Подробнее см. [Настройка мониторинга](#).

## Alerts не приходят

Если alerts не приходят, проверьте Alertmanager и Alerta.

### Проверьте Alertmanager

Проверьте, что Alertmanager обрабатывает alerts:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=alertmanager
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=alertmanager --tail=50
```

Проверьте alert rules:

```
kubectl get prometheusrules -n cozy-monitoring
```

### Проверьте конфигурацию Alerta

Убедитесь, что Alerta настроена корректно:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=alerta
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=alerta --tail=50
```

Проверьте конфигурацию routing в spec monitoring:

```
alerta:
 alerts:
 telegram:
 token: "your-token"
 chatID: "your-chat-id"
```

### Проблемы масштабируемости

Если alerts задерживаются из-за большого объема, настройте лимиты ресурсов:

```
alerta:
 resources:
 limits:
 cpu: 2
 memory: 2Gi
```

### Безопасность

- Используйте RBAC для ограничения доступа к alerts.

- 
- Включите TLS для alert endpoints.

Подробности конфигурации см. в [Alerting мониторинга](#).

## Проблемы Grafana

---

Диагностика проблем доступа и источников данных в Grafana.

### Проблемы доступа

Если Grafana недоступна:

- Проверьте service и ingress:

```
```bash
```

```
kubectl get svc,ingress -n <tenant-namespace> -l app.kubernetes.io/name=grafana
```

```
```
```

- Проверьте права RBAC для вашего пользователя.

### Конфигурация источников данных

Убедитесь, что источники данных подключены:

1. Войдите в Grafana.
2. Перейдите в Configuration > Data Sources.
3. Проверьте, что источник данных VictoriaMetrics находится в healthy-состоянии.

Если это не так, обновите URL и credentials.

### Лимиты ресурсов

При проблемах производительности увеличьте ресурсы Grafana:

```
grafana:
 resources:
 limits:
 cpu: 1
 memory: 1Gi
```

### Безопасность

- Включите authentication и authorization.
- Используйте TLS для доступа к Grafana.

Настройка dashboard описана в [Dashboard мониторинга](#).

## Проблемы сбора логов



---

Диагностика проблем Fluent Bit и VLogs.

## Проверьте Fluent Bit

Проверьте, что Fluent Bit собирает логи:

```
kubectll get pods -n cozy-monitoring -l app.kubernetes.io/name=fluent-bit
kubectll logs -n cozy-monitoring -l app.kubernetes.io/name=fluent-bit --tail=50
```

## Проверьте VLogs

Убедитесь, что VLogs сохраняет логи:

```
kubectll get pods -n cozy-monitoring -l app.kubernetes.io/name=vlogs
kubectll logs -n cozy-monitoring -l app.kubernetes.io/name=vlogs --tail=50
```

Проверьте прием логов:

```
kubectll exec -n cozy-monitoring -c vlogs deploy/vlogs -- curl -s http://localhost:9428/health
```

## Масштабируемость

Если логи не собираются из-за нагрузки, настройте ресурсы:

```
logsStorages:
- name: default
 storage: 50Gi # Увеличить хранилище
```

## Безопасность

- Используйте RBAC для доступа к логам.
- Включите TLS для передачи логов.

Дополнительная информация приведена в [Логах мониторинга](#).

---

# Устранение неполадок Kube-OVN

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/kube-ovn/>

## Получение информации о состоянии базы данных Kube-OVN

```
Northbound DB
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl \
 -t /var/run/ovn/ovnnb_db.ctl cluster/status OVN_Northbound
Southbound DB
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl \
 -t /var/run/ovn/ovnsb_db.ctl cluster/status OVN_Southbound
```

Пример вывода:

```
Name: OVN_Northbound
Cluster ID: abf6 (abf66f15-9382-4b2b-b14c-355d64ae1bda)
Server ID: 8d8a (8d8a2985-c444-43bb-99f6-21c82f05b58d)
Address: ssl:[10.200.1.22]:6643
Status: cluster member
Role: leader
Term: 3
Leader: self
Vote: self

Log: [2, 1569]
Entries not yet committed: 0
Entries not yet applied: 0
Connections: ->2a66 ->c23f <-2a66 <-c23f
Disconnections: 30
Servers:
 2a66 (2a66 at ssl:[10.200.1.18]:6643) next_index=1569 match_index=1568 last msg 17 ms ago
 8d8a (8d8a at ssl:[10.200.1.22]:6643) (self) next_index=1471 match_index=1568
 c23f (c23f at ssl:[10.200.1.1]:6643) next_index=1569 match_index=1568 last msg 18 ms ago
```

Чтобы исключить узел из кластера, например если он недоступен и его нужно удалить из кластера, используйте:

```
Northbound DB
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl -t /var/run/ovn/ovnnb_db.ctl c...
Southbound DB
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl -t /var/run/ovn/ovnsb_db.ctl c...
```

## Устранение падения pod Kube-OVN

В сложных случаях pod из DaemonSet Kube-OVN могут падать или не запускаться корректно. Обычно это указывает на поврежденную базу данных OVN. Подтвердить это можно по логам pod Kube-OVN CNI.

Получите список pod в namespace `cozy-kubeovn`:

```
kubectl get pod -n cozy-kubeovn
```

| NAME               | READY | STATUS  | RESTARTS    | AGE   |
|--------------------|-------|---------|-------------|-------|
| kube-ovn-cni-5rsvz | 0/1   | Running | 5 (35s ago) | 4m37s |
| kube-ovn-cni-jq2zz | 0/1   | Running | 5 (33s ago) | 4m39s |
| kube-ovn-cni-p4gz2 | 0/1   | Running | 3 (23s ago) | 4m38s |

Прочитайте логи pod по его имени (в этом примере `kube-ovn-cni-jq2zz`):

```
kubectl logs -n cozy-kubeovn kube-ovn-cni-jq2zz
W0725 08:21:12.479452 87678 ovs.go:35] 100.64.0.4 network not ready after 3 ping to gateway 100.64.0.1
W0725 08:21:15.479600 87678 ovs.go:35] 100.64.0.4 network not ready after 6 ping to gateway 100.64.0.1
W0725 08:21:18.479628 87678 ovs.go:35] 100.64.0.4 network not ready after 9 ping to gateway 100.64.0.1
W0725 08:21:21.479355 87678 ovs.go:35] 100.64.0.4 network not ready after 12 ping to gateway 100.64.0.1
W0725 08:21:24.479322 87678 ovs.go:35] 100.64.0.4 network not ready after 15 ping to gateway 100.64.0.1
W0725 08:21:27.479664 87678 ovs.go:35] 100.64.0.4 network not ready after 18 ping to gateway 100.64.0.1
W0725 08:21:30.478907 87678 ovs.go:35] 100.64.0.4 network not ready after 21 ping to gateway 100.64.0.1
W0725 08:21:33.479738 87678 ovs.go:35] 100.64.0.4 network not ready after 24 ping to gateway 100.64.0.1
W0725 08:21:36.479607 87678 ovs.go:35] 100.64.0.4 network not ready after 27 ping to gateway 100.64.0.1
W0725 08:21:39.479753 87678 ovs.go:35] 100.64.0.4 network not ready after 30 ping to gateway 100.64.0.1
W0725 08:21:42.479480 87678 ovs.go:35] 100.64.0.4 network not ready after 33 ping to gateway 100.64.0.1
W0725 08:21:45.478754 87678 ovs.go:35] 100.64.0.4 network not ready after 36 ping to gateway 100.64.0.1
W0725 08:21:48.479396 87678 ovs.go:35] 100.64.0.4 network not ready after 39 ping to gateway 100.64.0.1
```

Чтобы устранить проблему, можно очистить базу данных OVN. Для этого запускается DaemonSet, который удаляет файлы конфигурации OVN с каждого узла. Такая очистка безопасна: DaemonSet Kube-OVN автоматически пересоздаст нужные файлы из Kubernetes API.

Примените следующий YAML, чтобы развернуть cleanup DaemonSet:

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: ovn-cleanup
 namespace: cozy-kubeovn
spec:
 selector:
 matchLabels:
 app: ovn-cleanup
 template:
 metadata:
 labels:
 app: ovn-cleanup
 component: network
 type: infra
 spec:
 affinity:
 podAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchLabels:
 app: ovn-central
 topologyKey: "kubernetes.io/hostname"
 containers:
 - name: cleanup
 image: busybox
 command: ["/bin/sh", "-xc", "rm -rf /host-config-ovn/*; rm -rf /host-config-ovn/.*"]
 volumeMounts:
 - name: host-config-ovn
 mountPath: /host-config-ovn
 nodeSelector:
 kubernetes.io/os: linux
 node-role.kubernetes.io/control-plane: ""
 tolerations:
 - operator: "Exists"
 volumes:
 - name: host-config-ovn
 hostPath:
 path: /var/lib/ovn
 type: ""
 hostNetwork: true
 restartPolicy: Always
 terminationGracePeriodSeconds: 1

```

Проверьте, что DaemonSet запущен:

```

kubectl get pod -n cozy-kubeovn
ovn-cleanup-hjzxb 1/1 Running 0 6s
ovn-cleanup-wmzdvd 1/1 Running 0 6s
ovn-cleanup-ztm86 1/1 Running 0 6s

```

После завершения очистки удалите DaemonSet `ovn-cleanup` и перезапустите pod Kube-OVN CNI, чтобы применить новую конфигурацию:

```

Удалить cleanup DaemonSet
kubectl -n cozy-kubeovn delete ds ovn-cleanup

Перезапустить pod Kube-OVN через удаление
kubectl -n cozy-kubeovn delete pod -l app!=ovs

```



# Устранение crash loop у LINSTOR controller

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/linstor-controller/>

## Перезапуск controller

Если controller запущен, но выглядит простаивающим или не отвечает, попробуйте перезапустить его. Эта операция идемпотентна и безопасна: ожидающая или незавершенная работа продолжится после перезапуска.

## Crash loop LINSTOR controller

Если `linstor-controller` не может запуститься, а в логах нет полезной информации, можно повысить уровень логирования (максимальный уровень - `TRACE`).

Пример `INSTORCluster` CR с повышенным уровнем логирования:

```
apiVersion: piraeus.io/v1
kind: LINSTORCluster
spec:
 controller:
 podTemplate:
 spec:
 containers:
 - name: linstor-controller
 env:
 # обе настройки используются linstor-controller
 - name: LS_LOG_LEVEL
 value: TRACE
 - name: LS_LOG_LEVEL_LINSTOR
 value: TRACE
```

Примечание: если `linstor-controller` не находится в `crash loop`, но нужно повысить уровень логирования, это можно сделать временно во время выполнения следующей командой:

```
linstor controller set-log-level --global TRACE
```

Эта настройка вернется к исходному значению после перезапуска controller.

## LINSTOR перестает отвечать после истечения срока сертификатов

Если в LINSTOR настроена внутренняя TLS-коммуникация, сертификаты создаются и ротируются автоматически. Но есть открытая проблема [piraeusdatastore/piraeus-operator#701](#): компоненты не подхватывают новые сертификаты после ротации. Обходной путь - вручную перезапустить все компоненты LINSTOR.

---

Выполните шаги по порядку:

1. Перезапустите `linstor-controller`.
  2. Перезапустите каждый `satellite` по одному. Не перезапускайте их все одновременно. После перезапуска каждого `satellite` проверьте его логи на ошибки, прежде чем переходить к следующему.
  3. Снова перезапустите `linstor-controller`. Это нужно, потому что `controller` также иницирует подключения к `satellites` и может не переподключиться автоматически к `satellite`, который был перезапущен.
  4. Перезапустите все оставшиеся компоненты `LINSTOR`.
-

# Устранение LINSTOR CrashLoopBackOff из-за поврежденной базы данных

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/linstor-database/>

[!] Только для опытных пользователей

Это руководство предназначено для опытных пользователей, которым комфортны низкоуровневая диагностика и операции восстановления данных. Повреждение базы данных LINSTOR встречается редко и обычно указывает на серьезную базовую проблему.

Если вы столкнулись с такой ситуацией в production-окружении, настоятельно рекомендуем обратиться в квалифицированную поддержку, а не пытаться исправить проблему самостоятельно. Неверные действия могут привести к безвозвратной потере данных.

## Введение

При запуске вне Kubernetes LINSTOR controller использует SQL database того или иного типа. В Kubernetes `linstor-controller` не требует persistent volume или внешней базы данных. Вместо этого он хранит всю информацию как custom resources (CR) прямо в Kubernetes control plane. При запуске LINSTOR controller читает все CR и создает базу данных в памяти.

Эти CR находятся в API group `internal.linstor.linbit.com`:

```
kubectl get crds | grep internal.linstor.linbit.com
```

Пример вывода:

```
ebsremotes.internal.linstor.linbit.com 2024-12-28T00:39:50Z
files.internal.linstor.linbit.com 2024-12-28T00:39:24Z
keyvaluestore.internal.linstor.linbit.com 2024-12-28T00:39:24Z
layerbcachevolumes.internal.linstor.linbit.com 2024-12-28T00:39:24Z
layercachevolumes.internal.linstor.linbit.com 2024-12-28T00:39:25Z
layerdrbdresourcedefinitions.internal.linstor.linbit.com 2024-12-28T00:39:24Z
...
```

Если CR каким-то образом повреждаются, `linstor-controller` завершается с ошибкой и переходит в `CrashLoopBackOff`. Пока `pod controller` падает, остальные компоненты продолжают работать. Даже если падают `satellites`, `drbd` на узлах также продолжает работать. Но создание и удаление `volumes` становится невозможным.

Эти CR плохо читаются человеком, но по ним можно понять, что отсутствует или повреждено. Можно установить уровень логирования `TRACE`, чтобы увидеть процесс загрузки ресурсов в логах и убедиться, что проблема связана с CR.

CR database может повредиться, если `linstor-controller` был перезапущен во время очень долгой операции создания или удаления. “Очень долгая” также может означать “зависшая”. Если видно,



что что-то не может корректно удалиться, лучше разобраться и помочь операции завершиться, а не перезапускать controller.

## Пример логов

LINSTOR controller находится в crash loop:

```
kubectl get pod -n cozy-linstor
linstor-controller-6574668cf9-kjqt5 0/1 CrashLoopBackOff 2 (25s ago)
```

Просмотр логов:

```
kubectl logs -n cozy-linstor linstor-controller-6574668cf9-kjqt5
...
2025-10-31 13:08:36.016 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Initializing the k
2025-10-31 13:08:36.016 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Kubernetes-CRD con
2025-10-31 13:08:37.674 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Starting service i
2025-10-31 13:08:37.690 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Security objects l
2025-10-31 13:08:37.960 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Security objects l
2025-10-31 13:08:37.961 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load
2025-10-31 13:08:38.446 [Main] ERROR LINSTOR/Controller/ffffff SYSTEM - Unknown error duri
...
time="2025-10-31T13:08:38Z" level=fatal msg="failed to run" err="exit status 199"
```

Это указывает на поврежденную CRD database.

## Получение error report

LINSTOR controller создает внутри контейнера файл в каталоге `/var/log/linstor-controller/` с подробным stack trace. К сожалению, его сложно увидеть, потому что он сразу удаляется при перезапуске контейнера. Чтобы обойти это, нужно остановить Piraeus operator controller и изменить entrypoint контейнера linstor-controller.

Чтобы увидеть error report:

1. Остановите Piraeus operator controller (уменьшите его deployment до нуля).
2. Остановите deployment linstor-controller (уменьшите его до нуля).
3. Измените entrypoint deployment linstor-controller, чтобы он запускал `sleep infinity`.
4. Удалите probes, чтобы контейнер не перезапускался.
5. Дождитесь запуска контейнера, подключитесь к нему через `exec` и запустите controller вручную.
6. После падения controller прочитайте error report.

```
kubectl scale deployment -n cozy-linstor piraeus-operator-controller-manager --replicas 0
kubectl scale deployment -n cozy-linstor linstor-controller --replicas 0
kubectl edit deploy -n cozy-linstor linstor-controller
```

---

Откроется редактор. Удалите целиком секции `livenessProbe`, `readinessProbe` и `startupProbe`.  
Найдите секцию контейнера `linstor-controller` (основной контейнер). В ней замените секцию `args` на:

```
command:
- sleep
- infinity
```

После сохранения и выхода увеличьте deployment до одной реплики и дождитесь запуска pod:

```
kubectl scale deployment -n cozy-linstor linstor-controller --replicas 1
```

Запустите controller изнутри контейнера:

```
kubectl exec -ti -n cozy-linstor deploy/linstor-controller -- bash
root@linstor-controller-85fd6d6496-cdhbc:/# ./piraeus-entry.sh startController
```

Вы увидите error report с номером report:

```
2025-10-31 13:19:46.765 [Main] ERROR LINSTOR/Controller/ffffff SYSTEM - Unknown error during loading data f...
```

Теперь прочитайте error report:

```
root@linstor-controller-85fd6d6496-cdhbc:/# ls -l /var/log/linstor-controller/
root@linstor-controller-85fd6d6496-cdhbc:/# cat /var/log/linstor-controller/ErrorMessage-6904B76C-000000-000000
```

Пример error report:

## ERROR REPORT 6904B76C-00000-000000

```
=====
Application: LINBIT® LINSTOR
Module: Controller
Version: 1.31.3
Build ID: 3734cb10b55e71a97b4c3004877e43641e820f9e
Build time: 2025-07-10T06:00:07+00:00
Error time: 2025-10-31 13:19:46
Node: linstor-controller-85fd6d6496-cdhbc
Thread: Main
=====
```

### Reported error:

```

Category: Error
Class name: ImplementationError
Class canonical name: com.linbit.ImplementationError
Generated at: Method 'loadCoreObjects', Source file 'DatabaseLoader.java', Line number 150
```

Error message: Unknown error during loading data from DB

### Call backtrace:

| Method              | Native | Class:Line number                       |
|---------------------|--------|-----------------------------------------|
| ...                 |        |                                         |
| startSystemServices | N      | com.linbit.linstor.core.ApplicationLife |
| start               | N      | com.linbit.linstor.core.Controller:379  |
| main                | N      | com.linbit.linstor.core.Controller:635  |

### Caused by:

```
=====
Category: LinstorException
Class name: DatabaseException
Class canonical name: com.linbit.linstor.dbdrivers.DatabaseException
Generated at: Method 'getInstance', Source file 'ObjectProtectionFactory.java', Line number 64
```

Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not...

### ErrorContext:

### Call backtrace:

| Method                  | Native | Class:Line number                       |
|-------------------------|--------|-----------------------------------------|
| getInstance             | N      | com.linbit.linstor.security.ObjectProt  |
| getObjectProtection     | N      | com.linbit.linstor.dbdrivers.AbsDataaba |
| load                    | N      | com.linbit.linstor.core.objects.Resour  |
| load                    | N      | com.linbit.linstor.dbdrivers.AbsDataaba |
| loadAll                 | N      | com.linbit.linstor.dbdrivers.k8s.crd.K  |
| loadAll                 | N      | com.linbit.linstor.dbdrivers.AbsDataaba |
| loadCoreObjects         | N      | com.linbit.linstor.dbdrivers.DatabaseL  |
| ... (7 lines truncated) |        |                                         |

Ищите сообщения вроде этого:

Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not found

---

or

```
Error message: Database entry of table LAYER_DRBD_VOLUMES could not be restored.
ErrorContext: Details: Primary key: LAYER_RESOURCE_ID = '4804', VLM_NR = '0'
```

Они показывают, какой ресурс отсутствует или поврежден.

## Резервная копия и анализ

---

Перед любыми изменениями сохраните все CR для анализа и восстановления:

```
kubectl get crds | grep -o ".*\.internal\.linstor\.linbit\.com" | \
 xargs kubectl get crds -ojson > crds.json

kubectl get crds | grep -o ".*\.internal\.linstor\.linbit\.com" | \
 xargs -I{} sh -xc "kubectl get {} -ojson > {}.json"

tar czvf backup-$(date +%d.%m.%Y).tgz *.json
```

У CR трудночитаемые имена, поэтому удобно скачать их все в JSON и изучать удобными инструментами на рабочей машине.

## Исправление базы данных

---

[!] ДЕСТРУКТИВНОЕ ДЕЙСТВИЕ!

Если поврежденные CR нельзя исправить иначе, кроме удаления, можно удалить проблемные ресурсы обычно через `kubectl delete`. Учитывайте, что при удалении CR физические volumes останутся на storage nodes как orphan volumes. Они больше не будут автоматически управляться LINSTOR. При необходимости их нужно будет очистить вручную позже.

## Метод 1. Использование вспомогательного скрипта

---

Вспомогательный скрипт `linstor-find-db.sh` можно использовать для поиска ресурсов по разным критериям:

```
#!/usr/bin/env bash
set -euo pipefail

layer_resource_id=""
vln_nr=""
resource_name=""
node_name=""

for kv in "$@"; do
 k="${kv%%=*}"
 v="${kv#*=}"
 case "$k" in
 layer_resource_id) layer_resource_id="$v" ;;
 vln_nr) vln_nr="$v" ;;
 resource_name) resource_name="$v" ;;
 node_name) node_name="$v" ;;
 *) echo "Unknown key: $k" >&2; exit 2 ;;
 esac
done

shopt -s nullglob
files=(*.internal.linstor.linbit.com.json)
if (($#files[@]==0)); then
 echo "No *.internal.linstor.linbit.com.json files found" >&2
 exit 1
fi

if [[-n "$layer_resource_id" && -n "$vln_nr"]]; then
 cat "${files[@]}" \
 | jq -r --arg lrid "$layer_resource_id" --arg vlnr "$vln_nr" \
 '.items[] | select(.spec.layer_resource_id == ($lrid|tonumber) and .spec.vln_nr == ($vlnr|tonumber)) | ...'
 exit 0
fi

if [[-n "$resource_name" && -n "$node_name"]]; then
 cat "${files[@]}" \
 | jq -r --arg res "$resource_name" --arg node "$node_name" '
 .items[]
 | select((.spec.resource_name|tostring|ascii_upcase) == ($res|tostring|ascii_upcase)
 and (.spec.node_name|tostring|ascii_upcase) == ($node|tostring|ascii_upcase))
 | "\(.kind).\(.apiVersion|split("/")[0])/\(.metadata.name)'"
 exit 0
fi

echo "Usage:"
echo " $(basename "$0") layer_resource_id=NUM vln_nr=NUM"
echo " $(basename "$0") resource_name=NAME node_name=NAME" >&2
exit 2
```

Сохраните его как `linstor-find-db.sh`, сделайте исполняемым и используйте для поиска проблемных ресурсов.

## Пример 1. Отсутствует ObjProt

Если ошибка говорит об отсутствующем ObjProt для конкретного ресурса:

```
Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not found
```

---

Найдите все ресурсы для этого ресурса и узла:

```
./linstor-find-db.sh resource_name=PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D node_name=GLD-CSXHK-006
```

Пример вывода:

```
LayerResourceIds.internal.linstor.linbit.com/b3e34e9fe5a79d4e8753d0ad4107d0af969d8faaefb88fbd683169...
LayerResourceIds.internal.linstor.linbit.com/dcbff8f66de95d7c6148b3fbb3a9934d226ffb6dfd405...
Resources.internal.linstor.linbit.com/43319bec4ca2bbc21324663a9b716c3e4a7ba2607f0fa513dcc5...
Volumes.internal.linstor.linbit.com/4ef559b8fe14b58647c99a76a2d3a11f9bbf2b413a448eaf377177...
```

Удалите их все за один раз:

```
kubectl delete $(./linstor-find-db.sh resource_name=PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D node_name=GLD-...
```

## Пример 2. Отсутствует LayerDRBD volume

Если ошибка говорит об отсутствующем LayerDRBD volume:

```
Error message: Database entry of table LAYER_DRBD_VOLUMES could not be restored.
ErrorContext: Details: Primary key: LAYER_RESOURCE_ID = '4804', VLM_NR = '0'
```

Найдите ресурсы по `layer_resource_id` и `vlm_nr`:

```
./linstor-find-db.sh layer_resource_id=4804 vlm_nr=0
```

Пример вывода:

```
LayerStorageVolumes.internal.linstor.linbit.com/5b597f878f6bb586ddd7d7dc3bbddacbdabedf511861a411712...
```

Удалите его:

```
kubectl delete $(./linstor-find-db.sh layer_resource_id=4804 vlm_nr=0)
```

## Итеративное исправление

После удаления проблемных ресурсов снова попробуйте запустить controller изнутри контейнера:

```
piraeus-entry.sh startController
```

Если он снова падает, вы получите новый error report с другим ресурсом. Повторите процесс:

1. Прочитайте error report и определите проблемный ресурс.
2. Найдите и удалите его.
3. Попробуйте снова запустить controller.

---

Продолжайте, пока controller не запустится успешно:

```
2025-10-31 13:42:17.483 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load from data
2025-10-31 13:42:19.190 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load from data
...
2025-10-31 13:42:27.309 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Controller initial
```

## Перезапуск controller

---

После исправления CR-based database нужно вернуть deployment linstor-controller в исходное состояние:

```
kubectl delete deployment -n cozy-linstor linstor-controller
```

Верните piraeus operator controller:

```
kubectl scale deployment -n cozy-linstor piraeus-operator-controller-manager --replicas 1
```

Он выполнит reconcile deployment linstor-controller и запустит его с исходным endpoint.

## Восстановление исходного состояния

---

Если что-то пошло не так и ситуация стала непонятной, можно восстановить как минимум состояние, которое было до исправления.

```
получить сохраненные файлы в другом каталоге
mkdir restore; cd restore
tar xzf ../backup-*.tgz

удалить BCE CR по именам CRD, используя json-описания (при удалении CRD удаляются все CR)
kubectl delete -f crds.json
восстановить definitions (обратите внимание на `create` вместо обычного `apply`)
kubectl create -f crds.json

восстановить все ресурсы
kubectl get crds | grep -o ".*\.internal\.linstor\.linbit\.com" | xargs -I{} sh -xc "kubectl create -f {}.j..."
```

Теперь можно начать снова.

---

# Устранение неполадок Piraeus custom resources

<https://cozystack.ru/docs/v1.5/operations/troubleshooting/piraeus-custom-resources/>

## Введение

К Piraeus custom resources относятся:

- `LinstorCluster`
- `LinstorSatellite`
- `LinstorSatelliteConfiguration`
- `LinstorNodeConnection`

Piraeus controller защищает эти ресурсы от непреднамеренных изменений и удаления. При удалении ресурса controller сначала убеждается, что все нижестоящие ресурсы удалены.

Но по умолчанию webhook отклоняет любые изменения, если что-то идет не так с самим webhook. В нестабильной системе webhook может часто падать без полезных stacktrace.

## Отключение webhook

Если вы исправляете CR, но они “не редактируются”, нужно отключить webhook, а возможно и сам operator.

Несколько полезных способов отключить webhook:

- Если установкой piraeus управляет GitOps-система, можно остановить ее для этого release и просто удалить конфигурацию webhook. Когда вы снимете GitOps-систему с паузы, она пересоздаст конфигурацию webhook.
- Если вы не хотите удалять конфигурацию webhook, потому что она была установлена вручную, можно “сломать” ее selector.

Пример:

```
kubectl edit validatingwebhookconfigurations/piraeus-operator-validating-webhook-configuration
```

В редакторе замените все - `linstor*` на - `xlinstor`. Когда закончите, верните конфигурацию тем же способом. Примеры команд Vim: `%s/- linstor/- xlinstor/g` и `%s/- xlinstor/- linstor/g`.

Примечание: не отключайте webhook навсегда. Он нужен для защиты ресурсов.

## Отключение controller



---

Piraeus controller не только отслеживает собственные CR, но и обновляет их при необходимости. В большинстве случаев его тоже следует отключить. В частности, он постоянно пересоздает `LinstorSatellite` из `LinstorSatelliteConfiguration`.

Чтобы отключить controller, уменьшите его deployment до нуля реплик:

```
kubectl -n storage scale deployment piraeus-operator-controller-manager --replicas=0
```

После завершения обслуживания верните одну реплику:

```
kubectl -n storage scale deployment piraeus-operator-controller-manager --replicas=1
```

## Удаление finalizers

---

У каждого Piraeus CR есть finalizer, который все равно не работает после отключения controller. Если вы собираетесь удалить CR, удалите секцию finalizers обычным способом.

---

# Scheduling Classes

<https://cozystack.ru/docs/v1.5/operations/scheduling-classes/>

SchedulingClass - это cluster-scoped custom resource, который позволяет администраторам задавать политики размещения для tenant workload. Когда tenant назначен scheduling class, все его pod автоматически направляются в custom scheduler Cozystack, который объединяет ограничения, заданные классом, с ограничениями, уже указанными в pod.

Это позволяет операторам платформы закреплять tenant за конкретными дата-центрами, availability zone или группами узлов без изменения отдельных application charts.

## Как это работает

Функция состоит из двух компонентов:

1. Lineage-controller webhook (часть cozystack): mutating admission webhook, который перехватывает создание pod в tenant namespace. Если у namespace есть метка `scheduler.cozystack.io/scheduling-class`, webhook задает `schedulerName: cozystack-scheduler` и добавляет аннотацию `scheduler.cozystack.io/scheduling-class` на каждый pod. Если указанный SchedulingClass CR не существует, например scheduler не установлен, pod остаются без изменений и планируются обычным способом.
2. Cozystack scheduler (пакет cozystack-scheduler): custom Kubernetes scheduler, который работает рядом со стандартным scheduler. Во время планирования он находит SchedulingClass, указанный в аннотации pod, и объединяет ограничения CR (node affinity, pod affinity/anti-affinity, topology spread) с собственным spec pod полностью в памяти, не изменяя pod в API server.

## Предварительные требования

- Cozystack v1.2+
- The cozystack-scheduler system package (v0.2.0+)

## Установка scheduler

```
cozypkg add cozystack.cozystack-scheduler
```

## Создание SchedulingClass

SchedulingClass CR повторяет привычные примитивы планирования Kubernetes. Все поля необязательны: указывайте только те ограничения, которые вам нужны.

---

## Пример: закрепление workload за дата-центром

---

```
apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
metadata:
 name: dc-west
spec:
 nodeSelector:
 topology.kubernetes.io/region: us-west-2
```

Pod, назначенные этому классу, будут планироваться только на узлы с меткой `topology.kubernetes.io/region=us-west-2`.

---

## Пример: распределение по availability zone

---

```
apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
metadata:
 name: zone-spread
spec:
 topologySpreadConstraints:
 - maxSkew: 1
 topologyKey: topology.kubernetes.io/zone
 whenUnsatisfiable: DoNotSchedule
```

### Примечание

Если у `topologySpreadConstraint` или у условия `pod affinity/anti-affinity` значение `labelSelector` равно `nil`, scheduler автоматически заполняет его селектором, который соответствует identity labels приложения Cozystack (`apps.cozystack.io/application.group`, `.kind`, `.name`). Это позволяет задавать универсальные политики распределения или `anti-affinity` без жестко прописанных значений меток для каждого приложения.

---

## Пример: требование выделенных узлов с anti-affinity

---

```

apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
metadata:
 name: dedicated-nodes
spec:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: node-pool
 operator: In
 values:
 - dedicated
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - topologyKey: kubernetes.io/hostname

```

Это закрепляет workload за узлами из пула `dedicated` и распределяет pod по хостам. `labelSelector` для anti-affinity автоматически заполняется для каждого приложения, поэтому pod из разных приложений одного tenant все еще могут попадать на один узел.

## Полный справочник spec SchedulingClass

| Поле                                        | Тип                                                                                                                                                                                                                                                                   | Описание                                                                           |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>spec.nodeSelector</code>              | <code>map[string]string</code>                                                                                                                                                                                                                                        | Простые key-value метки узлов, которым должны соответствовать все подходящие узлы. |
| <code>spec.nodeAffinity</code>              | <a href="#">NodeAffinity</a>                                                                                                                                                                                                                                          | Обязательные и предпочтительные правила node affinity.                             |
| <code>spec.podAffinity</code>               | <a href="#">PodAffinity</a>                                                                                                                                                                                                                                           | Обязательные и предпочтительные правила совместного размещения pod.                |
| <code>spec.podAntiAffinity</code>           | <a href="#">PodAntiAffinity</a>                                                                                                                                                                                                                                       | Обязательные и предпочтительные правила разнесения pod.                            |
| <code>spec.topologySpreadConstraints</code> | <code>[]TopologySpreadConstraint</code> ( <a href="https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#topologyspreadconstraint-v1-core">https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#topologyspreadconstraint-v1-core</a> ) | Ограничения topology spread для равномерного распределения по failure domain.      |

## Назначение SchedulingClass tenant

При создании или редактировании tenant задайте параметр `schedulingClass` равным имени существующего SchedulingClass CR:

---

Через dashboard:

Выберите scheduling class в выпадающем списке формы создания tenant.

**\*\*Через Helm values ( values.yaml ):\*\***

```
schedulingClass: dc-west
```

Через tenant secret (наследование дочерними tenant):

Если родительскому tenant назначен scheduling class, все дочерние tenant наследуют его автоматически. Дочерний tenant не может переопределить scheduling class родителя: он может задать свой class только если у родителя его нет.

Назначение записывает метку scheduler.cozystack.io/scheduling-class в namespace tenant. Webhook читает эту метку (или определяет ее из owning Application CR), чтобы вставить имя scheduler и аннотацию в pod.

## Автоматически заполняемые label selectors

Scheduler (v0.2.0+) автоматически заполняет поля labelSelector со значением nil в условиях pod affinity, pod anti-affinity и topology spread constraint. Он использует identity labels приложения Cozystack у pod:

- apps.cozystack.io/application.group
- apps.cozystack.io/application.kind
- apps.cozystack.io/application.name

Это означает, что универсальный SchedulingClass, например:

```
spec:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - topologyKey: kubernetes.io/hostname
```

автоматически ограничит anti-affinity pod одного приложения: каждое приложение получит собственное поведение anti-affinity без отдельного SchedulingClass для каждого приложения.

Ключи меток по умолчанию можно переопределить в Helm values scheduler:

```
defaultLabelSelectorKeys:
- apps.cozystack.io/application.group
- apps.cozystack.io/application.kind
- apps.cozystack.io/application.name
```

Если у условия уже есть явный labelSelector, он сохраняется без изменений.

## Операторы без встроенной поддержки schedulerName

---

Некоторые операторы, используемые Cozystack, не предоставляют `schedulerName` в своих CRD. Подход на основе webhook обрабатывает такие случаи прозрачно, потому что изменяет pod напрямую во время admission, независимо от того, какой оператор их создал:

- `etcd-operator`
- `redis-operator` (spotahome)
- `mariadb-operator`
- `clickhouse-operator` (altinity)

Для workload, которыми управляют эти операторы, специальная конфигурация не требуется.

## Проверка настройки

---

1. Убедитесь, что scheduler запущен:

```
kubectl get pods -n cozy-system -l app.kubernetes.io/name=cozystack-scheduler
```

2. Убедитесь, что SchedulingClass существует:

```
kubectl get schedulingclasses
```

3. Проверьте, что namespace tenant содержит метку:

```
kubectl get ns tenant-example -o jsonpath='{.metadata.labels.scheduler\.cozystack\.io/scheduling-class}'
```

4. Проверьте, что pod в namespace tenant используют custom scheduler:

```
kubectl get pods -n tenant-example -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.schedulerName...}
```

---



---

# Managed Kubernetes

## Managed (Tenant) Kubernetes

<https://cozystack.ru/docs/v1.5/kubernetes/>

### Managed Kubernetes in Cozystack

---

Whenever you want to deploy a custom containerized application in Cozystack, it's best to deploy it to a managed Kubernetes cluster.

Cozystack deploys and manages Kubernetes-as-a-service as standalone applications within each tenant's isolated environment. In Cozystack, such clusters are named tenant Kubernetes clusters, while the base Cozystack cluster is called a management or root cluster. Tenant clusters are fully separated from the management cluster and are intended for deploying tenant-specific or customer-developed applications.

Within a tenant cluster, users can take advantage of LoadBalancer services and easily provision physical volumes as needed.

The control-plane operates within containers, while the worker nodes are deployed as virtual machines, all seamlessly managed by the application.

Kubernetes version in tenant clusters is independent of Kubernetes in the management cluster. Users can select the supported patch versions from 1.30 to 1.35.

### Why Use a Managed Kubernetes Cluster?

---

Kubernetes has emerged as the industry standard, providing a unified and accessible API, primarily utilizing YAML for configuration. This means that teams can easily understand and work with Kubernetes, streamlining infrastructure management.

Kubernetes leverages robust software design patterns, enabling continuous recovery in any scenario through the reconciliation method. Additionally, it ensures seamless scaling, allowing applications to grow alongside business needs.

The Managed Kubernetes Service in Cozystack offers a streamlined solution for efficiently managing these clusters, allowing users to focus on application development rather than infrastructure overhead.

### Starting Work

---



---

Once the tenant Kubernetes cluster is ready, you can get a kubeconfig file to work with it. It can be found in the [Secrets](#) section of your application's dashboard.

Open the Cozystack dashboard, switch to your tenant, find and open the application page. Copy one of the kubeconfig options from the Secrets section.

Run the following command (using the management cluster kubeconfig):

```
kubect! get secret -n tenant-<name> kubernetes-<clusterName>-admin-kubeconfig -o go-template='{{ printf "%s..."
```

There are several kubeconfig options available:

- `admin.conf` — The standard kubeconfig for accessing your new cluster. You can create additional Kubernetes users and groups later if needed.
- `admin.svc` — Same token as `admin.conf`, but with the API server address set to the internal service name. Use it for applications running within the management cluster.
- `super-admin.conf` — Similar to `admin.conf`, but with extended administrative permissions. Intended for troubleshooting and cluster maintenance.
- `super-admin.svc` — Same as `super-admin.conf`, but pointing to the internal API server address.

## Implementation Details

---

A tenant Kubernetes cluster in Cozystack is essentially Kubernetes-in-Kubernetes. Deploying it involves several key components:

- **Kamaji Control Plane:** [Kamaji](#) is an open-source project that facilitates the deployment of Kubernetes control planes as pods within the management cluster. This includes `kube-apiserver`, `controller-manager`, and `scheduler`, allowing for efficient multi-tenancy and resource utilization.
- **Etcd Cluster:** A dedicated etcd cluster is deployed using Aenix's [etcd-operator](#). It provides reliable and scalable key-value storage for the Kubernetes control plane.
- **Worker Nodes:** Virtual Machines are provisioned to serve as worker nodes using KubeVirt. These nodes are configured with a specific option (`blockSize.matchVolume`) to ensure compatibility with storage backends that use non-512-byte sectors, such as LINSTOR DRBD.
- **Cluster API:** Cozystack is using the [Kubernetes Cluster API](#) to provision the components of a cluster.

This architecture ensures isolated, scalable, and efficient tenant Kubernetes environments.

See the reference for components utilized in this service:

- [Kamaji Control Plane](#)
- [Kamaji — Cluster API](#)
- [github.com/clastix/kamaji](https://github.com/clastix/kamaji)
- [KubeVirt](#)
- [github.com/kubevirt/kubevirt](https://github.com/kubevirt/kubevirt)
- [github.com/aenix-io/etcd-operator](https://github.com/aenix-io/etcd-operator)
- [Kubernetes Cluster API](#)
- [github.com/kubernetes-sigs/cluster-api-provider-kubevirt](https://github.com/kubernetes-sigs/cluster-api-provider-kubevirt)

- [github.com/kubevirt/csi-driver](https://github.com/kubevirt/csi-driver)

## Breaking Changes

ephemeralStorage renamed to diskSize (v1.4): The `nodeGroups[name].ephemeralStorage` field has been renamed to `nodeGroups[name].diskSize` to better reflect its purpose (persistent disk for kubelet and containerd data). Existing clusters will be automatically migrated to use `diskSize`. Existing VMs will be automatically rolling-updated via CAPI when the new values are applied.

The top-level `storageClass` field is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it. The per-node-group `nodeGroups[name].storageClass` field is intentionally not annotated immutable: it is optional and undefaulted, so a strict `self == oldSelf` rule would block any future attempt to set it on an existing node group.

## Parameters

### Common Parameters

| Name                      | Description                                     | Type   | Default    |
|---------------------------|-------------------------------------------------|--------|------------|
| <code>storageClass</code> | Storage class for worker node persistent disks. | string | replicated |

### Application-specific Parameters

| Name                                       | Description                                           | Type              | Default   |
|--------------------------------------------|-------------------------------------------------------|-------------------|-----------|
| <code>nodeGroups</code>                    | Configuration for node groups.                        | map[string]object | {...}     |
| <code>nodeGroups[name].min Replicas</code> | Minimum number of replicas.                           | int               | 0         |
| <code>nodeGroups[name].max Replicas</code> | Maximum number of replicas.                           | int               | 10        |
| <code>nodeGroups[name].instanceType</code> | Instance type for worker nodes.                       | string            | u1.medium |
| <code>nodeGroups[name].disk Size</code>    | Persistent disk size for kubelet and containerd data. | quantity          | 20Gi      |

|                                               |                                                                                                         |          |       |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------|----------|-------|
| nodeGroups[name].storageClass                 | StorageClass for worker node persistent disks. When empty, uses the management cluster default.         | string   |       |
| nodeGroups[name].roles                        | Roles assigned to the node group.                                                                       | []string | []    |
| nodeGroups[name].resources                    | Explicit CPU and memory for each worker node, as an alternative to <a href="#">instanceType</a> sizing. | object   | {}    |
| nodeGroups[name].resources.cpu                | CPU quantity.                                                                                           | quantity |       |
| nodeGroups[name].resources.memory             | Memory quantity.                                                                                        | quantity |       |
| nodeGroups[name].gpus                         | GPU configuration.                                                                                      | []object | []    |
| nodeGroups[name].gpus[i].name                 | GPU device name.                                                                                        | string   |       |
| nodeGroups[name].kubenet                      | Kubelet configuration.                                                                                  | object   | {}    |
| nodeGroups[name].kubenet.systemReservedMemory | Memory reserved for system.                                                                             | string   |       |
| nodeGroups[name].kubenet.kubeReservedMemory   | Memory reserved for Kubernetes.                                                                         | string   |       |
| nodeGroups[name].kubenet.systemReservedCpu    | CPU reserved for system.                                                                                | string   |       |
| nodeGroups[name].kubenet.kubeReservedCpu      | CPU reserved for Kubernetes.                                                                            | string   |       |
| nodeGroups[name].kubenet.evictionHardMemory   | Hard eviction threshold for memory.                                                                     | string   | 7%    |
| nodeGroups[name].kubenet.evictionSoftMemory   | Soft eviction threshold for memory.                                                                     | string   | 10%   |
| version                                       | Kubernetes version.                                                                                     | string   | v1.35 |

|      |                                                                                                                          |        |  |
|------|--------------------------------------------------------------------------------------------------------------------------|--------|--|
| host | External hostname for Kubernetes cluster.<br>Defaults to <code>&lt;cluster-name&gt;.&lt;tenant-host&gt;</code> if empty. | string |  |
|------|--------------------------------------------------------------------------------------------------------------------------|--------|--|

## Cluster Addons

| Name                               | Description                                                                                    | Type                  | Default         |
|------------------------------------|------------------------------------------------------------------------------------------------|-----------------------|-----------------|
| addons                             | Configuration for cluster addons.                                                              | object                | <code>{}</code> |
| addons.certManager                 | cert-manager addon.                                                                            | object                | <code>{}</code> |
| addons.certManager.enabled         | Enable cert-manager.                                                                           | bool                  | false           |
| addons.certManager.valuesOverride  | Helm values override.                                                                          | object                | <code>{}</code> |
| addons.cilium                      | Cilium CNI addon.                                                                              | object                | <code>{}</code> |
| addons.cilium.valuesOverride       | Helm values override.                                                                          | object                | <code>{}</code> |
| addons.gatewayAPI                  | Gateway API support.                                                                           | object                | <code>{}</code> |
| addons.gatewayAPI.enabled          | Enable Gateway API.                                                                            | bool                  | false           |
| addons.ingressNginx                | Ingress NGINX controller.                                                                      | object                | <code>{}</code> |
| addons.ingressNginx.enabled        | Enable the controller (requires nodes labeled <code>ingress-nginx</code> ).                    | bool                  | false           |
| addons.ingressNginx.exposeMethod   | Method to expose the controller (Proxied, LoadBalancer).                                       | string                | Proxied         |
| addons.ingressNginx.hosts          | Domains routed to this tenant cluster when <code>exposeMethod</code> is <code>Proxied</code> . | <code>[]string</code> | <code>[]</code> |
| addons.ingressNginx.valuesOverride | Helm values override.                                                                          | object                | <code>{}</code> |
| addons.gpuOperator                 | NVIDIA GPU Operator.                                                                           | object                | <code>{}</code> |

|                                             |                           |        |       |
|---------------------------------------------|---------------------------|--------|-------|
| addons.gpuOperator.enabled                  | Enable GPU Operator.      | bool   | false |
| addons.gpuOperator.valuesOverride           | Helm values override.     | object | {}    |
| addons.hami                                 | HAMi for GPU sharing.     | object | {}    |
| addons.hami.enabled                         | Enable HAMi.              | bool   | false |
| addons.hami.valuesOverride                  | Helm values override.     | object | {}    |
| addons.fluxcd                               | Flux CD addon.            | object | {}    |
| addons.fluxcd.enabled                       | Enable Flux CD.           | bool   | false |
| addons.fluxcd.valuesOverride                | Helm values override.     | object | {}    |
| addons.monitoringAgents                     | Monitoring agents.        | object | {}    |
| addons.monitoringAgents.enabled             | Enable monitoring agents. | bool   | false |
| addons.monitoringAgents.valuesOverride      | Helm values override.     | object | {}    |
| addons.verticalPodAutoscaler                | VPA addon.                | object | {}    |
| addons.verticalPodAutoscaler.valuesOverride | Helm values override.     | object | {}    |
| addons.velero                               | Velero for backups.       | object | {}    |
| addons.velero.enabled                       | Enable Velero.            | bool   | false |
| addons.velero.valuesOverride                | Helm values override.     | object | {}    |
| addons.coredns                              | CoreDNS addon.            | object | {}    |
| addons.coredns.valuesOverride               | Helm values override.     | object | {}    |
| addons.ouroboros                            | Ouroboros addon.          | object | {}    |
| addons.ouroboros.enabled                    | Enable Ouroboros.         | bool   | false |
| addons.ouroboros.valuesOverride             | Helm values override.     | object | {}    |

## Kubernetes Control Plane Configuration

| Name                                            | Description                               | Type     | Default   |
|-------------------------------------------------|-------------------------------------------|----------|-----------|
| controlPlane                                    | Configuration for the control plane.      | object   | {}        |
| controlPlane.replicas                           | Number of control plane replicas.         | int      | 2         |
| controlPlane.apiServer                          | API server configuration.                 | object   | {}        |
| controlPlane.apiServer.resources                | Explicit resource requests/limits.        | object   | {}        |
| controlPlane.apiServer.resources.cpu            | CPU quantity.                             | quantity |           |
| controlPlane.apiServer.resources.memory         | Memory quantity.                          | quantity |           |
| controlPlane.apiServer.resources.Preset         | Preset if <code>resources</code> omitted. | string   | c1.medium |
| controlPlane.controllerManager                  | Controller manager configuration.         | object   | {}        |
| controlPlane.controllerManager.resources        | Explicit resource requests/limits.        | object   | {}        |
| controlPlane.controllerManager.resources.cpu    | CPU quantity.                             | quantity |           |
| controlPlane.controllerManager.resources.memory | Memory quantity.                          | quantity |           |
| controlPlane.controllerManager.resources.Preset | Preset if <code>resources</code> omitted. | string   | t1.micro  |
| controlPlane.scheduler                          | Scheduler configuration.                  | object   | {}        |
| controlPlane.scheduler.resources                | Explicit resource requests/limits.        | object   | {}        |
| controlPlane.scheduler.resources.cpu            | CPU quantity.                             | quantity |           |
| controlPlane.scheduler.resources.memory         | Memory quantity.                          | quantity |           |
| controlPlane.scheduler.resources.Preset         | Preset if <code>resources</code> omitted. | string   | t1.micro  |

|                                                   |                                           |          |          |
|---------------------------------------------------|-------------------------------------------|----------|----------|
| controlPlane.konnectivity                         | Konnectivity configuration.               | object   | {}       |
| controlPlane.konnectivity.server                  | Konnectivity server configuration.        | object   | {}       |
| controlPlane.konnectivity.server.resources        | Explicit resource requests/limits.        | object   | {}       |
| controlPlane.konnectivity.server.resources.cpu    | CPU quantity.                             | quantity |          |
| controlPlane.konnectivity.server.resources.memory | Memory quantity.                          | quantity |          |
| controlPlane.konnectivity.server.resourcesPreset  | Preset if <code>resources</code> omitted. | string   | t1.micro |
| images                                            | Custom container images.                  | object   | {}       |
| images.waitForKubeconfig                          | Image to wait for kubeconfig.             | string   |          |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (t1 1:0.5, c1 1:1, s1 1:2, u1 1:4, m1 1:8) and each series ships eight sizes (nano through 4xlarge). The legacy flat names (nano, micro, small, medium, large, xlarge, 2xlarge) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [docs/operations/resource-presets.md](https://docs.cozystack.com/operations/resource-presets.md) for the full size matrix and the legacy-to-instance-type mapping.

### instanceType Resources

The following instanceType resources are provided by Cozystack:

| Instance Type  | CPU | Memory     |
|----------------|-----|------------|
| cx1.2xlarge    | 8   | 16Gi       |
| cx1.2xlarge1gi | 8   | 16Gi + 1Gi |
| cx1.4xlarge    | 16  | 32Gi       |
| cx1.4xlarge1gi | 16  | 32Gi + 1Gi |
| cx1.8xlarge    | 32  | 64Gi       |
| cx1.8xlarge1gi | 32  | 64Gi + 1Gi |
| cx1.large      | 2   | 4Gi        |
| cx1.large1gi   | 2   | 4Gi + 1Gi  |
| cx1.medium     | 1   | 2Gi        |
| cx1.medium1gi  | 1   | 2Gi + 1Gi  |
| cx1.xlarge     | 4   | 8Gi        |
| cx1.xlarge1gi  | 4   | 8Gi + 1Gi  |
| d1.2xlarge     | 8   | 32Gi       |
| d1.2xmedium    | 2   | 8Gi        |
| d1.4xlarge     | 16  | 64Gi       |
| d1.8xlarge     | 32  | 128Gi      |
| d1.large       | 4   | 16Gi       |
| d1.medium      | 2   | 8Gi        |
| d1.micro       | 1   | 4Gi        |
| d1.nano        | 1   | 2Gi        |
| d1.small       | 1   | 4Gi        |
| d1.xlarge      | 4   | 16Gi       |
| gn1.2xlarge    | 8   | 32Gi       |
| gn1.4xlarge    | 16  | 64Gi       |
| gn1.8xlarge    | 32  | 128Gi      |
| gn1.xlarge     | 4   | 16Gi       |
| m1.2xlarge     | 8   | 64Gi       |
| m1.2xlarge1gi  | 8   | 64Gi + 1Gi |



|               |     |             |
|---------------|-----|-------------|
| m1.4xlarge    | 16  | 128Gi       |
| m1.4xlarge1gi | 16  | 128Gi + 1Gi |
| m1.8xlarge    | 32  | 256Gi       |
| m1.8xlarge1gi | 32  | 256Gi + 1Gi |
| m1.large      | 2   | 16Gi        |
| m1.large1gi   | 2   | 16Gi + 1Gi  |
| m1.xlarge     | 4   | 32Gi        |
| m1.xlarge1gi  | 4   | 32Gi + 1Gi  |
| n1.2xlarge    | 8   | 8Gi         |
| n1.4xlarge    | 16  | 16Gi        |
| n1.8xlarge    | 32  | 32Gi        |
| n1.large      | 2   | 2Gi         |
| n1.medium     | 1   | 1Gi         |
| n1.xlarge     | 4   | 4Gi         |
| o1.2xlarge    | 8   | 32Gi        |
| o1.4xlarge    | 16  | 64Gi        |
| o1.8xlarge    | 32  | 128Gi       |
| o1.large      | 4   | 16Gi        |
| o1.medium     | 2   | 8Gi         |
| o1.micro      | 1   | 4Gi         |
| o1.nano       | 0.5 | 2Gi         |
| o1.small      | 1   | 4Gi         |
| o1.xlarge     | 4   | 16Gi        |
| rt1.2xlarge   | 8   | 32Gi        |
| rt1.4xlarge   | 16  | 64Gi        |
| rt1.8xlarge   | 32  | 128Gi       |
| rt1.large     | 4   | 16Gi        |
| rt1.medium    | 2   | 8Gi         |
| rt1.micro     | 1   | 4Gi         |
| rt1.small     | 1   | 4Gi         |

|             |     |       |
|-------------|-----|-------|
| rt1.xlarge  | 4   | 16Gi  |
| u1.2xlarge  | 8   | 32Gi  |
| u1.2xmedium | 2   | 8Gi   |
| u1.4xlarge  | 16  | 64Gi  |
| u1.8xlarge  | 32  | 128Gi |
| u1.large    | 4   | 16Gi  |
| u1.medium   | 2   | 8Gi   |
| u1.micro    | 1   | 4Gi   |
| u1.nano     | 0.5 | 2Gi   |
| u1.small    | 1   | 4Gi   |
| u1.xlarge   | 4   | 16Gi  |

## Kubelet Resource Reservations

Each node group supports a `kubelet` object that configures how much memory and CPU the kubelet reserves for the host OS and the Kubernetes system itself.

When `systemReservedMemory` or `kubeReservedMemory` are left empty, they are auto-computed using the following formula:

- Determine the effective memory of the node:
  - If `resources.memory` is explicitly set, use that value.
  - Otherwise, look up the `instanceType` and use its `memory.guest` value.
  - If neither is available, the reservation falls back to the minimum (256Mi).
- Calculate 5% of the effective memory (in MiB, rounded down).
- Clamp the result to the range [256Mi, 1Gi]:
  - Nodes with 5 GiB or less get the minimum 256Mi reservation.
  - Nodes with 20 GiB or more get the maximum 1Gi reservation.

Both `systemReservedMemory` and `kubeReservedMemory` receive the same auto-computed value by default.

CPU reservations (`systemReservedCpu`, `kubeReservedCpu`) follow the same pattern: 5% of effective CPU, clamped to [50m, 500m]. Both are auto-computed when left empty.

## Kubelet Defaults

| Parameter                         | Default | Note |
|-----------------------------------|---------|------|
| <code>systemReservedMemory</code> | auto    |      |
| <code>kubeReservedMemory</code>   | auto    |      |

|                         |       |             |
|-------------------------|-------|-------------|
| systemReservedCpu       | auto  |             |
| kubeReservedCpu         | auto  |             |
| evictionHardMemory      | 7%    |             |
| evictionSoftMemory      | 10%   |             |
| evictionSoftGracePeriod | 1m30s | (hardcoded) |
| evictionMinimumReclaim  | 256Mi | (hardcoded) |

Eviction thresholds can be specified as percentages (e.g., 7%) or absolute values (e.g., 200Mi). Both thresholds must use the same unit type. The hard threshold must be strictly less than the soft threshold.

The `evictionSoftGracePeriod` and `evictionMinimumReclaim` parameters are currently hardcoded in the chart template and cannot be overridden through values.

## Capacity Annotation

When kubelet resource reservations are configured, both the `capacity.cluster-autoscaler.kubernetes.io/memory` and `capacity.cluster-autoscaler.kubernetes.io/cpu` annotations on MachineDeployments report allocatable values instead of total node resources. Memory allocatable subtracts system-reserved, kube-reserved, and eviction thresholds from the node's guest memory.

Upgrading from a version without this feature may cause the autoscaler to see reduced per-node capacity, potentially triggering an scale-up if current utilization is high.

Note: When neither `resources.memory` nor `instanceType` is set, eviction thresholds (default 7% hard / 10% soft) are still enforced by the kubelet at runtime, but the capacity annotations will not be able to account for them.

## Example: Override kubelet reservations

```
nodeGroups:
 md0:
 instanceType: u1.medium
 kubelet:
 systemReservedMemory: 512Mi
 kubeReservedMemory: 512Mi
 evictionHardMemory: 5%
 evictionSoftMemory: 8%
```

## U Series: Universal

The U Series is quite neutral and provides resources for general purpose applications.

U is the abbreviation for “Universal”, hinting at the universal attitude towards workloads.

VMs of instance types will share physical CPU cores on a time-slice basis with other VMs.

---

## U Series Characteristics

Specific characteristics of this series are:

- Burstable CPU performance — The workload has a baseline compute performance but is permitted to burst beyond this baseline, potentially using the entire physical core.
- vCPU-To-Memory Ratio (1:4) — A vCPU-to-Memory ratio of 1:4, for less noise per node.

## O Series: Overcommitted

---

The O Series is based on the U Series, with the only difference being that memory is overcommitted.

O is the abbreviation for “Overcommitted”.

## O Series Characteristics

Specific characteristics of this series are:

- Burstable CPU performance — The workload has a baseline compute performance but is permitted to burst beyond this baseline, potentially using the entire physical core.
- Overcommitted Memory — Memory is over-committed in order to achieve a higher workload density.
- vCPU-To-Memory Ratio (1:4) — A vCPU-to-Memory ratio of 1:4, for less noise per node.

## CX Series: Compute Exclusive

---

The CX Series provides exclusive compute resources for compute intensive applications.

CX is the abbreviation of “Compute Exclusive”.

The exclusive resources are given to the compute threads of the VM. In order to ensure this, some additional virtualization overhead is necessary.

## CX Series Characteristics

Specific characteristics of this series are:

- Hugepages — Hugepages are used in order to improve memory performance.
- Dedicated CPU — Physical cores are exclusively assigned to every vCPU in order to provide fixed and high compute performance.
- Isolated emulator threads — Hypervisor emulator threads are isolated from the vCPUs in order to reduce emulation related impact on the guest performance.
- vNUMA — Physical NUMA topology is reflected in the guest in order to optimize guest sided cache utilization.
- vCPU-To-Memory Ratio (1:2) — A vCPU-to-Memory ratio of 1:2.

---

## M Series: Memory

---

The M Series provides resources for memory intensive applications.

M is the abbreviation of “Memory”.

### M Series Characteristics

Specific characteristics of this series are:

- Hugepages — Hugepages are used in order to improve memory performance.
- Burstable CPU performance — The workload has a baseline compute performance but is permitted to burst beyond this baseline, potentially using the entire physical core.
- vCPU-To-Memory Ratio (1:8) — A vCPU-to-Memory ratio of 1:8, for much less noise per node.

## RT Series: RealTime

---

The RT Series provides resources for realtime applications, like Oslat.

RT is the abbreviation for “realtime”.

This series of instance types requires nodes capable of running realtime applications.

### RT Series Characteristics

Specific characteristics of this series are:

- Hugepages — Hugepages are used in order to improve memory performance.
  - Dedicated CPU — Physical cores are exclusively assigned to every vCPU in order to provide fixed and high compute performance.
  - Isolated emulator threads — Hypervisor emulator threads are isolated from the vCPUs in order to reduce emulation related impact on the guest performance.
  - vCPU-To-Memory Ratio (1:4) — A vCPU-to-Memory ratio of 1:4 starting from the medium size.
-

---

# GPU Sharing with HAMi

<https://cozystack.ru/docs/v1.5/kubernetes/gpu-sharing/>

**HAMi** (Heterogeneous AI Computing Virtualization Middleware) is a CNCF Sandbox project that enables fractional GPU sharing in Kubernetes. Instead of dedicating an entire GPU to a single workload, HAMi lets containers request specific amounts of GPU memory and compute cores.

This guide covers GPU sharing for containers in tenant Kubernetes clusters. For GPU passthrough to virtual machines on the management cluster, see [GPU Passthrough](#).

## How it works

---

HAMi sits between the Kubernetes scheduler and the NVIDIA GPU driver:

- A Scheduler Extender adds GPU-aware scheduling decisions (filtering and binding) so pods land on nodes with enough GPU capacity.
- A Device Plugin registers virtual GPU resources ([nvidia.com/gpu](https://nvidia.com/gpu), [nvidia.com/gpumem](https://nvidia.com/gpumem), [nvidia.com/gpucore](https://nvidia.com/gpucore)) with kubelet.
- A MutatingWebhook automatically routes GPU pods to the HAMi scheduler.
- HAMi-core ([libvgpu.so](https://github.com/NVIDIA/libvgpu)) is injected into workload containers via `LD_PRELOAD` to enforce memory and compute isolation at the CUDA API level.

When HAMi is enabled, GPU Operator's built-in device plugin is automatically disabled to avoid resource registration conflicts.

## Prerequisites

---

- A tenant Kubernetes cluster with GPU-enabled worker nodes (node groups with GPUs configured).
- GPU Operator add-on enabled on the tenant cluster.

## Enable HAMi

---

Enable both GPU Operator and HAMi in your tenant Kubernetes cluster configuration:

```

apiVersion: apps.cozystack.io/v1alpha1
kind: Kubernetes
metadata:
 name: my-cluster
 namespace: tenant-example
spec:
 nodeGroups:
 gpu-workers:
 minReplicas: 1
 maxReplicas: 3
 instanceType: u1.xlarge
 gpus:
 - name: nvidia.com/GA102GL_A10
 addons:
 gpuOperator:
 enabled: true
 hami:
 enabled: true

```

Apply this configuration:

```
kubectl apply -f my-cluster.yaml
```

## Request fractional GPU resources

Once HAMi is running, workloads can request fractional GPU resources:

```

apiVersion: v1
kind: Pod
metadata:
 name: gpu-workload
spec:
 containers:
 - name: cuda-app
 image: nvcr.io/nvidia/cuda:11.8.0-base-ubuntu20.04
 resources:
 limits:
 nvidia.com/gpu: 1
 nvidia.com/gpumem: 3000
 nvidia.com/gpucores: 30

```

The example above uses absolute memory (`gpumem`). Use `gpumem-percentage` for portability across GPU models with different memory sizes.

| Resource                                  | Description                              |
|-------------------------------------------|------------------------------------------|
| <code>nvidia.com/gpu</code>               | Number of virtual GPUs requested         |
| <code>nvidia.com/gpumem</code>            | GPU memory limit in MiB                  |
| <code>nvidia.com/gpucores</code>          | Percentage of GPU compute cores (1–100)  |
| <code>nvidia.com/gpumem-percentage</code> | GPU memory limit as a percentage (1–100) |

Use `nvidia.com/gpumem-percentage` instead of `nvidia.com/gpumem` when you want a portable limit that works across different GPU models without knowing exact memory sizes.

If `gpumem` and `gpubcores` are omitted, the container gets access to the full GPU's memory and compute capacity. Note that HAMi's virtualization layer is still active — this is not the same as bare-metal GPU passthrough.

## Custom configuration

HAMi's behavior can be tuned through `valuesOverride` in the addon configuration:

```
addons:
 hami:
 enabled: true
 valuesOverride:
 hami:
 devicePlugin:
 deviceSplitCount: 10
 deviceMemoryScaling: 1
 scheduler:
 defaultSchedulerPolicy:
 nodeSchedulerPolicy: binpack
 gpuSchedulerPolicy: spread
```

All parameters below are relative to the `valuesOverride.hami` key shown in the example above.

| Parameter                                                         | Description                                                        | Default              |
|-------------------------------------------------------------------|--------------------------------------------------------------------|----------------------|
| <code>devicePlugin.deviceSplitCount</code>                        | Maximum virtual GPUs per physical GPU                              | 10                   |
| <code>devicePlugin.deviceMemoryScaling</code>                     | Memory overcommit factor (>1.0 enables overcommit)                 | 1                    |
| <code>scheduler.defaultSchedulerPolicy.nodeSchedulerPolicy</code> | Node packing strategy: <code>binpack</code> or <code>spread</code> | <code>binpack</code> |
| <code>scheduler.defaultSchedulerPolicy.gpuSchedulerPolicy</code>  | GPU packing strategy: <code>binpack</code> or <code>spread</code>  | <code>spread</code>  |

## Known limitations

### glibc compatibility

HAMi-core relies on a private glibc symbol (`_dl_sym`) that was removed in glibc 2.34. This affects workload container images only — HAMi's own components and the host OS are not affected.

| Base image | glibc | Isolation |
|------------|-------|-----------|
|------------|-------|-----------|



|               |      |                                                          |
|---------------|------|----------------------------------------------------------|
| Ubuntu 20.04  | 2.31 | Full (memory + compute)                                  |
| Ubuntu 22.04  | 2.35 | Memory isolation only (compute isolation fails silently) |
| Ubuntu 24.04  | 2.39 | No isolation (HAMi-core fails to load silently)          |
| Alpine (musl) | N/A  | Incompatible                                             |

When HAMi-core fails to load, workloads still run but without any GPU resource limits. This can cause GPU out-of-memory errors for colocated workloads.

The distinction between Ubuntu 22.04 and 24.04 behavior is based on upstream testing — see [HAMi-core #174](#) for details.

Most current CUDA 12.x and PyTorch 2.x images use Ubuntu 22.04+, so compute isolation will not work with them. Use images based on Ubuntu 20.04 or older for full isolation until the [upstream fix](#) lands.

## Alpine / musl libc

HAMi-core is incompatible with musl libc. Only glibc-based container images (Debian, Ubuntu, RHEL) are supported.

# Backups with the Velero addon

<https://cozystack.ru/docs/v1.5/kubernetes/backups-with-velero-addon/>

The `velero` addon of the [Managed Kubernetes](#) application installs [Velero](#) inside a tenant Kubernetes cluster. Combined with a tenant [Bucket](#), it lets tenant users back up workloads to S3 and restore them later.

This guide is for the tenant-side Velero addon, which runs inside a tenant Kubernetes cluster and is operated by the tenant user.

For the platform-level Velero used by cluster administrators to back up `VMInstance` / `VMDisk` resources from the management cluster, see [Velero Backup Configuration](#).

## What the addon installs

When `spec.addons.velero.enabled` is `true` on a Kubernetes resource, Cozystack deploys Velero into the `cozy-velero` namespace of the tenant cluster. The chart wraps the upstream `vmware-tanzu/velero` chart as a subchart and pre-configures the AWS S3 plugin, the KubeVirt plugin, the node agent (file-system backup), and the CSI feature (`features: EnableCSI`). The `kubevirt-snapshots VolumeSnapshotClass` (driver `csi.kubevirt.io`) ships in every tenant cluster and is labelled for Velero's CSI plugin, so volume snapshots work out of the box.

The default install does not create a `BackupStorageLocation`. You provide one through `valuesOverride` — typically pointing at a [SeaweedFS Bucket](#) that lives in your tenant.

## Prerequisites

- SeaweedFS is enabled on your tenant (`spec.seaweedfs: true`). See [Object Storage](#).
- The [velero CLI](#) is installed locally.
- You have admin access to the tenant namespace in the management cluster (to create `Bucket` and `Kubernetes` resources) and can fetch the resulting `kubeconfig`.

## 1. Create a bucket for backups

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
 name: velero-backups
 namespace: tenant-example
spec:
 users:
 velero: {}
```

---

Cozystack creates a credentials secret named `bucket-<bucket-name>-<user>-credentials` with `accessKey`, `secretKey`, `endpoint`, and `bucketName` fields. Read the values you will plug into the addon configuration:

```
NS=tenant-example
SECRET=bucket-velero-backups-velero-credentials

BUCKET_NAME=$(kubectl get secret $SECRET -n $NS -o jsonpath='{.data.bucketName}' | base64 -d)
ACCESS_KEY=$(kubectl get secret $SECRET -n $NS -o jsonpath='{.data.accessKey}' | base64 -d)
SECRET_KEY=$(kubectl get secret $SECRET -n $NS -o jsonpath='{.data.secretKey}' | base64 -d)
ENDPOINT=$(kubectl get secret $SECRET -n $NS -o jsonpath='{.data.endpoint}' | base64 -d)

echo "$BUCKET_NAME / https://$ENDPOINT"
```

`endpoint` is a hostname only — Velero needs `https://$ENDPOINT` as the S3 URL.

## 2. Deploy a tenant cluster with the Velero addon

---

The addon chart embeds the upstream Velero chart **as a subchart under the `velero` key**. Override values **must** be nested under `velero`: — placing them at the top level (e.g. `valuesOverride.configuration.backupStorageLocation`) is silently ignored.

Render and apply a Kubernetes manifest using the values from the previous step:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps.cozystack.io/v1alpha1
kind: Kubernetes
metadata:
 name: my-cluster
 namespace: $NS
spec:
 host: my-cluster.tenant-example.cozystack.example.com
 addons:
 velero:
 enabled: true
 valuesOverride:
 velero:
 credentials:
 useSecret: true
 secretContents:
 cloud: |
 [default]
 aws_access_key_id=$ACCESS_KEY
 aws_secret_access_key=$SECRET_KEY
 configuration:
 backupStorageLocation:
 - name: default
 provider: aws
 bucket: $BUCKET_NAME
 default: true
 config:
 region: us-east-1
 s3ForcePathStyle: true
 s3Url: https://$ENDPOINT
 volumeSnapshotLocation:
 - name: default
 provider: aws
 config:
 region: us-east-1
EOF
```

Once the cluster is **Ready**, fetch its kubeconfig and point your shell at it:

```
kubectl get secret -n $NS kubernetes-my-cluster-admin-kubeconfig \
 -o go-template='{{ printf "%s\n" (index .data "admin.conf" | base64decode) }}' \
 > my-cluster-kubeconfig
export KUBECONFIG=$PWD/my-cluster-kubeconfig
```

The remaining steps run against the tenant cluster.

### 3. Verify Velero is running

```
kubectl -n cozy-velero get deploy
velero -n cozy-velero backup-location get
```

Expected output:

| NAME    | STATUS    | PROVIDER | BUCKET                                      | ACCESS MODE |
|---------|-----------|----------|---------------------------------------------|-------------|
| default | Available | aws      | bucket-91bbb59f-30ba-46fe-9a44-535d8332a464 | ReadWrite   |

---

STATUS: Available means Velero successfully connected to the bucket.

## 4. Back up a namespace

---

Create a sample workload (skip if you already have one to back up):

```
kubectl create namespace demo
kubectl -n demo create configmap demo-cm \
 --from-literal=marker=backup-restore-validation
```

Take the backup:

```
velero -n cozy-velero backup create demo-1 \
 --include-namespaces demo --snapshot-move-data
velero -n cozy-velero backup describe demo-1 --details
```

`--snapshot-move-data` uploads CSI snapshot data to the bucket so the backup is self-contained — restores no longer need access to the source PVs.

Wait until Phase: Completed.

## 5. Restore from the backup

---

Simulate a loss by deleting the namespace, then restore it from the backup:

```
kubectl delete namespace demo
velero -n cozy-velero restore create demo-1-restore \
 --from-backup demo-1
velero -n cozy-velero restore describe demo-1-restore --details
```

When the restore reaches Phase: Completed, the namespace and its objects are recreated. Verify:

```
kubectl -n demo get all,configmaps
```

The same pattern restores into a different tenant Kubernetes cluster as well — point a second cluster at the same bucket with an identical `addons.velero.valuesOverride`, and `velero backup get` in the second cluster will see `demo-1` once the bucket sync ticks (default interval one minute). See the upstream [migration scenario](#) for cross-cluster considerations.

## Related

---

- [Managed Kubernetes — addons.velero parameters](#)
  - [Buckets and Users](#)
  - [Velero Backup Configuration \(platform admin\)](#)
-

# How to relocate etcd replicas in tenant clusters

<https://cozystack.ru/docs/v1.5/kubernetes/relocate-etcd/>

Tenant Kubernetes clusters are using their own etcd clusters, not the one that is used by the management cluster. Such etcd clusters are deployed in tenants and are available to managed Kubernetes clusters deployed in the tenant and its sub-tenants. [Cozystack Documentation](#)

Replicas of a tenant etcd cluster can be relocated between nodes for maintenance reasons. Currently, management operations for tenant etcd clusters are not automated, but such a task can be done manually. [Cozystack Documentation](#)

First, you need to install the `kubectl-etcd` plugin for `kubectl`:

```
go install github.com/aenix-io/etcd-operator/cmd/kubectl-etcd@latest
```

Now you can manage etcd replicas. The example script shown below removes the `etcd-2` replica from the etcd cluster and then add it back.

```
tenant which the etcd cluster belongs to
NAMESPACE=tenant-demo

etcd replica
RM=etcd-2
POD=$(kubectl get pod -n "$NAMESPACE" -l app.kubernetes.io/name=etcd --no-headers | awk '$2 == "1/1" && $1 ...
RMID=$(kubectl etcd -n $NAMESPACE -p $POD members | awk '$2 == "'$RM'" {print $1}')

delete the replica
kubectl delete -n $NAMESPACE pvc/data-$RM pod/$RM
if [-n "$RMID"]; then
 kubectl etcd -n $NAMESPACE -p $POD remove-member "$RMID"
fi

add the replica back
kubectl etcd -n $NAMESPACE -p $POD add-member "https://$RM.etcd-headless.$NAMESPACE.svc:2380"

kubectl wait --for=condition=ready pod -n $NAMESPACE $RM --timeout=2m

kubectl etcd -n $NAMESPACE -p $RM members
```

To learn more about tenant nesting and shared services, read the [Tenants guide](#). [Cozystack Documentation](#)



---

# Managed Applications

## Managed Applications: Guides and Reference

<https://cozystack.ru/docs/v1.5/applications/>

### Available Application Versions

---

Cozystack deploys applications in two complementary ways:

- Operator-managed applications – Cozystack bundles a specific version of a Kubernetes Operator that installs and continuously reconciles the application. As a rule, the operator chooses one of the most recent stable versions of the application by default.
  - Chart-managed applications – When no mature operator exists, Cozystack packages an upstream (or in-house) Helm chart. The chart's `appVersion` pin tracks the latest stable upstream release, keeping deployments secure and up-to-date.
- 

### Application Backup and Recovery

---

Back up and restore managed databases (Postgres, MariaDB, ClickHouse) with BackupJob, Plan, and RestoreJob.

### Adding External Applications to Cozystack Catalog

---

Learn how to add managed applications from external sources

### Managed ClickHouse Service

---

### FoundationDB

---

### Managed Harbor Container Registry

---

### Managed Kafka Service

---



---

## Managed MariaDB Service

---

## Managed MongoDB Service

---

## Managed NATS Service

---

## Managed OpenBAO Service

---

## Managed PostgreSQL Service

---

## Managed Qdrant Service

---

## Managed RabbitMQ Service

---

## Managed Redis Service

---

## Tenant Application Reference

---

---

# Application Backup and Recovery

<https://cozystack.ru/docs/v1.5/applications/backup-and-recovery/>

This guide covers backing up and restoring Cozystack-managed databases — Postgres, MariaDB, and ClickHouse — as a tenant user: running one-off and scheduled backups, checking status, and restoring from a backup either in place or into a separate target instance.

**\*\*Storage, credentials, and the BackupClass are admin-provisioned.\*\*** Before you can run a BackupJob, an administrator provisions the S3 storage and the per-application credential Secrets your driver expects, and creates the cluster-scoped BackupClass you reference. Ask your administrator for:

\* the BackupClass name to use for your application Kind (you cannot list BackupClass resources under your tenant kubeconfig — they are cluster-scoped);

\* confirmation that the per-application credential Secrets exist in your namespace for every managed-DB application you want to back up.

>

Admins follow the [Managed Application Backup Configuration](#) guide.

These backups are data-only. Each strategy snapshots the database contents through the operator's native mechanism (CloudNativePG barman, mariadb-operator dumps, Altinity clickhouse-backup). They do not capture the `apps.cozystack.io/*` CR, its `HelmRelease`, chart values, or operator-managed Secrets.

>

To restore you must either:

\* keep the source application alive and restore in place (each driver re-bootstraps data into the existing instance)

\* or pre-provision an empty target application of the same Kind, then restore into it.

>

For backups that include the application's Helm release, CRs, and PVC snapshots (used for VMInstances), see [Backup and Recovery \(VMs\)](#).

## Prerequisites

- A BackupClass name handed to you by your administrator (for example, `postgres-data-backup` for a Postgres application).
- An existing managed-DB application (Postgres, MariaDB, or ClickHouse) in your tenant namespace.
- `kubectl` and a tenant kubeconfig with the `tenant-<ns>-admin` role.
- The examples below assume `tenant-user` for the tenant namespace; substitute your own.

## Run a backup

---

## One-off backup

Use a `BackupJob` for an ad-hoc backup (for example, before a risky change):

```
apiVersion: backups.cozystack.io/v1alpha1
kind: BackupJob
metadata:
 name: my-postgres-adhoc
 namespace: tenant-user
spec:
 applicationRef:
 apiGroup: apps.cozystack.io
 kind: Postgres
 name: my-postgres
 backupClassName: postgres-data-backup
```

When the `BackupJob` reaches phase: `Succeeded`, the driver creates a `Backup` object with the same name. That name is what you reference when restoring.

Replace `Postgres` / `postgres-data-backup` with `MariaDB` / `mariadb-data-backup` or `ClickHouse` / `clickhouse-data-backup` for the other drivers.

## Scheduled backup

Use a `Plan` for cron-driven recurring backups:

```
apiVersion: backups.cozystack.io/v1alpha1
kind: Plan
metadata:
 name: my-postgres-daily
 namespace: tenant-user
spec:
 applicationRef:
 apiGroup: apps.cozystack.io
 kind: Postgres
 name: my-postgres
 backupClassName: postgres-data-backup
 schedule:
 type: cron
 cron: "0 0 * * *" # every day at midnight
```

Each scheduled run creates a `BackupJob` (and, on success, a `Backup`) named after the `Plan` with a timestamp suffix.

```
kubectl -n tenant-user get backupjobs -l backups.cozystack.io/plan=my-postgres-daily
```

---

## Check backup status

List `BackupJob` and `Backup` resources in the namespace:

```
kubectll -n tenant-user get backupjobs
kubectll -n tenant-user get backups
```

Inspect a failed run:

```
kubectll -n tenant-user get backupjob my-postgres-adhoc -o jsonpath='{.status.message}'
kubectll -n tenant-user get events --field-selector involvedObject.name=my-postgres-adhoc
```

If `status.message` does not pinpoint the failure, hand the `BackupJob` name to your administrator and they will inspect the operator-native CR the driver created (see [Tenant escalation: driver-side diagnostics](#) in the admin guide).

## Restore in place

An in-place restore replays the backup into the same application. Use this to roll back accidental deletion or corruption on a live database you intend to keep using under the same name.

In-place restore is destructive. Each driver wipes or replaces existing data on the source application; any writes since the backup point are lost. If you cannot afford to lose recent writes, use [Restore to a copy](#) instead.

```
apiVersion: backups.cozystack.io/v1alpha1
kind: RestoreJob
metadata:
 name: my-postgres-restore-inplace
 namespace: tenant-user
spec:
 backupRef:
 name: my-postgres-adhoc
 # targetApplicationRef omitted: driver restores into Backup.spec.applicationRef.
 # options:
 # recoveryTime: "2024-01-01T00:00:00Z"
```

## Per-driver caveats

- Postgres (CNPG) — the driver deletes the live `cnpg.io/Cluster` and its PVCs, then re-bootstraps from the Barman archive. Connections drop for the duration. `spec.options.recoveryTime` (RFC3339) is supported for point-in-time recovery; omit it to restore to the latest WAL.
- MariaDB — the operator replays the logical dump into the live MariaDB via `mariadb-import`. Pre-existing tables will collide; pre-truncate the relevant schemas if your dump does not include `DROP TABLE`.
- ClickHouse — the Altinity strategy does not pass `clickhouse-backup --rm`. You are responsible for dropping conflicting tables on the source before submitting the `RestoreJob`; otherwise the operation fails with a duplicate-table error.

## Restore to a copy

---

A to-copy restore replays the backup into a different, freshly-provisioned application of the same Kind. Use this for disaster-recovery drills, side-by-side data comparison, or to recover data without impacting the live app.

First, provision an empty target application with the same Kind. For example, an empty `Postgres`:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Postgres
metadata:
 name: my-postgres-restored
 namespace: tenant-user
spec:
 # ...same shape as the source, no bootstrap data required...
```

Wait for the target to become Ready, then submit a `RestoreJob` that points at it:

```
apiVersion: backups.cozystack.io/v1alpha1
kind: RestoreJob
metadata:
 name: my-postgres-restore-to-copy
 namespace: tenant-user
spec:
 backupRef:
 name: my-postgres-adhoc
 targetApplicationRef:
 apiGroup: apps.cozystack.io
 kind: Postgres
 name: my-postgres-restored
```

The source application stays untouched. Cross-namespace restores are not supported — `targetApplicationRef` is a local reference; the target must live in the same namespace as the `RestoreJob`.

## Limitations and lifecycle

- Data-only scope. Application CRs, HelmReleases, chart values, and operator-managed Secrets (e.g. `cnpg.io` superuser secret, `clickhouse-installation` users) are not captured. Pre-provision the target application before a to-copy restore.
- Archive retention is driver-owned. Deleting a Cozystack `Backup` CR removes the artefact reference but leaves the actual S3 object intact. Each driver enforces its own retention:
  - CNPG: `retentionPolicy` on the strategy (admin-owned; default `30d` in the admin example).
  - MariaDB: `cleanupStrategy` on the operator-side `Backup` CR or rotation at the bucket level (admin-owned).
  - ClickHouse: governed by the in-pod sidecar's retention configuration. To purge an archive ahead of schedule, ask your administrator — the call goes against the `clickhouse-backup` HTTP API on the sidecar.
- ClickHouse depends on the in-chart sidecar. The Altinity strategy is a thin HTTP client; the backup itself runs inside each `chi-*` Pod via `clickhouse-backup`. Disabling `backup.enabled` on the application also disables the BackupClass flow.

---

## Troubleshooting

---

If a `BackupJob` or `RestoreJob` ends in phase: `Failed`, start with what you can see in your namespace:

```
kubectl -n tenant-user get backupjob my-postgres-adhoc -o jsonpath='{.status.message}'
kubectl -n tenant-user get restorejob my-postgres-restore-inplace -o jsonpath='{.status.message}'
kubectl -n tenant-user get events --field-selector involvedObject.name=my-postgres-adhoc
```

If those do not explain the failure, the next layer of diagnostics lives on the operator-native CR (e.g. [cnpg.io/Backup](#), [k8s.mariadb.com/Backup](#), or the [ClickHouse strategy Pod logs](#)). These resources are not reachable under the tenant kubeconfig — hand the `BackupJob` name to your administrator and they will follow [Tenant escalation: driver-side diagnostics](#).

## See also

---

- [Managed Application Backup Configuration](#) — how administrators define strategies and `BackupClass` resources.
  - [Backup and Recovery \(VMs\)](#) — the parallel guide for `VMInstance` / `VMDisk` backups (HelmRelease + CRs + PVC snapshots).
  - [Velero Backup Configuration](#) — administrator setup for the Velero-driven VM backups.
-

# Adding External Applications to Cozystack Catalog

<https://cozystack.ru/docs/v1.5/applications/external/>

Cozystack administrators can add applications from external sources in addition to the standard application catalog. These applications appear in the same catalog and behave like regular managed applications for platform users.

This guide explains the structure of an external application package and how to add it to a Cozystack cluster.

For a complete working example, see [github.com/cozystack/external-apps-example](https://github.com/cozystack/external-apps-example).

Just like standard Cozystack applications, this external application package uses Helm and FluxCD. To learn more about developing application packages, read the [Cozystack Developer Guide](#).

## Repository Structure

An external application repository has the following layout:

```
init.yaml # Bootstrap manifest (GitRepository + HelmRelease)
scripts/
 package.mk # Shared Makefile targets for app charts
packages/
 core/platform/ # Platform chart: namespaces, operators, HelmCharts, ApplicationDefinitions
 apps/<app-name>/ # Helm chart for each user-installable application
```

- `packages/core/platform` — a Helm chart deployed by FluxCD. It registers all applications via `ApplicationDefinition` CRDs, creates required namespaces, deploys operators, and defines `HelmChart` resources that point to the app charts in the same Git repository.
- `packages/apps/<app-name>` — standard Helm charts that template the actual Kubernetes resources (CRDs, ConfigMaps, Secrets, etc.).

## Platform Chart

The platform chart (`packages/core/platform/`) is the central piece. It contains templates for:

### Namespaces

Create namespaces for operators and system components:

---

```
apiVersion: v1
kind: Namespace
metadata:
 labels:
 cozystack.io/system: "true"
 name: external-<operator-name>
```

## HelmCharts

Define `HelmChart` resources that tell FluxCD where to find each app chart within the Git repository:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmChart
metadata:
 name: external-apps-<app-name>
 namespace: cozy-public
spec:
 interval: 5m
 chart: ./packages/apps/<app-name>
 sourceRef:
 kind: GitRepository
 name: external-apps
 reconcileStrategy: Revision
```

Use `reconcileStrategy: Revision` so that charts with a static `version: 0.0.0` are re-reconciled whenever the Git content changes.

## Operator Deployment

If your application requires an operator, deploy it via a `HelmRepository` and `HelmRelease`:



---

```

apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
 name: <operator-name>
 namespace: external-<operator-name>
spec:
 type: oci
 interval: 5m
 url: oci://ghcr.io/<org>/charts

apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
 name: <operator-name>
 namespace: external-<operator-name>
spec:
 interval: 5m
 releaseName: <operator-name>
 targetNamespace: external-<operator-name>
 chart:
 spec:
 chart: <operator-chart-name>
 sourceRef:
 kind: HelmRepository
 name: <operator-name>
 version: '>=1.0.0'

```

## ApplicationDefinitions

Register each application in the Cozystack dashboard with an `ApplicationDefinition`:

```

apiVersion: cozystack.io/v1alpha1
kind: ApplicationDefinition
metadata:
 name: <app-name>
spec:
 application:
 kind: <AppKind>
 singular: <appkind>
 plural: <appkinds>
 openAPISchema: '{"title":"Chart Values","type":"object","properties":{...}}'
 release:
 chartRef:
 kind: HelmChart
 name: external-apps-<app-name>
 namespace: cozy-public
 labels:
 cozystack.io/ui: "true"
 prefix: <app-name>-
 dashboard:
 category: <Category>
 singular: <Human-readable Name>
 plural: <Human-readable Names>
 description: <Short description.>
 tags:
 - <tag>
 icon: <base64-encoded SVG>
 keysOrder:
 - - apiVersion
 - - appVersion
 - - kind
 - - metadata
 - - metadata
 - - name
 - - spec
 - <field>

```

Follow these naming conventions (matching the main Cozystack repository):

| Field                | Convention                 | Example for my-app |
|----------------------|----------------------------|--------------------|
| metadata.name        | lowercase, hyphens allowed | my-app             |
| application.kind     | PascalCase, no hyphens     | MyApp              |
| application.singular | lowercase, no hyphens      | myapp              |
| application.plural   | lowercase, no hyphens      | myapps             |
| release.prefix       | <metadata.name>-           | my-app-            |
| openAPISchema title  | always "Chart Values"      | —                  |

The `openAPISchema` field contains a single-line JSON string with the schema for the application values. It intentionally omits `if / then / else` conditional rules because Kubernetes `apiextensions/v1 JSONSchemaProps` does not support these keywords. Use conditional validation only in the Helm chart's `values.schema.json`.

---

## Application Charts

---

Each application chart in `packages/apps/<app-name>/` is a standard Helm chart:

```
packages/apps/<app-name>/
 Chart.yaml
 Makefile
 values.yaml
 values.schema.json
 templates/
 <resource>.yaml
```

### Chart.yaml

```
apiVersion: v2
name: <app-name>
description: <Short description>
type: application
version: 0.0.0
appVersion: "1.0.0"
```

Use `version: 0.0.0` — the actual version is derived from the Git revision by FluxCD.

### Makefile

```
export NAME=<app-name>
export NAMESPACE=external-<operator-name>

include ../../../../scripts/package.mk
```

### values.schema.json

Define the JSON Schema (draft-07) for the application values. This schema is used by Helm for validation at install time and can include conditional rules (`if / then / else`) that are not supported at the `ApplicationDefinition` level.

---

## Bootstrap Manifest

---

The `init.yaml` file creates two FluxCD resources that bootstrap the entire catalog:

```

apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
 name: external-apps
 namespace: cozy-public
spec:
 interval: 1m0s
 ref:
 branch: main
 timeout: 60s
 url: https://github.com/<org>/<repo>.git

apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
 name: external-apps
 namespace: cozy-system
spec:
 interval: 5m
 targetNamespace: cozy-system
 chart:
 spec:
 chart: ./packages/core/platform
 sourceRef:
 kind: GitRepository
 name: external-apps
 namespace: cozy-public
 reconcileStrategy: Revision
```

Apply it to your Cozystack cluster:

```
kubectl apply -f init.yaml
```

After FluxCD reconciles, the applications will appear in the Cozystack dashboard.

## FluxCD Reference

---

These FluxCD documents will help you understand the resources used in this guide:

- [GitRepository](#)
  - [HelmRelease](#)
  - [HelmChart](#)
-

# Managed ClickHouse Service

<https://cozystack.ru/docs/v1.5/applications/clickhouse/>

**ClickHouse** is an open source high-performance and column-oriented SQL database management system (DBMS). It is used for online analytical processing (OLAP).

## Backup orchestration

Two backup paths coexist on the chart; pick one per release.

### Recommended: BackupClass + Plan via the Altinity strategy

The cluster-scoped **Altinity** strategy (`strategy.backups.cozystack.io/v1alpha1`) wraps **Altinity's clickhouse-backup** in a one-shot `batch/v1.Job` per `BackupJob` / `RestoreJob`. It is engine-aware (`FREEZE + upload`), supports both in-place and `targetApplicationRef` (to-copy) restore, and does not require any in-chart `CronJob`.

Wiring per release:

1. Set `backup.enabled: true` plus the `s3*` (or `s3CredentialsSecret.name`) fields on the chart values. The chart materialises a Secret named `<release>-backup-s3` carrying bucket coordinates and credentials. That Secret is consumed by the chart-emitted `clickhouse-backup` sidecar (rendered into the `ClickHouseInstallation` Pod by `templates/clickhouse.yaml`); the strategy Pod itself is a `curl/jq` client that reaches the sidecar's HTTP API on port 7171 and never reads the Secret directly.
2. Cluster admin one-time installs an **Altinity** strategy and a **BackupClass** that maps `apps.cozystack.io/ClickHouse` to it (see `examples/backups/clickhouse/01-create-strategy.sh` and `02-create-backupclass.sh`).
3. Tenant creates a **Plan** (cron schedule) or submits an ad-hoc **BackupJob** referencing the **BackupClass**. Restoring is a **RestoreJob** referencing the resulting **Backup**; omit `targetApplicationRef` for in-place, set it to a second ClickHouse instance for to-copy.

### Bringing your own S3 credentials Secret

Setting `backup.s3CredentialsSecret.name` makes the chart skip the materialisation of `<release>-backup-s3` and points the sidecar at the referenced Secret instead. The Secret must hold the five S3 fields as separate string keys, not a JSON blob; the per-key field names default to `bucketName` / `endpoint` / `region` / `accessKey` / `secretKey` and can be remapped via `backup.s3CredentialsSecret.{bucketKey,endpointKey,...}`.

Example:

```

apiVersion: v1
kind: Secret
metadata:
 name: my-s3-creds
 namespace: tenant-test
type: Opaque
stringData:
 bucketName: my-clickhouse-archive
 endpoint: https://s3.example.org
 region: us-east-1
 accessKey: AKIAIOSFODNN7EXAMPLE
 secretKey: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

In ClickHouse values.yaml:
backup:
 enabled: true
 s3CredentialsSecret:
 name: my-s3-creds

```

The Cozystack Bucket app's BucketInfo Secret (bucket-`<name>-<user>`, single BucketInfo key holding a JSON blob) is not directly consumable as s3CredentialsSecret. Either let the chart read the raw values via backup.s3\* (as examples/backups/clickhouse/03-create-bucket.sh does — extract coordinates from BucketInfo and pass them to the chart values), or materialise an intermediate Secret with the five string keys above.

## Legacy: chart-emitted Restic CronJob

The chart still ships a CronJob that streams per-table `SHOW CREATE TABLE + SELECT * FORMAT TabSeparated` into a Restic repository. This path is kept for backward compatibility; new installations should prefer the Altinity strategy.

To opt in:

Set `backup.enabled: true`, `backup.schedule: "..."` (or another non-empty cron), `backup.s3*` and `backup.restrictPassword`.

To restore manually:

```

restic -r s3:s3.example.org/clickhouse-backups/table_name snapshots
restic -r s3:s3.example.org/clickhouse-backups/table_name restore latest --target /tmp

```

For more details, read [Restic: Effective Backup from Stdin](#).

The legacy `backup.schedule`, `backup.cleanupStrategy`, and `backup.restrictPassword` values are deprecated and will be removed once the Altinity strategy is the default in all reference deployments. The chart-emitted CronJob renders only when `backup.schedule` is non-empty.

`storageClass` is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it.

## Parameters

## Common parameters

| Name                          | Description                                                                                                                                     | Type     | Value                 |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| <code>replicas</code>         | Number of ClickHouse replicas.                                                                                                                  | int      | 2                     |
| <code>shards</code>           | Number of ClickHouse shards.                                                                                                                    | int      | 1                     |
| <code>resources</code>        | Explicit CPU and memory configuration for each ClickHouse replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| <code>resources.cpu</code>    | CPU available to each replica.                                                                                                                  | quantity | ""                    |
| <code>resources.memory</code> | Memory (RAM) available to each replica.                                                                                                         | quantity | ""                    |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                              | string   | <code>t1.small</code> |
| <code>size</code>             | Persistent Volume Claim size available for application data.                                                                                    | quantity | <code>10Gi</code>     |
| <code>storageClass</code>     | StorageClass used to store the data.                                                                                                            | string   | ""                    |

## Application-specific parameters

| Name                              | Description                                                                          | Type              | Value            |
|-----------------------------------|--------------------------------------------------------------------------------------|-------------------|------------------|
| <code>logStorageSize</code>       | Size of Persistent Volume for logs.                                                  | quantity          | <code>2Gi</code> |
| <code>logTTL</code>               | TTL (expiration time) for <code>query_log</code> and <code>query_thread_log</code> . | int               | 15               |
| <code>users</code>                | Map of users to their configurations.                                                | map[string]object | <code>{}</code>  |
| <code>users[name].password</code> | User password.                                                                       | string            | ""               |

|                                   |                                |      |       |
|-----------------------------------|--------------------------------|------|-------|
| <code>users[name].readonly</code> | Whether the user is read-only. | bool | false |
|-----------------------------------|--------------------------------|------|-------|

## Backup parameters

| Name                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Type   | Value           |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|-----------------|
| <code>backup</code>                 | Backup configuration.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | object | <code>{}</code> |
| <code>backup.enabled</code>         | Enable backup integration. Materialises the chart-emitted <code>&lt;release&gt;-backup-s3</code> Secret consumed by the Altinity backup strategy and, when <code>schedule</code> is non-empty, also renders the legacy chart-managed CronJob.                                                                                                                                                                                                                                                             | bool   | false           |
| <code>backup.useSystemBucket</code> | Opt-in: when true, the chart-emitted <code>&lt;release&gt;-backup-s3</code> Secret is skipped and the <code>clickhouse-backup</code> sidecar reads bucket coordinates + S3 credentials from the platform-projected <code>cozy-backups-creds</code> Secret. <code>S3_PATH</code> is set to <code>&lt;namespace&gt;/&lt;release&gt;</code> for cross-tenant isolation. Use together with the platform <code>cozy-default BackupClass</code> — tenants do not need to fill any <code>s3*</code> field below. | bool   | false           |



|                                     |                                                                                                                                                                                                                                                                                                |        |                                                      |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|------------------------------------------------------|
| <code>backup.s3Region</code>        | DEPRECATED. Per-tenant S3 configuration is being phased out in favour of the platform-managed <code>cozy-default BackupClass</code> . Optional now so tenants on <code>useSystemBucket: true</code> can omit it.                                                                               | string | us-east-1                                            |
| <code>backup.s3Bucket</code>        | DEPRECATED. Optional; see <code>s3Region</code> .                                                                                                                                                                                                                                              | string | s3.example.org/clickhouse-backups                    |
| <code>backup.endpoint</code>        | S3 endpoint URL.                                                                                                                                                                                                                                                                               | string | ""                                                   |
| <code>backup.s3PathOverride</code>  | DEPRECATED. Object-key prefix inside the legacy <code>s3Bucket</code> ; the new platform flow scopes by namespace automatically.                                                                                                                                                               | string | ""                                                   |
| <code>backup.schedule</code>        | Legacy. Cron schedule for the chart-emitted <code>CronJob</code> that runs the <code>dump+restic backup</code> . Empty (default) skips rendering it; the <code>BackupClass + Plan</code> from <code>backups.cozystack.io</code> already drives backup orchestration via the Altinity strategy. | string | ""                                                   |
| <code>backup.cleanupStrategy</code> | Legacy. Restic retention policy passed to the legacy <code>CronJob</code> ( <code>restic forget ...</code> ). Unused by the Altinity strategy.                                                                                                                                                 | string | --keep-last=3 --keep-daily=3 --keep-within-weekly=1m |
| <code>backup.s3AccessKey</code>     | DEPRECATED. Tenants no longer supply S3 keys; the platform-managed Bucket Secret is the source of truth. With <code>useSystemBucket: true</code> tenants can omit it.                                                                                                                          | string | <your-access-key>                                    |

|                                                            |                                                                                                                                                                     |        |                                      |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------------------------------------|
| <code>backup.s3SecretKey</code>                            | DEPRECATED. Optional; see <code>s3AccessKey</code> .                                                                                                                | string | <code>&lt;your-secret-key&gt;</code> |
| <code>backup.resticPassword</code>                         | Restic password for the legacy backup.                                                                                                                              | string | <code>&lt;password&gt;</code>        |
| <code>backup.s3CredentialsSecret</code>                    | DEPRECATED. Pre-existing Secret reference for the legacy chart-emitted sidecar. The platform flow projects <code>cozy-backups-creds</code> instead.                 | object | <code>{}</code>                      |
| <code>backup.s3CredentialsSecret.name</code>               | Name of the Secret in the application namespace. Empty means the chart materialises <code>&lt;release&gt;-backup-s3</code> from the legacy <code>s3*</code> fields. | string | <code>""</code>                      |
| <code>backup.s3CredentialsSecret.bucketKey</code>          | Key in the Secret holding the bucket name. Defaults to <code>bucketName</code> .                                                                                    | string | <code>""</code>                      |
| <code>backup.s3CredentialsSecret.endpointKey</code>        | Key in the Secret holding the S3 endpoint URL. Defaults to <code>endpoint</code> .                                                                                  | string | <code>""</code>                      |
| <code>backup.s3CredentialsSecret.regionKey</code>          | Key in the Secret holding the S3 region. Defaults to <code>region</code> .                                                                                          | string | <code>""</code>                      |
| <code>backup.s3CredentialsSecret.accessKeyIDKey</code>     | Key in the Secret holding the access key ID. Defaults to <code>accessKey</code> .                                                                                   | string | <code>""</code>                      |
| <code>backup.s3CredentialsSecret.secretAccessKeyKey</code> | Key in the Secret holding the secret access key. Defaults to <code>secretKey</code> .                                                                               | string | <code>""</code>                      |

## ClickHouse Keeper parameters

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

|                                               |                                                     |          |                 |
|-----------------------------------------------|-----------------------------------------------------|----------|-----------------|
| <code>clickhouseKeeper</code>                 | ClickHouse Keeper configuration.                    | object   | <code>{}</code> |
| <code>clickhouseKeeper.enabled</code>         | Enable ClickHouse Keeper.                           | bool     | true            |
| <code>clickhouseKeeper.size</code>            | Persistent Volume Claim size for ClickHouse Keeper. | quantity | 1Gi             |
| <code>clickhouseKeeper.resourcesPreset</code> | Resources preset for ClickHouse Keeper.             | string   | t1.micro        |
| <code>clickhouseKeeper.replicas</code>        | Number of ClickHouse Keeper replicas.               | int      | 3               |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

| Preset  | CPU  | Memory |
|---------|------|--------|
| nano    | 250m | 128Mi  |
| micro   | 500m | 256Mi  |
| small   | 1    | 512Mi  |
| medium  | 1    | 1Gi    |
| large   | 2    | 2Gi    |
| xlarge  | 4    | 4Gi    |
| 2xlarge | 8    | 8Gi    |

# FoundationDB

<https://cozystack.ru/docs/v1.5/applications/foundationdb/>

A managed FoundationDB service for Cozystack.

## Overview

FoundationDB is a distributed database designed to handle large volumes of structured data across clusters of commodity servers. It organizes data as an ordered key-value store and employs ACID transactions for all operations.

This package provides a managed FoundationDB cluster deployment using the FoundationDB Kubernetes Operator.

## Features

- High Availability: Multi-instance deployment with automatic failover
- ACID Transactions: Full ACID transaction support across the cluster
- Scalable: Easily scale storage and compute resources
- Backup Integration: Optional S3-compatible backup storage
- Monitoring: Built-in monitoring and alerting through WorkloadMonitor
- Flexible Configuration: Support for custom FoundationDB parameters

## Configuration

### Basic Configuration

```
Cluster process configuration
cluster:
 version: "7.3.63"
 processCounts:
 storage: 3 # Number of storage processes (determines cluster size)
 stateless: -1 # Automatically calculated
 cluster_controller: 1
 faultDomain:
 key: "kubernetes.io/hostname"
 valueFrom: "spec.nodeName"
```

## Storage

```
storage:
 size: "16Gi" # Storage size per instance
 storageClass: "" # Storage class (optional)
```

## Resources

```
Use preset sizing
resourcesPreset: "medium" # small, medium, large, xlarge, 2xlarge

Or custom resource configuration
resources:
 cpu: "2000m"
 memory: "4Gi"
```

## Backups

The recommended backup flow uses the Cozystack backups framework: a cluster-scoped `BackupClass` binds `apps.cozystack.io/FoundationDB` to a `strategy.backups.cozystack.io/FoundationDB` strategy, and tenants drive runs via `BackupJob` / `RestoreJob`. See [\[examples/backups/foundationdb/\]\(https://cozystack.ru/docs/examples/backups/foundationdb/\)](https://cozystack.ru/docs/examples/backups/foundationdb/) for an end-to-end walkthrough (admin setup, backup, in-place restore, to-copy restore).

The legacy in-chart `backup.*` values (`backup.enabled`, `backup.s3`, `backup.retentionPolicy`) are DEPRECATED but still render the original `FoundationDBBackup` CR unchanged when `backup.enabled=true`, for backward compatibility. New deployments should leave `backup.enabled=false` (the default) and use the `BackupClass` flow above. Mixing the two on the same cluster is unsupported — the `FoundationDB` operator only permits one running backup directory per cluster, and the backup driver fails fast with `Ready=False/ConflictingInChartBackup` when it sees a chart-rendered Running backup against the same target.

## Advanced Configuration

```
Custom FoundationDB parameters
customParameters:
 - "knob_disable_posix_kernel_aio=1"

Image type (split is the default; matches existing clusters' effective value)
imageType: "split"

Enable automatic pod replacements
automaticReplacements: true

Security context configuration
securityContext:
 runAsUser: 4059
 runAsGroup: 4059
```

## Prerequisites

- 
- FoundationDB Operator must be installed in the cluster
  - Sufficient storage and compute resources
  - For backups: S3-compatible storage credentials

## Deployment

---

1. Install the FoundationDB operator (system package)
2. Deploy this application package with your desired configuration
3. The cluster will be automatically provisioned and configured

## Monitoring

---

This package includes WorkloadMonitor integration for cluster health monitoring and resource tracking. Monitoring can be disabled by setting:

```
monitoring:
 enabled: false
```

## Security

---

- All containers run with restricted security contexts
- No privilege escalation allowed
- Read-only root filesystem where possible
- Custom security context configurations supported

## Fault Tolerance

---

FoundationDB is designed for high availability:

- Automatic failure detection and recovery
- Data replication across instances
- Configurable fault domains for rack/zone awareness
- Transaction log redundancy

The included `WorkloadMonitor` is automatically configured based on the `cluster.redundancyMode` value. It sets the `minReplicas` property on the `WorkloadMonitor` resource to ensure the cluster's health status accurately reflects its fault tolerance level. The number of tolerated failures is as follows:

- `single`: 0 failures
- `double`: 1 failure
- `triple` and datacenter-aware modes: 2 failures

---

For example, with the default configuration (`redundancyMode: double` and 3 storage pods), `minReplicas` will be set to 2.

## Performance Considerations

---

- Use SSD storage for better performance
- Consider dedicating nodes for storage processes
- Monitor cluster metrics for optimization opportunities
- Scale storage and stateless processes based on workload

## Support

---

For issues related to FoundationDB itself, refer to the [FoundationDB documentation](#).

For Cozystack-specific issues, consult the Cozystack documentation or support channels.

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(https://cozystack.ru/docs/docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

## Parameters

---

### Common parameters

| Name                                                  | Description                                            | Type   | Value           |
|-------------------------------------------------------|--------------------------------------------------------|--------|-----------------|
| <code>cluster</code>                                  | Cluster configuration.                                 | object | <code>{}</code> |
| <code>cluster.processCounts</code>                    | Process counts for different roles.                    | object | <code>{}</code> |
| <code>cluster.processCounts.stateless</code>          | Number of stateless processes (-1 for automatic).      | int    | -1              |
| <code>cluster.processCounts.storage</code>            | Number of storage processes (determines cluster size). | int    | 3               |
| <code>cluster.processCounts.cluster_controller</code> | Number of cluster controller processes.                | int    | 1               |
| <code>cluster.version</code>                          | Version of FoundationDB to use.                        | string | 7.3.63          |

|                                            |                                                                                                                                                    |          |                        |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------------|
| <code>cluster.redundancyMode</code>        | Database redundancy mode (single, double, triple, three_datacenter, three_datacenter_fallback).                                                    | string   | double                 |
| <code>cluster.storageEngine</code>         | Storage engine (ssd-2, ssd-redwood-v1, ssd-rocksdb-v1, memory).                                                                                    | string   | ssd-2                  |
| <code>cluster.faultDomain</code>           | Fault domain configuration.                                                                                                                        | object   | {}                     |
| <code>cluster.faultDomain.key</code>       | Fault domain key.                                                                                                                                  | string   | kubernetes.io/hostname |
| <code>cluster.faultDomain.valueFrom</code> | Fault domain value source.                                                                                                                         | string   | spec.nodeName          |
| <code>storage</code>                       | Storage configuration.                                                                                                                             | object   | {}                     |
| <code>storage.size</code>                  | Size of persistent volumes for each instance.                                                                                                      | quantity | 16Gi                   |
| <code>storage.storageClass</code>          | Storage class (if not set, uses cluster default).                                                                                                  | string   | ""                     |
| <code>resources</code>                     | Explicit CPU and memory configuration for each FoundationDB instance. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | {}                     |
| <code>resources.cpu</code>                 | CPU available to each instance.                                                                                                                    | quantity | ""                     |
| <code>resources.memory</code>              | Memory (RAM) available to each instance.                                                                                                           | quantity | ""                     |
| <code>resourcesPreset</code>               | Default sizing preset used when <code>resources</code> is omitted.                                                                                 | string   | c1.small               |



|                                                    |                                                                                                             |        |                 |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------|-----------------|
| <code>backup</code>                                | DEPRECATED Backup configuration (use the Cozystack backups framework: BackupClass + FoundationDB strategy). | object | <code>{}</code> |
| <code>backup.enabled</code>                        | DEPRECATED Enable in-chart backups (superseded by BackupClass + FoundationDB strategy).                     | bool   | false           |
| <code>backup.s3</code>                             | DEPRECATED S3 configuration for backups.                                                                    | object | <code>{}</code> |
| <code>backup.s3.bucket</code>                      | DEPRECATED S3 bucket name.                                                                                  | string | ""              |
| <code>backup.s3.endpoint</code>                    | DEPRECATED S3 endpoint URL.                                                                                 | string | ""              |
| <code>backup.s3.region</code>                      | DEPRECATED S3 region.                                                                                       | string | us-east-1       |
| <code>backup.s3.credentials</code>                 | DEPRECATED S3 credentials.                                                                                  | object | <code>{}</code> |
| <code>backup.s3.credentials.accessKeyId</code>     | DEPRECATED S3 access key ID.                                                                                | string | ""              |
| <code>backup.s3.credentials.secretAccessKey</code> | DEPRECATED S3 secret access key.                                                                            | string | ""              |
| <code>backup.retentionPolicy</code>                | DEPRECATED Retention policy for backups.                                                                    | string | 7d              |
| <code>monitoring</code>                            | Monitoring configuration.                                                                                   | object | <code>{}</code> |
| <code>monitoring.enabled</code>                    | Enable WorkloadMonitor integration.                                                                         | bool   | true            |

## FoundationDB configuration

| Name                          | Description                                | Type     | Value           |
|-------------------------------|--------------------------------------------|----------|-----------------|
| <code>customParameters</code> | Custom parameters to pass to FoundationDB. | []string | <code>[]</code> |
| <code>imageType</code>        | Container image deployment type.           | string   | split           |

---

|                                         |                                    |        |                 |
|-----------------------------------------|------------------------------------|--------|-----------------|
| <code>securityContext</code>            | Security context for containers.   | object | <code>{}</code> |
| <code>securityContext.runAsUser</code>  | User ID to run the container.      | int    | 4059            |
| <code>securityContext.runAsGroup</code> | Group ID to run the container.     | int    | 4059            |
| <code>automaticReplacements</code>      | Enable automatic pod replacements. | bool   | true            |

---

# Managed Harbor Container Registry

<https://cozystack.ru/docs/v1.5/applications/harbor/>

Harbor is an open-source trusted cloud-native registry project that stores, signs, and scans content.

[Cozystack Documentation](#)

## Prerequisites

The Cozystack Harbor app stores its registry data exclusively in S3-compatible object storage: the chart pins the registry backend to S3 and exposes no filesystem option. That bucket is provisioned through COSI (`objectstorage.k8s.io`) from a SeaweedFS deployment, so before deploying Harbor the tenant must have SeaweedFS available — enabled on the same tenant or inherited from a parent tenant (the resolved class is propagated down the tenant tree, surfaced as the `namespace.cozystack.io/seaweedfs` namespace annotation). [Cozystack Documentation](#)

Enable it by setting `seaweedfs: true` on the tenant (or a parent tenant):

```
seaweedfs: true
```

Without object storage in the tenant chain, Harbor cannot provision its registry bucket: the `<release>-registry BucketClaim/BucketAccess` never produces the `<release>-registry-bucket` credentials secret, so the Harbor `HelmRelease` stays unreconciled, waiting on `BucketInfo`. [Cozystack Documentation](#)

`storageClass` is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it. [Cozystack Documentation](#)

## Parameters

### Common parameters

| Name         | Description                                                                                  | Type   | Value |
|--------------|----------------------------------------------------------------------------------------------|--------|-------|
| host         | Hostname for external access to Harbor (defaults to 'harbor' subdomain for the tenant host). | string | ""    |
| storageClass | StorageClass used to store the data.                                                         | string | ""    |

[Cozystack Documentation](#)

## Component configuration

| Name                      | Description                                                                                                         | Type     | Value                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| core                      | Core API server configuration.                                                                                      | object   | <code>{}</code>       |
| core.resources            | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| core.resources.cpu        | Number of CPU cores allocated.                                                                                      | quantity | <code>""</code>       |
| core.resources.memory     | Amount of memory allocated.                                                                                         | quantity | <code>""</code>       |
| core.resourcesPreset      | Default sizing preset used when <code>resources</code> is omitted.                                                  | string   | <code>t1.small</code> |
| registry                  | Container image registry configuration.                                                                             | object   | <code>{}</code>       |
| registry.resources        | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| registry.resources.cpu    | Number of CPU cores allocated.                                                                                      | quantity | <code>""</code>       |
| registry.resources.memory | Amount of memory allocated.                                                                                         | quantity | <code>""</code>       |
| registry.resourcesPreset  | Default sizing preset used when <code>resources</code> is omitted.                                                  | string   | <code>t1.small</code> |
| jobservice                | Background job service configuration.                                                                               | object   | <code>{}</code>       |

|                                          |                                                                                                                     |                       |                      |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------|
| <code>jobservice.resources</code>        | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>      |
| <code>jobservice.resources.cpu</code>    | Number of CPU cores allocated.                                                                                      | <code>quantity</code> | <code>""</code>      |
| <code>jobservice.resources.memory</code> | Amount of memory allocated.                                                                                         | <code>quantity</code> | <code>""</code>      |
| <code>jobservice.resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                  | <code>string</code>   | <code>t1.nano</code> |
| <code>trivy</code>                       | Trivy vulnerability scanner configuration.                                                                          | <code>object</code>   | <code>{}</code>      |
| <code>trivy.enabled</code>               | Enable or disable the vulnerability scanner.                                                                        | <code>bool</code>     | <code>true</code>    |
| <code>trivy.size</code>                  | Persistent Volume size for vulnerability database cache.                                                            | <code>quantity</code> | <code>5Gi</code>     |
| <code>trivy.resources</code>             | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>      |
| <code>trivy.resources.cpu</code>         | Number of CPU cores allocated.                                                                                      | <code>quantity</code> | <code>""</code>      |
| <code>trivy.resources.memory</code>      | Amount of memory allocated.                                                                                         | <code>quantity</code> | <code>""</code>      |
| <code>trivy.resourcesPreset</code>       | Default sizing preset used when <code>resources</code> is omitted.                                                  | <code>string</code>   | <code>t1.nano</code> |
| <code>database</code>                    | PostgreSQL database configuration.                                                                                  | <code>object</code>   | <code>{}</code>      |
| <code>database.size</code>               | Persistent Volume size for database storage.                                                                        | <code>quantity</code> | <code>5Gi</code>     |
| <code>database.replicas</code>           | Number of database instances.                                                                                       | <code>int</code>      | <code>2</code>       |

---

|                             |                                           |                       |                  |
|-----------------------------|-------------------------------------------|-----------------------|------------------|
| <code>redis</code>          | Redis cache configuration.                | <code>object</code>   | <code>{}</code>  |
| <code>redis.size</code>     | Persistent Volume size for cache storage. | <code>quantity</code> | <code>1Gi</code> |
| <code>redis.replicas</code> | Number of Redis replicas.                 | <code>int</code>      | <code>2</code>   |

[Cozystack Documentation](#)

---

# Managed Kafka Service

<https://cozystack.ru/docs/v1.5/applications/kafka/>

Both `kafka.storageClass` and `zookeeper.storageClass` are annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it.

## Parameters

### Common parameters

| Name                  | Description                                                                                                                                                                                                                                                                                                                                                                                   | Type                | Value              |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|--------------------|
| <code>external</code> | Enable external access from outside the cluster.                                                                                                                                                                                                                                                                                                                                              | <code>bool</code>   | <code>false</code> |
| <code>tls</code>      | TLS configuration. Strimzi manages the cluster PKI automatically (no cert-manager is involved for this chart): the operator auto-creates <code>&lt;release&gt;-cluster-ca-cert</code> and <code>&lt;release&gt;-clients-ca-cert</code> secrets, both exposed for client trust setup. The internal TLS listener on 9093 is always on; this toggle only controls the external listener on 9094. | <code>object</code> | <code>{}</code>    |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                |                    |                   |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|
| <code>tls.enabled</code> | Enable TLS on the external listener. When unset, inherits the value of <code>external</code> (TLS is on when external access is enabled). Warning: setting this to false while external is true exposes Kafka over plaintext on a public IP via LoadBalancer. Strimzi does not provide authentication on this listener unless SCRAM, mTLS, or OAuth is separately configured. Use only in controlled networks. | <code>*bool</code> | <code>null</code> |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|

## Application-specific parameters

| Name                              | Description           | Type                  | Value           |
|-----------------------------------|-----------------------|-----------------------|-----------------|
| <code>topics</code>               | Topics configuration. | <code>[]object</code> | <code>[]</code> |
| <code>topics[i].name</code>       | Topic name.           | <code>string</code>   | <code>""</code> |
| <code>topics[i].partitions</code> | Number of partitions. | <code>int</code>      | <code>0</code>  |
| <code>topics[i].replicas</code>   | Number of replicas.   | <code>int</code>      | <code>0</code>  |
| <code>topics[i].config</code>     | Topic configuration.  | <code>object</code>   | <code>{}</code> |

## Kafka configuration

| Name                        | Description               | Type                | Value           |
|-----------------------------|---------------------------|---------------------|-----------------|
| <code>kafka</code>          | Kafka configuration.      | <code>object</code> | <code>{}</code> |
| <code>kafka.replicas</code> | Number of Kafka replicas. | <code>int</code>    | <code>3</code>  |



|                                     |                                                                                                                     |          |                       |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| <code>kafka.resources</code>        | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| <code>kafka.resources.cpu</code>    | CPU available to each replica.                                                                                      | quantity | <code>" "</code>      |
| <code>kafka.resources.memory</code> | Memory (RAM) available to each replica.                                                                             | quantity | <code>" "</code>      |
| <code>kafka.resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                  | string   | <code>cl.small</code> |
| <code>kafka.size</code>             | Persistent Volume size for Kafka.                                                                                   | quantity | <code>10Gi</code>     |
| <code>kafka.storageClass</code>     | StorageClass used to store the Kafka data.                                                                          | string   | <code>" "</code>      |

## ZooKeeper configuration

| Name                                    | Description                                                                                                         | Type     | Value                 |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| <code>zookeeper</code>                  | ZooKeeper configuration.                                                                                            | object   | <code>{}</code>       |
| <code>zookeeper.replicas</code>         | Number of ZooKeeper replicas.                                                                                       | int      | <code>3</code>        |
| <code>zookeeper.resources</code>        | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| <code>zookeeper.resources.cpu</code>    | CPU available to each replica.                                                                                      | quantity | <code>" "</code>      |
| <code>zookeeper.resources.memory</code> | Memory (RAM) available to each replica.                                                                             | quantity | <code>" "</code>      |
| <code>zookeeper.resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                  | string   | <code>cl.small</code> |

|                                     |                                                |          |     |
|-------------------------------------|------------------------------------------------|----------|-----|
| <code>zookeeper.size</code>         | Persistent Volume size for ZooKeeper.          | quantity | 5Gi |
| <code>zookeeper.storageClass</code> | StorageClass used to store the ZooKeeper data. | string   | " " |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [docs/operations/resource-presets.md](#) for the full size matrix and the legacy-to-instance-type mapping.

### Authentication

This chart does not configure listener authentication. When TLS is enabled on the external listener, clients can connect without credentials. To require authentication, use Strimzi's `KafkaUser` resource with an appropriate authentication type (`tls`, `scram-sha-512`, or `oauth`) outside this chart. See the [Strimzi documentation on KafkaUser](#) for details.

### topics

---

```
topics:
- name: Results
 partitions: 1
 replicas: 3
 config:
 min.insync.replicas: 2
- name: Orders
 config:
 cleanup.policy: compact
 segment.ms: 3600000
 max.compaction.lag.ms: 5400000
 min.insync.replicas: 2
 partitions: 1
 replicas: 3
```

---

# Managed MariaDB Service

<https://cozystack.ru/docs/v1.5/applications/mariadb/>

The Managed MariaDB Service offers a powerful and widely used relational database solution. This service allows you to create and manage a replicated MariaDB cluster seamlessly.

## Deployment Details

This managed service is controlled by mariadb-operator, ensuring efficient management and seamless operation.

- Docs: <https://mariadb.com/kb/en/documentation/>
- GitHub: <https://github.com/mariadb-operator/mariadb-operator>

storageClass is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it.

## HowTos

### How to switch master/slave replica

```
kubectl edit mariadb <instance>
```

update:

```
spec:
 replication:
 primary:
 podIndex: 1
```

check status:

| NAME       | READY | STATUS  | PRIMARY POD | AGE |
|------------|-------|---------|-------------|-----|
| <instance> | True  | Running | app-db1-1   | 41d |

### How to back up and restore a MariaDB application

The recommended path is the Cozystack BackupClass / Plan / RestoreJob flow with the operator-native `strategy.backups.cozystack.io/MariaDB` strategy. See [examples/backups/mariadb](#) for a numbered, end-to-end walkthrough (bucket, source, strategy, BackupClass, Plan, ad-hoc BackupJob, and both in-place + to-copy RestoreJob fixtures).

---

The chart's `backup.*` block (mariadb-dump + restic CronJob) is deprecated and remains supported for backward compatibility only. Existing tenants can keep using it unchanged; new deployments should use the operator-native flow above.

## How to restore from a deprecated restic-based backup

find snapshot:

```
restic -r s3:s3.example.org/mariadb-backups/database_name snapshots
```

restore:

```
restic -r s3:s3.example.org/mariadb-backups/database_name restore latest --target /tmp/
```

more details:

- <https://blog.aenix.io/restic-effective-backup-from-stdin-4bc1e8f083c1>

## Known issues

- Replication can't be finished with various errors
- **\*\*Replication can't be finished in case if `binlog` purged\*\***

Until `mariadbbackup` is not used to bootstrap a node by mariadb-operator (this feature is not implemented yet), follow these manual steps to fix it:

<https://github.com/mariadb-operator/mariadb-operator/issues/141#issuecomment-1804760231>

- Corrupted indices

Sometimes some indices can be corrupted on master replica, you can recover them from slave:

```
mysqldump -h <slave> -P 3306 -u<user> -p<password> --column-statistics=0 <database> <table> ~/tmp/fix-table...
mysql -h <master> -P 3306 -u<user> -p<password> <database> < ~/tmp/fix-table.sql
```

## Parameters

### Common parameters

| Parameter | Description                 | Type | Default |
|-----------|-----------------------------|------|---------|
| replicas  | Number of MariaDB replicas. | int  | 2       |

|                  |                                                                                                                                              |          |                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------|
| resources        | Explicit CPU and memory configuration for each MariaDB replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>      |
| resources.cpu    | CPU available to each replica.                                                                                                               | quantity |                      |
| resources.memory | Memory (RAM) available to each replica.                                                                                                      | quantity |                      |
| resourcesPreset  | Default sizing preset used when <code>resources</code> is omitted.                                                                           | string   | <code>t1.nano</code> |
| size             | Persistent Volume Claim size available for application data.                                                                                 | quantity | <code>10Gi</code>    |
| storageClass     | StorageClass used to store the data.                                                                                                         | string   |                      |
| external         | Enable external access from outside the cluster.                                                                                             | bool     | <code>false</code>   |
| version          | MariaDB major.minor version to deploy                                                                                                        | string   | <code>v11.8</code>   |

## Application-specific parameters

| Parameter                      | Description                          | Type                           | Default         |
|--------------------------------|--------------------------------------|--------------------------------|-----------------|
| users                          | Users configuration map.             | <code>map[string]object</code> | <code>{}</code> |
| users[name].password           | Password for the user.               | string                         |                 |
| users[name].maxUserConnections | Maximum number of connections.       | int                            | <code>0</code>  |
| databases                      | Databases configuration map.         | <code>map[string]object</code> | <code>{}</code> |
| databases[name].roles          | Roles assigned to users.             | object                         | <code>{}</code> |
| databases[name].roles.admin    | List of users with admin privileges. | <code>[]string</code>          | <code>[]</code> |

|                                |                                          |          |    |
|--------------------------------|------------------------------------------|----------|----|
| databases[name].roles.readonly | List of users with read-only privileges. | []string | [] |
|--------------------------------|------------------------------------------|----------|----|

## Backup parameters (DEPRECATED)

| Parameter              | Description                                                                                         | Type   | Default                                              |
|------------------------|-----------------------------------------------------------------------------------------------------|--------|------------------------------------------------------|
| backup                 | DEPRECATED: Backup configuration. Prefer the BackupClass / Plan flow under examples/backups/mariadb | object | {}                                                   |
| backup.enabled         | DEPRECATED: Enable regular backups (default: false).                                                | bool   | false                                                |
| backup.s3Region        | DEPRECATED: AWS S3 region where backups are stored.                                                 | string | us-east-1                                            |
| backup.s3Bucket        | DEPRECATED: S3 bucket used for storing backups.                                                     | string | s3.example.org/mariadb-backups                       |
| backup.schedule        | DEPRECATED: Cron schedule for automated backups.                                                    | string | 0 2 *                                                |
| backup.cleanupStrategy | DEPRECATED: Retention strategy for cleaning up old backups.                                         | string | --keep-last=3 --keep-daily=3 --keep-within-weekly=1m |
| backup.s3AccessKey     | DEPRECATED: Access key for S3 authentication.                                                       | string | <your-access-key>                                    |
| backup.s3SecretKey     | DEPRECATED: Secret key for S3 authentication.                                                       | string | <your-secret-key>                                    |
| backup.resticPassword  | DEPRECATED: Password for Restic backup encryption.                                                  | string | <password>                                           |

## Parameter examples and reference

### resources and resourcesPreset

---

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [docs/operations/resource-presets.md](https://docs.cozystack.io/operations/resource-presets.md) for the full size matrix and the legacy-to-instance-type mapping.

## users

```
users:
 user1:
 maxUserConnections: 1000
 password: hackme
 user2:
 maxUserConnections: 1000
 password: hackme
```

## databases

```
databases:
 myapp1:
 roles:
 admin:
 - user1
 readonly:
 - user2
```



---

# Managed MongoDB Service

<https://cozystack.ru/docs/v1.5/applications/mongodb/>

**MongoDB** is a popular document-oriented NoSQL database known for its flexibility and scalability. The Managed MongoDB Service provides a self-healing replicated cluster managed by the Percona Operator for MongoDB.

## Deployment Details

---

This managed service is controlled by the Percona Operator for MongoDB, ensuring efficient management and seamless operation.

- Docs: <https://docs.percona.com/percona-operator-for-mongodb/>
- Github: <https://github.com/percona/percona-server-mongodb-operator>

## Deployment Modes

---

### Replica Set Mode (default)

By default, MongoDB deploys as a replica set with the specified number of replicas. This mode is suitable for most use cases requiring high availability.

### Sharded Cluster Mode

Enable `sharding: true` for horizontal scaling across multiple shards. Each shard is a replica set, and mongos routers handle query routing.

## Notes

---

### External Access

When `external: true` is enabled:

- Replica Set mode: Traffic is load-balanced across all replica set members. This works well for read operations, but write operations require connecting to the primary. MongoDB drivers handle primary discovery automatically using the replica set connection string.
- Sharded mode: Traffic is routed through mongos routers, which handle both reads and writes correctly.

### Credentials

---

On first install, the credentials secret will be empty until the Percona operator initializes the cluster. Run `helm upgrade` after MongoDB is ready to populate the credentials secret with the actual password.

## Data lifecycle

When the MongoDB release is uninstalled, the operator finalizers reclaim release-scoped resources:

**\*\*Reclaimed by the `percona.com/delete-psmdb-pvc` finalizer:\*\***

- All PVCs backing the replica set storage. Whether the underlying PersistentVolume and on-disk data are actually deleted depends on the StorageClass `reclaimPolicy` (`Delete` removes them, `Retain` leaves them for manual cleanup).
- Operator-managed secrets:
  - `<release>-percona-server-mongodb-users` — operator users credentials
  - `internal-<release>` — internal operator state
  - `internal-<release>-users` — operator-internal users data
  - `<release>-mongodb-encryption-key` — at-rest encryption key

**\*\*Reclaimed by `helm uninstall`:**

- `<release>-credentials` — connection string for application code
- `<release>-user-<username>` — per-user passwords
- `<release>-s3-creds` — backup destination credentials (if backups are configured)

Not reclaimed automatically:

- TLS secrets `<release>-ssl` and `<release>-ssl-internal` (issued by cert-manager) remain in the namespace after uninstall. Delete them manually if no longer needed.

Recovery from a stuck deletion:

If the `psmdb-operator` is uninstalled before MongoDB CRs are deleted, the finalizers cannot run and the `PerconaServerMongoDB` CR hangs in `Terminating`. To recover, clear the finalizers manually:

```
kubectl --namespace <namespace> patch psmdb <release> --type merge --patch '{"metadata":{"finalizers":null}}'
```

Note that this skips the operator-driven cleanup — PVCs and operator-managed secrets will remain orphaned and must be removed manually.

If you need to retain data, take a backup before deletion. Refer to the [Percona Operator for MongoDB documentation](#) for backup/restore workflows.

## Upgrading from earlier versions

Earlier versions of this chart referenced a namespace-shared system users secret (`percona-server-mongodb-users`). Upgrading to a release that scopes this secret per CR (`<release>-percona-server-mongodb-users`) triggers a password rotation for the operator-managed system users. The rotation is performed in place by the Percona operator via `db.changeUserPassword()`

---

against the running mongod (operator log: `Secret data changed. Updating users...`); pods are not restarted and the cluster stays available.

Rotated automatically on upgrade:

- The five operator-managed system accounts: `databaseAdmin`, `userAdmin`, `backup`, `clusterAdmin`, `clusterMonitor`.
- Secret `<release>-percona-server-mongodb-users` (newly created, per-CR) and `internal-<release>-users` receive the new values.
- Secret `<release>-credentials` is regenerated; its `password` and `uri` keys reflect the new `databaseAdmin` password.

Not affected:

- Custom users defined under `users:` in chart values. Their `<release>-user-<name>` secrets are not touched.
- The at-rest encryption key (`<release>-mongodb-encryption-key`) and replica set keyfile (`<release>-mongodb-keyfile`) are unchanged, so on-disk data remains readable.

Action required after upgrade:

Workloads that mount `<release>-credentials` keep using the cached old password until they re-read the secret. Restart those pods, or run a controller such as [Reloader](#) to roll them automatically. Without this, application connections fail with authentication errors once their existing sessions expire.

Orphaned legacy secret:

The previous namespace-shared secret `percona-server-mongodb-users` is no longer referenced by any MongoDB CR after upgrade, but the operator does not garbage-collect it. If multiple MongoDB releases in the same namespace previously shared it, all of them rotate to their own per-CR secrets — passwords are no longer shared across CRs in the namespace, which is the intended outcome. Confirm no other consumers reference it, then remove it manually:

```
kubectl --namespace <namespace> delete secret percona-server-mongodb-users
```

`storageClass` is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it.

## Parameters

---

### Common parameters

| Name     | Description                                | Type | Value |
|----------|--------------------------------------------|------|-------|
| replicas | Number of MongoDB replicas in replica set. | int  | 3     |

|                  |                                                                                                                                              |          |          |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
| resources        | Explicit CPU and memory configuration for each MongoDB replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | {}       |
| resources.cpu    | CPU available to each replica.                                                                                                               | quantity | ""       |
| resources.memory | Memory (RAM) available to each replica.                                                                                                      | quantity | ""       |
| resourcesPreset  | Default sizing preset used when <code>resources</code> is omitted.                                                                           | string   | t1.small |
| size             | Persistent Volume Claim size available for application data.                                                                                 | quantity | 10Gi     |
| storageClass     | StorageClass used to store the data.                                                                                                         | string   | ""       |
| external         | Enable external access from outside the cluster.                                                                                             | bool     | false    |
| version          | MongoDB major version to deploy.                                                                                                             | string   | v8       |

## Sharding configuration

| Name                            | Description                                                        | Type     | Value |
|---------------------------------|--------------------------------------------------------------------|----------|-------|
| sharding                        | Enable sharded cluster mode. When disabled, deploys a replica set. | bool     | false |
| shardingConfig                  | Configuration for sharded cluster mode.                            | object   | {}    |
| shardingConfig.configServers    | Number of config server replicas.                                  | int      | 3     |
| shardingConfig.configServerSize | PVC size for config servers.                                       | quantity | 3Gi   |
| shardingConfig.mongos           | Number of mongos router replicas.                                  | int      | 2     |

|                                   |                                    |          |       |
|-----------------------------------|------------------------------------|----------|-------|
| shardingConfig.shards             | List of shard configurations.      | []object | [...] |
| shardingConfig.shards[i].name     | Shard name.                        | string   | ""    |
| shardingConfig.shards[i].replicas | Number of replicas for this shard. | int      | 0     |
| shardingConfig.shards[i].size     | PVC size for this shard.           | quantity | ""    |

## Users configuration

| Name                 | Description                                        | Type              | Value |
|----------------------|----------------------------------------------------|-------------------|-------|
| users                | Users configuration map.                           | map[string]object | {}    |
| users[name].password | Password for the user (auto-generated if omitted). | string            | ""    |

## Databases configuration

| Name                           | Description                                                | Type              | Value |
|--------------------------------|------------------------------------------------------------|-------------------|-------|
| databases                      | Databases configuration map.                               | map[string]object | {}    |
| databases[name].roles          | Roles assigned to users.                                   | object            | {}    |
| databases[name].roles.admin    | List of users with admin privileges (readWrite + dbAdmin). | []string          | []    |
| databases[name].roles.readonly | List of users with read-only privileges.                   | []string          | []    |

## Backup parameters

| Name            | Description                     | Type   | Value |
|-----------------|---------------------------------|--------|-------|
| backup          | Backup configuration.           | object | {}    |
| backup.enabled  | Enable regular backups.         | bool   | false |
| backup.schedule | Backup schedule in cron format. | string | 0 2 * |

|                        |                                                        |        |                                   |
|------------------------|--------------------------------------------------------|--------|-----------------------------------|
| backup.retentionPolicy | Retention policy (e.g. "30d").                         | string | 30d                               |
| backup.destinationPath | Destination path for backups (e.g. s3://bucket/path/). | string | s3://bucket/path/to/folder/       |
| backup.endpointURL     | S3 endpoint URL for uploads.                           | string | http://minio-gateway-service:9000 |
| backup.s3AccessKey     | Access key for S3 authentication.                      | string | ""                                |
| backup.s3SecretKey     | Secret key for S3 authentication.                      | string | ""                                |

### Bootstrap (recovery) parameters

| Name                   | Description                                               | Type   | Value |
|------------------------|-----------------------------------------------------------|--------|-------|
| bootstrap              | Bootstrap configuration.                                  | object | {}    |
| bootstrap.enabled      | Whether to restore from a backup.                         | bool   | false |
| bootstrap.recoveryTime | Timestamp for point-in-time recovery; empty means latest. | string | ""    |
| bootstrap.backupName   | Name of backup to restore from.                           | string | ""    |

# Managed NATS Service

<https://cozystack.ru/docs/v1.5/applications/nats/>

NATS is an open-source, simple, secure, and high performance messaging system. It provides a data layer for cloud native applications, IoT messaging, and microservices architectures.

`storageClass` is annotated as immutable in the chart schema — see [docs/storage-immutability.md](#) for the contract and which consumers enforce it.

## Parameters

### Common parameters

| Name                          | Description                                                                                                                               | Type                  | Value                |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------|
| <code>replicas</code>         | Number of replicas.                                                                                                                       | <code>int</code>      | <code>2</code>       |
| <code>resources</code>        | Explicit CPU and memory configuration for each NATS replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>      |
| <code>resources.cpu</code>    | CPU available to each replica.                                                                                                            | <code>quantity</code> | <code>""</code>      |
| <code>resources.memory</code> | Memory (RAM) available to each replica.                                                                                                   | <code>quantity</code> | <code>""</code>      |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                        | <code>string</code>   | <code>t1.nano</code> |
| <code>storageClass</code>     | StorageClass used to store the data.                                                                                                      | <code>string</code>   | <code>""</code>      |
| <code>external</code>         | Enable external access from outside the cluster.                                                                                          | <code>bool</code>     | <code>false</code>   |
| <code>tls</code>              | TLS configuration. When omitted, TLS follows the external flag.                                                                           | <code>object</code>   | <code>{}</code>      |

|                          |                                                                                            |                    |                   |
|--------------------------|--------------------------------------------------------------------------------------------|--------------------|-------------------|
| <code>tls.enabled</code> | Enable TLS. When omitted, TLS is enabled automatically when <code>external</code> is true. | <code>*bool</code> | <code>null</code> |
|--------------------------|--------------------------------------------------------------------------------------------|--------------------|-------------------|

## Application-specific parameters

| Name                              | Description                                                   | Type                           | Value             |
|-----------------------------------|---------------------------------------------------------------|--------------------------------|-------------------|
| <code>users</code>                | Users configuration map.                                      | <code>map[string]object</code> | <code>{}</code>   |
| <code>users[name].password</code> | Password for the user.                                        | <code>string</code>            | <code>""</code>   |
| <code>jetstream</code>            | Jetstream configuration.                                      | <code>object</code>            | <code>{}</code>   |
| <code>jetstream.enabled</code>    | Enable or disable Jetstream for persistent messaging in NATS. | <code>bool</code>              | <code>true</code> |
| <code>jetstream.size</code>       | Jetstream persistent storage size.                            | <code>quantity</code>          | <code>10Gi</code> |
| <code>config</code>               | NATS configuration.                                           | <code>object</code>            | <code>{}</code>   |
| <code>config.merge</code>         | Additional configuration to merge into NATS config.           | <code>*object</code>           | <code>{}</code>   |
| <code>config.resolver</code>      | Additional resolver configuration to merge into NATS config.  | <code>*object</code>           | <code>{}</code>   |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.



---

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [docs/operations/resource-presets.md](https://docs.cozystack.io/operations/resource-presets.md) for the full size matrix and the legacy-to-instance-type mapping.

---

# Managed OpenBAO Service

<https://cozystack.ru/docs/v1.5/applications/openbao/>

OpenBAO is an open-source secrets management solution forked from HashiCorp Vault. It provides identity-based secrets and encryption management for cloud infrastructure. ([Cozystack Documentation](#))

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(https://cozystack.ru/docs/docs/storage-immutability.md\)](#) for the contract and which consumers enforce it. ([Cozystack Documentation](#))

## Parameters

### Common parameters

| Name                          | Description                                                                                                                                                                                            | Type                  | Value                 |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------|
| <code>replicas</code>         | Number of OpenBAO replicas. HA with Raft is automatically enabled when <code>replicas &gt; 1</code> . Switching between standalone (file storage) and HA (Raft storage) modes requires data migration. | <code>int</code>      | <code>1</code>        |
| <code>resources</code>        | Explicit CPU and memory configuration for each OpenBAO replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied.                                                           | <code>object</code>   | <code>{}</code>       |
| <code>resources.cpu</code>    | CPU available to each replica.                                                                                                                                                                         | <code>quantity</code> | <code>" "</code>      |
| <code>resources.memory</code> | Memory (RAM) available to each replica.                                                                                                                                                                | <code>quantity</code> | <code>" "</code>      |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                                                                                     | <code>string</code>   | <code>t1.small</code> |

|              |                                                  |          |       |
|--------------|--------------------------------------------------|----------|-------|
| size         | Persistent Volume Claim size for data storage.   | quantity | 10Gi  |
| storageClass | StorageClass used to store the data.             | string   | ""    |
| external     | Enable external access from outside the cluster. | bool     | false |

## Application-specific parameters

| Name | Description                | Type | Value |
|------|----------------------------|------|-------|
| ui   | Enable the OpenBAO web UI. | bool | true  |

(Source: [Cozystack Documentation](#))

---

# Managed PostgreSQL Service

<https://cozystack.ru/docs/v1.5/applications/postgres/>

PostgreSQL is currently the leading choice among relational databases, known for its robust features and performance. The Managed PostgreSQL Service takes advantage of platform-side implementation to provide a self-healing replicated cluster. This cluster is efficiently managed using the highly acclaimed CloudNativePG operator, which has gained popularity within the community.

## Deployment Details

---

This managed service is controlled by the CloudNativePG operator, ensuring efficient management and seamless operation.

- Docs: <https://cloudnative-pg.io/docs/>
- Github: <https://github.com/cloudnative-pg/cloudnative-pg>

## Operations

---

PostgreSQL backups have two layers, and the recommended setup uses both together rather than picking one:

| Layer | What it does | Configured via |
|-------|--------------|----------------|
|-------|--------------|----------------|

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Archive plumbing (chart, legacy)   | <p>Renders <code>spec.backup.barmanObjectStore</code> on the <code>cnpg.io</code> Cluster from <code>helm-install</code> onwards. CNPG starts postgres with <code>archive_command=barman-cloud-wal-archive</code>, every WAL switch ships to object storage, and any backup driver gets a complete WAL chain to start replay from. Renders only when <code>backup.enabled=true</code>, <code>useSystemBucket=false</code>, <code>destinationPath</code> is non-empty, AND inline-or-external creds are supplied. **Skipped in the platform <code>useSystemBucket=true</code> mode** — the CNPG backup driver SSA-patches <code>barmanObjectStore</code> onto the live Cluster at first BackupJob time; until that first BackupJob fires, <code>archive_command</code> is not active and WAL accumulates on the PVC. Fire an ad-hoc BackupJob immediately after enabling the flag on existing releases.</p> | <p>Legacy: <code>backup.enabled=true</code> plus <code>backup.destinationPath</code>, <code>backup.endpointURL</code>, and either <code>backup.s3AccessKey</code> + <code>backup.s3SecretKey</code> or <code>backup.s3CredentialsSecret</code>. Platform: <code>backup.enabled=true</code> plus <code>backup.useSystemBucket=true</code> — no S3 fields.</p> |
| Backup orchestration (recommended) | <p>Drives ad-hoc and scheduled backups, retention, and restores from <code>backups.cozystack.io</code> resources that can span multiple Postgres apps in a tenant. The driver SSA-merges its own <code>barmanObjectStore</code> shape over the chart's with <code>ForceOwnership</code> (so compression / <code>serverName</code> / target settings flow from the strategy template); chart-managed values are ignored.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | <p><code>strategy.backups.cozystack.io/CNPG + BackupClass + Plan</code> (recurring) or <code>BackupJob</code> (ad-hoc), restores via <code>RestoreJob</code></p>                                                                                                                                                                                             |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                              |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
|  | <p>The chart can also emit a <code>cnpg.io/ScheduledBackup</code> directly. Superseded by the BackupClass + Plan path, kept around for clusters that did not migrate yet. Off by default - rendered only when both <code>backup.enabled</code> and <code>backup.schedule</code> are non-empty. With <code>backup.schedule=</code> the archive plumbing still runs; only the chart-emitted scheduled backup is silent.</p> | <p><code>backup.enabled=true</code> plus <code>backup.schedule</code> (CNPG 6-field cron, e.g. <code>0 0 0 * * *</code>)</p> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|

The canonical setup depends on whether the cluster ships the platform `cozy-default` BackupClass.

Platform-managed flow (recommended for new clusters) — opt in via `backup.useSystemBucket: true` and reference `cozy-default` from BackupJob/Plan/RestoreJob. The chart leaves S3 coordinates blank; the CNPG driver SSA-patches `barmanObjectStore` from the platform-managed bucket coordinates at first BackupJob time.

```
spec:
 backup:
 enabled: true
 useSystemBucket: true
 # platform projects cozy-backups-creds + driver SSA-patches barmanObjectStore
```

Legacy chart-managed flow — for clusters that pre-date the platform BackupClass or use a tuned non-default bucket. Provide all S3 coordinates (or `s3CredentialsSecret.name`); `barmanObjectStore` renders from helm-install onwards.

```
spec:
 backup:
 enabled: true
 # archive WAL to object storage
 destinationPath: s3://my-bucket/pg-src/
 endpointURL: https://seaweedfs-s3.tenant-foo:8333
 s3CredentialsSecret:
 name: pg-src-cnpg-backup-creds
 endpointCA:
 name: pg-src-cnpg-backup-ca
 # backup.schedule intentionally left empty - the BackupClass / Plan
 # below drives recurring backups, no chart-emitted ScheduledBackup
 # should compete with that schedule.
```

... paired with a `strategy.backups.cozystack.io/CNPG + BackupClass + Plan` in the same tenant. The end-to-end e2e fixture under `examples/backups/postgres/` is the canonical reference (`05-postgres-src.yaml` shows the chart side, `10-cnpg-strategy.yaml` and `15-backupclass.yaml` show the orchestration side, `25-backupjob-adhoc.yaml` and `40-restorejob-to-copy.yaml` show ad-hoc backup and restore).

---

Why both layers and not one? WAL archive is a postmaster setting - CNPG can swap `archive_command` at runtime via SIGHUP, but any WAL that closed before the swap was “archived” by `archive_command=/bin/true` and is gone for good. A backup taken from such a cluster is missing the WAL its `begin_wal` points at, and recovery later fails with `WAL not found`. Letting the chart bootstrap `archive_command` from `helm-install` removes the race.

## How to enable backups (preferred: BackupClass + Plan)

---

End-to-end manifests live under `examples/backups/postgres/`. Briefly, the moving parts are:

1. A `strategy.backups.cozystack.io/CNPG` describing the destination bucket and templating the `barmanObjectStore` (including a Secret reference to S3 credentials - the credentials never appear on the Postgres CR `.spec`; see Security note below).
2. A `backups.cozystack.io/BackupClass` that names the strategy and is selected by an `applicationRef` matching the Postgres app’s Kind / Name.
3. A `backups.cozystack.io/Plan` (recurring) or `BackupJob` (ad-hoc) that references the BackupClass. The controller materialises a `Backup` artifact when the `cnpg.io Backup` completes; restores then reference that Backup via `RestoreJob`.

Both in-place restores (overwrite the source app’s data) and to-copy restores (restore into a separate app instance) are supported via the `RestoreJob.spec.targetApplicationRef` field.

Security: With the BackupClass path, S3 credentials live in a tenant-readable Secret referenced from the strategy; the strategy controller injects `spec.backup.s3CredentialsSecret` on restore, so access keys never land in the Postgres CR `.spec`, etcd object store, or `kubectl get -o yaml` output. Prefer this over the chart-managed path whenever possible.

## How to enable chart-managed scheduled backups (legacy)

---

The chart can also emit a `cnpg.io/ScheduledBackup` directly, without a BackupClass. Superseded by the BackupClass + Plan path above and kept around for clusters that did not migrate yet.

`backup.schedule` defaults to an empty string, which gates the chart’s ScheduledBackup template off. To turn it on, supply a 6-field cron string:

```

@param backup.enabled Enable archive_command + render the chart-managed ScheduledBackup
@param backup.schedule Cron schedule (CNPg 6-field). Empty means no chart-managed ScheduledBackup
@param backup.retentionPolicy Retention policy
@param backup.destinationPath Path to store the backup (i.e. s3://bucket/path/to/folder)
@param backup.endpointURL S3 Endpoint used to upload data to the cloud
@param backup.s3AccessKey Access key for S3, used for authentication
@param backup.s3SecretKey Secret key for S3, used for authentication
backup:
 enabled: true
 retentionPolicy: 30d
 destinationPath: s3://bucket/path/to/folder/
 endpointURL: http://minio-gateway-service:9000
 schedule: "0 0 0 * * *" # opt in - empty (the default) means no chart-managed schedule
 s3AccessKey: oobaiRus9pah8PhohL1ThaeTa4UVa7gu
 s3SecretKey: ju3eum4dekeich9ahM1te8waeGai0oog

```

## How to recover a backup (preferred: RestoreJob)

For BackupClass-managed backups, create a `backups.cozystack.io/RestoreJob` that references the desired Backup. See `examples/backups/postgres/35-restorejob-in-place.yaml` and `examples/backups/postgres/40-restorejob-to-copy.yaml`. On a to-copy restore the controller replaces the target app's `spec.databases` and `spec.users` with the source-spec snapshot persisted in Backup. `status.underlyingResources`, so the chart's post-install init-job does not drop the recovered roles or databases.

e2e coverage: the CI e2e (`hack/e2e-apps/backup-postgres.bats`) exercises the same-namespace to-copy restore (Steps 0-7), which leaves the source running and is the most complex path. The CNPG strategy controller logic is covered by `TestClusterHasRecoveryBootstrap_TerminatingCluster`, `TestCNPGBackupWALArchived`, `TestCNPGPurgeNeeded`, and the rest of the `internal/backupcontroller/cnpgstrategy_controller_test.go` suite.

## How to recover a backup (chart-managed bootstrap)

CloudNativePG supports point-in-time-recovery. Recovering a backup is done by creating a new database cluster and specifying the recovery target.

Create a new PostgreSQL application with a different name, but identical configuration. Set `bootstrap.enabled` to `true` and fill in the name of the database instance to recover from and the recovery time:

```

@param bootstrap.enabled Restore database cluster from a backup
@param bootstrap.recoveryTime Timestamp (PITR) up to which recovery will proceed, expressed in RFC3339 f...
@param bootstrap.oldName Name of database cluster before deleting
##
bootstrap:
 enabled: false
 recoveryTime: "" # leave empty for latest or exact timestamp; example: 2020-11-26 15:22:00.000000+00
 oldName: ""

```



---

## How to switch primary/secondary replica

---

See: [https://cloudnative-pg.io/documentation/1.15/rolling\\_update/#manual-updates-supervised](https://cloudnative-pg.io/documentation/1.15/rolling_update/#manual-updates-supervised)

`storageClass` is annotated as immutable in the chart schema — see `docs/storage-immutability.md` for the contract and which consumers enforce it.

---

## TLS for server connections

---

CNPG manages the cert chain end-to-end. The operator auto-generates a self-signed CA, signs server, and signs client certificates for its built-in `users` and `databases` (the chart does not materialise `cert-manager.io` Issuer/Certificate objects — that path is mutually exclusive with the operator-managed chain on the CNPG admission webhook).

What the chart contributes: when TLS is on and `external: true`, the chart sets `spec.certificates.serverAltDNSNames` on the CNPG Cluster CR to inject the external hostname `<release>.<_namespace>.host` into the auto-generated server certificate's SAN list. CNPG's default SAN coverage already includes the internal Cluster IPs and Service names (`-rw`, `-r`, `-ro`) across the four DNS forms (`<svc>`, `<svc>.<ns>`, `<svc>.<ns>.svc`, `<svc>.<ns>.svc.<cluster-domain>`); only the external hostname needs to be added.

The tri-state `tls.enabled` controls whether the chart injects `serverAltDNSNames`:

- `tls.enabled: null` (the default) — TLS posture inherits from `external`. When `external: true`, the chart injects the external hostname into the operator-managed cert.
- `tls.enabled: true` with `external: true` — same effect as the default.
- `tls.enabled: true` with `external: false` — no `serverAltDNSNames` injection is needed (there is no external hostname to add); CNPG's auto-generated cert covers internal access by default.
- `tls.enabled: false` — the chart skips `serverAltDNSNames` injection.

Note: CNPG keeps its built-in TLS on the wire regardless of this flag; this toggle only controls whether the external hostname is added to the SAN list. To turn off SSL entirely on the server side, you would need to set `postgresql.parameters.ssl = 'off'` at the CNPG layer, which is out of scope for this flag.

### Retrieving the CA bundle for client verification:

CNPG bundles the CA certificate in every user-credentials Secret it creates under the key `ca.crt`. Retrieve it from the `<release>-credentials` Secret, which is already accessible to tenants via the dashboard RBAC:

```
kubectl --context <ctx> --namespace <tenant> get secret <release>-credentials --output jsonpath='{.data.ca\...
```

### Connecting with full verification (psql example):

```
psql "sslmode=verify-full sslrootcert=ca.crt host=<release>.<_namespace>.host user=<user> dbname=<db>"
```

---

For `sslmode=verify-full` to work, the CA bundle retrieved above must be saved to `ca.crt`. Without it, use `sslmode=require` (encrypts but does not verify the server certificate).

## Parameters

---

### Common parameters

| Parameter                     | Description                                                                                                                                     | Type     | Default               |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| <code>replicas</code>         |                                                                                                                                                 | int      | 2                     |
| <code>resources</code>        | Explicit CPU and memory configuration for each PostgreSQL replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object   | <code>{}</code>       |
| <code>resources.cpu</code>    |                                                                                                                                                 | quantity |                       |
| <code>resources.memory</code> |                                                                                                                                                 | quantity |                       |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                              | string   | <code>t1.micro</code> |
| <code>size</code>             |                                                                                                                                                 | quantity | <code>10Gi</code>     |
| <code>storageClass</code>     |                                                                                                                                                 | string   |                       |
| <code>external</code>         |                                                                                                                                                 | bool     | <code>false</code>    |
| <code>version</code>          |                                                                                                                                                 | string   | <code>v18</code>      |

### TLS configuration

| Parameter        | Description | Type   | Default         |
|------------------|-------------|--------|-----------------|
| <code>tls</code> |             | object | <code>{}</code> |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                    |                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|
| <code>tls.enabled</code> | <p>Tri-state switch controlling whether the chart injects the external hostname into the operator-managed server certificate. Defaults to <code>null</code>, which enables injection when <code>external: true</code> and skips it otherwise. Set explicitly to <code>true</code> to inject regardless of <code>external</code> (no-op when <code>external: false</code> since there is no external hostname to add). Set to <code>false</code> to skip injection. Note that CNPG keeps its built-in TLS on the wire regardless of this flag — this toggle does NOT set <code>postgresql.parameters.ssl = 'off'</code> at the CNPG layer.</p> | <code>*bool</code> | <code>null</code> |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|

## Application-specific parameters

| Parameter                          | Description                                                                                                                                                                                                                                                                                                                                                            | Type                                | Default         |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------|
| <code>postgresql</code>            |                                                                                                                                                                                                                                                                                                                                                                        | object                              | <code>{}</code> |
| <code>postgresql.parameters</code> | <p>PostgreSQL server parameters. Values may be strings or integers; integers are coerced to strings by the operator (both <code>max_connections: 100</code> and <code>max_connections: "100"</code> are accepted). BLOCKED (enable arbitrary code execution): <code>archive_command</code> and, <code>restore_command</code>, <code>ssl_passphrase_command</code>.</p> | <code>map[string]intOrString</code> | <code>{}</code> |

---

## Quorum-based synchronous replication

| Parameter              | Description | Type   | Default |
|------------------------|-------------|--------|---------|
| quorum                 |             | object | {}      |
| quorum.minSyncReplicas |             | int    | 0       |
| quorum.maxSyncReplicas |             | int    | 0       |

## Users configuration

| Parameter               | Description | Type              | Default |
|-------------------------|-------------|-------------------|---------|
| users                   |             | map[string]object | {}      |
| users[name].password    |             | string            |         |
| users[name].replication |             | bool              | false   |

## Databases configuration

| Parameter                      | Description | Type              | Default |
|--------------------------------|-------------|-------------------|---------|
| databases                      |             | map[string]object | {}      |
| databases[name].roles          |             | object            | {}      |
| databases[name].roles.admin    |             | []string          | []      |
| databases[name].roles.readonly |             | []string          | []      |
| databases[name].extensions     |             | []string          | []      |

## Backup parameters

| Parameter      | Description | Type   | Default |
|----------------|-------------|--------|---------|
| backup         |             | object | {}      |
| backup.enabled |             | bool   | false   |

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |      |       |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| backup.useSystemBucket | <p>Opt-in: when true, the chart-emitted <code>&lt;release&gt;-s3-creds</code> Secret is skipped AND <code>spec.backup.barmanObjectStore</code> is left UNSET in the chart-rendered Cluster — the <code>cozy-default</code> BackupClass driver SSA-patches the platform-managed bucket coordinates onto the live Cluster at first BackupJob time. Note that <code>archive_command</code> is NOT active until that first BackupJob fires; WAL accumulates on the PVC in the meantime, so fire an ad-hoc BackupJob immediately after installation. Requires the platform-managed <code>cozy-default</code> BackupClass — tenants do not need to fill <code>s3AccessKey</code> / <code>s3SecretKey</code> or <code>destinationPath</code> / <code>endpointURL</code>. The destination path automatically scopes to <code>s3://cozy-backups/&lt;namespace&gt;/&lt;release&gt;/. </code></p> | bool | false |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|

|                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |        |                                          |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|------------------------------------------|
| <code>backup.schedule</code>        | Legacy. Cron schedule (CNPg 6-field format) for the chart-emitted ScheduledBackup. Empty means no chart-managed schedule. Prefer the BackupClass from <code>backups.cozystack.io</code> which already drives backup orchestration. In the legacy chart-managed flow <code>spec.backup.barmanObjectStore</code> is rendered when <code>backup.enabled=true</code> AND <code>useSystemBucket=false</code> AND <code>destinationPath</code> is non-empty AND inline-or-external creds are supplied; in the platform <code>useSystemBucket=true</code> flow the chart skips emitting <code>barmanObjectStore</code> and the CNPg driver SSA-patches it onto the live Cluster at first BackupJob time. | string |                                          |
| <code>backup.retentionPolicy</code> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | string | 30d                                      |
| <code>backup.destinationPath</code> | DEPRECATED. Per-tenant S3 configuration is superseded by the platform-managed <code>cozy-default</code> BackupClass and the <code>cozy-backups</code> system bucket. Leave empty for new installations; the BackupClass driver picks up the system-managed bucket automatically.                                                                                                                                                                                                                                                                                                                                                                                                                  | string | <code>s3://bucket/path/to/folder/</code> |

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |        |                                                |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|------------------------------------------------|
| <code>backup.endpointURL</code>         | DEPRECATED. See <code>destinationPath</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | string | <code>http://minio-gateway-service:9000</code> |
| <code>backup.s3AccessKey</code>         | DEPRECATED. Tenants no longer supply S3 keys; the system Bucket Secret is projected into the tenant namespace and referenced by the Strategy controller. Use the BackupClass instead. Security note: <code>barmanObjectStore</code> renders when <code>s3CredentialsSecret.name</code> is set or <code>useSystemBucket</code> is true. The chart skips materialising <code>&lt;release&gt;-s3-creds</code> whenever this field is empty so a default install does not leak placeholder credentials into the tenant. | string |                                                |
| <code>backup.s3SecretKey</code>         | DEPRECATED. See <code>s3AccessKey</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | string |                                                |
| <code>backup.s3CredentialsSecret</code> | DEPRECATED. Pre-existing Secret with S3 credentials. Use the platform-managed <code>cozy-default</code> BackupClass instead. When set, the chart references this Secret directly (legacy chart-managed flow) and the Strategy controller avoids landing keys in the Cluster <code>.spec</code> .                                                                                                                                                                                                                    | object | <code>{}</code>                                |

|                                                            |                                                                                                                                                                                      |        |                 |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|-----------------|
| <code>backup.s3CredentialsSecret.name</code>               | Name of the Secret in the application namespace. Empty means the chart materialises <code>&lt;release&gt;-s3-creds</code> from <code>s3AccessKey</code> / <code>s3SecretKey</code> . | string |                 |
| <code>backup.s3CredentialsSecret.accessKeyIDKey</code>     | Key in the Secret holding the access key ID. Defaults to <code>AWS_ACCESS_KEY_ID</code> .                                                                                            | string |                 |
| <code>backup.s3CredentialsSecret.secretAccessKeyKey</code> | Key in the Secret holding the secret access key. Defaults to <code>AWS_SECRET_ACCESS_KEY</code> .                                                                                    | string |                 |
| <code>backup.endpointCA</code>                             |                                                                                                                                                                                      | object | <code>{}</code> |
| <code>backup.endpointCA.name</code>                        |                                                                                                                                                                                      | string |                 |
| <code>backup.endpointCA.key</code>                         | Key within the Secret containing the CA bundle. Defaults to <code>ca.crt</code> .                                                                                                    | string |                 |

## Bootstrap (recovery) parameters

| Parameter                           | Description | Type   | Default            |
|-------------------------------------|-------------|--------|--------------------|
| <code>bootstrap</code>              |             | object | <code>{}</code>    |
| <code>bootstrap.enabled</code>      |             | bool   | <code>false</code> |
| <code>bootstrap.recoveryTime</code> |             | string |                    |
| <code>bootstrap.oldName</code>      |             | string |                    |
| <code>bootstrap.serverName</code>   |             | string |                    |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.



---

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1` 1:0.5, `c1` 1:1, `s1` 1:2, `u1` 1:4, `m1` 1:8) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [docs/operations/resource-presets.md](#) for the full size matrix and the legacy-to-instance-type mapping.

## users

```
users:
 user1:
 password: strongpassword
 user2:
 password: hackme
 airflow:
 password: qwerty123
 debezium:
 replication: true
```

## databases

```
databases:
 myapp:
 roles:
 admin:
 - user1
 - debezium
 readonly:
 - user2
 airflow:
 roles:
 admin:
 - airflow
 extensions:
 - hstore
```

# Managed Qdrant Service

<https://cozystack.ru/docs/v1.5/applications/qdrant/>

Qdrant is a high-performance vector database and similarity search engine designed for AI and machine learning applications. It provides efficient storage and retrieval of high-dimensional vectors with advanced filtering capabilities, making it ideal for recommendation systems, semantic search, and RAG (Retrieval-Augmented Generation) applications.

## Deployment Details

Service deploys Qdrant as a StatefulSet with automatic cluster mode when multiple replicas are configured.

- Docs: <https://qdrant.tech/documentation/>
- GitHub: <https://github.com/qdrant/qdrant>

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(https://cozystack.ru/docs/docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

## Parameters

### Common parameters

| Name                       | Description                                                                                                                                 | Type                  | Value            |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------------|
| <code>replicas</code>      | Number of Qdrant replicas. Cluster mode is automatically enabled when <code>replicas &gt; 1</code> .                                        | <code>int</code>      | <code>1</code>   |
| <code>resources</code>     | Explicit CPU and memory configuration for each Qdrant replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>  |
| <code>resources.cpu</code> | CPU available to each replica.                                                                                                              | <code>quantity</code> | <code>" "</code> |

|                               |                                                                    |                       |                       |
|-------------------------------|--------------------------------------------------------------------|-----------------------|-----------------------|
| <code>resources.memory</code> | Memory (RAM) available to each replica.                            | <code>quantity</code> | <code>""</code>       |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted. | <code>string</code>   | <code>t1.small</code> |
| <code>size</code>             | Persistent Volume Claim size available for vector data storage.    | <code>quantity</code> | <code>10Gi</code>     |
| <code>storageClass</code>     | StorageClass used to store the data.                               | <code>string</code>   | <code>""</code>       |
| <code>external</code>         | Enable external access from outside the cluster.                   | <code>bool</code>     | <code>false</code>    |

## TLS parameters

| Name                     | Description                                                                                                                                                                             | Type                | Value             |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------|
| <code>tls</code>         | TLS configuration. When omitted, the effective TLS state follows the value of <code>external</code> .                                                                                   | <code>object</code> | <code>{}</code>   |
| <code>tls.enabled</code> | Enable TLS. When unset, inherits the value of <code>external</code> (TLS is on when external access is enabled). Set explicitly to <code>true</code> or <code>false</code> to override. | <code>*bool</code>  | <code>null</code> |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

---

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1` 1:0.5, `c1` 1:1, `s1` 1:2, `u1` 1:4, `m1` 1:8) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [`docs/operations/resource-presets.md`](<https://cozystack.ru/docs/docs/operations/resource-presets.md>) for the full size matrix and the legacy-to-instance-type mapping.

---

# Managed RabbitMQ Service

<https://cozystack.ru/docs/v1.5/applications/rabbitmq/>

RabbitMQ is a robust message broker that plays a crucial role in modern distributed systems. Our Managed RabbitMQ Service simplifies the deployment and management of RabbitMQ clusters, ensuring reliability and scalability for your messaging needs.

## Deployment Details

The service utilizes official RabbitMQ operator. This ensures the reliability and seamless operation of your RabbitMQ instances.

- Github: <https://github.com/rabbitmq/cluster-operator/>
- Docs: <https://www.rabbitmq.com/kubernetes/operator/operator-overview.html>

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]](#)(<https://cozystack.ru/docs/docs/storage-immutability.md>) for the contract and which consumers enforce it.

## Parameters

### Common parameters

| Name                          | Description                                                                                                                                   | Type                  | Value                |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------|
| <code>replicas</code>         | Number of RabbitMQ replicas.                                                                                                                  | <code>int</code>      | <code>3</code>       |
| <code>resources</code>        | Explicit CPU and memory configuration for each RabbitMQ replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>      |
| <code>resources.cpu</code>    | CPU available to each replica.                                                                                                                | <code>quantity</code> |                      |
| <code>resources.memory</code> | Memory (RAM) available to each replica.                                                                                                       | <code>quantity</code> |                      |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                            | <code>string</code>   | <code>t1.nano</code> |

|              |                                                              |          |       |
|--------------|--------------------------------------------------------------|----------|-------|
| size         | Persistent Volume Claim size available for application data. | quantity | 10Gi  |
| storageClass | StorageClass used to store the data.                         | string   |       |
| external     | Enable external access from outside the cluster.             | bool     | false |
| version      | RabbitMQ major.minor version to deploy                       | string   | v4.2  |

## Application-specific parameters

| Name                        | Description                      | Type              | Value |
|-----------------------------|----------------------------------|-------------------|-------|
| users                       | Users configuration map.         | map[string]object | {}    |
| users[name].password        | Password for the user.           | string            |       |
| vhosts                      | Virtual hosts configuration map. | map[string]object | {}    |
| vhosts[name].roles          | Virtual host roles list.         | object            | {}    |
| vhosts[name].roles.admin    | List of admin users.             | []string          | []    |
| vhosts[name].roles.readonly | List of readonly users.          | []string          | []    |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

| Preset name | CPU | memory |
|-------------|-----|--------|
|-------------|-----|--------|

---

|         |      |       |
|---------|------|-------|
| nano    | 100m | 128Mi |
| micro   | 250m | 256Mi |
| small   | 500m | 512Mi |
| medium  | 500m | 1Gi   |
| large   | 1    | 2Gi   |
| xlarge  | 2    | 4Gi   |
| 2xlarge | 4    | 8Gi   |

---

# Managed Redis Service

<https://cozystack.ru/docs/v1.5/applications/redis/>

Redis is a highly versatile and blazing-fast in-memory data store and cache that can significantly boost the performance of your applications. Managed Redis Service offers a hassle-free solution for deploying and managing Redis clusters, ensuring that your data is always available and responsive.

## Deployment Details

Service utilizes the Spotahome Redis Operator for efficient management and orchestration of Redis clusters.

- Docs: <https://redis.io/docs/>
- GitHub: <https://github.com/spotahome/redis-operator>

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(https://cozystack.ru/docs/docs/storage-immutability.md\)](https://cozystack.ru/docs/docs/storage-immutability.md) for the contract and which consumers enforce it.

## Parameters

### Common parameters

| Name                          | Description                                                                                                                                | Type                  | Value                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|----------------------|
| <code>replicas</code>         | Number of Redis replicas.                                                                                                                  | <code>int</code>      | <code>2</code>       |
| <code>resources</code>        | Explicit CPU and memory configuration for each Redis replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code>   | <code>{}</code>      |
| <code>resources.cpu</code>    | CPU available to each replica.                                                                                                             | <code>quantity</code> |                      |
| <code>resources.memory</code> | Memory (RAM) available to each replica.                                                                                                    | <code>quantity</code> |                      |
| <code>resourcesPreset</code>  | Default sizing preset used when <code>resources</code> is omitted.                                                                         | <code>string</code>   | <code>t1.nano</code> |



|              |                                                              |          |       |
|--------------|--------------------------------------------------------------|----------|-------|
| size         | Persistent Volume Claim size available for application data. | quantity | 1Gi   |
| storageClass | StorageClass used to store the data.                         | string   |       |
| external     | Enable external access from outside the cluster.             | bool     | false |
| version      | Redis major version to deploy                                | string   | v8    |

## Application-specific parameters

| Name        | Description                 | Type | Value |
|-------------|-----------------------------|------|-------|
| authEnabled | Enable password generation. | bool | true  |

## Parameter examples and reference

### resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

```
resources:
 cpu: 4000m
 memory: 4Gi
```

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [\[docs/operations/resource-presets.md\]](https://cozystack.ru/docs/docs/operations/resource-presets.md)(https://cozystack.ru/docs/docs/operations/resource-presets.md) for the full size matrix and the legacy-to-instance-type mapping.

# Tenant Application Reference

<https://cozystack.ru/docs/v1.5/applications/tenant/>

A tenant is the main unit of security on the platform. The closest analogy would be Linux kernel namespaces.

Tenants can be created recursively and are subject to the following rules:

## Tenant naming

Tenant names must be alphanumeric:

- Lowercase letters (a-z) and digits (0-9) only
- Must start with a lowercase letter
- Dashes (-) are not allowed, unlike with other services
- Maximum length depends on the cluster configuration (Helm release prefix and root domain)

This restriction exists to keep consistent naming in tenants, nested tenants, and services deployed in them. A tenant cannot be named `foo-bar` because parsing internal resource names like `tenant-foo-bar` would be ambiguous.

For example:

- The root tenant is named `root`, but internally it's referenced as `tenant-root`.
- A nested tenant could be named `foo`, which would result in `tenant-foo` in service names and URLs.

## Unique domains

Each tenant has its own domain. By default, (unless otherwise specified), it inherits the domain of its parent with a prefix of its name. For example, if the parent had the domain `example.org`, then `tenant-foo` would get the domain `foo.example.org` by default.

Kubernetes clusters created in this tenant namespace would get domains like:

`kubernetes-cluster.foo.example.org`

Example:

```
tenant-root (example.org)
 tenant-foo (foo.example.org)
 kubernetes-cluster1 (kubernetes-cluster1.foo.example.org)
```

## Nesting tenants and reusing parent services

---

Tenants can be nested. A tenant administrator can create nested tenants using the “Tenant” application from the catalogue.

Higher-level tenants can view and manage the applications of all their children tenants. If a tenant does not run their own cluster services, it can access ones of its parent.

For example, you create:

- Tenant `tenant-u1` with a set of services like `etcd`, `ingress`, `monitoring`.
- Tenant `tenant-u2` nested in `tenant-u1`.

Let's see what will happen when you run Kubernetes and Postgres under `tenant-u2` namespace.

Since `tenant-u2` does not have its own cluster services like `etcd`, `ingress`, and `monitoring`, the applications running in `tenant-u2` will use the cluster services of the parent tenant.

This in turn means:

- The Kubernetes cluster data will be stored in `etcd` for `tenant-u1`.
- Access to the cluster will be through the common `ingress` of `tenant-u1`.
- Essentially, all metrics will be collected in the `monitoring` from `tenant-u1`, and only that tenant will have access to them.

Example:

```
tenant-u1
 etcd
 ingress
 monitoring
 tenant-u2
 kubernetes-cluster1
 postgres-db1
```

## Parameters

---

### Common parameters

| Name                    | Description                                                                                                                | Type                | Value              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------|--------------------|
| <code>host</code>       | The hostname used to access tenant services (defaults to using the tenant name as a subdomain for its parent tenant host). | <code>string</code> | <code>""</code>    |
| <code>etcd</code>       | Deploy own Etcd cluster.                                                                                                   | <code>bool</code>   | <code>false</code> |
| <code>monitoring</code> | Deploy own Monitoring Stack.                                                                                               | <code>bool</code>   | <code>false</code> |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                  |                    |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|--------------------|
| <code>ingress</code>         | Deploy own Ingress Controller.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <code>bool</code>                | <code>false</code> |
| <code>gateway</code>         | Deploy own Gateway API Gateway (backed by Cilium Gateway API controller). When unset (the default), the chart auto-enables the Gateway for tenants whose apex is derived from the parent (i.e. <code>host</code> is empty), and leaves it off for tenants with a custom non-derived apex. Set to <code>true</code> or <code>false</code> explicitly to override that auto-behaviour. Note: leave the key absent (do not write <code>gateway: null</code> ) — the chart distinguishes “unset” via missing-key, not via null value, to satisfy the JSON schema generated from this comment. | <code>bool</code>                | <code>false</code> |
| <code>seaweedfs</code>       | Deploy own SeaweedFS.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <code>bool</code>                | <code>false</code> |
| <code>schedulingClass</code> | The name of a SchedulingClass CR to apply scheduling constraints for this tenant’s workloads.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <code>string</code>              | <code>""</code>    |
| <code>resourceQuotas</code>  | Define resource quotas for the tenant.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <code>map[string]quantity</code> | <code>{}</code>    |

## Configuration

### Resource Quotas

The `resourceQuotas` parameter allows you to limit resources available to the tenant. Supported keys include:

---

Compute resources (converted to `requests.X` and `limits.X`):

- `cpu` - Total CPU cores (e.g., "4" or "500m")
- `memory` - Total memory (e.g., "4Gi" or "512Mi")
- `ephemeral-storage` - Ephemeral storage limit (e.g., "10Gi")
- `storage` - Total persistent storage (e.g., "100Gi")

Object count quotas (passed as-is):

- `pods` - Maximum number of pods
- `services` - Maximum number of services
- `services.loadbalancers` - Maximum number of LoadBalancer services
- `services.nodeports` - Maximum number of NodePort services
- `configmaps` - Maximum number of ConfigMaps
- `secrets` - Maximum number of Secrets
- `persistentvolumeclaims` - Maximum number of PVCs

Example:

```
resourceQuotas:
 cpu: 4
 memory: 4Gi
 storage: 10Gi
 services.loadbalancers: "3"
 pods: "50"
```



# Виртуализация

## Возможности виртуализации в Cozystack

<https://cozystack.ru/docs/v1.5/virtualization/>

В этом руководстве объясняется, как работает виртуализация в Cozystack.

### Пакеты виртуализации

Каталог Cozystack включает два пакета, связанных с виртуализацией:

- `vm-disk` - диск виртуальной машины
- `vm-instance` - экземпляр виртуальной машины

### Диск виртуальной машины

Прежде чем создать экземпляр виртуальной машины, необходимо создать диск, с которого будет загружаться VM.

Этот пакет определяет диск виртуальной машины, используемый для хранения данных. Вы можете использовать подготовленный образ (также известный как `golden image`), загрузить образ на диск по HTTP или загрузить его из локального образа. Также можно создать пустой образ.

#### 1. Golden Image:

```
``yaml
```

```
@param source The source image location used to create a disk
```

```
source:
```

```
image:
```

```
name: ubuntu
```

```
``
```

#### 2. HTTP:

```
``yaml
```

```
source:
```

```
http:
```

```
url: "https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img"
```

```
``
```

### 3. Upload:

```
```yaml
```

```
source:
```

```
upload: {}
```

```
```
```

После создания диска будет сгенерирована команда для загрузки с помощью инструмента virtctl.

Note:

Если вы хотите, чтобы virtctl знал о правильной конечной точке для загрузки образов, необходимо настроить кластер, указав для него конечную точку:

>

1. Пропатчите Platform Package, чтобы открыть доступ к cdi-uploadproxy вместе с панелью управления:

```
```bash
```

```
kubectrl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
```

```
-p '[{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "cdi-uploadproxy"}]'
```

```
```
```

2. Укажите действительную конечную точку CDI uploadproxy, пропатчив Package kubevirt-cdi:

```
```bash
```

```
kubectrl patch packages.cozystack.io cozystack.kubevirt-cdi --type=merge -p '{
```

```
"spec": {
```

```
"components": {
```

```
"kubevirt-cdi": {
```

```
"values": {
```

```
"uploadProxyURL": "https://cdi-uploadproxy.example.org"
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}'
```

```
```
```

### 4. Empty:

```
```yaml
```

```
source: {}
```

```
```
```



---

При желании можно указать, что диск является оптическим CD-ROM:

```
optical: true
```

Созданные диски можно подключать к экземпляру виртуальной машины.

См. справочник по приложению: `[vm-disk]`(<https://cozystack.ru/docs/v1.5/virtualization/vm-disk/>).

## Экземпляр виртуальной машины

---

Этот пакет определяет экземпляр виртуальной машины, для которого требуется указать ранее созданный `vm-disk`. Первый диск всегда является загрузочным, и ВМ будет пытаться загрузиться с него.

```
disks:
- name: example-system
- name: example-data
```

Остальные параметры аналогичны Virtual Machine (simple).

См. справочник по приложению: `[vm-instance]`(<https://cozystack.ru/docs/v1.5/virtualization/vm-instance/>).

## Доступ к виртуальным машинам

---

Получить доступ к виртуальной машине можно с помощью инструмента `virtctl`:

- [KubeVirt User Guide - Virtctl Client Tool](#)

Для доступа к последовательной консоли:

```
virtctl console <vm>
```

Для доступа к ВМ через VNC:

```
virtctl vnc <vm>
```

Для подключения к ВМ по SSH:

```
virtctl ssh <user>@<vm>
```

---

## Виртуальная машина

---

---

## Диск виртуальной машины

---

## Создание и использование именованных образов VM

---

Руководство по созданию, управлению и использованию золотых (именованных) образов VM в Cozystack для ускорения развёртывания виртуальных машин.

## Клонлируемые виртуальные машины

---

Создание клонируемых виртуальных машин

## Запуск VM с пробросом GPU (passthrough)

---

Запуск VM с пробросом GPU (passthrough)

## Резервное копирование и восстановление

---

Как создавать резервные копии ресурсов VMInstance и VMDisk и управлять ими с помощью BackupJob и Plan.

## Запуск виртуальных машин Windows в Cozystack

---

Запуск виртуальных машин Windows в Cozystack

## Запуск MikroTik RouterOS в Cozystack

---

Развёртывание MikroTik RouterOS (CHR) как виртуального устройства в Cozystack

## Миграция виртуальных машин из Proxmox

---

Пошаговое руководство по миграции виртуальных машин из Proxmox VE в Cozystack

## Ресурсы виртуальных машин

---

Справочник по типам инстансов и профилям инстансов VM

---

# Виртуальная машина

<https://cozystack.ru/docs/v1.5/virtualization/vm-instance/>

Виртуальная машина (ВМ) имитирует аппаратное обеспечение компьютера, позволяя запускать различные операционные системы и приложения в изолированной среде.

## Детали развёртывания

Виртуальная машина управляется и размещается с помощью KubeVirt, что позволяет использовать преимущества виртуализации в рамках вашей экосистемы Kubernetes.

- Документация: [KubeVirt User Guide](#)
- GitHub: [KubeVirt Repository](#)

## Доступ к виртуальной машине

Вы можете получить доступ к виртуальной машине с помощью инструмента virtctl:

[KubeVirt User Guide - Virtctl Client Tool](#)

Для доступа к последовательной консоли:

```
virtctl console <vm>
```

Для доступа к ВМ по VNC:

```
virtctl vnc <vm>
```

Для подключения к ВМ по SSH:

```
virtctl ssh <user>@<vm>
```

## Параметры

### Общие параметры

| Имя            | Описание                                | Тип    |
|----------------|-----------------------------------------|--------|
| external       | Включить внешний доступ извне кластера. | bool   |
| externalMethod | Метод передачи трафика к ВМ.            | string |

|                   |                                                                                                                                                                                                                                  |          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| externalPorts     | Порты для проброса извне кластера.                                                                                                                                                                                               | []int    |
| externalAllowICMP | Принимать ли ICMP-трафик к ВМ в режиме PortList (сохраняет ping и обнаружение PMTU). Не действует в режиме WholeIP. По умолчанию true, чтобы ping работал так, как ожидают пользователи, даже когда действует фильтрация портов. | bool     |
| runStrategy       | Запрошенное состояние работы VirtualMachineInstance                                                                                                                                                                              | string   |
| instanceType      | Тип инстанса виртуальной машины.                                                                                                                                                                                                 | string   |
| instanceProfile   | Профиль предпочтений виртуальной машины.                                                                                                                                                                                         | string   |
| disks             | Список дисков для подключения.                                                                                                                                                                                                   | []object |
| disks[i].name     | Имя диска.                                                                                                                                                                                                                       | string   |
| disks[i].bus      | Тип шины диска (например, "sata").                                                                                                                                                                                               | string   |
| networks          | Сети, к которым подключается ВМ.                                                                                                                                                                                                 | []object |
| subnets           | Устарело: используйте networks вместо этого.                                                                                                                                                                                     | []object |
| subnets[i].name   | Имя сетевого подключения (network attachment).                                                                                                                                                                                   | string   |
| gpus              | Список GPU для подключения (драйвер NVIDIA требует не менее 4 ГиБ RAM).                                                                                                                                                          | []object |
| gpus[i].name      | Имя ресурса GPU для подключения.                                                                                                                                                                                                 | string   |
| cpuModel          | Model задаёт модель CPU внутри VMI. Список доступных моделей <a href="https://github.com/libvirt/libvirt/tree/master/src/cpu_map">https://github.com/libvirt/libvirt/tree/master/src/cpu_map</a>                                 | string   |

|                   |                                                |          |
|-------------------|------------------------------------------------|----------|
| resources         | Конфигурация ресурсов для виртуальной машины.  | object   |
| resources.cpu     | Количество выделенных ядер CPU.                | quantity |
| resources.memory  | Объём выделенной памяти.                       | quantity |
| resources.sockets | Количество сокетов CPU (топология vCPU).       | quantity |
| sshKeys           | Список открытых SSH-ключей для аутентификации. | []string |
| cloudInit         | Пользовательские данные cloud-init.            | string   |
| cloudInitSeed     | Seed-строка для генерации SMBIOS UUID для ВМ.  | string   |

## Серия U

Серия U довольно нейтральна и предоставляет ресурсы для приложений общего назначения.

U — это сокращение от «Universal», что намекает на универсальное отношение к рабочим нагрузкам.

ВМ этих типов инстансов делят физические ядра CPU на основе квантования времени с другими ВМ.

### Характеристики серии U

Специфические характеристики этой серии:

- Пакетная (burstable) производительность CPU — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
- Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4, для меньшего «шума» на узел.

## Серия O

Серия O основана на серии U, с единственным отличием, заключающимся в том, что память переподписана (overcommitted).

O — это сокращение от «Overcommitted».

### Характеристики серии UO

Специфические характеристики этой серии:

- 
- Пакетная (burstable) производительность CPU — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
  - Переподписанная память — память переподписана для достижения более высокой плотности рабочих нагрузок.
  - Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4, для меньшего «шума» на узел.

## Серия CX

Серия CX предоставляет эксклюзивные вычислительные ресурсы для вычислительно интенсивных приложений.

CX — это сокращение от «Compute Exclusive».

Эксклюзивные ресурсы предоставляются вычислительным потокам VM. Чтобы обеспечить это, будут запрошены и выделены физические ядра CPU специально для VM. Кроме того, в этой серии топология NUMA используемых ядер предоставляется VM.

### Характеристики серии CX

Специфические характеристики этой серии:

- Nuberpages — Nuberpages используются для улучшения производительности памяти.
- Выделенный CPU — физические ядра эксклюзивно назначаются каждому vCPU, чтобы предоставить фиксированные и высокие гарантии вычислений для рабочей нагрузки.
- Изолированные потоки эмулятора — потоки эмулятора гипервизора изолированы от vCPU, чтобы уменьшить влияние, связанное с эмуляцией, на рабочую нагрузку.
- vNUMA — физическая топология NUMA отражается в гостевой системе, чтобы оптимизировать использование кэша на стороне гостя.
- Соотношение vCPU к памяти (1:2) — соотношение vCPU к памяти 1:2.

## Серия M

Серия M предоставляет ресурсы для приложений, интенсивно использующих память.

M — это сокращение от «Memory».

### Характеристики серии M

Специфические характеристики этой серии:

- Nuberpages — Nuberpages используются для улучшения производительности памяти.
- Пакетная (burstable) производительность CPU — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
- Соотношение vCPU к памяти (1:8) — соотношение vCPU к памяти 1:8, для гораздо меньшего «шума» на узел.

---

## Серия RT

Серия RT предоставляет ресурсы для приложений реального времени, таких как Oslat.

RT — это сокращение от «realtime».

Эта серия типов инстансов требует узлов, способных запускать приложения реального времени.

### Характеристики серии RT

Специфические характеристики этой серии:

- Hugepages — Hugepages используются для улучшения производительности памяти.
- Выделенный CPU — физические ядра эксклюзивно назначаются каждому vCPU, чтобы предоставить фиксированные и высокие гарантии вычислений для рабочей нагрузки.
- Изолированные потоки эмулятора — потоки эмулятора гипервизора изолированы от vCPU, чтобы уменьшить влияние, связанное с эмуляцией, на рабочую нагрузку.
- Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4, начиная с размера medium.

## Разработка

Чтобы начать настройку или создание собственных instancetypes и preferences, см. [DEVELOPMENT.md](#).

## Ресурсы

Следующие ресурсыinstancetype предоставляются Cozystack:

| Name           | vCPUs | Memory |
|----------------|-------|--------|
| cx1.2xlarge    | 8     | 16Gi   |
| cx1.4xlarge    | 16    | 32Gi   |
| cx1.4xlargeigi | 16    | 32Gi   |
| cx1.8xlarge    | 32    | 64Gi   |
| cx1.8xlargeigi | 32    | 64Gi   |
| cx1.large      | 2     | 4Gi    |
| cx1.largeigi   | 2     | 4Gi    |
| cx1.medium     | 1     | 2Gi    |
| cx1.mediumigi  | 1     | 2Gi    |
| cx1.xlarge     | 4     | 8Gi    |
| cx1.xlargeigi  | 4     | 8Gi    |

|               |    |       |
|---------------|----|-------|
| d1.2xlarge    | 8  | 32Gi  |
| d1.2xmedium   | 2  | 4Gi   |
| d1.4xlarge    | 16 | 64Gi  |
| d1.8xlarge    | 32 | 128Gi |
| d1.large      | 2  | 8Gi   |
| d1.medium     | 1  | 4Gi   |
| d1.micro      | 1  | 1Gi   |
| d1.nano       | 1  | 512Mi |
| d1.small      | 1  | 2Gi   |
| d1.xlarge     | 4  | 16Gi  |
| gn1.2xlarge   | 8  | 32Gi  |
| gn1.xlarge    | 4  | 16Gi  |
| m1.2xlarge    | 8  | 64Gi  |
| m1.2xlargelgi | 8  | 64Gi  |
| m1.4xlarge    | 16 | 128Gi |
| m1.4xlargelgi | 16 | 128Gi |
| m1.8xlarge    | 32 | 256Gi |
| m1.8xlargelgi | 32 | 256Gi |
| m1.large      | 2  | 16Gi  |
| m1.largelgi   | 2  | 16Gi  |
| m1.xlarge     | 4  | 32Gi  |
| m1.xlargelgi  | 4  | 32Gi  |
| n1.2xlarge    | 16 | 32Gi  |
| n1.4xlarge    | 32 | 64Gi  |
| n1.8xlarge    | 64 | 128Gi |
| n1.large      | 4  | 8Gi   |
| n1.medium     | 4  | 4Gi   |
| n1.xlarge     | 8  | 16Gi  |
| o1.2xlarge    | 8  | 32Gi  |
| o1.4xlarge    | 16 | 64Gi  |



|             |    |       |
|-------------|----|-------|
| o1.8xlarge  | 32 | 128Gi |
| o1.large    | 2  | 8Gi   |
| o1.medium   | 1  | 4Gi   |
| o1.micro    | 1  | 1Gi   |
| o1.nano     | 1  | 512Mi |
| o1.small    | 1  | 2Gi   |
| o1.xlarge   | 4  | 16Gi  |
| rt1.2xlarge | 8  | 32Gi  |
| rt1.4xlarge | 16 | 64Gi  |
| rt1.8xlarge | 32 | 128Gi |
| rt1.large   | 2  | 8Gi   |
| rt1.medium  | 1  | 4Gi   |
| rt1.micro   | 1  | 1Gi   |
| rt1.small   | 1  | 2Gi   |
| rt1.xlarge  | 4  | 16Gi  |
| u1.2xlarge  | 8  | 32Gi  |
| u1.2xmedium | 2  | 4Gi   |
| u1.4xlarge  | 16 | 64Gi  |
| u1.8xlarge  | 32 | 128Gi |
| u1.large    | 2  | 8Gi   |
| u1.medium   | 1  | 4Gi   |
| u1.micro    | 1  | 1Gi   |
| u1.nano     | 1  | 512Mi |
| u1.small    | 1  | 2Gi   |
| u1.xlarge   | 4  | 16Gi  |

Следующие ресурсы preference предоставляются Cozystack:

| Name                    | Guest OS         |
|-------------------------|------------------|
| alpine                  | Alpine           |
| centos.stream10         | CentOS Stream 10 |
| centos.stream10.desktop | CentOS Stream 10 |

|                          |                                             |
|--------------------------|---------------------------------------------|
| centos.stream9           | CentOS Stream 9                             |
| centos.stream9.desktop   | CentOS Stream 9                             |
| centos.stream9.dpdk      | CentOS Stream 9                             |
| cirros                   | Cirros                                      |
| debian                   | Debian                                      |
| fedora                   | Fedora (amd64)                              |
| fedora.arm64             | Fedora (arm64)                              |
| fedora.s390x             | Fedora (s390x)                              |
| legacy                   | Legacy Guest                                |
| linux                    | Linux Guest                                 |
| linux.efi                | Linux EFI Guest                             |
| linux.virtiotransitional | Linux Virtio Transitional Guest             |
| opensuse.leap            | OpenSUSE Leap                               |
| opensuse.tumbleweed      | OpenSUSE Tumbleweed                         |
| oraclelinux              | Oracle Linux                                |
| rhel.10.arm64            | Red Hat Enterprise Linux 10 (arm64)         |
| rhel.10.s390x            | Red Hat Enterprise Linux 10 (s390x)         |
| rhel.7                   | Red Hat Enterprise Linux 7                  |
| rhel.7.desktop           | Red Hat Enterprise Linux 7                  |
| rhel.8                   | Red Hat Enterprise Linux 8                  |
| rhel.8.desktop           | Red Hat Enterprise Linux 8                  |
| rhel.8.dpdk              | Red Hat Enterprise Linux 8                  |
| rhel.9                   | Red Hat Enterprise Linux 9 (amd64)          |
| rhel.9.arm64             | Red Hat Enterprise Linux 9 (arm64)          |
| rhel.9.desktop           | Red Hat Enterprise Linux 9 Desktop (amd64)  |
| rhel.9.dpdk              | Red Hat Enterprise Linux 9 DPDK (amd64)     |
| rhel.9.realtime          | Red Hat Enterprise Linux 9 Realtime (amd64) |
| rhel.9.s390x             | Red Hat Enterprise Linux 9 (s390x)          |
| sles                     | SUSE Linux Enterprise Server                |
| ubuntu                   | Ubuntu                                      |

|                     |                                                |
|---------------------|------------------------------------------------|
| windows.10          | Microsoft Windows 10                           |
| windows.10.virtio   | Microsoft Windows 10 (virtio)                  |
| windows.11          | Microsoft Windows 11                           |
| windows.11.virtio   | Microsoft Windows 11 (virtio)                  |
| windows.2k12        | Microsoft Windows Server 2012/2012 R2          |
| windows.2k12.virtio | Microsoft Windows Server 2012/2012 R2 (virtio) |
| windows.2k16        | Microsoft Windows Server 2016                  |
| windows.2k16.virtio | Microsoft Windows Server 2016 (virtio)         |
| windows.2k19        | Microsoft Windows Server 2019                  |
| windows.2k19.virtio | Microsoft Windows Server 2019 (virtio)         |
| windows.2k22        | Microsoft Windows Server 2022                  |
| windows.2k22.virtio | Microsoft Windows Server 2022 (virtio)         |
| windows.2k25        | Microsoft Windows Server 2025                  |
| windows.2k25.virtio | Microsoft Windows Server 2025 (virtio)         |
| windows.2k3         | Microsoft Windows Server 2003                  |
| windows.2k8         | Microsoft Windows Server 2008/2008 R2          |
| windows.2k8.virtio  | Microsoft Windows Server 2008/2008 R2 (virtio) |
| windows.7           | Microsoft Windows 7                            |
| windows.7.virtio    | Microsoft Windows 7 (virtio)                   |
| windows.xp          | Microsoft Windows XP                           |

# Диск виртуальной машины

<https://cozystack.ru/docs/v1.5/virtualization/vm-disk/>

## Диск виртуальной машины

`storageClass` помечен как неизменяемый (`immutable`) в схеме чарта — см. [\[docs/storage-immutability.md\]](https://cozystack.ru/docs/docs/storage-immutability.md)(<https://cozystack.ru/docs/docs/storage-immutability.md>), где описан этот контракт и какие потребители его обеспечивают.

## Параметры

### Общие параметры

| Имя                            | Описание                                                         | Тип                  | Значение          |
|--------------------------------|------------------------------------------------------------------|----------------------|-------------------|
| <code>source</code>            | Расположение исходного образа, используемого для создания диска. | <code>object</code>  | <code>{}</code>   |
| <code>source.image</code>      | Использовать образ по имени из коллекции по умолчанию.           | <code>*object</code> | <code>null</code> |
| <code>source.image.name</code> | Имя используемого образа.                                        | <code>string</code>  | <code>""</code>   |
| <code>source.upload</code>     | Загрузить локальный образ.                                       | <code>*object</code> | <code>null</code> |
| <code>source.http</code>       | Скачать образ из источника по HTTP.                              | <code>*object</code> | <code>null</code> |
| <code>source.http.url</code>   | URL для скачивания образа.                                       | <code>string</code>  | <code>""</code>   |
| <code>source.disk</code>       | Клонировать существующий <code>vm-disk</code> .                  | <code>*object</code> | <code>null</code> |
| <code>source.disk.name</code>  | Имя копируемого <code>vm-disk</code> .                           | <code>string</code>  | <code>""</code>   |

---

|              |                                                   |          |            |
|--------------|---------------------------------------------------|----------|------------|
| optical      | Определяет, следует ли считать диск оптическим.   | bool     | false      |
| storage      | Размер диска, выделенного для виртуальной машины. | quantity | 5Gi        |
| storageClass | StorageClass, используемый для хранения данных.   | string   | replicated |

---

# Создание и использование именованных образов ВМ

<https://cozystack.ru/docs/v1.5/virtualization/vm-image/>

Золотые образы (golden images) в Cozystack позволяют администраторам подготовить именованные образы операционных систем, которые пользователи смогут впоследствии повторно использовать при создании виртуальных машин. В этом руководстве объясняются преимущества золотых образов, способы их создания и использования при развёртывании ВМ.

По умолчанию каждый раз, когда пользователь создаёт виртуальную машину, Cozystack загружает требуемый образ из его источника по URL. Это может стать узким местом, когда несколько ВМ создаются в быстрой последовательности. Золотые образы решают эту проблему, кэшируя образ локально, что устраняет повторные загрузки и ускоряет развёртывание.

## Соглашения об именовании (важно)

Cozystack автоматически добавляет префиксы к внутренним ресурсам Kubernetes:

| Имя, видимое пользователю | Resource Kind                            | Фактическое имя ресурса   |
|---------------------------|------------------------------------------|---------------------------|
| <image>                   | DataVolume в cozy-public (золотой образ) | vm-default-images-<image> |
| <disk>                    | DataVolume, созданный из VMDisk          | vm-disk-<disk>            |
| <vm>                      | VMInstance                               | vm-instance-<vm>          |

Это означает, что если вы создадите VMInstance с именем `ubuntu`, VirtualMachine в Kubernetes будет называться `vm-instance-ubuntu`.

## Коллекция образов по умолчанию (подключаемый пакет)

Cozystack поставляется с необязательным пакетом `vm-default-images`, который предоставляет тщательно подобранную коллекцию готовых золотых образов (Ubuntu, Rocky Linux, AlmaLinux, Debian, CentOS Stream, openSUSE, Alpine) в пространстве имён `cozy-public`. Пакет отключён по умолчанию и должен быть включён явно.

## Storage requirements

Набор образов по умолчанию запрашивает примерно 320Gi хранилища (16 образов × 20Gi каждый). Перед включением сократите список образов или уменьшите размеры хранилища для каждого образа, чтобы они соответствовали ёмкости вашего кластера.

Коллекция по умолчанию включает:

| Имя образа         | Описание                           |
|--------------------|------------------------------------|
| ubuntu-20.04       | Ubuntu 20.04 LTS (Focal Fossa)     |
| ubuntu-22.04       | Ubuntu 22.04 LTS (Jammy Jellyfish) |
| ubuntu-24.04       | Ubuntu 24.04 LTS (Noble Numbat)    |
| debian-12          | Debian 12 (Bookworm)               |
| debian-13          | Debian 13 (Trixie)                 |
| rocky-8            | Rocky Linux 8                      |
| rocky-9            | Rocky Linux 9                      |
| rocky-10           | Rocky Linux 10                     |
| almalinux-8        | AlmaLinux 8                        |
| almalinux-9        | AlmaLinux 9                        |
| almalinux-10       | AlmaLinux 10                       |
| centos-stream-9    | CentOS Stream 9                    |
| centos-stream-10   | CentOS Stream 10                   |
| opensuse-leap-15.6 | openSUSE Leap 15.6                 |
| opensuse-leap-16.0 | openSUSE Leap 16.0                 |
| alpine-3.21        | Alpine Linux 3.21                  |

Вы можете вывести список всех доступных образов с помощью:

```
kubectl -n cozy-public get dv -l app.kubernetes.io/managed-by=cozystack
```

## Включение пакета

Добавьте `cozystack.vm-default-images` в `bundles.enabledPackages` в [Platform Package](#):

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json -p '[{"op": "add", "path": "/s...
```

Подождите минуту, пока чарт платформы согласует состояние, затем проверьте `HelmRelease` и `DataVolumes`:

```
kubectl -n cozy-public get hr cozystack.vm-default-images
kubectl -n cozy-public get dv
```

DataVolumes, предоставляемые пакетом, именуются `vm-default-images-<image>` и доступны арендаторам как золотые образы с именем `<image>` (например, `ubuntu-24.04`, `debian-12`).

## Настройка списка образов

Переопределите список по умолчанию, отредактировав `Package cozystack.vm-default-images` и задав значения в `spec.components.vm-default-images.values`. Схема определена в файле [values.yaml](#) чарта:

- `storageClass` — `StorageClass` по умолчанию для всех образов; при пустом значении используется `StorageClass` кластера по умолчанию.
- `images[]` — список записей золотых образов. Каждая запись содержит:
  - `name` — имя образа в том виде, в котором оно доступно пользователям (например, `ubuntu-24.04`).
  - `url` — HTTP(S)-URL источника образа.
  - `storage` — выделяемый размер хранилища (например, `20Gi`).
  - `storageClass` — переопределение глобального `StorageClass` для конкретного образа.
  - `os.family`, `os.name`, `os.version`, `architecture`, `description` — необязательные метаданные, отображаемые в UI.

Пример: сократить список по умолчанию до двух образов и зафиксировать `StorageClass`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
 name: cozystack.vm-default-images
spec:
 variant: default
 components:
 vm-default-images:
 values:
 storageClass: replicated
 images:
 - name: ubuntu-24.04
 url: https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
 storage: 20Gi
 os:
 family: Linux
 name: Ubuntu
 version: "24.04"
 architecture: amd64
 description: "Ubuntu 24.04 LTS (Noble Numbat) cloud image"
 - name: debian-12
 url: https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-generic-amd64.qcow2
 storage: 20Gi
 os:
 family: Linux
 name: Debian
 version: "12"
 architecture: amd64
 description: "Debian 12 (Bookworm) generic cloud image"
```



---

Чтобы удалить образ после установки пакета, уберите его из `images[]` и удалите осиротевший `DataVolume`:

```
kubectl -n cozy-public delete dv vm-default-images-<name>
```

## Добавление пользовательских золотых образов

---

Для создания дополнительных именованных образов ВМ требуется учётная запись администратора в Cozystack. Простейший способ добавить пользовательский образ — использовать CLI-скрипт. Скрипт `cdi_golden_image_create.sh` можно скачать из тега релиза Cozystack v1.5.0:

```
curl -L0 https://raw.githubusercontent.com/cozystack/cozystack/v1.5.0/hack/cdi_golden_image_create.sh
chmod +x cdi_golden_image_create.sh
```

Этот скрипт использует вашу конфигурацию `kubectl`. Перед его запуском убедитесь, что ваша конфигурация указывает на целевой кластер Cozystack. Чтобы добавить пользовательский образ, запустите скрипт с именем образа и его URL:

```
cdi_golden_image_create.sh '<name>' 'https://<image-url>'
```

Например, чтобы добавить образ Talos:

```
cdi_golden_image_create.sh 'talos' 'https://github.com/siderolabs/talos/releases/download/v1.7.6/nocloud-am...
```

Внутри скрипт создаёт ресурс Kubernetes `kind: DataVolume` в пространстве имён `cozy-public`. Имя ресурса — это имя образа с префиксом `vm-default-images-`. Например, ресурс `vm-default-images-talos` создаёт образ, доступный как `talos`.

Вы можете отслеживать ход выполнения с помощью:

```
kubectl -n cozy-public get dv
kubectl -n cozy-public describe dv vm-default-images-talos
```

## Использование золотых образов

---

### Создание VMDisk из золотого образа

---

Чтобы использовать золотой образ в качестве источника для диска ВМ, создайте `VMDisk` с `source.image.name`, ссылающимся на имя образа:

---

```
kubectl -n tenant-root create -f- <<EOF
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
 name: ubuntu
spec:
 source:
 image:
 name: ubuntu-24.04
 storage: 20Gi
EOF
```

Вы можете отслеживать процесс с помощью следующих команд:

```
kubectl -n tenant-root get vmdisk
kubectl -n tenant-root get dv
kubectl -n tenant-root describe dv vm-disk-ubuntu
```

## Подключение диска к ВМ

---

Далее создайте VMInstance, использующий этот диск:

```
kubectl -n tenant-root create -f- <<EOF
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
 name: ubuntu
spec:
 disks:
 - name: ubuntu
EOF
```

Вы можете проверить статус VirtualMachine с помощью:

```
kubectl -n tenant-root get vminstance
```

Чтобы подключиться к ВМ, выполните:

```
virtctl -n tenant-root console ubuntu
```

---

# Клонируемые виртуальные машины

<https://cozystack.ru/docs/v1.5/virtualization/cloneable-vms/>

Чтобы создать клонируемую VM, вам потребуется создать `VMDisk` и `VMInstance`. В этом руководстве в качестве примера используется базовый образ `ubuntu`.

## 1. Создайте VMDisk

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
 name: ubuntu-source
 namespace: tenant-root
spec:
 optical: false
 source:
 http:
 url: https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
 storage: 20Gi
 storageClass: replicated
```

Поскольку расширение диска может быть сложной задачей, мы рекомендуем создавать его с запасом места для будущего роста.

## 2. Создайте VMInstance

Поскольку пользовательский ресурс `VirtualMachine` не предоставляет удобного способа работы с несколькими дисками, используйте вместо него `VMInstance`.

Создайте `VMInstance`, используя следующий шаблон:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
 name: sourcevm
 namespace: tenant-root
spec:
 externalMethod: PortList
 disks:
 - name: ubuntu-source
 externalPorts:
 - 22
 instanceProfile: ubuntu
 instanceType: ""
 running: true
 sshKeys:
 - <paste your ssh public key here>
 external: true
 resources:
 cpu: "2"
 memory: 4Gi
```

---

После создания VM она получит внешний IP-адрес балансировщика нагрузки. Вы можете получить его с помощью:

```
kubectl get svc -n tenant-root vm-instance-sourcevm
```

### 3. Подключитесь к VM по SSH

Теперь вы можете подключиться к VM по SSH, используя внешний IP-адрес. Пользователь по умолчанию для базового образа ubuntu — `ubuntu`.

```
ssh ubuntu@<external IP>
```

Настройте виртуальную машину перед клонированием.

### 4. Удалите VMInstance

Данные на диске будут сохранены.

```
kubectl delete vminstance -n tenant-root sourcevm
```

### 5. Создайте образ диска

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
 name: "vm-image-sourcevm" # prefix vm-image is necessary
 namespace: cozy-public # do not change
 annotations:
 cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
 source:
 pvc:
 name: vm-disk-ubuntu-source
 namespace: tenant-root
 storage:
 resources:
 requests:
 storage: 20Gi
 storageClassName: replicated
```

Это займёт некоторое время. Подождите, прежде чем продолжить. Вы можете проверить ход выполнения с помощью:

```
kubectl get datavolume -n cozy-public vm-image-sourcevm
```

Пример вывода, когда всё готово:

| NAME | PHASE | PROGRESS | RESTARTS | AGE |
|------|-------|----------|----------|-----|
|------|-------|----------|----------|-----|

|                   |           |        |  |       |
|-------------------|-----------|--------|--|-------|
| vm-image-sourcevm | Succeeded | 100.0% |  | 7m32s |
|-------------------|-----------|--------|--|-------|

## 6. Создайте VMDisk из клонированного образа

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
 name: ubuntu-cloned-1
 namespace: tenant-root
spec:
 optical: false
 source:
 image:
 name: sourcevm # image name without prefix
 storage: 20Gi # size greater or equal to the disk image size
 storageClass: replicated
```

## 7. Создайте VMInstance из клонированного диска

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
 name: cloned-vm
 namespace: tenant-root
spec:
 external: true
 externalMethod: PortList
 cloudInit:
 cloudInitSeed: "1"
 userData: |
 #cloud-config
 hostname: my-cloned-vm
 disks:
 - name: ubuntu-cloned-1
 externalPorts:
 - 22
 instanceProfile: ubuntu
 running: true
```

Чтобы сетевые функции клонированной VM работали корректно, вы должны переопределить её `hostname` через `.spec.cloudInit` и указать уникальный `.spec.cloudInitSeed`, чтобы избежать конфликтов с исходной VM.

# Запуск ВМ с пробросом GPU (passthrough)

<https://cozystack.ru/docs/v1.5/virtualization/gpu/>

В этом разделе показано, как развёртывать виртуальные машины (ВМ) с пробросом GPU (passthrough) с помощью Cozystack. Сначала мы развернём GPU Operator, чтобы настроить рабочий узел для проброса GPU (passthrough). Затем мы развернём ВМ KubeVirt, запрашивающую GPU.

По умолчанию для обеспечения проброса GPU (passthrough) GPU Operator развёртывает следующие компоненты:

- VFIO Manager для привязки драйвера `vfio-pci` ко всем GPU на узле.
- Sandbox Device Plugin для обнаружения проброшенных GPU и их анонсирования kubelet.
- Sandbox Validator для проверки остальных операндов.

## Предварительные требования

- Кластер Cozystack с хотя бы одним узлом с поддержкой GPU.
- Установленный `kubectl` и настроенные учётные данные для доступа к кластеру.

## 1. Установка GPU Operator

Выполните следующие шаги:

1. Явно пометьте рабочий узел меткой для рабочих нагрузок с пробросом GPU (passthrough):

```
``bash
kubectl label node <node-name> --overwrite nvidia.com/gpu.workload.config=passthrough
``
```

2. Включите GPU Operator в вашем Platform Package, добавив его в список включённых пакетов:

```
``bash
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "gpu-operator"}]'
``
```

Это развернёт компоненты (операнды).

3. Убедитесь, что все поды находятся в состоянии `running` и все проверки компонента `sandbox-validator` проходят успешно:

```
``bash
kubectl get pods -n cozy-gpu-operator
```

...

Пример вывода (имена ваших подов могут отличаться):

```
```text
```

```
NAME READY STATUS RESTARTS AGE
```

...

```
nvidia-sandbox-device-plugin-daemonset-4mxsc 1/1 Running 0 40s
```

```
nvidia-sandbox-validator-vxj7t 1/1 Running 0 40s
```

```
nvidia-vfio-manager-thfwf 1/1 Running 0 78s
```

...

Чтобы проверить привязку GPU, получите доступ к узлу с помощью `kubectl node-shell -n cozy-system -x` или `kubectl debug node` и выполните:

```
lspci -nnk -d 10de:
```

Под `vfio-manager` привяжет все GPU на узле к драйверу `vfio-pci`. Пример вывода:

```
3b:00.0 3D controller [0302]: NVIDIA Corporation Device [10de:2236] (rev a1)
        Subsystem: NVIDIA Corporation Device [10de:1482]
        Kernel driver in use: vfio-pci
86:00.0 3D controller [0302]: NVIDIA Corporation Device [10de:2236] (rev a1)
        Subsystem: NVIDIA Corporation Device [10de:1482]
        Kernel driver in use: vfio-pci
```

`sandbox-device-plugin` обнаружит эти ресурсы и анонсирует их `kubelet`. В этом примере узел показывает два GPU A10 как доступные ресурсы:

```
kubectl describe node <node-name>
```

Пример вывода:

```
...
Capacity:
  ...
  nvidia.com/GA102GL_A10:          2
  ...
Allocatable:
  ...
  nvidia.com/GA102GL_A10:          2
  ...
```

Примечание: Имена ресурсов формируются путём объединения столбцов `device` и `device_name` из [базы данных PCI IDs](#). Например, запись в базе данных для A10 выглядит как 2236 GA102GL [A10], что приводит к имени ресурса `nvidia.com/GA102GL_A10`.

2. Обновление Custom Resource KubeVirt

Далее мы обновим Custom Resource KubeVirt, как описано в [руководстве пользователя KubeVirt](#), чтобы проброшенные GPU были разрешены и могли запрашиваться VM KubeVirt.

Подберите значения `pciVendorSelector` и `resourceName` под вашу конкретную модель GPU. Установка `externalResourceProvider=true` указывает, что этот ресурс предоставляется внешним device plugin, в данном случае `sandbox-device-plugin`, который развёртывается оператором.

```
kubectrl edit kubevirt -n cozy-kubevirt
```

пример конфигурации:

```
...
spec:
  configuration:
    permittedHostDevices:
      pciHostDevices:
        - externalResourceProvider: true
          pciVendorSelector: 10DE:2236
          resourceName: nvidia.com/GA102GL_A10
...

```

3. Создание виртуальной машины

Теперь мы готовы создать VM.

1. Создайте тестовую виртуальную машину с помощью следующей спецификации VMI, которая запрашивает ресурс `nvidia.com/GA102GL_A10`.

`vmi-gpu.yaml`:

```
```yaml
```

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: VirtualMachine
```

```
metadata:
```

```
name: gpu
```

```
namespace: tenant-example
```

```
spec:
```

```
running: true
```

```
instanceProfile: ubuntu
```

```
instanceType: u1.medium
```

```
systemDisk:
```

```
image: ubuntu
```

```
storage: 5Gi
```

```
storageClass: replicated
```

```
gpus:
```



---

```
- name: nvidia.com/GA102GL_A10
```

```
cloudInit: |
```

```
#cloud-config
```

```
password: ubuntu
```

```
chpasswd: { expire: False }
```

```
...
```

```
```bash
```

```
kubectl apply -f vmi-gpu.yaml
```

```
...
```

Пример вывода:

```
```text
```

```
virtualmachines.apps.cozystack.io/gpu created
```

```
...
```

2. Проверьте статус VM:

```
```bash
```

```
kubectl get vmi
```

```
...
```

3. Войдите в VM и убедитесь, что у неё есть доступ к GPU:

```
```bash
```

```
virtctl console virtual-machine-gpu
```

```
...
```

Пример вывода:

```
```text
```

```
Successfully connected to vmi-gpu console. The escape sequence is ^]
```

```
vmi-gpu login: ubuntu
```

```
Password:
```

```
ubuntu@virtual-machine-gpu:~$ lspci -nnk -d 10de:
```

```
08:00.0 3D controller [0302]: NVIDIA Corporation GA102GL [A10] [10de:2236] (rev a1)
```

```
Subsystem: NVIDIA Corporation GA102GL [A10] [10de:1851]
```

```
Kernel driver in use: nvidia
```

```
Kernel modules: nvidiafb, nvidia_drm, nvidia
```

```
...
```

Совместное использование GPU виртуальными машинами

Проброс GPU (passthrough) назначает весь физический GPU одной ВМ. Чтобы разделить один GPU между несколькими ВМ, требуется NVIDIA vGPU.

vGPU (виртуальный GPU)

NVIDIA vGPU использует опосредованные устройства (mdev) для создания виртуальных GPU, назначаемых ВМ. Это единственное готовое к промышленной эксплуатации решение для совместного использования GPU между ВМ.

Требования:

- Лицензия NVIDIA vGPU (коммерческая, приобретается у NVIDIA)
- NVIDIA vGPU Manager, установленный на узлах-хостах

Почему не MIG? MIG (Multi-Instance GPU) разделяет GPU на изолированные экземпляры, но это логические разделы внутри одного устройства PCIe. VFIO не может пробросить их в ВМ — MIG работает только с контейнерами. Чтобы использовать MIG с ВМ, нужен vGPU поверх разделов MIG (всё равно требуется лицензия).

Open-Source vGPU (экспериментально)

NVIDIA разрабатывает поддержку vGPU с открытым исходным кодом для ядра Linux. После её включения это может обеспечить совместное использование GPU без лицензии.

- Статус: стадия RFC, не включено в основную ветку ядра
 - Поддерживает: Ada Lovelace и новее (L4, L40 и т. д.)
 - Ссылки: [анонс Phoronix](#), [патчи ядра](#)
-

Резервное копирование

<https://cozystack.ru/docs/v1.5/virtualization/backup-and-recovery/>

Содержимое этой страницы недоступно в офлайн-версии. Откройте онлайн-версию по ссылке выше.

Запуск виртуальных машин Windows в Cozystack

<https://cozystack.ru/docs/v1.5/virtualization/windows/>

Cozystack может запускать виртуальные машины Windows. В этом руководстве описаны предварительные требования и шаги, необходимые для загрузки виртуальной машины с ОС Windows.

Предварительные требования

- ISO-образ установки Windows.
- ISO-образ драйверов Virtio.
- Клиент KubeVirt `virtctl` [установленный в вашем локальном окружении](#) и настроенный для пространства имён вашего тенанта.
- Cozystack версии v0.34.2 или новее.

Установка

Создание виртуальной машины с ОС Windows начинается с создания объектов `VMDisk` и продолжается созданием `VMInstance`.

1. Создание объектов `VMDisk`

Вам понадобится три диска:

1. Установочный ISO – оптический.
2. ISO с драйверами Virtio – оптический.
3. Системный диск – не оптический.

В следующем примере используются минимально рекомендованные тома хранилища.

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: win2k25-iso
spec:
  source:
    http:
      url: https://software-static.download.prss.microsoft.com/dbazure/888969d5-f34g-4e03-ac9d-1f9786c66749...
  optical: true
  storage: 6Gi
  storageClass: replicated
---
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: virtio-drivers
spec:
  source:
    http:
      url: https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso
  optical: true
  storage: 1Gi
  storageClass: replicated
---
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: win2k25-system
spec:
  optical: false
  storage: 50Gi
  storageClass: replicated
```

2. Создание VMInstance

Выберите профиль инстанса с поддержкой Virtio и подключите пустой системный диск. Выберите из доступных профилей Virtio:

```
windows.10.virtio
windows.11.virtio
windows.2k16.virtio
windows.2k19.virtio
windows.2k22.virtio
windows.2k25.virtio
```

Создайте объект `VMInstance`, как показано в этом примере:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
  name: win2k25-demo
spec:
  running: true
  instanceType: "u1.xlarge"
  instanceProfile: windows.2k25.virtio # выбран из списка выше
  disks:
    - name: win2k25-system
    - name: win2k25-iso
    - name: virtio-drivers
```

3. Установка Windows

1. Откройте консоль с помощью клиента `virtctl`:

```
``bash
```

```
virtctl vnc vm-instance-win2k25-demo
```

```
``
```

2. Продолжите стандартную установку Windows.
3. Когда появится запрос "Where do you want to install Windows?" (Куда вы хотите установить Windows?), выберите Load driver, затем перейдите к CD-ROM с Virtio, например `E:\viosstor\amd64\`.
4. После появления виртуального диска продолжите установку и дайте Windows перезагрузиться.
5. После первой перезагрузки отключите установочный диск Windows (`win2k25-iso`) и драйверы Virtio (`virtio-drivers`) от VMInstance.

Преобразование существующего образа Windows

Если у вас уже есть диск Windows, созданный на VMware, Hyper-V или в другом облаке, вы можете воспользоваться этим путём, чтобы подготовить его к работе с Virtio в Cozystack.

1. Создание фиктивного VMDisk для драйвера Virtio

Создайте фиктивный VMDisk, который будет использоваться для установки драйвера Virtio:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: dummy-disk-for-virtio
spec:
  optical: false
  storage: "1Gi"
  storageClass: "replicated"
```

2. Запуск с диском и не-Virtio шиной

При создании `VMInstance` подключите системный диск с шиной `sata`, а фиктивный диск — с неуказанной шиной — тогда для диска по умолчанию будет использована шина `Virtio SCSI`. Вы также можете одновременно смонтировать `Virtio ISO`, чтобы упростить установку драйвера.

```
spec:
  instanceProfile: windows.2k25.virtio
  disks:
    - name: win2k19-system
      bus: sata
    - name: dummy-disk-for-virtio
    - name: virtio-drivers
      bus: sata
```

3. Установка драйверов хранилища Virtio

Выполните следующие шаги для установки драйверов Virtio:

1. Смонтируйте `virtio-win.iso` внутри гостевой системы.
2. Запустите мастер установки для установки драйверов.
3. Убедитесь, что установка драйвера прошла успешно:
 1. Откройте Диспетчер устройств (Device Manager).
 2. Вы должны увидеть устройство `SCSI` в списке с иконкой восклицательного знака рядом с ним.
 3. Если драйвер не установлен, щёлкните правой кнопкой мыши по устройству и выберите `Update Driver`.
 4. Выберите `Install the hardware that I manually select from a list`, затем `Show All Devices`, затем `Next`.
 5. Нажмите `Have Disk...`, перейдите к файлу `.inf` и завершите работу мастера.
 4. В качестве альтернативы щёлкните правой кнопкой мыши по файлу `.inf` в Проводнике и выберите `Install`.

4. Переключение на шину Virtio

После установки драйверов вам нужно переключиться на шину `Virtio`. Выполните следующие шаги:

1. Выключите `ВМ`.
2. Отредактируйте `VMInstance` и удалите строку `bus: sata` из системного диска, а также фиктивный диск:

```
``yaml
spec:
disks:
  - name: win2k19-system
```

```
# bus: sata
```

```
#- name: dummy-disk-for-virtio
```

```
'''
```

3. Примените манифест и снова включите ВМ. Windows должна нормально загрузиться с использованием Virtio.

4. Теперь фиктивный диск можно удалить:

```
'''bash
```

```
kubectrl delete vmdisk dummy-disk-for-virtio
```

```
'''
```

Соображения по поводу MTU сети

Cozystack устанавливает размер MTU равным 1400 на каждом vNIC. Windows корректно определяет это только при использовании VirtioNet. С устаревшими сетевыми драйверами вы можете столкнуться с потерей пакетов.

Чтобы заставить Windows учитывать MTU 1400, выполните следующие команды в PowerShell:

```
# List interfaces
Get-NetIPInterface
# Set MTU permanently
Set-NetIPInterface -InterfaceAlias "Ethernet Instance 0" -NlMtuBytes 1400
```

Настоятельно рекомендуется использовать профиль Virtio.

Запуск MikroTik RouterOS в Cozystack

<https://cozystack.ru/docs/v1.5/virtualization/mikrotik/>

Предварительные требования

- ISO-образ MikroTik RouterOS (установочный образ CHR или NPK), например, `mikrotik-7.19.3.iso`.
- Свободный статический IP-адрес или DHCP в подключённой сети тенанта.
- Клиент KubeVirt [`virtctl`](https://kubevirt.io/user-guide/user_workloads/virtctl_client_tool/), установленный в вашей локальной среде и настроенный для пространства имён вашего тенанта.
- Cozystack версии v0.34.2 или новее.

Установка

1. Подготовка дисков

Вам нужно два диска:

1. Установочный ISO – оптический.
2. Системный диск – неоптический.

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: mikrotik-iso
spec:
  source:
    http:
      url: https://download.mikrotik.com/routeros/7.19.3/mikrotik-7.19.3.iso
    optical: true
  storage: 1Gi
  storageClass: replicated
---
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: mikrotik-system
spec:
  optical: false
  storage: 1Gi
  storageClass: replicated
```

2. Создание VMInstance

RouterOS не требует специального профиля экземпляра. Используйте лёгкий профиль Linux, такой как `ubuntu`, с небольшим типом экземпляра, например `u1.medium`:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
  name: mikrotik-demo
spec:
  running: true
  instanceType: "u1.medium"
  instanceProfile: ubuntu
  disks:
  - name: mikrotik-system
    bus: sata
  - name: mikrotik-iso
    bus: sata
```

3. Установка RouterOS

1. Запустите консоль:

```
virtctl vnc mikrotik-demo -n tenant-test
```

2. При запросе выбора пакетов выберите нужный набор (обычно `_system_`, `_routing_`, `_security_`). Подтвердите форматирование системного диска.
3. После завершения установки извлеките установочный ISO.

4. Настройка MTU (необязательно)

Виртуальные сетевые интерфейсы Cozystack по умолчанию используют MTU 1400. RouterOS учитывает это автоматически на адаптерах Virtio?Net, но вы можете проверить или изменить значение:

```
/interface ethernet print detail
/interface ethernet set [ find default-name~"ether1" ] mtu=1400
```

Избегайте устаревших драйверов `e1000/vmxnet`, поскольку они игнорируют значения MTU, отличные от 1500, и могут отбрасывать большие пакеты.

Миграция виртуальных машин из Proxmox

<https://cozystack.ru/docs/v1.5/virtualization/proxmox-migration/>

В этом руководстве описывается процесс миграции виртуальных машин из Proxmox VE в Cozystack путём экспорта образов дисков VM и их загрузки в целевую среду.

Note: Миграция выполняется путём экспорта дисков VM в файлы и их загрузки в Cozystack. Состояние VM и снимки (snapshot) при миграции не сохраняются.

Предварительные требования

Перед началом миграции убедитесь, что у вас есть:

1. **Клиент KubeVirt `virtctl`**, установленный на вашей локальной машине:

- Руководство по установке: [KubeVirt User Guide - Virtctl Client Tool](#)

2. Настроенный доступ к прокси загрузки в вашем кластере Cozystack:

- Примените патч к Platform Package, чтобы открыть `cdi-uploadproxy`:

```
```bash
```

```
kubectrl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
```

```
-p [{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/cdi-uploadproxy", "value": {"type": "ClusterIP"}}]
```

```
```
```

- Настройте конечную точку прокси загрузки CDI, применив патч к Package `kubevirt-cdi`:

```
```bash
```

```
kubectrl patch packages.cozystack.io cozystack.kubevirt-cdi --type=merge -p \
```

```
'{"spec": {"components": {"cdi": {"values": {"config": {"uploadProxyURLOverride": "https://cdi-uploadproxy.example.org"}}}}}]'
```

```
```
```

3. Настройка DNS или файла `hosts` для доступа к прокси загрузки:

- При необходимости добавьте запись в `/etc/hosts` на вашей локальной машине:

```
```text
```

```
1.2.3.4 cdi-uploadproxy.example.org
```

```
```
```

Шаг 1: Экспорт дисков VM из Proxmox

Перед экспортом убедитесь, что виртуальные машины остановлены в Proxmox.

Экспортируйте диск VM в файл в формате `qcow2` (или другом формате, поддерживаемом KubeVirt):

```
# Пример: Экспорт диска VM из хранилища Proxmox
qm disk export <vmid> <disk> /tmp/vm-disk.qcow2
```

Результатом должен стать файл образа диска (например, `vm-disk.qcow2`), готовый к загрузке.

Note: Конкретные команды для экспорта дисков могут различаться в зависимости от бэкенда хранилища Proxmox и его конфигурации. Подробнее см. в [документации Proxmox VE](#).

Шаг 2: Создание VMDisk для загрузки

Создайте ресурс `VMDisk` в Cozystack с `source.upload`, чтобы подготовиться к загрузке образа:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: proxmox-vm-disk
  namespace: tenant-root
spec:
  source:
    upload: {}
  storage: 10Gi
  storageClass: replicated
```

Примените манифест:

```
kubectl apply -f vmdisk.yaml
```

Отслеживайте статус создания диска:

```
kubectl get vmdisk -n tenant-root
```

Шаг 3: Загрузка образа диска

Как только `VMDisk` создан и готов к загрузке, используйте `virtctl` для загрузки образа диска:

```
virtctl image-upload dv vm-disk-proxmox-vm-disk \
  -n tenant-root \
  --image-path=./vm-disk.qcow2 \
  --uploadproxy-url https://cdi-uploadproxy.example.org
```

Note: Имя `DataVolume` следует шаблону `vm-disk-<vmdisk-name>`. Если ваш `VMDisk` называется `proxmox-vm-disk`, то `DataVolume` будет `vm-disk-proxmox-vm-disk`.

Дождитесь завершения загрузки. Вы можете отслеживать прогресс:

```
kubectl get dv -n tenant-root
kubectl describe dv vm-disk-proxmox-vm-disk -n tenant-root
```

Загрузка завершена, когда статус показывает Succeeded.

Шаг 4: Создание VMInstance

После завершения загрузки диска создайте VMInstance для загрузки с загруженного диска:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
  name: migrated-vm
  namespace: tenant-root
spec:
  running: true
  instanceType: ul.medium
  disks:
    - name: proxmox-vm-disk
  # Optional: configure network, cloud-init, etc.
```

Примените манифест:

```
kubectl apply -f vminstance.yaml
```

Убедитесь, что VM запущена:

```
kubectl get vminstance -n tenant-root
```

Шаг 5: Доступ к мигрированной VM

Получите доступ к консоли VM с помощью virtctl:

```
# Serial console
virtctl console migrated-vm -n tenant-root

# VNC access
virtctl vnc migrated-vm -n tenant-root

# SSH (if configured)
ssh user@vm-ip
```

Контрольный список миграции

Используйте этот контрольный список для отслеживания прогресса миграции:

- [] Экспортировать диски VM из Proxmox (в формате qcow2 или совместимом)
- [] Установить `virtctl` на вашей локальной машине
- [] Настроить доступ к прокси загрузки в Cozystack
- [] Добавить запись DNS/hosts для прокси загрузки (при необходимости)
- [] Создать VMDisk с `source.upload` в Cozystack

-
- [] Загрузить образ диска с помощью `virtctl image-upload`
 - [] Дождаться завершения загрузки (статус: Succeeded)
 - [] Создать VMInstance с загруженным диском
 - [] Убедиться, что VM успешно загружается
 - [] Проверить сетевую связность и работоспособность VM

Устранение неполадок

Загрузка завершается с ошибкой подключения

Проблема:

`virtctl image-upload` завершается с ошибкой `connection refused` или тайм-аутом.

Решение:

- Убедитесь, что прокси загрузки доступен: `curl -k https://cdi-uploadproxy.example.org`
- Проверьте, что запись в `/etc/hosts` соответствует IP-адресу прокси загрузки
- Убедитесь, что Platform Package включает `cdi-uploadproxy` в `publishing.exposedServices`

Загрузка застряла на 0%

Проблема:

Загрузка начинается, но не продвигается.

Решение:

- Проверьте статус DataVolume: `kubectl describe dv vm-disk-<name> -n tenant-root`
- Убедитесь, что у класса хранилища есть доступная ёмкость
- Проверьте логи пода CDI: `kubectl logs -n cozy-system -l app=cdi-uploadproxy`

ВМ не загружается после миграции

Проблема:

ВМ загружается, но не запускается корректно.

Решение:

- Проверьте, что диск ВМ подключён как первый диск в `spec VMInstance`
- Убедитесь, что формат диска совместим (`qcow2`, `raw`)
- Просмотрите логи ВМ: `virtctl console vm-instance-<name> -n tenant-root`
- Убедитесь, что драйверы ВМ совместимы с KubeVirt (рекомендуется VirtIO)

Дальнейшие шаги

После успешной миграции:

-
- Настройте [cloud-init](#) для автоматизированной настройки VM
 - Изучите [типы инстансов и профили](#) для оптимального распределения ресурсов
 - Рассмотрите возможность создания [золотых образов](#) для будущих развёртываний VM
-

Ресурсы виртуальных машин

<https://cozystack.ru/docs/v1.5/virtualization/resources/>

У каждой виртуальной машины есть две следующие настройки конфигурации: ([Cozystack Documentation](#))

- `instanceType` определяет ресурсы, предоставляемые виртуальной машине.
- `instanceProfile` определяет набор предпочтений для виртуальных машин в соответствии с используемой ОС.

Ресурсы типа инстанса

Справочная таблица

Следующие ресурсы `instancetype` предоставляются Cozystack:

| Имя | vCPUs | память |
|-------------|-------|--------|
| cx1.2xlarge | 8 | 16Gi |
| cx1.4xlarge | 16 | 32Gi |
| cx1.8xlarge | 32 | 64Gi |
| cx1.large | 2 | 4Gi |
| cx1.medium | 1 | 2Gi |
| cx1.xlarge | 4 | 8Gi |
| gn1.2xlarge | 8 | 32Gi |
| gn1.4xlarge | 16 | 64Gi |
| gn1.8xlarge | 32 | 128Gi |
| gn1.xlarge | 4 | 16Gi |
| m1.2xlarge | 8 | 64Gi |
| m1.4xlarge | 16 | 128Gi |
| m1.8xlarge | 32 | 256Gi |
| m1.large | 2 | 16Gi |
| m1.xlarge | 4 | 32Gi |
| n1.2xlarge | 16 | 32Gi |
| n1.4xlarge | 32 | 64Gi |

| | | |
|-------------|----|-------|
| n1.8xlarge | 64 | 128Gi |
| n1.large | 4 | 8Gi |
| n1.medium | 4 | 4Gi |
| n1.xlarge | 8 | 16Gi |
| o1.2xlarge | 8 | 32Gi |
| o1.4xlarge | 16 | 64Gi |
| o1.8xlarge | 32 | 128Gi |
| o1.large | 2 | 8Gi |
| o1.medium | 1 | 4Gi |
| o1.micro | 1 | 1Gi |
| o1.nano | 1 | 512Mi |
| o1.small | 1 | 2Gi |
| o1.xlarge | 4 | 16Gi |
| rt1.2xlarge | 8 | 32Gi |
| rt1.4xlarge | 16 | 64Gi |
| rt1.8xlarge | 32 | 128Gi |
| rt1.large | 2 | 8Gi |
| rt1.medium | 1 | 4Gi |
| rt1.micro | 1 | 1Gi |
| rt1.small | 1 | 2Gi |
| rt1.xlarge | 4 | 16Gi |
| u1.2xlarge | 8 | 32Gi |
| u1.2xmedium | 2 | 4Gi |
| u1.4xlarge | 16 | 64Gi |
| u1.8xlarge | 32 | 128Gi |
| u1.large | 2 | 8Gi |
| u1.medium | 1 | 4Gi |
| u1.micro | 1 | 1Gi |
| u1.nano | 1 | 512Mi |
| u1.small | 1 | 2Gi |

| | | |
|-----------|---|------|
| u1.xlarge | 4 | 16Gi |
|-----------|---|------|

Серия U

Серия U достаточно нейтральна и предоставляет ресурсы для приложений общего назначения. ([Cozystack Documentation](#))

U — это сокращение от «Universal» (универсальный), намекающее на универсальное отношение к рабочим нагрузкам. VM этих типов инстансов будут совместно использовать физические ядра CPU с другими VM на основе разделения по времени.

Особые характеристики этой серии:

- Burstable-производительность CPU — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4 для меньшего уровня помех на узел.

Серия O

Серия O основана на серии U, единственное отличие заключается в том, что память переподписана (overcommitted). ([Cozystack Documentation](#))

O — это сокращение от «Overcommitted» (переподписанный).

Особые характеристики этой серии:

- Burstable-производительность CPU — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- Переподписанная память — память переподписана для достижения более высокой плотности рабочих нагрузок.
- Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4 для меньшего уровня помех на узел.

Серия CX

Серия CX предоставляет эксклюзивные вычислительные ресурсы для вычислительно-интенсивных приложений. ([Cozystack Documentation](#))

CX — это сокращение от «Compute Exclusive» (эксклюзивные вычисления). Эксклюзивные ресурсы предоставляются вычислительным потокам VM. Чтобы это обеспечить, будут запрошены некоторые дополнительные ядра (в зависимости от количества дисков и NIC) для разгрузки потоков ввода-вывода с ядер, выделенных под рабочую нагрузку. Кроме того, в этой серии топология NUMA используемых ядер предоставляется VM.

Особые характеристики этой серии:

-
- Hugepages — Hugepages используются для повышения производительности памяти.
 - Выделенный CPU — физические ядра эксклюзивно назначаются каждому vCPU, чтобы обеспечить фиксированные и высокие вычислительные гарантии для рабочей нагрузки.
 - Изолированные потоки эмулятора — потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.
 - vNUMA — физическая топология NUMA отражается в гостевой системе для оптимизации использования кэша на стороне гостя.
 - Соотношение vCPU к памяти (1:2) — соотношение vCPU к памяти 1:2.

Серия M

Серия M предоставляет ресурсы для приложений, интенсивно использующих память. ([Cozystack Documentation](#))

M — это сокращение от «Memory» (память).

Особые характеристики этой серии:

- Hugepages — Hugepages используются для повышения производительности памяти.
- Burstable-производительность CPU — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- Соотношение vCPU к памяти (1:8) — соотношение vCPU к памяти 1:8 для гораздо меньшего уровня помех на узел.

Серия RT

Серия RT предоставляет ресурсы для приложений реального времени, таких как Oslat. ([Cozystack Documentation](#))

RT — это сокращение от «realtime» (реальное время). Этой серии типов инстансов требуются узлы, способные выполнять приложения реального времени.

Особые характеристики этой серии:

- Hugepages — Hugepages используются для повышения производительности памяти.
- Выделенный CPU — физические ядра эксклюзивно назначаются каждому vCPU, чтобы обеспечить фиксированные и высокие вычислительные гарантии для рабочей нагрузки.
- Изолированные потоки эмулятора — потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.
- Соотношение vCPU к памяти (1:4) — соотношение vCPU к памяти 1:4, начиная с размера medium.

Ресурсы профиля инстанса

Следующие ресурсы предпочтений предоставляются Cozystack: ([Cozystack Documentation](#))

| Имя | Гостевая ОС |
|-------------------------|---|
| alpine | Alpine |
| centos.7 | CentOS 7 |
| centos.7.desktop | CentOS 7 |
| centos.stream10 | CentOS Stream 10 |
| centos.stream10.desktop | CentOS Stream 10 |
| centos.stream8 | CentOS Stream 8 |
| centos.stream8.desktop | CentOS Stream 8 |
| centos.stream8.dpdk | CentOS Stream 8 |
| centos.stream9 | CentOS Stream 9 |
| centos.stream9.desktop | CentOS Stream 9 |
| centos.stream9.dpdk | CentOS Stream 9 |
| cirros | Cirros |
| fedora | Fedora (amd64) |
| fedora.arm64 | Fedora (arm64) |
| opensuse.leap | OpenSUSE Leap |
| opensuse.tumbleweed | OpenSUSE Tumbleweed |
| rhel.10 | Red Hat Enterprise Linux 10 Beta (amd64) |
| rhel.10.arm64 | Red Hat Enterprise Linux 10 Beta (arm64) |
| rhel.7 | Red Hat Enterprise Linux 7 |
| rhel.7.desktop | Red Hat Enterprise Linux 7 |
| rhel.8 | Red Hat Enterprise Linux 8 |
| rhel.8.desktop | Red Hat Enterprise Linux 8 |
| rhel.8.dpdk | Red Hat Enterprise Linux 8 |
| rhel.9 | Red Hat Enterprise Linux 9 (amd64) |
| rhel.9.arm64 | Red Hat Enterprise Linux 9 (arm64) |
| rhel.9.desktop | Red Hat Enterprise Linux 9 Desktop (amd64) |
| rhel.9.dpdk | Red Hat Enterprise Linux 9 DPDK (amd64) |
| rhel.9.realtime | Red Hat Enterprise Linux 9 Realtime (amd64) |
| sles | SUSE Linux Enterprise Server |

| | |
|---------------------|--|
| ubuntu | Ubuntu |
| windows.10 | Microsoft Windows 10 |
| windows.10.virtio | Microsoft Windows 10 (virtio) |
| windows.11 | Microsoft Windows 11 |
| windows.11.virtio | Microsoft Windows 11 (virtio) |
| windows.2k16 | Microsoft Windows Server 2016 |
| windows.2k16.virtio | Microsoft Windows Server 2016 (virtio) |
| windows.2k19 | Microsoft Windows Server 2019 |
| windows.2k19.virtio | Microsoft Windows Server 2019 (virtio) |
| windows.2k22 | Microsoft Windows Server 2022 |
| windows.2k22.virtio | Microsoft Windows Server 2022 (virtio) |
| windows.2k25 | Microsoft Windows Server 2025 |
| windows.2k25.virtio | Microsoft Windows Server 2025 (virtio) |

Разработка типов и профилей инстансов

Чтобы начать настройку или создание собственных instancetypes и предпочтений, см.

[Руководство разработчика. \(Cozystack Documentation\)](#)

Хранилище

Руководства по подсистеме хранения

<https://cozystack.ru/docs/v1.5/storage/>

Эти руководства покажут, как выполнять типовые задачи, связанные с системой хранения LINSTOR в Cozystack.

Подготовка дисков для пулов хранения LINSTOR

Как очистить метаданные дисков и подготовить физическое хранилище для LINSTOR

Настройка выделенной сети для LINSTOR

Перенаправление трафика репликации LINSTOR на выделенный сетевой интерфейс для повышения надёжности и производительности.

Настройка контроллера ресинхронизации DRBD в LINSTOR

Узнайте, как настроить параметры контроллера ресинхронизации DRBD в LINSTOR для ускорения синхронизации

Использование NFS-ресурсов в Cozystack

Настройка дополнительного модуля `nfs-driver` для заказа томов из NFS-ресурсов в Cozystack

LINSTOR GUI

Включение и доступ к дополнительной веб-консоли LINSTOR для управления узлами хранения, ресурсами и томами.

Создание зашифрованного хранилища на LINSTOR

Узнайте, как настроить и использовать шифрование постоянных томов `at-rest` в LINSTOR

Подготовка дисков для пулов хранения LINSTOR

<https://cozystack.ru/docs/v1.5/storage/disk-preparation/>

В этом руководстве описано, как подготовить физические диски к использованию с LINSTOR, если на них остались старые метаданные, мешающие автоматическому обнаружению.

Описание проблемы

При настройке хранилища на новых или повторно используемых узлах на физических дисках могут оставаться следы предыдущих установок:

- суперблоки RAID
- таблицы разделов
- сигнатуры LVM
- метаданные файловых систем

Эти старые метаданные не позволяют LINSTOR определить диски как доступное хранилище.

Симптомы

1. `linstor physical-storage list` выводит пустой список или в нём отсутствуют диски
2. Диски отображаются с неожиданными типами файловых систем (например, `linux_raid_member`)
3. В пулах хранения виден только `DfltDisklessStorPool` без фактического хранилища

Диагностика

Настройка псевдонима для LINSTOR

Для удобного доступа к командам LINSTOR настройте псевдоним:

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

Проверка узлов LINSTOR

Выведите список узлов и проверьте их готовность:


```
linstor node list
```

В ожидаемом выводе все узлы должны находиться в состоянии `Online`.

Проверка пулов хранения

Проверьте текущие пулы хранения:

```
linstor storage-pool list
```

Проверка доступного физического хранилища

Проверьте, какие физические диски видит LINSTOR:

```
linstor physical-storage list
```

Если команда выводит пустой список или в нём нет ожидаемых дисков, скорее всего, на дисках остались старые метаданные и их нужно очистить.

Проверка состояния дисков на узле

Проверьте состояние дисков на конкретном узле через под `satellite`:

```
# Вывести список подов LINSTOR satellite
kubectl get pod -n cozy-linstor -l app.kubernetes.io/component=linstor-satellite

# Проверить состояние дисков
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \
lsblk -f
```

В ожидаемом выводе у чистых дисков поле `FSTYPE` должно быть пустым:

```
NAME      FSTYPE LABEL UUID MOUNTPOINT
nvme0n1
nvme1n1
```

Если вы видите `linux_raid_member`, `LVM2_member` или другие типы файловых систем, диски нужно очистить.

Решение: очистка метаданных дисков

ВНИМАНИЕ: очистка дисков уничтожает все данные на указанных устройствах. Очищайте только те диски, которые НЕ используются для операционной системы или установки Talos.

Шаг 1: определите системные диски

Перед очисткой определите, на каком диске установлен Talos. Проверьте конфигурацию Talos в файле `nodes/<node-name>.yaml`:

```
# In nodes/<node-name>.yaml
machine:
  install:
    disk: /dev/sda # This disk should NOT be wiped
```

Как правило, системный диск — это `/dev/sda`, `/dev/vda` или похожее устройство.

Шаг 2: найдите конфигурацию узла

Если для развёртывания кластера вы использовали [Talm](#), конфигурации узлов хранятся в файлах `nodes/*.yaml` в каталоге конфигурации кластера.

Каждый файл соответствует конкретному узлу (например, `nodes/node1.yaml`, `nodes/node2.yaml`).

Шаг 3: очистите диски

Выведите список всех дисков на узле:

```
talm -f nodes/<node-name>.yaml get disks
```

Очистите все несистемные диски:

```
talm -f nodes/<node-name>.yaml wipe disk nvme0n1 nvme1n1 nvme2n1 ...
```

Note: Перечислите все диски, которые нужно очистить, в одной команде. НЕ указывайте системный диск (например, `sda`, если на нём установлен Talos).

Шаг 4: убедитесь, что диски чистые

После очистки убедитесь, что диски стали видны LINSTOR:

```
linstor physical-storage list
```

Теперь в ожидаемом выводе должны появиться ваши диски:

```
+-----+
| Device      | Size      | Rotational |
|=====|
/dev/nvme0n1	3.49 TiB	False
/dev/nvme1n1	3.49 TiB	False
...	...	...
+-----+
```

Также можно проверить непосредственно на узле:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \
lsblk -f
```

У чистых дисков поле `FSTYPE` должно быть пустым.

Создание пулов хранения

Когда диски очищены, создайте пул хранения LINSTOR.

Для пулов хранения ZFS из нескольких дисков:

```
linstor physical-storage create-device-pool zfs <node-name> \
/dev/nvme0n1 /dev/nvme1n1 /dev/nvme2n1 ... \
--pool-name data \
--storage-pool data
```

Note: Укажите все диски в одной команде, чтобы создать единый пул ZFS. Повторный запуск команды с тем же именем пула завершится ошибкой.

Убедитесь, что пул хранения создан:

```
linstor storage-pool list
```

Ожидаемый вывод:

```
+-----+
| StoragePool | Node  | Driver | PoolName | FreeCapacity | TotalCapacity | State |
|=====|
| data        | node1 | ZFS    | data     | 47.34 TiB    | 47.62 TiB     | Ok    |
| data        | node2 | ZFS    | data     | 47.34 TiB    | 47.62 TiB     | Ok    |
+-----+
```

Устранение неполадок

После очистки на дисках по-прежнему видны старые метаданные

Попробуйте очистку методом ZEROES для более тщательной обработки:

```
taln -f nodes/<node-name>.yaml wipe disk --method ZEROES nvme0n1
```

Этот метод записывает на диск нули: он занимает больше времени, но гарантирует полное удаление метаданных.

Ошибка «Zpool name already used»

Если нужно пересоздать пул хранения:

1. Удалите его из LINSTOR:

```
linstor storage-pool delete <node-name> <pool-name>
```

2. Уничтожьте пул ZFS на узле:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \
  zpool destroy <pool-name>
```

3. Пересоздайте пул, указав все диски в одной команде.

Отказ в доступе на рабочих узлах

Рабочие узлы могут не разрешать прямой доступ к API Talos. Используйте под satellite для проверки состояния дисков:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- lsblk -f
```

Если требуется очистить диски на рабочих узлах, убедитесь, что конфигурация узла разрешает доступ, или обратитесь к администратору кластера.

Краткая справка

| Команда | Описание |
|---|---|
| <code>linstor sp l</code> | Вывести список пулов хранения |
| <code>linstor ps l</code> | Вывести список доступного физического хранилища |
| <code>linstor ps cdp zfs <node> <disks> --pool-name <name> --storage-pool <name></code> | Создать пул хранения ZFS |
| <code>taln -f nodes/<node>.yaml wipe disk <disks></code> | Очистить метаданные дисков |
| <code>taln -f nodes/<node>.yaml get disks</code> | Вывести список дисков на узле |

Связанная документация

- [Развёртывание Cozystack с помощью Talm](#)
 - [Настройка выделенной сети для LINSTOR](#)
 - [Настройка контроллера ресинхронизации DRBD](#)
 - [Устранение неполадок LINSTOR](#)
-

Настройка выделенной сети для LINSTOR

<https://cozystack.ru/docs/v1.5/storage/dedicated-network/>

В этом руководстве описано, как повысить надёжность и производительность хранилища, перенаправив трафик репликации LINSTOR на выделенный сетевой интерфейс.

Введение

Платформа Cozystack создана для поддержки высокодоступных (HA) рабочих нагрузок, а значит, система должна продолжать работать, даже если один или несколько узлов выйдут из строя. Kubernetes хорошо справляется с этим для нагрузок без сохранения состояния (stateless). Однако нагрузкам с сохранением состояния (stateful), таким как файловые хранилища или диски виртуальных машин, требуется надёжная система хранения.

Когда вы выбираете класс хранения `replicated` для PersistentVolumeClaim (PVC) или DataVolume, LINSTOR создаёт несколько синхронных реплик данных на разных узлах. Благодаря этому при отказе узла (из-за отключения питания, проблем с оборудованием и т.п.) данные мгновенно остаются доступными на другом узле.

За такой уровень надёжности приходится платить: репликация хранилища может создавать значительный сетевой трафик в зависимости от интенсивности рабочей нагрузки.

Если у узлов несколько сетевых интерфейсов, производительность можно повысить, выделив один из них под трафик хранилища. Это изолирует трафик репликации от остальных нагрузок и предотвращает возможные узкие места в сети.

Когда нужна выделенная сеть для хранилища?

Не всегда необходимо настраивать выделенную сеть для трафика хранилища с самого начала. В небольших кластерах и кластерах для проверки концепции (PoC) стандартной конфигурации сети, как правило, достаточно. Преждевременная оптимизация может привести к неоправданному усложнению.

Однако в более крупных кластерах трафик репликации хранилища часто становится узким местом производительности. Он может исчерпать доступную пропускную способность и мешать другим нагрузкам, использующим ту же сеть.

Если у узлов только один сетевой интерфейс и вы не уверены, нужен ли выделенный интерфейс, начните с наблюдения за фактическим трафиком хранилища под реалистичной нагрузкой. Один из практических подходов - выделить VLAN на существующем интерфейсе под трафик репликации хранилища. Это позволит отслеживать использование пропускной способности без добавления оборудования.

Если уровень трафика остаётся стабильно высоким или влияет на производительность приложений, переход на выделенную сеть становится оправданной оптимизацией.

Термины и определения

Прежде чем продолжить, полезно уточнить терминологию, используемую для сетевых интерфейсов в конфигурации на основе LINSTOR:

- Маршрут по умолчанию узла Kubernetes

Это маршрут по умолчанию, используемый для исходящего трафика узла. Его можно посмотреть командой `ip route show default`.

- IP-адрес источника по умолчанию узла Kubernetes

Это IP-адрес, связанный с маршрутом по умолчанию. Обычно это адрес источника для всего исходящего трафика; его можно проверить той же командой.

- Внутренний IP-адрес узла Kubernetes

Это адрес, используемый Kubernetes для внутреннего взаимодействия. Часто он совпадает с адресом источника по умолчанию, но не всегда. Его можно проверить командой `kubectl get nodes -o wide`.

- Интерфейс по умолчанию LINSTOR satellite

Здесь конфигурация становится менее очевидной. Когда оператор Piraeus запускает LINSTOR Satellite, он задаёт один или два интерфейса по умолчанию: `default-ipv4` и `default-ipv6`. Как правило, доступен только IPv4. Интерфейс по умолчанию устанавливается в IP-адрес источника по умолчанию узла при запуске и не меняется до перезапуска Satellite. Этот интерфейс можно изменить во время работы Satellite, но оператор Piraeus сбросит его при следующем перезапуске.

- Дополнительные интерфейсы LINSTOR satellite

К LINSTOR Satellite можно вручную добавить дополнительные интерфейсы. Они хранятся в базе данных контроллера LINSTOR и сохраняются между перезапусками. Оператор Piraeus не изменяет и не удаляет их. На момент написания оператор Piraeus не поддерживает объявление дополнительных интерфейсов через пользовательские ресурсы Kubernetes.

- Активное подключение LINSTOR satellite

Контроллер LINSTOR использует ровно один интерфейс для взаимодействия с каждым Satellite. По умолчанию это интерфейс `default-ipv4`. Хотя CLI LINSTOR позволяет его изменить, оператор Piraeus вернёт его обратно при следующем перезапуске Satellite.

- Соединение узлов LINSTOR (путь)

Оно определяет, как LINSTOR Satellite взаимодействуют друг с другом при синхронной репликации. Именно эту настройку мы будем выполнять в данном руководстве. Соединения узлов можно определять либо через CRD Piraeus, либо командами CLI. Оператор Piraeus не управляет ими и не переопределяет их. При этом приоритет имеет конфигурация, применённая последней.

Как назначаются IP-адреса по умолчанию

Вот типичный вывод `node list` в LINSTOR:

```
INSTOR ==> node list
```

| +-----+ | | | | |
|---------|-----------|-------------------------|--------|--|
| Node | NodeType | Addresses | State | |
| +-----+ | | | | |
| node01 | SATELLITE | 10.78.22.146:3367 (SSL) | Online | |
| node02 | SATELLITE | 10.78.22.147:3367 (SSL) | Online | |
| node03 | SATELLITE | 10.78.22.148:3367 (SSL) | Online | |
| node04 | SATELLITE | 10.78.22.149:3367 (SSL) | Online | |
| node05 | SATELLITE | 10.78.22.150:3367 (SSL) | Online | |
| +-----+ | | | | |

Эти IP-адреса обычно совпадают с внутренними IP-адресами узлов Kubernetes и адресами маршрута по умолчанию. По умолчанию именно они используются для трафика репликации хранилища.

Может показаться, что достаточно изменить их, чтобы перенаправить трафик хранилища на другой интерфейс. Однако это не сработает.

Оператор Piraeus получает текущий IP-адрес пода Satellite при каждом перезапуске. Затем он использует этот IP - как правило, внутренний адрес узла Kubernetes - для настройки интерфейса INSTOR по умолчанию.

Даже если попытаться изменить внутренний IP-адрес Kubernetes, трафик репликации хранилища не перейдёт на другой интерфейс. Более того, такое изменение затронет весь трафик узла, а не только репликацию INSTOR.

Чтобы корректно изолировать трафик хранилища от остальных нагрузок, используйте соединения узлов, как описано в следующем разделе.

Интерфейсы Satellite

По умолчанию INSTOR настраивает только один интерфейс на каждый satellite: `default-ipv4`, полученный из IP-адреса по умолчанию узла Kubernetes. Вот пример того, как выглядит список интерфейсов по умолчанию INSTOR satellite:

```
INSTOR ==> node interface list node01
```

| +-----+ | | | | |
|-----------|--------------|--------------|------|----------------|
| node01 | NetInterface | IP | Port | EncryptionType |
| +-----+ | | | | |
| + StltCon | default-ipv4 | 10.78.22.146 | 3367 | SSL |
| +-----+ | | | | |

Этот IP берётся с узла Kubernetes при запуске Satellite. Если на узле есть дополнительные интерфейсы, INSTOR о них не знает.

Чтобы разрешить трафик хранилища через другую сеть, можно вручную добавить второй интерфейс:


```
LINSTOR ==> node interface create node01 optic-san 10.78.24.201
SUCCESS:
Description:
  New netInterface 'optic-san' on node 'node01' registered.
Details:
  NetInterface 'optic-san' on node 'node01' UUID is: aa28f583-ca91-4cac-b749-67fe54e72c10
```

```
LINSTOR ==> node interface list node01
+-----+
| node01 | NetInterface | IP           | Port | EncryptionType |
+-----+
| + StltCon | default-ipv4 | 10.78.22.146 | 3367 | SSL             |
| +         | optic-san    | 10.78.24.201 |      |                 |
+-----+
```

Знать имя базового сетевого интерфейса Linux не обязательно. Можно использовать любое имя - LINSTOR не проверяет, действительно ли этот IP назначен узлу.

Флагом `StltCon` помечен только интерфейс `default-ipv4`. Это означает, что именно его контроллер LINSTOR использует для взаимодействия с Satellite.

Хотя активное подключение Satellite можно изменить вручную через CLI LINSTOR, оператор Piraeus сбросит его при следующем перезапуске. Это поведение относится только к установкам вне Kubernetes.

Чтобы использовать новый интерфейс для репликации хранилища, нужно определить соединения узлов. Но сначала повторите создание интерфейса на остальных узлах:

```
LINSTOR ==> node interface create node02 optic-san 10.78.24.202
LINSTOR ==> node interface create node03 optic-san 10.78.24.203
LINSTOR ==> node interface create node04 optic-san 10.78.24.204
LINSTOR ==> node interface create node05 optic-san 10.78.24.205
```

Соединения узлов

Соединения узлов LINSTOR определяют, как satellite взаимодействуют друг с другом при синхронной репликации данных. Для каждой пары satellite должен быть определён путь соединения.

В отличие от интерфейсов satellite, соединения узлов можно определять как вручную через CLI, так и декларативно через пользовательские ресурсы Kubernetes. Для небольших кластеров создание соединений вручную обычно не вызывает трудностей. Однако по мере роста кластера эффективнее использовать соглашение об именовании и описать все пути соединений в одном определении ресурса.

В следующих разделах рассматриваются оба подхода.

Ручной способ

Соединения узлов можно создать вручную с помощью CLI LINSTOR. Вот как посмотреть синтаксис команды:

```
LINSTOR ==> node-connection path create -h
usage: linstor node-connection path create [-h]
                                         node_a node_b path_name
                                         netinterface_a netinterface_b

Creates a new node connection path.

positional arguments:
  node_a              1. Node of the connection
  node_b              2. Node of the connection
  path_name           Name of the created path
  netinterface_a      NetInterface name to use for 1. node
  netinterface_b      NetInterface name to use for the 2. node
```

Пример: создание соединения между `node01` и `node02` с использованием интерфейса `optic-san` с обеих сторон:

```
LINSTOR ==> node-connection path create node01 node02 optic-san optic-san optic-san
SUCCESS:
Description:
  Node connection between nodes 'node01' and 'node02' modified.
Details:
  Node connection between nodes 'node01' and 'node02' UUID is: c1f4ee6a-776e-46ba-9e74-9b2f6385d568
SUCCESS:
  (node01) Node changes applied.
SUCCESS:
  (node02) Node changes applied.
SUCCESS:
  (node02) Resource 'pvc-6f535d3a-82c1-46ab-80fe-5a59ee8bffa4' [DRBD] adjusted.
....
```

После создания соединения LINSTOR сразу обновляет все затронутые ресурсы DRBD, чтобы они использовали новый путь.

Путь создаётся один раз для каждой пары узлов. Определять отдельный обратный путь не нужно.

Проверить конфигурацию можно так:

```
LINSTOR ==> node-connection path list node01 node02
```

```
+-----+
| Key                | Value          |
+-----+-----+
| Paths/node01-02/node01 | optic-san      |
| Paths/node01-02/node02 | optic-san      |
+-----+-----+
```

```
LINSTOR ==> node-connection path list node02 node01
```

```
+-----+
| Key                | Value          |
+-----+-----+
| Paths/node01-02/node01 | optic-san      |
| Paths/node01-02/node02 | optic-san      |
+-----+-----+
```

```
LINSTOR ==> node-connection list node01 node02
```

```
+-----+
| Node A | Node B | Properties                               |
+-----+-----+
| node01 | node02 | node01=optic-san,node02=optic-san         |
+-----+-----+
```

Способ с CRD

Рассмотрим способ с использованием Kubernetes CRD (Custom Resource Definition).

Внимание

На момент написания у оператора Piraeus есть известная проблема с обработкой отсутствующих интерфейсов. Между попытками согласования нет задержки (backoff). Если отсутствует хотя бы один интерфейс, контроллер может потреблять 100% своего лимита CPU.

>

Всегда проверяйте, что все необходимые интерфейсы существуют, прежде чем применять CRD. Следите за подом контроллера, чтобы убедиться, что после изменения он не перегружен.

С увеличением числа узлов количество необходимых соединений быстро растёт. Для 3 узлов нужно 3 соединения. Для 5 узлов - 10. Для 10 узлов - 45, и так далее.

Вместо создания каждого соединения вручную можно определить все пути сразу с помощью одного пользовательского ресурса `LinstorNodeConnection`.

Для использования этого способа должны выполняться следующие условия:

- На всех участвующих узлах интерфейс должен называться одинаково.
- Интерфейс должен быть создан на всех узлах до применения CRD.

Пример пользовательского ресурса `LinstorNodeConnection`:

```

apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
  name: dedicated
spec:
  paths:
    - name: dedicated
      interface: optic-san

```

Примените CRD командой:

```
kubectl apply -f linstor-node-connections.yaml
```

После применения можно проверить соединения:

```

LINSTOR ==> node-connection list
+-----+
| Node A | Node B | Properties |
+-----+
node01	node02	last-applied=["Paths/dedicated/optic-san"],node01=optic-san...
node01	node03	last-applied=["Paths/dedicated/optic-san"],node01=optic-san...
node01	node04	last-applied=["Paths/dedicated/optic-san"],node01=optic-san...
node01	node05	last-applied=["Paths/dedicated/optic-san"],node01=optic-san...
node02	node03	last-applied=["Paths/dedicated/optic-san"],node02=optic-san...
node02	node04	last-applied=["Paths/dedicated/optic-san"],node02=optic-san...
node02	node05	last-applied=["Paths/dedicated/optic-san"],node02=optic-san...
node03	node04	last-applied=["Paths/dedicated/optic-san"],node03=optic-san...
node03	node05	last-applied=["Paths/dedicated/optic-san"],node03=optic-san...
node04	node05	last-applied=["Paths/dedicated/optic-san"],node04=optic-san...
+-----+

```

Старые пути, созданные вручную, могут по-прежнему отображаться. Приоритет имеет соединение, применённое последним.

В этом примере видны старые пути для `node01` и `node02`:

```

LINSTOR ==> node-connection path list node01 node03
+-----+
| Key | Value |
+-----+
| Paths/dedicated/node01 | optic-san |
| Paths/dedicated/node03 | optic-san |
+-----+

```

Чтобы конфигурация оставалась аккуратной, старый ручной путь можно удалить:

```

LINSTOR ==> linstor node-connection path delete node01 node02 node01-02
SUCCESS:
  Successfully deleted property key(s): Paths/node01-02/node02,Paths/node01-02/node01
....

```

После удаления:

```
LINSTOR ==> node-connection path list node01 node02
```

```
+-----+
| Key                | Value      |
+-----+-----+
| Paths/dedicated/node01 | optic-san  |
| Paths/dedicated/node02 | optic-san  |
+-----+-----+
```

Расширенный способ с CRD

См. пример в руководстве [Выделенная сеть хранилища для нескольких дата-центров](#)

Настройка контроллера ресинхронизации DRBD в LINSTOR

<https://cozystack.ru/docs/v1.5/storage/drbd-tuning/>

Администраторы Cozystack могут регулировать производительность синхронизации DRBD, задавая параметры настройки для контроллера LINSTOR.

Это позволяет оптимизировать скорость ресинхронизации, не перегружая сеть репликации и систему хранения.

Подробное описание всех доступных параметров и рекомендации по настройке приведены в официальном руководстве LINBIT: [Tuning the DRBD Resync Controller](#).

Для конфигурации с несколькими дата-центрами также прочитайте руководство [Настройка DRBD для нескольких дата-центров](#).

Рекомендуемые настройки для сетей 10G

Мы считаем следующие значения оптимальными для кластеров, соединённых 10-гигабитной сетью:

```
linstor controller set-property DrbdOptions/Net/max-buffers 36864
linstor controller set-property DrbdOptions/Net/rcvbuf-size 10485760
linstor controller set-property DrbdOptions/Net/sndbuf-size 10485760
linstor controller set-property DrbdOptions/PeerDevice/c-fill-target 2048
linstor controller set-property DrbdOptions/PeerDevice/c-max-rate 737280
linstor controller set-property DrbdOptions/PeerDevice/c-min-rate 245760
linstor controller set-property DrbdOptions/PeerDevice/resync-rate 245760
linstor controller set-property DrbdOptions/PeerDevice/c-plan-ahead 10
```

- `c-max-rate` указывается в КиБ/с и должен соответствовать максимальной устойчивой пропускной способности дисков или сети (в зависимости от того, что меньше). Значение 737280 в примере соответствует 720 МиБ/с.
- `c-min-rate` и `resync-rate` также указываются в КиБ/с; их следует устанавливать примерно в одну треть от `c-max-rate`.

Использование NFS-ресурсов в Cozystack

<https://cozystack.ru/docs/v1.5/storage/nfs/>

Включение драйвера NFS

Добавьте `cozystack.nfs-driver` в `bundles.enabledPackages` в **Platform Package**:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozyst...']
```

Подождите около минуты, пока чарт платформы выполнит согласование, затем убедитесь, что `HelmRelease` создан:

```
kubectl get helmrelease --namespace cozy-nfs-driver nfs-driver
```

Экспорт общего ресурса

```
apt install nfs-server
mkdir /data
chmod 777 /data
echo '/data *(rw,sync,no_subtree_check)' >> /etc/exports
exportfs -a
```

Настройка подключения

```
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.244.57.210
  share: /data
reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: true
mountOptions:
  - nfsvers=4.1
```

Заказ тома

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: nfs
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 3Gi
```

LINSTOR GUI

<https://cozystack.ru/docs/v1.5/storage/linstor-gui/>

Пакет `linstor-gui` развёртывает **LINSTOR GUI** от LINBIT — веб-консоль для просмотра и управления узлами LINSTOR, определениями ресурсов, томами, пулами хранения и снимками. Интерфейс проксирует REST API контроллера LINSTOR внутри кластера с использованием mTLS, поэтому учётные данные никогда не попадают в браузер. [Cozystack Documentation](#)

Пакет является опциональным и включается по требованию. Рабочий процесс с CLI не меняется — включение GUI никак не влияет на поведение LINSTOR. [Cozystack Documentation](#)

Включение пакета

Добавьте `cozystack.linstor-gui` в `bundles.enabledPackages` в **Platform Package**:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
  -p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozyst...']'
```

Подождите около минуты, пока чарт платформы выполнит согласование, затем убедитесь, что `HelmRelease` создан:

```
kubectl get helmrelease --namespace cozy-linstor linstor-gui
```

[Cozystack Documentation](#)

Доступ к интерфейсу

Вариант 1 - Ingress, защищённый Keycloak (рекомендуется)

Если включена **аутентификация OIDC**, интерфейс можно опубликовать по адресу `https://linstor-gui.<root-host>` за realm Keycloak кластера. Добавьте `linstor-gui` в `publishing.exposedServices` в Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
  -p '[{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "lin...']'
```

Ingress создаётся только при выполнении обоих условий: `linstor-gui` указан в `publishing.exposedServices` и включён OIDC (`authentication.oidc.enabled: true`). Без Keycloak перед прокси REST API LINSTOR нет слоя аутентификации, поэтому чарт намеренно не создаёт Ingress. [Cozystack Documentation](#)

Доступ ограничен участниками группы Keycloak `cozystack-cluster-admin` — той же группы, которая выдаёт права `cluster-admin` RBAC в управляющем кластере. После включения откройте

<https://linstor-gui.<root-host>> в браузере и войдите, используя учётные данные Keycloak.
[Cozystack Documentation](#)

Вариант 2 - Проброс порта

Для разового доступа без Keycloak пробросьте порт сервиса ClusterIP:

```
kubectl -n cozy-linstor port-forward svc/linstor-gui 3373:80
```

Затем откройте <http://localhost:3373>. [Cozystack Documentation](#)

Создание зашифрованного хранилища на LINSTOR

<https://cozystack.ru/docs/v1.5/storage/disk-encryption/>

Администраторы Cozystack могут включить зашифрованное хранилище, создав собственный StorageClass. В этом руководстве описано, как задать парольную фразу шифрования, создать зашифрованный класс хранения и использовать его в приложениях.

LINSTOR обеспечивает шифрование постоянных томов at-rest с помощью [LUKS](#). Это гарантирует, что данные на диске хранятся в зашифрованном виде и доступны только тогда, когда том смонтирован и разблокирован.

Настройка шифрования в LINSTOR

Чтобы начать использовать шифрование, задайте парольную фразу шифрования в LINSTOR.

```
kubectl exec -i -t -n cozy-linstor deploy/linstor-controller -- linstor encryption create-passphrase
```

[!] Сохраните парольную фразу в надёжном месте.

>

Если вы потеряете парольную фразу шифрования, все зашифрованные данные будут безвозвратно утеряны.

Парольную фразу нужно вводить каждый раз после перезапуска контроллера LINSTOR. Для ввода парольной фразы используйте следующую команду:

```
kubectl exec -i -t -n cozy-linstor deploy/linstor-controller -- linstor encryption enter-passphrase
```

Создание зашифрованного класса хранения

Создайте StorageClass для зашифрованного хранилища:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-encrypted
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: "data"
  linstor.csi.linbit.com/layerList: "luks storage"
  linstor.csi.linbit.com/encryption: "true"
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "false"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: replicated-encrypted
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: "data"
  linstor.csi.linbit.com/autoPlace: "3"
  linstor.csi.linbit.com/layerList: "drbd luks storage"
  linstor.csi.linbit.com/encryption: "true"
  linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
  property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
  property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

Теперь вы можете использовать этот `StorageClass` для создания `PersistentVolumeClaims` (PVC) под шифрованное хранилище.

Сеть

Сетевые возможности

<https://cozystack.ru/docs/v1.5/networking/>

Этот раздел документации описывает настройку сети, виртуальные маршрутизаторы, балансировщики нагрузки и другие сетевые возможности Cozystack.

Сетевая архитектура

Обзор сетевой архитектуры кластера Cozystack: балансировка нагрузки MetalLB, сеть Cilium на eBPF и изоляция тенантов с Kube-OVN.

VPC

Выделенные подсети

Управляемый сервис VPN

Управляемый сервис VPN упрощает развёртывание VPN-сервера и управление им, позволяя легко устанавливать защищённые соединения.

Gateway API (Cilium)

Ingress на основе Gateway API для отдельных тенантов с использованием Cilium - CRD TenantGateway, наследование через метки пространств имён, интеграция с cert-manager, терминатция и сквозная передача TLS, двухуровневая модель безопасности.

Управляемый сервис HTTP-кеширования на основе Nginx

Сервис HTTP-кеширования на основе Nginx предназначен для оптимизации веб-трафика и повышения производительности веб-приложений.

PROXY-protocol и исправление hairpin-NAT

Включение PROXY-protocol на ingress-nginx в Cozystack и исправление возникающей проблемы hairpin-NAT с помощью входящего в состав платформы компонента ouroboros.

Публикация конечной точки Kubernetes API через external-dns

Преобразование EndpointSlice default/kubernetes в аннотированные headless-сервисы, чтобы external-dns мог публиковать конечную точку Kubernetes API в DNS с помощью входящего в состав платформы компонента kuberture.

Управляемый сервис балансировки TCP-нагрузки

Управляемый сервис балансировки TCP-нагрузки упрощает развёртывание балансировщиков нагрузки и управление ими.

Виртуальные маршрутизаторы

Развёртывание виртуального маршрутизатора в VM

Сетевая архитектура

<https://cozystack.ru/docs/v1.5/networking/architecture/>

Обзор

Cozystack использует многослойный сетевой стек, разработанный для bare-metal-кластеров Kubernetes. Архитектура объединяет несколько компонентов, каждый из которых отвечает за свой уровень сети:

| Уровень | Компонент | Назначение |
|--------------------------------|----------------------------|--|
| Внешняя балансировка нагрузки | MetalLB | Публикация сервисов во внешние сети |
| Балансировка нагрузки сервисов | Cilium eBPF | Замена kube-proxy, DNAT внутри ядра |
| Сетевые политики | Cilium eBPF | Изоляция тенантов и обеспечение безопасности |
| Сеть подов (CNI) | Kube-OVN | Централизованный IPAM, оверлейная сеть |
| Проброс IP в VM | cozy-proxy | Проброс внешних IP-адресов внутрь виртуальных машин |
| Вторичные интерфейсы VM | Multus CNI | Подключение вторичных L2-интерфейсов к виртуальным машинам |
| Наблюдаемость | Hubble (опционально) | Видимость сетевого трафика (по умолчанию отключено) |

Сетевая конфигурация кластера

| Параметр | Значение по умолчанию |
|----------------|-----------------------|
| Pod CIDR | 10.244.0.0/16 |
| Service CIDR | 10.96.0.0/16 |
| Join CIDR | 100.64.0.0/16 |
| Домен кластера | cozy.local |
| Тип оверлея | GENEVE |
| CNI | Kube-OVN |

| | |
|-------------------|-------------|
| Замена kube-proxy | Cilium eBPF |
|-------------------|-------------|

Варианты сетевого стека

Cozystack поддерживает несколько вариантов сетевого стека для разных типов кластеров. Вариант выбирается через `bundles.system.variant` в конфигурации платформы.

| Вариант | Компоненты | Целевая платформа |
|-------------------------------------|-----------------------------------|--------------------|
| <code>kubeovn-cilium</code> | Kube-OVN + Cilium (по умолчанию) | Talos Linux |
| <code>kubeovn-cilium-generic</code> | Kube-OVN + Cilium | kubeadm, k3s, RKE2 |
| <code>cilium</code> | Только Cilium | Talos Linux |
| <code>cilium-generic</code> | Только Cilium | kubeadm, k3s, RKE2 |
| <code>cilium-kilo</code> | Cilium + Kilo | Talos Linux |
| <code>noop</code> | Нет (используйте собственный CNI) | Любая |

В вариантах с Kube-OVN Cilium работает как цепочечный CNI (режим `generic-veth`): Kube-OVN отвечает за сеть подов и IPAM, а Cilium обеспечивает балансировку нагрузки сервисов, применение сетевых политик и опциональную наблюдаемость через Hubble.

В вариантах только с Cilium он выступает одновременно и как CNI, и как балансировщик нагрузки сервисов.

Далее в этом документе описывается вариант по умолчанию `kubeovn-cilium`.

Выделение Pod CIDR (Kube-OVN)

Kube-OVN использует модель общего Pod CIDR:

- Все поды получают адреса из единого общего пула IP-адресов (10.244.0.0/16)
- IP-адреса выделяются централизованно через IPAM Kube-OVN
- Нет разбиения CIDR по узлам (в отличие от Calico или Flannel)
- Поскольку IP-адреса не привязаны к CIDR-блокам конкретных узлов, поды можно переносить на другие узлы с сохранением адресов
- Взаимодействие подов между узлами использует туннели GENEVE (Join CIDR: 100.64.0.0/16)

Приём внешнего трафика через MetalLB

MetalLB - реализация балансировщика нагрузки для bare-metal-кластеров Kubernetes. Он назначает внешние IP-адреса сервисам типа `LoadBalancer`, позволяя внешнему трафику достигать кластера.

Режим Layer 2 (ARP)

В режиме L2 MetalLB отвечает на ARP-запросы для внешнего IP-адреса сервиса. Один узел становится «лидером» для этого IP и принимает весь трафик.

Как это работает:

1. Спикер MetalLB на одном из узлов забирает внешний IP себе
2. Спикер отвечает на ARP-запросы: «IP X находится по MAC-адресу aa:bb:cc:dd:ee:ff»
3. Весь трафик для этого IP идёт на узел-лидер
4. Cilium на узле выполняет DNAT к нужному поду

В режиме L2 трафик для конкретного IP сервиса обрабатывает только один узел. При отказе узла-лидера происходит переключение, но настоящей балансировки нагрузки между узлами для одного сервиса нет.

Режим BGP

В режиме BGP MetalLB устанавливает BGP-сессии с вышестоящими маршрутизаторами и анонсирует маршруты /32 для IP-адресов сервисов. Это обеспечивает настоящую балансировку нагрузки ECMP между узлами.

Как это работает:

1. Спикеры MetalLB устанавливают BGP-сессии с вышестоящими маршрутизаторами
2. Каждый спикер анонсирует IP сервиса как маршрут /32
3. У маршрутизатора появляется несколько next-hop для одного префикса
4. ECMP распределяет трафик между узлами
5. Cilium на принимающем узле выполняет DNAT к нужному поду

Интеграция VLAN для внешнего трафика

Внешний трафик может доставляться в кластер через дополнительные VLAN (клиентские VLAN, DMZ, публичные сети и т.п.), откуда он направляется к сервисам через MetalLB и Cilium.

Cilium как замена kube-proxy

Cilium заменяет kube-proxy, подключая программы eBPF непосредственно в ядре Linux. Это обеспечивает более эффективную обработку пакетов и расширенные возможности.

Традиционный kube-proxy (iptables) против Cilium eBPF

Ключевые отличия:

| Аспект | kube-proxy (iptables) | Cilium (eBPF) |
|------------------------|---|-------------------------------|
| Сложность поиска | Обход правил за $O(n)$ | Поиск по хешу за $O(1)$ |
| Контекст выполнения | Накладные расходы в пользовательском пространстве | Нативно в ядре |
| Переключения контекста | Требуются | Отсутствуют |
| Масштабируемость | Деградирует с ростом числа сервисов | Постоянная производительность |

Архитектура eBPF

(Секция описывает взаимодействие eBPF Programs (TC, XDP, Socket-level) и eBPF Maps (Service Tables, Endpoint Maps, Policy Maps) в Kernel Space).

Изоляция тенантов с Kube-OVN и Cilium

В мультитенантном кластере Cozystack все тенанты используют общий Pod CIDR. Это безопасно, потому что изоляция обеспечивается политиками Cilium eBPF на уровне ядра, а не сегментацией сети. Тенанты не могут взаимодействовать друг с другом, хотя используют общий пул IP-адресов. Kube-OVN выделяет IP-адреса из этого общего пула централизованно, без разбиения CIDR по узлам.

Архитектура CNI

Kube-OVN является основным CNI-плагином для сети подов и IPAM. Собственный механизм сетевых политик Kube-OVN отключён (`ENABLE_NP: false`), и всё применение политик делегировано Cilium. Cilium работает как цепочечный CNI-компонент (режим `generic-veth`), который применяет сетевые политики через eBPF и заменяет kube-proxy для балансировки нагрузки сервисов.

Модель изоляции тенантов

(Секция описывает логику разрешения трафика внутри тенанта и запрета между тенантами через Cilium eBPF Policy Engine).

Безопасность на основе идентичностей

Cilium присваивает каждой конечной точке (поду) идентичность безопасности на основе её меток. Политики применяются с использованием этих идентичностей, а не IP-адресов.

Применение политик в ядре

Когда пакет передаётся между подами, Cilium применяет политики полностью в пространстве ядра. Всё это происходит в пространстве ядра примерно за 100 наносекунд.

Почему применение политик через eBPF безопасно

| Свойство | Описание |
|--|---|
| Верификатор | Программы eBPF проверяются перед загрузкой - без сбоев и бесконечных циклов |
| Изоляция | Программы выполняются в ограниченном контексте ядра |
| Нет обхода из пользовательского пространства | Весь сетевой трафик обязан проходить через хуки eBPF |
| Атомарные обновления | Изменения политик атомарны - без состояний гонки |
| Внутри ядра | Не нужны переключения контекста, быстрее, чем в пользовательском пространстве |

Применение на уровне ядра

Трафик не может обойти политику — он ОБЯЗАН проходить через ядро, где работают верифицированные eBPF программы.

Запрет по умолчанию и изоляция пространств имён

По умолчанию Kubernetes разрешает весь трафик между подами. Cozystack автоматически применяет ресурсы `CiliumNetworkPolicy` для ограничения трафика.

Для изоляции Cozystack использует иерархические метки тенантов. Политики сопоставляются по меткам пространств имён `tenant.cozystack.io/*`, что позволяет родительским тенантам включать пространства имён дочерних тенантов. Пример:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-internal-communication
  namespace: tenant-example
spec:
  endpointSelector: {}
  ingress:
    - fromEndpoints:
        - matchLabels:
            k8s:io.cilium.k8s.namespace.labels.tenant.cozystack.io/tenant-example: ""
  egress:
    - toEndpoints:
        - matchLabels:
            k8s:io.cilium.k8s.namespace.labels.tenant.cozystack.io/tenant-example: ""
    - toEntities:
        - kube-apiserver
        - cluster
```

Наблюдаемость с Hubble

Hubble обеспечивает видимость сетевого трафика для плоскости данных Cilium. Он входит в сетевой стек, но по умолчанию отключён, чтобы минимизировать потребление ресурсов.

Во включённом состоянии Hubble предоставляет:

- Журналы потоков в реальном времени для всего трафика между подами и внешнего трафика
- Видимость DNS-запросов
- Метрики уровня запросов HTTP/gRPC
- Интеграцию с метриками Prometheus
- Веб-интерфейс для визуализации трафика

Чтобы включить Hubble, задайте следующее в конфигурации Cilium:

```
cilium:
  hubble:
    enabled: true
    relay:
      enabled: true
    ui:
      enabled: true
```

Полные сведения о настройке см. в разделе [Enabling Hubble](#).

Сводка потоков трафика

(Секция содержит визуальные схемы внешнего доступа и изоляции tenants).

VPC

<https://cozystack.ru/docs/v1.5/networking/vpc/>

VPC предоставляет набор выделенных подсетей вместе со связанными с ними сетевыми сервисами. По мере развития сервиса будет появляться больше способов изоляции рабочих нагрузок.

Детали сервиса

Для работы сервису требуются kube-ovn и multus CNI, поэтому по умолчанию он работает только с бандлом `paas-full`. Kube-ovn предоставляет ресурсы VPC и Subnet, а также обеспечивает изоляцию и обслуживание сети, например DHCP. Внутри он использует виртуальные маршрутизаторы и виртуальные коммутаторы ovn. Multus добавляет поддержку нескольких сетевых интерфейсов, поэтому под или VM могут иметь два и более сетевых интерфейса.

Сейчас каждая рабочая нагрузка подключается к управляющей сети по умолчанию, в которой также находится шлюз по умолчанию, и большая часть трафика идёт через неё. Подсети VPC пока являются дополнительными выделенными сетевыми пространствами.

Замечания по развёртыванию

Имя VPC должно быть уникальным в пределах тенанта. Имя подсети и диапазон IP-адресов должны быть уникальными в пределах VPC. Адресное пространство подсети не должно пересекаться с диапазоном IP-адресов управляющей сети по умолчанию; рекомендуется использовать подмножества 172.16.0.0/12. Сейчас защитных проверок нет, но они запланированы на будущее.

Разные VPC могут содержать подсети с пересекающимися диапазонами IP-адресов.

VM или под могут быть подключены к нескольким вторичным подсетям одновременно. Каждое вторичное подключение будет представлено дополнительным сетевым интерфейсом.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|---------|-------------|-------------------|----------|
| subnets | Подсети VPC | map[string]object | {...} |

| | | | |
|---------------------------------|--|-------------------|-----------------|
| <code>subnets[name].cidr</code> | CIDR подсети,
например 192.168.0.0/24 | <code>cidr</code> | <code>{}</code> |
|---------------------------------|--|-------------------|-----------------|

Примеры

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VirtualPrivateCloud
metadata:
  name: vpc00
spec:
  subnets:
    sub00:
      cidr: 172.16.0.0/24
    sub01:
      cidr: 172.16.1.0/24
    sub02:
      cidr: 172.16.2.0/24
```

Управляемый сервис VPN | Cozystack

<https://cozystack.ru/docs/v1.5/networking/vpn/>

Виртуальная частная сеть (VPN) - важнейший инструмент для обеспечения безопасного и приватного обмена данными через интернет. Управляемый сервис VPN упрощает развёртывание VPN-сервера и управление им, позволяя легко устанавливать защищённые соединения.

- Клиентские приложения VPN: <https://shadowsocks5.github.io/en/download/clients.html>

Детали развёртывания

Сервис VPN работает на основе Outline Server - продвинутого и удобного VPN-решения. Его внутреннее название - «Shadowbox»; он упрощает процесс настройки и совместного использования серверов Shadowsocks. Он работает, запуская экземпляры Shadowsocks по требованию. Кроме того, Shadowbox совместим со стандартными клиентами Shadowsocks, что обеспечивает гибкость и удобство использования VPN.

- Документация: <https://shadowsocks.org/>
- Документация: <https://github.com/Jigsaw-Code/outline-server/tree/master/src/shadowbox>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|--|-----------------------|------------------|
| <code>replicas</code> | Количество реплик VPN-сервера. | <code>int</code> | 2 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики VPN-сервера. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |

| | | | |
|------------------------------|--|---------------------|----------------------|
| <code>resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>external</code> | Разрешить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

Параметры приложения

| Имя | Описание | Тип | Значение |
|-----------------------------------|--|--------------------------------|-----------------|
| <code>host</code> | Хост, подставляемый в генерируемые URL. | <code>string</code> | <code>""</code> |
| <code>users</code> | Карта конфигурации пользователей. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Пароль пользователя (генерируется автоматически, если не указан). | <code>string</code> | <code>""</code> |
| <code>externalIPs</code> | Список внешних IP-адресов (<code>externalIPs</code>) для сервиса. Необязательный. Если не указан, по умолчанию использует сервис LoadBalancer. | <code>[]string</code> | <code>[]</code> |

Примеры параметров и справка

`resources` и `resourcesPreset`

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

```
resources:
  cpu: 4000m
  memory: 4Gi
```

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют принятому в облаках соглашению об именовании `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая

серия включает восемь размеров (от `nano` до `4xlarge`). Старые плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов с соотношением 1:1.

Полная матрица размеров и соответствие старых имён типам приведены в [docs/operations/resource-presets.md](https://docs.cozycloud.org/en/stable/operations/resource-presets.md).

users

```
users:
  user1:
    password: hackme
  user2: {} # autogenerated password
```

externalIPs

```
externalIPs:
  - "11.22.33.44"
  - "11.22.33.45"
  - "11.22.33.46"
```

Gateway API (Cilium)

<https://cozystack.ru/docs/v1.5/networking/gateway-api/>

Обзор

Cozystack предоставляет поддержку Gateway API как опциональную альтернативу ingress-nginx. Когда она включена, арендатор, явно выбравший её через `tenant.spec.gateway: true`, получает собственный Gateway (собственный сервис LoadBalancer, собственный IP балансировщика, собственные Issuer и Certificate для арендатора), материализованный в его собственном пространстве имён. Все остальные арендаторы дерева публикуются через Gateway ближайшего предка, владеющего им, — по той же схеме, что и существующее наследование `_namespace.ingress`.

Чарт не рендерит ресурсы Gateway, Issuer или Certificate напрямую. Вместо этого он рендерит один CR `gateway.cozystack.io/v1alpha1 TenantGateway` на каждый включивший опцию арендатор, а `cozystack-controller` согласует из него все нижестоящие объекты Gateway API и `cert-manager`. Это устраняет гонку между Helm и контроллером за `Gateway.spec.listeners`, которую иначе вызывала бы динамическая материализация слушателей на основе маршрутов.

Эта страница описывает архитектуру, модель наследования, выбор режима сертификатов (HTTP-01 по умолчанию или wildcard DNS-01 по выбору), двухуровневую модель безопасности и сценарий миграции с ingress-nginx.

Gateway API и ingress-nginx сосуществуют в одном кластере — режимы выбираются для каждого сервиса / арендатора, а не глобально. Существующие кластеры обновляются с `gateway.enabled=false` и не видят изменений в поведении.

Поверхность API для арендаторов

Арендаторы в Cozystack взаимодействуют с платформой исключительно через ресурсы `apps.cozystack.io/*`, обслуживаемые `cozystack-api`; RBAC арендатора не даёт права записи в `gateway.networking.k8s.io/*`, базовые Namespaces или `cozystack.io/Package`. В разделе [Безопасность](#) объясняется, как уровни допуска (admission) выстроены с учётом этого ограничения.

Архитектура

Поток согласования

1. **Чарт `extra/gateway`** (вызывается из `apps/tenant`, когда `tenant.spec.gateway=true`) рендерит `TenantGateway` CR (`gateway.cozystack.io/v1alpha1`).
2. **`cozystack-controller (TenantGatewayReconciler)`**:

-
- Материализует Gateway (на каждый тенант, динамические слушатели), Issuer (учётная запись ACME на тенант), Certificate(s) (HTTP-01: для каждого слушателя; DNS-01: один wildcard + SAN для потомков) и HTTPRoute (владелец, для перенаправления http->https).
 - Патчит метки пространств имён (`namespace.cozystack.io/gateway` на `attachedNamespaces`).
 - Наблюдает за HTTPRoute / TLSRoute (принадлежат приложениям).

Контроллер:

- Материализует Gateway, Issuer тенанта, HTTPRoute для перенаправления и Certificate(ы) из `TenantGateway.spec`.
- Наблюдает за ресурсами HTTPRoute и TLSRoute по всему кластеру. Для каждого маршрута, прикреплённого к его Gateway, он собирает имена хостов и генерирует Certificate для каждого приложения.
- В режиме DNS-01 расширяет wildcard-Certificate SAN-записями `<child-apex> + *.<child-apex>` для каждого тенанта, наследующего через этот Gateway (они обнаруживаются перечислением пространств имён с меткой `namespace.cozystack.io/gateway = <owner>` и чтением их `namespace.cozystack.io/host`), и добавляет по одному HTTPS-слушателю `*.<child-apex>` на каждого наследующего потомка.
- Проставляет метку `namespace.cozystack.io/gateway = <owner>` на каждое пространство имён из `TenantGateway.spec.attachedNamespaces` (системные пространства имён `cozy-*`, публикуемые через Gateway). Патч сопровождается аннотацией `cozystack.io/gateway-attached-by`, чтобы контроллер знал, какие метки записал он сам, а какие принадлежат чарту `apps/tenant` — метки, записанные чартом, никогда не трогаются. Метки, записанные контроллером, вычищаются, когда пространства имён исчезают из `attachedNamespaces`.
- Разрешает межпространственные конфликты имён хостов: пространства имён `cozy-*` (платформенные сервисы под управлением администратора кластера) выигрывают у пространств имён тенантов. В случае поражения тенанта контроллер записывает условие `HostnameConflict` под именем контроллера в `Status.Parents`.
- Отказывается молча присваивать уже существующие объекты Gateway, Issuer, Certificate или HTTPRoute перенаправления, которые носят выведенное контроллером имя, но не имеют `OwnerReference` на `TenantGateway`. Вместо перезаписи вручную закреплённой конфигурации операторы видят явное условие `Ready=False/ReconcileError`.

Путь трафика

- `GatewayClass` задаётся для каждого `TenantGateway` через настраиваемое оператором поле чарта `gatewayClassName` (по умолчанию `cilium`). У тенантов нет RBAC на запись CR `TenantGateway`, поэтому самостоятельно выбрать класс они не могут.
- Один Gateway на тенанта-владельца в пространстве имён этого тенанта. HTTPRoute / TLSRoute всех наследующих потомков прикрепляются к этому общему Gateway.
- Envoy работает как DaemonSet Cilium (`cilium.envoy.enabled=true`) и обеспечивает как терминацию TLS (HTTPS-слушатели), так и сквозную передачу TLS (выделенные слушатели Passthrough). `envoy.enabled=true` — значение по умолчанию для новых установок Cozystack; операторам, обновляющим существующий кластер, нужно включить его перед `gateway.enabled`.
- IP LoadBalancer выделяется тем механизмом балансировки, который администратор кластера настроил на уровне платформы: `IPAddressPool` / `L2Advertisement` / `BGPAdvertisement` /

`CiliumLoadBalancerIPPool`. Администраторы подключают тот аллокатор, который подходит их окружению (пул MetalLB с L2 / BGP, Cilium LB-IPAM, [robotlb](#) для Hetzner Robot или `Service.spec.externalIPs` как способ ручного закрепления). API тенанта остаётся независимым от механизма — поля `gatewayIP` в CR Tenant нет. Чтобы закрепить конкретный адрес, оператор заранее создаёт сервис LoadBalancer с `loadBalancerIP` или выдаёт тенанту ссылку на именованный пул, управляемый администратором.

- `externalTrafficPolicy`: сервис LoadBalancer, обслуживающий Gateway, создаётся Cilium и использует значение Kubernetes по умолчанию (`Cluster`). Поэтому исходные IP внешних клиентов транслируются (NAT) в адрес принимающего узла. Это отличается от прежней схемы с ingress (`publishing.exposure: loadBalancer`), которая устанавливает `externalTrafficPolicy: Local` и ограничивает IP балансировщика узлами с подами ingress.

Раскладка слушателей на Gateway тенанта

Gateway тенанта всегда материализует HTTP-слушатель:

| Имя | Порт | Протокол | Хост | Режим | Описание |
|------|------|----------|------|-------|--|
| http | 80 | HTTP | * | - | ACME /.well-known/acme-challenge/* + HTTPRoute перенаправления HTTP->HTTPS |

Плюс HTTPS-слушатели, зависящие от режима сертификатов:

- Режим HTTP-01 (по умолчанию): по одному HTTPS-слушателю на каждое имя хоста прикреплённого HTTPRoute, с именем `https-<first-label>-<8-hex>`. Шестнадцатеричный суффикс — это первые 32 бита `sha256(hostname)`, поэтому два разных имени хоста с одинаковой первой меткой (`harbor.foo.example.com` и `harbor.alice.example.com`) получают разные имена слушателей. `tls.certificateRefs` каждого слушателя указывает на его собственный `Certificate` с именем `<tgw>-<first-label>-<8-hex>-tls`, также выпускаемый автоматически.
- Режим DNS-01 (по выбору): слушатели `https (*.<owner apex>)` и `https-apex (<owner apex>)`, использующие один wildcard-Certificate, плюс по одному слушателю `https-child-<first-label>-<8-hex>` на apex каждого наследующего потомка (ссылающемуся на тот же wildcard-сертификат, чьи `dnsNames` расширяются до `<child-apex> + *.<child-apex>`).
- Плюс по одному дополнительному слушателю на каждый сервис со сквозной передачей TLS (см. [TLSRoute \(TLS passthrough\)](#)).

`allowedRoutes.namespaces` слушателей использует два разных селектора в зависимости от роли слушателя:

1. HTTPS-слушатели и слушатели сквозного TLS сопоставляются по метке `namespace.cozystack.io/gateway` и допускают маршруты из любого пространства имён, чья метка равна имени пространства имён тенанта-владельца (например, `tenant-root` или `tenant-alice` — имя пространства имён, а не «голое» имя тенанта). Это точка опоры наследования — пространства имён тенантов-потомков получают эту метку автоматически (из чарта

`apps/tenant`), а системные пространства имён `cozy-*` из `attachedNamespaces` получают ту же метку от контроллера.

2. Простой HTTP-слушатель (порт 80) использует строго более узкий белый список по встроенной метке `kubernetes.io/metadata.name` — только само пространство имён тенанта-владельца (где живёт принадлежащий контроллеру HTTPRoute перенаправления) и `cozy-cert-manager` (HTTPRoute для проверок ACME HTTP-01). Поэтому HTTPRoute приложений, прикрепляющиеся к Gateway по порту 80, никогда не будут приняты.

HTTPS-слушатели дополнительно ограничивают `allowedRoutes.kinds` до HTTPRoute (а слушатели сквозного TLS — до TLSRoute), не позволяя GRPCRoute / TCPRoute / UDPRoute прикрепляться вне зоны покрытия VAP по именам хостов.

Включение Gateway API

Gateway API включается на двух уровнях. Оба значения по умолчанию остаются `false`; обновления не переключают тенантов незаметно.

1. Флаг на уровне платформы

Установите `gateway.enabled: true` в Package `cozystack.cozystack-platform`. Полные таблицы значений `gateway.*` и `publishing.certificates.dns01.*` см. в справочнике [Platform Package](#).

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        publishing:
          host: example.org
          gateway:
            enabled: true
            attachedNamespaces:
              - cozy-cert-manager
              - cozy-dashboard
              - cozy-keycloak
              - cozy-system
              - cozy-harbor
              - cozy-bucket
              - cozy-kubevirt
              - cozy-kubevirt-cdi
              - cozy-monitoring
              - cozy-linstor-gui
              - default
```

Пространство имён `default` включено потому, что TLSRoute Kubernetes API (поставляемый пакетом `cozystack-api`) живёт рядом с сервисом `kubernetes`, на который он указывает, а тот всегда находится в `default`.

Включение `gateway.enabled=true` задействует три вещи:

1. `ClusterIssuer.spec.acme.solvers` в `cert-manager` переключается с `http01.ingress.ingressClassName` на `http01.gatewayHTTPRoute`, прикрепляющийся к Gateway публикующего тенанта.
2. Шаблоны публикуемых сервисов (`dashboard`, `keycloak`, `grafana`, `alerta`) перестают рендерить свой `Ingress` и начинают рендерить свой `HTTPRoute`.
3. Сервисы со сквозной передачей TLS (`cozystack-api`, `vm-exportproxy`, `cdi-uploadproxy`) перестают рендерить `Ingress` и начинают рендерить `TLSRoute`, прикреплённый к выделенному слушателю `Passthrough`.

Список `attachedNamespaces` перечисляет системные пространства имён `cozy-*`, маршруты которых должны публиковаться через Gateway тенанта-владельца. Контроллер проставляет `namespace.cozystack.io/gateway = <owner>` на каждую запись.

2. Gateway для отдельного тенанта

Тенант получает собственный CR `TenantGateway` (а через контроллер — собственные `Gateway`, `Issuer`, `Certificate(ы)` и сервис `LoadBalancer`) только тогда, когда явно запросил это через `tenant.spec.gateway: true`. Все остальные тенанты дерева публикуются через Gateway ближайшего предка, владеющего им. По умолчанию `gateway` не задан, что трактуется как `false` (наследовать).

Отдельный Gateway имеет смысл, когда:

- Тенанту нужен собственный IP балансировщика.
- Архив тенанта не выводится из родительского (задан собственный `tenant.spec.host`).
- Тенант хочет собственную учётную запись ACME / Issuer.

```
# Примеры конфигурации в Tenant:
spec:
  gateway: true
```

Наследование

Чарт `apps/tenant` записывает `namespace.cozystack.io/gateway: <owner-namespace>` на каждое пространство имён тенанта. Чтобы проверить, через какой Gateway сейчас наследуется данное пространство имён:

```
kubectl get namespace <tenant-ns> -o jsonpath='{.metadata.labels.namespace\.cozystack\.io/gateway}'
```

В режиме DNS-01 контроллер расширяет wildcard-Certificate Gateway-владельца SAN-записями `<child-архив> + *.<child-архив>` для каждого наследующего потомка.

Режим сертификатов: HTTP-01 (по умолчанию) или DNS-01 (по выбору)

`publishing.certificates.solver` определяет, как Issuer тенанта получает TLS-сертификаты.

HTTP-01 (по умолчанию)

Работает из коробки. Контроллер рендерит ACME Issuer с солвером `http01.gatewayHTTPRoute`. Для каждого приложения выпускается отдельный Certificate.

DNS-01 (по выбору)

Установите `publishing.certificates.solver: dns01` и выберите провайдера:

| Провайдер | Настройка в чарте | Секреты |
|--------------|---------------------------------|--|
| cloudflare | - | <code>cloudflare-api-token-secret</code> |
| route53 | <code>route53.region</code> | IAM ключи или IRSA |
| digitalocean | - | <code>digitalocean-api-token-secret</code> |
| rfc2136 | <code>rfc2136.nameserver</code> | TSIG ключи |

Маршрутизация по сервисам

HTTPRoute (терминация TLS на Gateway)

| Сервис | Пространство имён | Бэкенд | Условие рендера |
|-----------------|-------------------|----------------------------------|--|
| cozy-dashboard | cozy-dashboard | incloud-web-gatekeeper:8000 | <code>gateway.enabled</code> и <code>dashboard</code> в <code>exposedServices</code> |
| cozy-keycloak | cozy-keycloak | keycloak-http:80 | <code>gateway.enabled</code> |
| cozy-monitoring | cozy-monitoring | grafana-service:3000 / alerta:80 | <code>gateway.enabled</code> |

TLSRoute (TLS passthrough)

| Сервис | Пространство имён | Бэкенд | Порт | Имя TLSRoute |
|-----------------|-------------------|---------------------|------|---------------------|
| Kubernetes API | default | kubernetes:443 | 443 | tls-api |
| vm-exportproxy | cozy-kubevirt | vm-exportproxy:443 | 443 | tls-vm-exportproxy |
| cdi-uploadproxy | cozy-kubevirt-cdi | cdi-uploadproxy:443 | 443 | tls-cdi-uploadproxy |

Безопасность

Защиты выстроены на уровне `ValidatingAdmissionPolicy` (VAP) и `cozystack-api`.

1. **Уровень 1** — Селектор пространств имён `allowedRoutes` слушателя: Ограничивает прикрепление маршрутов только к разрешенным пространствам имён по метке `namespace.cozystack.io/gateway`.
2. **Уровень 2** — `cozystack-gateway-hostname-policy`: Отклоняет любой слушатель Gateway, чьё имя хоста не соответствует метке `namespace.cozystack.io/host` пространства имён.
3. **Уровень 3** — `cozystack-tenant-host-policy`: Отклоняет установку или изменение `spec.host` в ресурсе `Tenant`, если вызывающий не является доверенным (администратором).
4. **Уровень 4** — `cozystack-namespace-host-label-policy`: Делает метку `namespace.cozystack.io/host` на `Namespace` фактически неизменяемой.
5. **Уровень 5** — `cozystack-route-hostname-policy`: Отклоняет `HTTPRoute` / `TLSRoute`, если их `spec.hostnames` не соответствуют разрешенному арех пространства имён.

Разрешение HostnameConflict

В случае конфликта имен хостов:

- Маршрут из `cozy-*` всегда выигрывает.
- При равном приоритете выигрывает маршрут с лексикографически меньшим `<namespace>/<name>`.

Ограничения частоты

Let's Encrypt применяет квоты (50 новых сертификатов на домен в неделю). Рекомендуется использовать режим DNS-01 с wildcard-сертификатами для больших инсталляций, чтобы не исчерпать лимиты.

Миграция с ingress-nginx

Для существующего кластера

1. Для каждого тенанта-владельца установите `tenant.spec.gateway: true`.
2. Дождитесь готовности Gateway:

```
```bash
```

```
kubectl -n <owner-tenant-ns> wait gateway/cozystack --for=condition=Programmed --timeout=600s
```

```
```
```

3. Включите `gateway.enabled: true` в `Package` платформы. Это переключит системные сервисы на новые маршруты без простоя.

Известные ограничения

-
- Общий IP балансировщика: Несколько арендаторов-владельцев не могут разделить один IP на текущем Cilium из-за коллизии порта 443.
 - TLSRoute v1alpha2: Используется экспериментальная версия API.
 - DNS-01: Требуется делегирование поддомена или доступа к DNS-провайдеру для всех уровней иерархии.

Устранение неполадок

- Проверьте список TenantGateway: `kubectl get tenantgateway -A`.
- TenantGateway в ReconcileError: Проверьте, нет ли уже существующих объектов с такими же именами, не принадлежащих контроллеру.
- Gateway в Programmed=False: Проверьте логи Cilium operator: `kubectl -n cozy-cilium logs deploy/cilium-operator`.
- Certificate в Ready=False: Проверьте доступность порта 80 для HTTP-01 или настройки DNS-провайдера для DNS-01.

См. также

- gateway-api.sigs.k8s.io
 - docs.cilium.io
 - [Let's Encrypt Rate Limits](#)
-

Управляемый сервис HTTP-кеширования на основе Nginx

<https://cozystack.ru/docs/v1.5/networking/http-cache/>

Сервис HTTP-кеширования на основе Nginx предназначен для оптимизации веб-трафика и повышения производительности веб-приложений. Сервис объединяет специально собранные экземпляры Nginx и HAProxy для эффективного кеширования и балансировки нагрузки.

Информация о развёртывании

Экземпляры Nginx включают следующие модули и возможности:

- модуль VTS для статистики
- интеграция с ip2location
- интеграция с ip2proxy
- поддержка 51Degrees
- функция очистки кеша

HAProxy играет ключевую роль в этой схеме: он направляет входящий трафик на конкретные экземпляры Nginx на основе консистентного хеша, вычисляемого от URL. Каждый экземпляр Nginx использует Persistent Volume Claim (PVC) для хранения кешированного содержимого, обеспечивая быстрый и надёжный доступ к часто используемым ресурсам.

Детали развёртывания

Архитектура развёртывания показана на схеме ниже:



Известные проблемы

- Модуль VTS показывает неверное время ответа апстрима, github.com/vozt/nginx-module-vts#198

storageClass помечен как неизменяемый (immutable) в схеме чарта - см. docs/storage-immutability.md, где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|--------------|--|----------|----------|
| size | Размер Persistent Volume Claim, доступный для данных приложения. | quantity | 10Gi |
| storageClass | StorageClass, используемый для хранения данных. | string | " " |

| | | | |
|-----------------------|--|-------------------|--------------------|
| <code>external</code> | Разрешить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |
|-----------------------|--|-------------------|--------------------|

Параметры приложения

| Имя | Описание | Тип | Значение |
|------------------------|--|-----------------------|-----------------|
| <code>endpoints</code> | Конфигурация конечных точек в виде списка <code>ip:port</code> . | <code>[]string</code> | <code>[]</code> |

Параметры HAProxy

| Имя | Описание | Тип | Значение |
|---------------------------------------|---|-----------------------|----------------------|
| <code>haproxy</code> | Конфигурация HAProxy. | <code>object</code> | <code>{}</code> |
| <code>haproxy.replicas</code> | Количество реплик HAProxy. | <code>int</code> | <code>2</code> |
| <code>haproxy.resources</code> | Явная конфигурация CPU и памяти. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>haproxy.resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>haproxy.resources.мемору</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>haproxy.resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |

Параметры Nginx

| Имя | Описание | Тип | Значение |
|-----------------------------|--------------------------|---------------------|-----------------|
| <code>nginx</code> | Конфигурация Nginx. | <code>object</code> | <code>{}</code> |
| <code>nginx.replicas</code> | Количество реплик Nginx. | <code>int</code> | <code>2</code> |

| | | | |
|-------------------------------------|---|-----------------------|----------------------|
| <code>nginx.resources</code> | Явная конфигурация CPU и памяти. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>nginx.resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>nginx.resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>nginx.resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |

Примеры параметров и справка

`resources` и `resourcesPreset`

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

```
resources:
  cpu: 4000m
  memory: 4Gi
```

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

| Имя пресета | CPU | Память |
|-------------|------|--------|
| nano | 250m | 128Mi |
| micro | 500m | 256Mi |
| small | 1 | 512Mi |
| medium | 1 | 1Gi |
| large | 2 | 2Gi |
| xlarge | 4 | 4Gi |
| 2xlarge | 8 | 8Gi |

endpoints

`endpoints` - плоский список IP-адресов:

```
endpoints:  
- 10.100.3.1:80  
- 10.100.3.11:80  
- 10.100.3.2:80  
- 10.100.3.12:80  
- 10.100.3.3:80  
- 10.100.3.13:80
```

PROXY-protocol и исправление hairpin-NAT

<https://cozystack.ru/docs/v1.5/networking/hairpin-proxy-protocol/>

О чём эта страница

PROXY-protocol на ingress-nginx управляющего кластера, порождаемая им проблема hairpin-NAT, входящее в состав Cozystack исправление ([ouroboros](#), реализация [compumike/hairpin-proxy](#) на Go - см. [lexfrei/ouroboros](#)), единый флаг платформы, связывающий обе части, аддон для того же исправления внутри кластеров tenants и несимметричные пути отключения на каждом уровне.

Почему PROXY-protocol ломает трафик внутри кластера

Когда вышестоящий балансировщик L4 (облачный LB, hcloud-CCM, F5, haproxy за MetalLB и т.д.) добавляет [заголовок PROXY-protocol v1](#) к каждому соединению, приходящему на ingress-nginx, ingress-nginx настраивается с `use-proxy-protocol: "true"` и принимает только соединения с заголовком. Внешний трафик работает.

Трафик внутри кластера - нет. Под, который резолвит публичное имя самого кластера (self-check и HTTP-01 в cert-manager, внутренние вызовы `https://` по публичному DNS, обращения ArgoCD к самому себе и т.д.), обращается к CoreDNS и получает в ответ IP LoadBalancer. kube-proxy / Cilium-KPR видит этот IP LB и срезает путь напрямую к локальному сервису, минуя вышестоящий балансировщик. Соединение приходит на ingress-nginx без заголовка PROXY, который тот теперь требует. Ingress-nginx закрывает соединение. Hairpin молча ломается.

[KEP-1860](#) (`status.loadBalancer.ingress[].ipMode: Proxy`) - это апстрим-решение для кластеров, стоящих за внешним прокси-балансировщиком под управлением CCM. Оно не подходит для настроек Cozystack по умолчанию: при `kubeProxyReplacement: true` и IP-адресах LB, анонсируемых Cilium, Cilium полностью убирает фронтенд LB, когда `ipMode != VIP`, ломая сервис и для внутрикластерного, и для внешнего трафика. Вместо этого Cozystack использует [ouroboros](#).

Как это исправляет ouroboros

ouroboros - небольшой контроллер внутри кластера, который отслеживает имена хостов в Ingress (и опционально Gateway / HTTPRoute) и переписывает внутрикластерный DNS так, чтобы эти имена резолвились в небольшой внутрикластерный TCP-прокси. Он состоит из двух частей:

- контроллер, который отслеживает ресурсы Ingress / Gateway и добавляет в CoreDNS строки `rewrite name <host> ouroboros-proxy.<namespace>.svc.cluster.local.;`
- TCP-прокси, который слушает порты 8080/8443, добавляет заголовок PROXY-protocol v1 и пересылает трафик на настоящий ingress-nginx.

Cozystack использует два режима работы в зависимости от уровня:

| Режим | Имя | Описание |
|-------|-----------------------------|---|
| | <code>coredns</code> | Рабочий Corefile <code>kube-system/coredns</code> , между маркерами <code># === BEGIN ouroboros (do not edit by hand) ===</code> . CoreDNS управляющего кластера управляется Talos и не содержит директивы <code>import</code> , поэтому альтернативный режим там молча ничего бы не делал. |
| | <code>coredns-import</code> | Только строки <code>rewrite name</code> в ключ данных <code>ouroboros.override</code> ConfigMap <code>kube-system/coredns-custom</code> - отдельного ConfigMap, который поставляется чартом <code>cozystack-coredns</code> и подключается в Corefile через <code>import /etc/coredns/custom/*.override</code> . <code>ouroboros</code> никогда не трогает Corefile, отрендеренный чартом, поэтому не конфликтует с чартом при обновлении. |

Включение в управляющем кластере

Один флаг платформы включает PROXY-protocol на ingress-nginx управляющего кластера и автоматически развёртывает `ouroboros` на стороне управляющего кластера:

```
publishing:
  proxyProtocol: true
```

По умолчанию `false` - кластеры, не использующие PROXY-protocol, не получают ни новых ресурсов, ни изменений RBAC, ни исправлений DNS.

Предусловие: вышестоящий балансировщик

Флаг не настраивает за вас балансировщик L4 перед ingress-nginx - тот находится вне кластера (облачный LB или ваше собственное оборудование). Балансировщик обязан уже добавлять заголовки PROXY-protocol v1 до переключения флага.

Кадры PROXY-protocol передаются от вышестоящего балансировщика к ingress-nginx (балансировщик открывает TCP, пишет кадр, затем пишет данные). Прямые запросы `curl / nc` с ноутбука их никогда не увидит. Есть два практических способа проверки, выберите один:

1. Запустите `tcpdump --interface any port 80 or port 443 -A` на узле с ingress-nginx до переключения флага и посмотрите, есть ли ведущий кадр `PROXY TCP4 ... \r\n` во входящем TCP-трафике от балансировщика. Если кадр есть, балансировщик выполняет вставку и флаг можно включать.
2. Выполните любой внешний HTTP-запрос после изменения на стороне балансировщика, но до переключения `publishing.proxyProtocol`, и посмотрите в логах ingress-nginx (`kubectl --namespace cozy-ingress-nginx logs deploy/ingress-nginx-controller`) сообщение `client sent invalid proxy protocol header` - эта ошибка означает, что балансировщик начал вставлять кадры PROXY, а ingress-nginx ещё не настроен их принимать. Ошибка `broken header` (после переключения флага, когда балансировщик не вставляет кадры) сигнализирует, что предусловие на самом деле не выполнялось, и из этого состояния нужно немедленно откатиться назад.

Без выполнения предусловия каждый внешний запрос к ingress-nginx ломается в момент включения флага. Сначала настройте балансировщик, затем переключайте `publishing.proxyProtocol`.

Окно нестабильности при перерендеринге Talos

CoreDNS управляющего кластера - это Deployment под управлением Talos; ouroboros изменяет его рабочий Corefile, патча ConfigMap. При каждом обновлении конфигурации Talos (обновление версии Talos, изменение параметров ядра и т.п.) Talos перерендеривает ConfigMap по своему шаблону и затирает блок `rewrite`. Переписывание имён хостов ломается на короткое время (`controller.resync: 10m`, сверху ограниченное ближайшим событием Ingress/Gateway), пока контроллер не применит блок заново. На tenants этой проблемы нет.

Обёртка cozystack-coredns

Cozystack предоставляет собственную тонкую обёртку над апстрим-пакетом `coredns/helm-charts`: в Corefile добавляется директива `import /etc/coredns/custom/*.override` в серверный блок `:53`, а Deployment CoreDNS монтирует пустой ConfigMap `coredns-custom` в `/etc/coredns/custom/` (`optional: true`). Шаблон обёртки рендерит этот ConfigMap вообще без поля `data`: - поэтому трёхстороннему слиянию Helm не с чем сравнивать со стороны шаблона при каждом согласовании. Записи `data.ouroboros.override`, которые ouroboros добавляет на стороне apiserver во время работы, переживают любое обновление чарта.

`helm.sh/resource-policy: keep` на том же ConfigMap - отдельная, более слабая мера: она лишь не даёт `helm uninstall` удалить ConfigMap, но не защищает от затирания при обновлении чарта. Инвариант закреплён проверкой `notExists: data` в `packages/system/coredns/tests/coredns_custom_test.yaml` - будущее изменение чарта, добавляющее хотя бы `data: {}` «для явности», уронит эту проверку до того, как регрессия попадёт в релиз. Сегодня эта схема используется для всех tenants (где ouroboros работает в режиме `coredns-import`); CoreDNS управляющего кластера

управляется Talos и никогда не использует этот чарт, поэтому устанавливает `ouroboros` в режиме `coredns` с рабочим `Corefile`.

Домен кластера DNS у tenants

Чарт `ouroboros` начиная с версии 0.7.0 поддерживает два пути определения `clusterDomain`: явное закрепление через `controller.clusterDomain` или автоопределение во время работы из `/etc/resolv.conf`. Cozystack закрепляет в обёртке аддона `tenant` `controller.clusterDomain: cluster.local`, потому что автоопределение возвращает неверное значение на `tenants` Cozystack. Kubelet `tenant` подставляет `clusterDomain` платформы (часто `cozy.local`) в `resolv.conf` пода как поисковый домен, тогда как управляемый Kamaji CoreDNS `tenant` обслуживает собственный домен (`cluster.local` согласно `TenantControlPlane.networkProfile.clusterDomain`). Автоопределение собрал бы `<service>.<namespace>.svc.cozy.local.`, который CoreDNS `tenant` не обслуживает, и `/readyz` прокси вечно получал бы NXDOMAIN. Закреплённое значение совпадает с тем, что CoreDNS `tenant` действительно обслуживает, и даёт верный аргумент `--proxy-fqdn=<service>.<namespace>.svc.cluster.local..`

Операторы `tenants` с нестандартным доменом кластера `tenant` (федерации, пользовательский `TenantControlPlane.networkProfile.clusterDomain` в Kamaji и т.п.) могут переопределить его через `addons.ouroboros.valuesOverride.ouroboros.controller.clusterDomain` в CR Kubernetes. Чарт учитывает переопределение и выдаёт `--proxy-fqdn=<service>.<namespace>.svc.<cluster-domain>..` Указывать `_cluster.cluster-domain` (соглашение платформы Cozystack на стороне управляющего кластера, часто `cozy.local`) здесь было бы неправильно: это значение отражает `clusterDomain` платформы управляющего кластера, а не `tenant`.

Включение для отдельного tenant

`Tenants`, которые запускают собственный `ingress-nginx` с `PROXY-protocol`, нужен `ouroboros` внутри кластера `tenant`. `addons.ouroboros` в CR Kubernetes конкретного `tenant` включает там этот чарт:

```

apiVersion: apps.cozystack.io/v1alpha1
kind: Kubernetes
metadata:
  name: production
  namespace: tenant-acme
spec:
  addons:
    ingressNginx:
      enabled: true
      hosts:
        - acme.example.org
      valuesOverride:
        ingress-nginx:
          controller:
            config:
              use-proxy-protocol: "true"
              use-forwarded-headers: "true"
              compute-full-forwarded-for: "true"
              real-ip-header: "proxy_protocol"
              enable-real-ip: "true"
              # SECURITY: when use-forwarded-headers is on, ALWAYS pair it with
              # proxy-real-ip-cidr scoped to the upstream LB's CIDR. Without
              # that pairing, any client can spoof X-Forwarded-For and
              # bypass IP-based controls. The host platform deliberately
              # omits use-forwarded-headers / compute-full-forwarded-for
              # for this reason – turn them on at the tenant only when you
              # have a paired CIDR.
              # proxy-real-ip-cidr:
    ouroboros:
      enabled: true

```

Прежде чем переключать `enabled` обратно в `false`, прочитайте раздел [Отключение](#) ниже - блок `rewrite`, который `ouroboros` записал в CoreDNS тенанта, не вычищается автоматически.

- `addons.ouroboros.enabled: true` требует `addons.ingressNginx.enabled: true`. Иначе рендер чарта тенанта завершается понятной ошибкой - без `ingress-nginx` исправлению `hairpin` не на кого проксировать трафик.
- Включение аддона не включает автоматически `PROXY-protocol` на `ingress-nginx` тенанта - конфигурация `use-proxy-protocol` / `use-forwarded-headers` / `real-ip-header` задаётся вручную через `valuesOverride`, потому что у тенантов часто разные конфигурации вышестоящих балансировщиков (одни получают кадры `PROXY v1`, другие `PROXY v2`, третьи используют `X-Forwarded-For` через `HTTP-балансировщик`).

Отключение

Отключение устроено по-разному на двух уровнях, и это сделано намеренно. Путь управляющего кластера защищён от случайного затирания `Corefile` (через поле подтверждения `publishing.proxyProtocolAcknowledgeUnclean`), потому что `helm.sh/resource-policy: keep` на CR Package платформы создаёт асимметрию «перестали генерировать, но осталось установленным», которую платформа сама разрешить не имеет прав. Уровень тенанта этой защиты не имеет - `addons.ouroboros.enabled: false` напрямую запускает `helm uninstall`, `pre-delete`-хук чарта выполняет очистку при удалении, а шаблон тенанта `Cozystack` намеренно не содержит `lookup`

(проверка на остаточный HelmRelease заблокировала бы обычное отключение: рендер родителя выполняется раньше, чем Flux удаляет дочерний HR).

Уровень управляющего кластера.

Переключение `publishing.proxyProtocol` из `true` в `false` делает две вещи:

1. Со стороны ingress-nginx: следующее согласование регенерирует `cozystack.ingress-application` без `use-proxy-protocol / real-ip-header / enable-real-ip` (Cozystack намеренно НЕ устанавливает `use-forwarded-headers` и `compute-full-forwarded-for` в управляющем кластере: эти ключи позволили бы любому вышестоящему прокси подделывать `X-Forwarded-For` без парного `proxy-real-ip-cidr`). ingress-nginx перестает принимать заголовки `PROXY-protocol`. **Если вышестоящий балансировщик L4 в этот момент всё ещё добавляет кадры PROXY, каждый внешний запрос к ingress-nginx будет завершаться ошибкой `400 Bad Request`.** Сначала выключите PROXY на балансировщике, затем переключайте этот флаг.
2. Со стороны ouroboros: платформа перестает генерировать CR Package `cozystack.ouroboros`, но каждый CR Package несёт `helm.sh/resource-policy: keep`. Helm оставляет существующий Package в кластере, ouroboros остаётся установленным, а блок `rewrite` в CoreDNS продолжает направлять трафик на `ouroboros-proxy`. Само по себе переключение флага не удаляет ouroboros.

Платформа отказывается рендерить простое переключение флага, когда CR Package `cozystack.ouroboros` уже есть в кластере - `helm template / helm upgrade` сразу завершается ошибкой, указывающей на `kubectl delete package.cozystack.io cozystack.ouroboros` (что запускает `helm uninstall` и `pre-delete`-хук очистки чарта) и на поле подтверждения. Вендоренный чарт ouroboros содержит Job очистки (`charts/ouroboros/templates/coredns-cleanup-hook.yaml`), который останавливает контроллер и с помощью `sed` вырезает блок `# === BEGIN ouroboros ... END ouroboros ===` из `kube-system/coredns` автоматически, когда `helm` действительно удаляет чарт, - операторам не нужно выполнять ручной рецепт с `sed` при обычном отключении. Полная последовательность отключения в управляющем кластере:

1. Выключите вставку `PROXY-protocol` на вышестоящем балансировщике (предусловие для внешнего трафика).
2. Удалите CR Package командой `kubectl delete package.cozystack.io cozystack.ouroboros` (или добавьте его в `bundles.disabledPackages`). Это запускает `helm uninstall`, который выполняет `pre-delete`-хук чарта и автоматически патчит `kube-system/coredns`.
3. Установите `publishing.proxyProtocol: false` в значениях платформы. Рендер-защита теперь проходит (lookup для `cozystack.ouroboros` возвращает `nil` после шага 2), поэтому `publishing.proxyProtocolAcknowledgeUnclean` остаётся в значении по умолчанию `false`.

Если у оператора есть причина переключить `publishing.proxyProtocol: false` ДО удаления CR Package (строгий GitOps, где `kubectl delete` не входит в основной процесс, параллельные откаты и т.п.), установите `publishing.proxyProtocolAcknowledgeUnclean: true` вместе с переключением флага в том же коммите, а удаление Package выполните отдельно. Верните `proxyProtocolAcknowledgeUnclean` в `false`, когда кластер пробудет чистым один цикл согласования. Это аварийный клапан, а не рекомендуемый путь. Рецепт очистки управляющего кластера ниже - ручной запасной вариант для редкого случая, когда `pre-delete`-хук не смог восстановить `kube-system/coredns` после удаления пакета.

Известная дыра: оператор, который удаляет CR Package до переключения флага, полностью обходит защиту (`lookup` возвращает `nil`, рендер платформы проходит). Этот путь в основном безопасен - `kubectl delete package` запускает `helm uninstall`, и `pre-delete`-хук чарта патчит `Corefile` при удалении. Дыра - это редкий случай дрейфа, когда Package удалён, флаг `true`, а Ingress только что создан: он резолвится в старый IP LB и ломается по `hairpin`.

Уровень тенанта.

Переключение `addons.ouroboros.enabled` из `true` в `false` прекращает рендер `HelmRelease`, и Flux удаляет чарт при следующем согласовании на стороне тенанта. Политика `helm.sh/resource-policy: keep`, упомянутая выше для управляющего кластера, находится на CR Package платформы - на стороне тенанта на `HelmRelease` она не действует, поэтому отключение там действительно удаляет рабочую нагрузку, а не просто прекращает её генерацию. Очистку выполняет `pre-delete`-хук чарта, обнуляя ключ `ouroboros.override` в `kube-system/coredns-custom` до удаления пода контроллера. Полная последовательность отключения на тенанте состоит из одного шага:

1. Установите `addons.ouroboros.enabled: false` в CR Kubernetes тенанта. Flux выполнит `helm uninstall`, и `pre-delete`-хук чарта сам патчит `kube-system/coredns-custom`.

Если `pre-delete`-хук чарта не сработал (под контроллера застрял в `CrashLoop`, дрейф RBAC на `ConfigMap`, `kubectl delete hr` в обход `helm uninstall`), симптомом будет устаревшая запись `rewrite` в `ConfigMap kube-system/coredns-custom` тенанта, указывающая на уже несуществующий сервис. Для восстановления выполните рецепт очистки тенанта ниже. `Ingress-nginx` тенанта не затрагивается переключением одного лишь `addons.ouroboros - PROXY-protocol` на `ingress` тенанта настраивается вручную через `valuesOverride` и остаётся таким, каким его задал оператор.

Рецепт очистки управляющего кластера (запасной вариант)

Этот рецепт - ручной запасной вариант для редкого случая, когда `pre-delete`-хук чарта не сработал (штатный путь на управляющем кластере вообще не требует ручного рецепта). Требования: `kubectl` с `kubeconfig` управляющего кластера, а также `jq` и `sed` на рабочей станции оператора. Блок ограничен строками `# === BEGIN ouroboros (do not edit by hand) ===` и `# === END ouroboros ===` (пояснение `(do not edit by hand)` есть только в строке `BEGIN`). Диапазон `sed` ниже использует сопоставление по префиксу с обеих сторон, поэтому маркеры находятся независимо от их точного окончания.

```
# === BEGIN cleanup-recipe ===
existing=$(kubectl --namespace kube-system get configmap coredns --output jsonpath='{.data.Corefile}')
cleaned=$(printf '%s\n' $existing | sed '/# === BEGIN ouroboros/,/# === END ouroboros/d')
kubectl --namespace kube-system patch configmap coredns --type merge --patch $(jq --null-input --arg c "$cleaned" '{.data.Corefile: $c}')
# === END cleanup-recipe ===
```

Рецепт использует JSON merge-patch, поэтому метки, аннотации и другие ключи данных `ConfigMap coredns` (включая любые метаданные, управляемые Talos) сохраняются.

Рецепт очистки тенанта (запасной вариант)

Этот рецепт - ручной запасной вариант для редкого случая, когда `pre-delete`-хук чарта не смог самостоятельно обнулить `ouroboros.override` (штатный путь на тенанте оставляет очистку чарту и вообще не требует ручного рецепта). Требования: `kubectl` с `admin-kubeconfig` тенанта (`jq` / `sed` не нужны - рецепт для тенанта состоит из одного разового патча). Выполните его после неудавшегося удаления аддона:

```
kubectl --kubeconfig <tenant-admin-kubeconfig> --namespace kube-system patch configmap coredns-custom --typ...
```

`ouroboros.override` - единственный ключ, которым `ouroboros` владеет внутри `coredns-custom`; его обнуление не затрагивает остальные ключи (принадлежащие оператору фрагменты `*.override`, будущие дополнения `Cozystack`).

Операторы изолированных сред

Чарт и образ загружаются напрямую из апстрим-реестра `lexfrei/ouroboros` - они не зеркалируются в `ghcr.io/cozystack/*`. Операторам изолированных (`air-garped`) сред нужно отзеркалировать два дополнительных источника:

1. `oci://ghcr.io/lexfrei/charts/ouroboros:<version>` (чарт, дайджест зафиксирован в `packages/system/ouroboros/Makefile` как `OUROBOROS_CHART_DIGEST=sha256:...` - точные значения `OUROBOROS_CHART_VERSION` и `OUROBOROS_CHART_DIGEST` берите из `Makefile` того релиза `Cozystack`, который вы зеркалируете);
2. `ghcr.io/lexfrei/ouroboros:<version>@sha256:...` (образ, дайджест зафиксирован в `packages/system/ouroboros/values.yaml` в `image.tag` - передайте именно эту ссылку в `regsync / crane copy / skopeo copy`).

Ссылка на образ в `Cozystack` включает дайджест `@sha256:...`, указанный выше. Инструменты зеркалирования должны либо сохранить этот дайджест на всём пути (стандартное поведение `regsync`, `crane copy`, `skopeo copy`), либо убрать закрепление `@sha256:...` из `values.yaml` после зеркалирования - иначе `kubelet` при загрузке будет резолвить апстрим-дайджест, не найдёт его в локальном реестре и выдаст `ErrImagePull`.

Почему в настройках Cozystack по умолчанию не используется KEP-1860

Значение по умолчанию `kubeProxyReplacement: true` в `Cozystack` означает, что `Cilium` полностью убирает фронтенд LB, когда `ipMode != VIP`, ломая приём трафика сервисов для IP-адресов, анонсируемых по L2/BGP. Режим `Proxy` из `KEP-1860` предназначен для кластеров за внешним прокси-балансировщиком под управлением CCM, который перенаправляет трафик на `NodePort`. В `Cozystack` балансировщик L4 часто направляет трафик напрямую на IP-адреса сервисов `Cilium` (через BGP), поэтому `Cozystack` использует `ouroboros`.

Публикация конечной точки Kubernetes API через external-dns

<https://cozystack.ru/docs/v1.5/networking/kuberture/>

О чём эта страница

Как опубликовать конечную точку Kubernetes API самого кластера в виде DNS-записи с помощью `external-dns`, почему очевидный подход (направить `external-dns` напрямую на `EndpointSlice default/kubernetes`) не работает, что представляет собой входящий в состав Cozystack мост (`kuberture`), как одна установка `kuberture` может одновременно обслуживать несколько экземпляров `external-dns`, а также как всё это отключить.

Почему external-dns не может читать EndpointSlice напрямую

`external-dns` считывает значимое для DNS состояние из источников Kubernetes — `service`, `ingress`, `gateway-httproute` и ряда других. Источник `EndpointSlice` он не поддерживает. `EndpointSlice` управляющего слоя, который Kubernetes поддерживает в `default/kubernetes`, — это канонический, всегда актуальный список адресов узлов, обслуживающих API; кроме того, это единственный полноценный объект, который несёт эту информацию без меток или селекторов, задаваемых оператором. Операторы, которые хотят публиковать конечную точку API своего кластера в публичный DNS через `external-dns` (проверки `dns01` в `cert-manager`, внутренняя автоматизация, обращающаяся к API по FQDN, обнаружение сервисов между кластерами), сталкиваются с тем, что автоматизировать это нечем.

Обычно предлагаемые обходные пути — поддерживаемый вручную `Service` типа `ExternalName`, поддерживаемый вручную `headless-Service` с вручную закреплёнными `Endpoints`, внешний скрипт, опрашивающий API и обновляющий запись, — имеют один и тот же недостаток: они перестают соответствовать действительности, как только узел управляющего слоя добавляется, удаляется или меняет адрес. `EndpointSlice` уже корректно отслеживает это состояние. `kuberture` соединяет одно с другим.

Как это решает kuberture

`kuberture` — небольшой контроллер внутри кластера, который следит за `EndpointSlice` и создаёт аннотированные `headless-объекты Service`. Поток данных таков:

```
EndpointSlice (kubernetes) -> kuberture -> headless Service(s) with annotations -> external-dns
```

Для каждого настроенного оператором выхода (output) `kuberture` создаёт `headless-сервис` (`spec.clusterIP: None`) в собственном пространстве имён (`cozy-kuberture`) и предоставляет на нём три

аннотации. Каждый ключ состоит из заданного оператором `annotationPrefix` и фиксированного суффикса:

| Полный ключ аннотации | Источник |
|---|--|
| <code><annotationPrefix>hostname</code> | заданные оператором имена хостов для этого выхода |
| <code><annotationPrefix>target</code> | IP-адреса через запятую, полученные из <code>EndpointSlice</code> или из узлов, на которые он указывает |
| <code><annotationPrefix>tTL</code> | заданный оператором <code>recordTTL</code> ; если он не указан, используется значение по умолчанию контроллера |

С префиксом по умолчанию это даёт `external-dns.alpha.kubernetes.io/hostname`, `external-dns.alpha.kubernetes.io/target`, `external-dns.alpha.kubernetes.io/ttl` — ключи, которые платформенный `external-dns` читает из коробки. `external-dns` читает сервис, видит аннотации и создаёт DNS-запись. У сервиса нет селектора и вручную управляемых `Endpoints` — он существует исключительно как носитель аннотаций для `external-dns`.

Контроллер не создаёт объекты `EndpointSlice` или `Endpoints` (у него есть только право чтения `EndpointSlice`) и не трогает исходный `EndpointSlice default/kubernetes`. Он строго аддитивен: отдельный сервис на каждый выход, в собственном пространстве имён, без побочных эффектов для существующего состояния кластера.

Включение в управляющем кластере

`kuberture` — опциональный системный пакет, включаемый через `bundles.enabledPackages`. Сам пакет не содержит пригодных значений по умолчанию, кроме включения `ServiceMonitor` для платформенного `Prometheus`, — развёртывание сразу завершается ошибкой, если `config.outputs` пуст. Оператор обязан объявить хотя бы один выход, потому что единственное, чего `kuberture` не может определить сам, — это DNS-имя, которое оператор действительно хочет опубликовать.

Один выход, потребляемый платформенным `external-dns`:

```

apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
    platform:
      values:
        bundles:
          enabledPackages:
            - cozystack.external-dns
            - cozystack.kuberture
---
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.kuberture
spec:
  variant: default
  components:
    kuberture:
      values:
        kuberture:
          config:
            outputs:
              - name: api
                hostname:
                  - api.k8s.example.com
                serviceName: kuberture-api
                addressSource: endpointslice
                recordTTL: 60

```

`annotationPrefix` здесь не указан, поэтому контроллер использует значение по умолчанию `external-dns.alpha.kubernetes.io/` — префикс, за которым платформенный `external-dns` следит из коробки. Когда включены оба пакета, HelmRelease `external-dns` подхватывает сервис `kuberture-api` в `cozy-kuberture` и создаёт запись `api.k8s.example.com` у настроенного провайдера.

Пакет `cozystack.external-dns` — это общекластерный системный экземпляр (он работает с `namespaced: false` и наблюдает за сервисами по всему кластеру). `cozystack.external-dns-application` — вариант, ограниченный пространством имён тенанта, и он не подхватит сервисы за пределами своего пространства имён. Поэтому используйте `kuberture` в паре с `cozystack.external-dns`, а не с `cozystack.external-dns-application`.

Маршрутизация на несколько экземпляров external-dns

Одна установка `kuberture` может обслуживать любое количество экземпляров `external-dns`, если задавать разный `annotationPrefix` для каждого выхода. Каждый экземпляр `external-dns` запускается с флагом `--annotation-prefix=<your-prefix>/`, из-за чего он строит все ключи аннотаций `hostname` / `target` / `ttl` под этим префиксом и игнорирует всё остальное; `kuberture` проставляет соответствующий префикс на каждом сервисе. Это описанный в апстриме паттерн [Split Horizon DNS](#) — в отличие от `--annotation-filter`, который является селектором меток Kubernetes и фильтрует, какие сервисы рассматривает экземпляр, а не из какого префикса он читает данные.

```

apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.kuberture
spec:
  variant: default
  components:
    kuberture:
      values:
        kuberture:
          config:
            outputs:
              - name: public
                hostname:
                  - api.k8s.example.com
                serviceName: kuberture-public
                addressSource: endpointslice
              - name: internal
                hostname:
                  - api-internal.k8s.example.internal
                annotationPrefix: internal-dns.example.com/
                serviceName: kuberture-internal
                addressSource: endpointslice
                recordTTL: 300

```

В результате в `cozy-kuberture` появляются два headless-сервиса:

- `kuberture-public` несёт стандартные аннотации `external-dns.alpha.kubernetes.io/*` и потребляется платформенным `external-dns`.
- `kuberture-internal` несёт аннотации `internal-dns.example.com/*` и невидим для платформенного `external-dns` (который читает префикс по умолчанию). Его потребляет второй экземпляр `external-dns`, запущенный с `--annotation-prefix=internal-dns.example.com/`: этот флаг указывает экземпляру искать `hostname / target / ttl` под `internal-dns.example.com/*`, поэтому он видит `kuberture-internal` и не видит `kuberture-public`.

Каждый сервис несёт только аннотации своего префикса — перекрёстного смешивания между выходами нет.

`annotationPrefix` допускает две формы: не указывать поле, чтобы унаследовать значение по умолчанию контроллера `external-dns.alpha.kubernetes.io/`, либо задать непустую строку, оканчивающуюся на `/`. Пустая строка `"` отклоняется схемой значений чарта; единственный способ вернуться к префиксу по умолчанию — не указывать поле.

Стратегии определения адресов

`addressSource` определяет, откуда каждый выход берёт целевые IP-адреса:

| addressSource | Что публикует kuberture |
|------------------------------|---|
| endpointslice (по умолчанию) | Адреса из EndpointSlice <code>default/kubernetes</code> как есть. Используйте, когда IP-адреса из EndpointSlice — это именно те адреса, которые должны попасть в DNS. |

| | |
|----------------------------|---|
| <code>node-internal</code> | InternalIP каждого узла, на котором находится конечная точка EndpointSlice. Используйте, когда EndpointSlice содержит IP-адреса сети подов, а <code>external-dns</code> должен публиковать внутренние адреса узлов. |
| <code>node-external</code> | ExternalIP каждого узла. Используйте для облачных кластеров, где у узлов публичные адреса на другом интерфейсе, нежели тот, на котором слушает API. |
| <code>node-public</code> | (В документации оставлено пустым) |

`addressType` (IPv4 / IPv6) дополнительно фильтрует полученный набор; по умолчанию IPv4.

Отключение

Удаление `cozystack.kuberture` из `bundles.enabledPackages` прекращает генерацию CR Package платформенным Helm-чартом. Как и у любого опционального системного пакета, существующий CR Package не удаляется автоматически: платформенный помощник проставляет `helm.sh/resource-policy: keep` на каждый сгенерированный Package, поэтому Helm/Flux оставляет его на месте, когда тот перестаёт рождаться из шаблона.

Чтобы окончательно удалить `kuberture`:

1. Удалите Package вручную: `kubectl delete package cozystack.kuberture`.

Каскадное удаление убирает HelmRelease в `cozy-kuberture`, что удаляет Deployment, созданные по конфигурации оператора выходные сервисы и (со временем) само пространство имён. Записи в `external-dns`, ранее опубликованные из этих сервисов, подчиняются собственной политике удаления `external-dns` (`policy: upsert-only` — значение по умолчанию в Cozystack, при котором записи не отзываются при удалении сервиса; заранее переключитесь на `policy: sync`, если записи должны вычищаться автоматически).

Замечания о цепочке поставки

Чарт загружается из `oci://ghcr.io/lexfrei/kuberture/charts/kuberture`, а образ контроллера — из `ghcr.io/lexfrei/kuberture`. Оба находятся в личном пространстве имён мейнтейнера и намеренно не зеркалируются в `ghcr.io/cozystack/*`. Операторы изолированных (air-gapped) сред должны отзеркалировать и чарт, и образ контроллера в свой реестр и переопределить `kuberture.image.repository` в значениях Package. Версия чарта и дайджест OCI-манифеста зафиксированы в Makefile пакета `cozystack`; тег и дайджест образа контроллера зафиксированы в `values.yaml` пакета. Фиксации обновляются синхронно с каждым релизом апстрима.

Полный перечень значений, процедура обновления и причина отключённого по умолчанию NetworkPolicy описаны в README пакета: [packages/system/kuberture/](#).

См. также

- [Включение и отключение компонентов](#) — используемый здесь механизм `bundles.enabledPackages` / `bundles.disabledPackages`.
 - [Справочник Platform Package](#) — справка по полю `bundles.enabledPackages`.
 - [lexfrei/kuberture](#) — исходный код контроллера и чарт, справочник по конфигурации и планы развития.
 - [packages/system/kuberture/](#) — пакет `cozystack`: прослойка значений, фикстуры `helm-unittest`, сквозной `smoke-тест`.
-

Управляемый сервис балансировки TCP-нагрузки

<https://cozystack.ru/docs/v1.5/networking/tcp-balancer/>

Управляемый сервис балансировки TCP-нагрузки упрощает развёртывание балансировщиков нагрузки и управление ими. Он эффективно распределяет входящий TCP-трафик между несколькими бэкенд-серверами, обеспечивая высокую доступность и оптимальное использование ресурсов.

Детали развёртывания

Управляемый сервис балансировки TCP-нагрузки использует для балансировки HAProxy. HAProxy — проверенное и надёжное решение для распределения входящего TCP-трафика между несколькими бэкенд-серверами, обеспечивающее высокую доступность и эффективное использование ресурсов. Такой выбор гарантирует бесперебойную и надёжную работу вашей инфраструктуры балансировки нагрузки.

- Документация: <https://www.haproxy.com/documentation/>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|---|-----------------------|------------------|
| <code>replicas</code> | Количество реплик HAProxy. | <code>int</code> | 2 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики балансировщика TCP. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |

| | | | |
|------------------------------|--|---------------------|----------------------|
| <code>resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>external</code> | Разрешить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

Параметры приложения

| Имя | Описание | Тип | Значение |
|---|--|-----------------------|--------------------|
| <code>httpAndHttps</code> | Конфигурация HTTP и HTTPS. | <code>object</code> | <code>{}</code> |
| <code>httpAndHttps.mode</code> | Режим работы балансировщика. | <code>string</code> | <code>tcp</code> |
| <code>httpAndHttps.targetPorts</code> | Конфигурация целевых портов. | <code>object</code> | <code>{}</code> |
| <code>httpAndHttps.targetPorts.http</code> | Номер порта HTTP. | <code>int</code> | <code>80</code> |
| <code>httpAndHttps.targetPorts.https</code> | Номер порта HTTPS. | <code>int</code> | <code>443</code> |
| <code>httpAndHttps.endpoints</code> | Список адресов конечных точек. | <code>[]string</code> | <code>[]</code> |
| <code>whitelistHTTP</code> | Защитить HTTP с помощью списка разрешённых клиентских сетей (по умолчанию: <code>false</code>). | <code>bool</code> | <code>false</code> |
| <code>whitelist</code> | Список разрешённых клиентских сетей. | <code>[]string</code> | <code>[]</code> |

Примеры параметров и справка

`resources` и `resourcesPreset`

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

```
resources:
  cpu: 4000m
  memory: 4Gi
```

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют принятому в облаках соглашению об именовании <серия>.<размер>. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1` 1:0.5, `c1` 1:1, `s1` 1:2, `u1` 1:4, `m1` 1:8), и каждая серия включает восемь размеров (от `nano` до `4xlarge`). Старые плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов с соотношением 1:1.

Полная матрица размеров и соответствие старых имён типам приведены в [docs/operations/resource-presets.md](https://docs.cozystack.io/operations/resource-presets.md).

Виртуальные маршрутизаторы

<https://cozystack.ru/docs/v1.5/networking/virtual-router/>

Начиная с версии **v0.27.0** Cozystack может развёртывать виртуальные маршрутизаторы (также известные как «router appliances» или «middlebox appliances»). Эта возможность позволяет создать виртуальный маршрутизатор на основе экземпляра виртуальной машины. Виртуальный маршрутизатор может маршрутизировать трафик между разными сетями.

Создание виртуального маршрутизатора

Для создания виртуального маршрутизатора требуется учётная запись администратора Cozystack.

1. Создайте экземпляр VM

Используйте стандартные пакеты `vm-instance` и `virtual-machine`, чтобы создать экземпляр виртуальной машины.

2. Отключите защиту от подмены адресов (anti-spoofing)

Чтобы экземпляр VM мог работать как виртуальный маршрутизатор, у него должна быть отключена защита от подмены адресов:

```
```bash
kubectl patch virtualmachines.kubevirt.io virtual-machine-example --type=merge \
-p '{"spec":{"template":{"metadata":{"annotations":{"ovn.kubernetes.io/port_security": "false"}}}}}'
...
```
```

3. Перезапустите виртуальную машину

```
```bash
virtctl stop virtual-machine-example
virtctl start virtual-machine-example
...
```
```

4. Получите IP-адрес VM

```
```bash
kubectl get vmi
...
```
```

В выводе будет строка с IP-адресом новой VM:

```
```text
NAME AGE PHASE IP NODENAME READY
virtual-machine-example 3d4h Running 10.244.8.56 gld-csxhk-003 True
```
```

...

5. Настройте пользовательские маршруты для тенанта

Отредактируйте пространство имён тенанта:

```
```bash
```

```
kubectl edit namespace tenant-example
```

...

Добавьте следующую аннотацию, указав полученный ранее IP-адрес маршрутизатора в `gw` и маску подсети, которую должен обслуживать маршрутизатор, в `dst`:

```
```yaml
```

```
ovn.kubernetes.io/routes: |
```

```
{
```

```
"gw": "10.244.8.56",
```

```
"dst": "10.10.13.0/24"
```

```
}]
```

```
```
```

Теперь эти пользовательские маршруты будут применяться ко всем подам в пространстве имён тенанта.

---



---

# Cozystack API

## Cozystack API

<https://cozystack.ru/docs/v1.5/cozystack-api/>

### Cozystack API

---

Cozystack предоставляет мощный API, который позволяет развёртывать сервисы с помощью различных инструментов. Вы можете управлять ресурсами через `kubectl`, Terraform или программно с помощью Go.

Оптимальный способ изучения Cozystack API:

1. Используйте дашборд для развёртывания приложения.
2. Изучите развёрнутый ресурс в Cozystack API и используйте его как пример.
3. Параметризируйте и воспроизведите пример ресурса, чтобы создавать собственные ресурсы через API.

### Обнаружение ресурсов

---

Вы можете получить список всех доступных ресурсов с помощью `kubectl`:

```
kubectl api-resources | grep apps.cozystack
buckets apps.cozystack.io/v1alpha1 true Bucket
clickhouses apps.cozystack.io/v1alpha1 true ClickHouse
etcds apps.cozystack.io/v1alpha1 true Etcd
foundationdb apps.cozystack.io/v1alpha1 true FoundationDB
harbors apps.cozystack.io/v1alpha1 true Harbor
httpcaches apps.cozystack.io/v1alpha1 true HTTPCache
infos apps.cozystack.io/v1alpha1 true Info
ingresses apps.cozystack.io/v1alpha1 true Ingress
kafkas apps.cozystack.io/v1alpha1 true Kafka
kuberneteses apps.cozystack.io/v1alpha1 true Kubernetes
mariadbs apps.cozystack.io/v1alpha1 true MariaDB
mongodbs apps.cozystack.io/v1alpha1 true MongoDB
monitorings apps.cozystack.io/v1alpha1 true Monitoring
natses apps.cozystack.io/v1alpha1 true NATS
openbaos apps.cozystack.io/v1alpha1 true OpenBAO
postgreses apps.cozystack.io/v1alpha1 true Postgres
qdrants apps.cozystack.io/v1alpha1 true Qdrant
rabbitmq apps.cozystack.io/v1alpha1 true RabbitMQ
redises apps.cozystack.io/v1alpha1 true Redis
seaweedfses apps.cozystack.io/v1alpha1 true SeaweedFS
tcpbalancers apps.cozystack.io/v1alpha1 true TCPBalancer
tenants apps.cozystack.io/v1alpha1 true Tenant
virtualprivate apps.cozystack.io/v1alpha1 true VirtualPrivateCloud
vmdisks apps.cozystack.io/v1alpha1 true VMDisk
vminstances apps.cozystack.io/v1alpha1 true VMInstance
vpns apps.cozystack.io/v1alpha1 true VPN
```

## Использование kubectl

Запросите конкретный тип ресурса в пространстве имён вашего тенанта:

```
kubectl get postgreses -n tenant-test
NAME READY AGE VERSION
test True 46s 0.7.1
```

Просмотрите вывод в формате YAML:

```
kubectl get postgreses -n tenant-test test -o yaml
```

```
apiVersion: apps.cozystack.io/v1alpha1
appVersion: 0.7.1
kind: Postgres
metadata:
 name: test
 namespace: tenant-test
spec:
 databases: {}
 replicas: 2
 size: 10Gi
 storageClass: ""
 users: {}
status:
 conditions:
 - lastTransitionTime: "2024-12-10T09:53:32Z"
 message: Helm install succeeded for release tenant-test/postgres-test.v1 with chart postgres@0.7.1
 reason: InstallSucceeded
 status: "True"
 type: Ready
 - lastTransitionTime: "2024-12-10T09:53:32Z"
 message: Helm install succeeded for release tenant-test/postgres-test.v1 with chart postgres@0.7.1
 reason: InstallSucceeded
 status: "True"
 type: Released
 version: 0.7.1
```

Вы можете использовать этот ресурс как пример для создания аналогичного сервиса через API. Просто сохраните вывод в файл, обновите `name` и нужные параметры, затем используйте `kubectl` для создания нового экземпляра Postgres:

```
kubectl apply -f postgres.yaml
```

## Использование Terraform

Cozystack интегрируется с Terraform. Для создания ресурсов в Cozystack API можно использовать стандартный провайдер `kubernetes`.

Пример:

```
provider "kubernetes" {
 config_path = "~/.kube/config"
}

resource "kubernetes_manifest" "vm_disk_iso" {
 manifest = {
 "apiVersion" = "apps.cozystack.io/v1alpha1"
 "appVersion" = "0.7.1"
 "kind" = "Postgres"
 "metadata" = {
 "name" = "test2"
 "namespace" = "tenant-test"
 }
 "spec" = {
 "replicas" = 2
 "size" = "10Gi"
 }
 }
}
```

Затем выполните:

```
terraform plan
terraform apply
```

Ваш новый кластер Postgres будет развёрнут.

---

## Использование кода на Go

Cozystack публикует пользовательские типы ресурсов Kubernetes в виде модуля Go, что позволяет управлять ресурсами Cozystack из любого кода на Go. Подробности и примеры см. на странице [Go Types](#).

---

## Go Types

Программное управление ресурсами Cozystack с помощью типов Go

---

## REST API Reference

Cozystack REST API Reference

---

## Справочник ApplicationDefinition

Как ресурсы ApplicationDefinition описывают типы приложений и как искать их из клиентского кода

---

# Go Types

<https://cozystack.ru/docs/v1.5/cozystack-api/go-types/>

## Типы Go

Cozystack публикует свои типы ресурсов Kubernetes в виде Go-модуля, что позволяет управлять ресурсами Cozystack из любого Go-кода. Типы доступны по адресу [pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1](https://pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1).

## Установка

Добавьте зависимость в ваш Go-модуль:

```
go get github.com/cozystack/cozystack/api/apps/v1alpha1@v1.5.0
```

## Сценарии использования

Типы Go полезны для:

- Создания инструментов автоматизации — разработки скриптов или приложений, программно развёртывающих ресурсы Cozystack и управляющих ими
- Интеграции с внешними системами — подключения Cozystack к собственным CI/CD-конвейерам, системам мониторинга или оркестрации
- Валидации конфигураций — проверки спецификаций ресурсов перед их применением к кластеру
- Генерации документации — разбора и анализа существующих ресурсов Cozystack
- Создания дашбордов — разработки пользовательских интерфейсов для управления Cozystack

## Доступные пакеты

Модуль содержит пакеты для каждого типа ресурса; вы можете изучить их для вашей конкретной версии по адресу [pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1](https://pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1)

## Простой пример

Для базового использования достаточно импортировать нужный пакет:



---

```
package main

import (
 "fmt"

 "github.com/cozystack/cozystack/api/apps/v1alpha1/vmdisk"
)

func main() {
 // Создание источника VMDisk из именованного образа
 image := vmdisk.SourceImage{Name: "ubuntu"}
 fmt.Printf("Source: %+v\n", image)
}
```

## Сложный пример

---

Этот пример демонстрирует создание и маршалинг нескольких типов ресурсов Cozystack:

```

package main

import (
 "encoding/json"
 "fmt"

 "github.com/cozystack/cozystack/api/apps/v1alpha1/postgresql"
 "github.com/cozystack/cozystack/api/apps/v1alpha1/redis"
 "github.com/cozystack/cozystack/api/apps/v1alpha1/vminstance"
 metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
 "k8s.io/apimachinery/pkg/api/resource"
)

func main() {
 // Создание конфигурации PostgreSQL с пользователями и базами данных
 pgConfig := postgresql.Config{
 TypeMeta: metav1.TypeMeta{
 APIVersion: "apps.cozystack.io/v1alpha1",
 Kind: "Postgres",
 },
 ObjectMeta: metav1.ObjectMeta{
 Name: "my-app-db",
 Namespace: "tenant-myapp",
 },
 Spec: postgresql.ConfigSpec{
 Replicas: 3,
 Size: resource.MustParse("50Gi"),
 Version: postgresql.Version("v16"),
 Users: map[string]postgresql.User{
 "appuser": {
 Password: "secretpassword",
 Replication: false,
 },
 "readonly": {
 Password: "readonlypass",
 },
 },
 },
 Databases: map[string]postgresql.Database{
 "myapp": {
 Extensions: []string{"pg_trgm", "uuid-ossf"},
 Roles: postgresql.DatabaseRoles{
 Admin: []string{"appuser"},
 ReadOnly: []string{"readonly"},
 },
 },
 },
 Backup: postgresql.Backup{
 Enabled: true,
 DestinationPath: "s3://backups/postgres",
 EndpointURL: "http://minio:9000",
 RetentionPolicy: "30d",
 S3AccessKey: "myaccesskey",
 S3SecretKey: "mysecretkey",
 Schedule: "0 2 * * *",
 },
 },
 ... (73 lines truncated)
}

```

## Развёртывание ресурсов

---

После создания конфигураций ресурсов их можно развернуть с помощью:

1. `kubectl` — маршалинг в YAML и применение:

```
yamlData, _ := json.Marshal(yourConfig)
// Используйте библиотеку маршалинга YAML для конвертации в YAML
```

2. Прямой клиент Kubernetes — использование `client-go`:

```
import (
 "k8s.io/client-go/kubernetes"
 "k8s.io/apimachinery/pkg/runtime"
)

scheme := runtime.NewScheme()
// Зарегистрируйте ваши типы в схеме
```

## Дополнительные ресурсы

---

- [Документация Go-пакета](#)
  - [Репозиторий Cozystack на GitHub](#)
  - [Справочник Kubernetes API](#)
-

# REST API Reference

<https://cozystack.ru/docs/v1.5/cozystack-api/rest/>

Cozystack apps.cozystack.io API api/apps/v1alpha1/v1.5.2-1-g7cc5df513 OAS 3.0

Download OpenAPI specification: </docs/v1.5/cozystack-api/api.json>

## appsCozystackIo\_v1alpha1

### Operations

- GET /apis/apps.cozystack.io/v1alpha1/
- GET /apis/apps.cozystack.io/v1alpha1/bootboxes
- GET /apis/apps.cozystack.io/v1alpha1/buckets
- GET /apis/apps.cozystack.io/v1alpha1/clickhouses
- GET /apis/apps.cozystack.io/v1alpha1/etcds
- GET /apis/apps.cozystack.io/v1alpha1/externaldns
- GET /apis/apps.cozystack.io/v1alpha1/foundationdb
- GET /apis/apps.cozystack.io/v1alpha1/harbors
- GET /apis/apps.cozystack.io/v1alpha1/httpcaches
- GET /apis/apps.cozystack.io/v1alpha1/infos
- GET /apis/apps.cozystack.io/v1alpha1/ingresses
- GET /apis/apps.cozystack.io/v1alpha1/kafkas
- GET /apis/apps.cozystack.io/v1alpha1/kuberneteses
- GET /apis/apps.cozystack.io/v1alpha1/mariadbs
- GET /apis/apps.cozystack.io/v1alpha1/mongodbs
- GET /apis/apps.cozystack.io/v1alpha1/monitorings
  - Description: list or watch objects of kind Monitoring
  - Parameters

| Name                | Type            | Description                                                                     |
|---------------------|-----------------|---------------------------------------------------------------------------------|
| allowWatchBookmarks | boolean (query) | allowWatchBookmarks requests watch events with type "BOOKMARK".                 |
| continue            | string (query)  | The continue option should be set when retrieving more results from the server. |
| fieldSelector       | string (query)  | A selector to restrict the list of returned objects by their fields.            |

|                      |                 |                                                                                           |
|----------------------|-----------------|-------------------------------------------------------------------------------------------|
| labelSelector        | string (query)  | A selector to restrict the list of returned objects by their labels.                      |
| limit                | integer (query) | limit is a maximum number of responses to return for a list call.                         |
| pretty               | string (query)  | If 'true', then the output is pretty printed.                                             |
| resourceVersion      | string (query)  | resourceVersion sets a constraint on what resource versions a request may be served from. |
| resourceVersionMatch | string (query)  | resourceVersionMatch determines how resourceVersion is applied to list calls.             |
| sendInitialEvents    | boolean (query) | may be set together with watch=true.                                                      |
| timeoutSeconds       | integer (query) | Timeout for the list/watch call.                                                          |
| watch                | boolean (query) | Watch for changes to the described resources.                                             |

- Responses

- 200 OK: Media type Controls Accept header.

- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes
- POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes/{name}
- PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes/{name}
- DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes/{name}
- PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/bootboxes/{name}
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets
- POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets/{name}
- PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets/{name}
- DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets/{name}
- PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/buckets/{name}
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses
- POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses/{name}
- PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses/{name}
- DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses/{name}
- PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/clickhouses/{name}
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds

- 
- POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/etcds/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/externaldns/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/foundationdb/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/harbors/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/httpcaches/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/infos/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/ingresses/{name}

- 
- GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kafkas/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/kuberneteses/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mariadbs/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/mongodbs/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/monitorings/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/natses/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos/{name}

- 
- PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/openbaos/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/opensearches/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/postgreses/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/qdrants/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/rabbitmq/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/redises/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/seaweedfses/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers/{name}



- 
- DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tcpbalancers/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/tenants/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/virtualprivateclouds/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vmdisks/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vminstances/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns
  - POST /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns
  - GET /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns/{name}
  - PUT /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns/{name}
  - DELETE /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns/{name}
  - PATCH /apis/apps.cozystack.io/v1alpha1/namespaces/{namespace}/vpns/{name}
  - GET /apis/apps.cozystack.io/v1alpha1/natses
  - GET /apis/apps.cozystack.io/v1alpha1/openbaos
  - GET /apis/apps.cozystack.io/v1alpha1/opensearches
  - GET /apis/apps.cozystack.io/v1alpha1/postgreses
  - GET /apis/apps.cozystack.io/v1alpha1/qdrants
  - GET /apis/apps.cozystack.io/v1alpha1/rabbitmqes
  - GET /apis/apps.cozystack.io/v1alpha1/redises
  - GET /apis/apps.cozystack.io/v1alpha1/seaweedfses
  - GET /apis/apps.cozystack.io/v1alpha1/tcpbalancers

- 
- GET /apis/apps.cozystack.io/v1alpha1/tenants
  - GET /apis/apps.cozystack.io/v1alpha1/virtualprivateclouds
  - GET /apis/apps.cozystack.io/v1alpha1/vmdisks
  - GET /apis/apps.cozystack.io/v1alpha1/vminstances
  - GET /apis/apps.cozystack.io/v1alpha1/vpns

## Watch Operations

Includes GET watch operations for all resources listed above under /watch/ paths.

## Schemas

---

The following schemas are defined in the API:

- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.BootBox
- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.BootBoxList
- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.BootBoxStatus
- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.Bucket
- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.BucketList
- com.github.cozystack.cozystack.pkg.apis.apps.v1alpha1.BucketStatus
- ... (List continues for all managed resources including ClickHouse, Etcd, ExternalDNS, FoundationDB, Harbor, HTTPCache, Ingress, Kafka, Kubernetes, MariaDB, MongoDB, Monitoring, Nats, OpenBao, OpenSearch, Postgres, Qdrant, RabbitMQ, Redis, SeaweedFS, TCPBalancer, Tenant, VirtualPrivateCloud, VMDisk, VMInstance, VPN)

---

Information extracted from [Cozystack Documentation](#).

---

# Справочник ApplicationDefinition

<https://cozystack.ru/docs/v1.5/cozystack-api/application-definitions/>

## Обзор

`ApplicationDefinition` (`applicationdefinitions.cozystack.io/v1alpha1`) — это CRD кластерного масштаба, описывающий каждый тип приложения, предоставляемый платформой. Каждое определение объявляет `Kind` Kubernetes, который используют tenants в агрегированном API (`spec.application.kind`), схему OpenAPI для отображения формы на дашборде и валидации пользовательского ввода (`spec.application.openAPISchema`), а также метаданные дашборда: категорию, иконку и отображаемые имена (`spec.dashboard`).

Агрегированный API-сервер (`cozystack-api`) перечисляет все `ApplicationDefinition` один раз при запуске и регистрирует соответствующий ресурс под `apps.cozystack.io/v1alpha1`. Набор tenant-ориентированных `Kind` не меняется пока работает API-сервер — добавление, удаление или переименование `ApplicationDefinition` вступает в силу только после перезапуска `cozystack-api`.

Специальный контроллер (`applicationdefinition-controller`, поставляемый вместе с Cozystack) следит за `ApplicationDefinition` и автоматически инициирует этот перезапуск: при любом изменении набора он вычисляет контрольную сумму SHA-256 по отсортированным определениям и записывает её в аннотацию `cozystack.io/config-hash` шаблона пода в `Deployment` `cozy-system/cozystack-api`, после чего Kubernetes выполняет плавный перезапуск. События дебаунсятся в течение короткого окна, и если контрольная сумма не изменилась — перезапуск пропускается. Операторам не нужно вручную выполнять `kubectl rollout restart`.

Когда пользователь создаёт CR `Postgres` через дашборд, `kubectl` или Go-клиент, агрегированный слой транслирует его в `Flux HelmRelease`, использующий чарт, указанный в определении.

## Соглашение об именовании

`ApplicationDefinition` использует два независимых стиля именования. Каждое определение задаёт их явно, и связь между ними не может быть выведена никаким строковым преобразованием:

| Поле                               | Стиль                           | Пример (HTTP-кэш)       | Пример (диск VM)     | Пример (TCP-балансировщик) |
|------------------------------------|---------------------------------|-------------------------|----------------------|----------------------------|
| <code>metadata.name</code>         | строчные-с-дефисами             | <code>http-cache</code> | <code>vm-disk</code> | <code>tcp-balancer</code>  |
| <code>spec.application.kind</code> | CamelCase, акронимы сохраняются | <code>HTTPCache</code>  | <code>VMDisk</code>  | <code>TCPBalancer</code>   |

|                           |                       |            |         |              |
|---------------------------|-----------------------|------------|---------|--------------|
| spec.application.singular | строчные, без дефисов | httpcache  | vmdisk  | tcpbalancer  |
| spec.application.plural   | строчные, без дефисов | httpcaches | vmdisks | tcpbalancers |

Обратите внимание, что `metadata.name` не является функцией от `spec.application.kind`. Расположение дефисов (`tcp-balancer`, `vm-disk`, `http-cache`) и их отсутствие в `singular/plural` (`tcpbalancer`, `vmdisk`, `httpcache`) — это соглашения, выбранные для каждого приложения отдельно, а не результат работы общего алгоритма. `strings.ToLower(kind)` даёт `httpcache`, что совпадает с `spec.application.singular`, но не с `metadata.name`. Прямой поиск по Kind в нижнем регистре поэтому завершится ошибкой:

```
Ресурс агрегированного API использует plural в нижнем регистре:
$ kubectl get httpcaches --namespace tenant-demo
NAME READY AGE VERSION
frontend True 2m 1.2.0

Но ApplicationDefinition, лежащий в его основе, хранится под другим именем:
$ kubectl get applicationdefinition httpcache
Error from server (NotFound): applicationdefinitions.cozystack.io "httpcache" not found

$ kubectl get applicationdefinition http-cache
NAME AGE
http-cache 14d
```

Акронимы делают это особенно наглядным: `TCPBalancer`, `HTTPCache` и `VMDisk` — все теряют заглавные буквы в имени агрегированного ресурса (`tcpbalancers`, `httpcaches`, `vmdisks`), но сохраняют дефисы в имени CRD (`tcp-balancer`, `http-cache`, `vm-disk`).

## Рекомендуемый шаблон поиска

Клиентский код, которому нужно разрешить Kind Cozystack — например дашборд, получающий `HTTPCache` из метки `HelmRelease` и желающий отрендерить соответствующую форму — должен получить список всех `ApplicationDefinition` и фильтровать по `spec.application.kind` вместо попытки прямого `Get` по Kind в нижнем регистре. Набор определений невелик (порядка десятков элементов) и изменяется редко, поэтому такой подход дешёв и надёжен. Возвращайте весь найденный объект, чтобы вызывающий код мог читать `spec.application.openAPISchema`, `spec.dashboard` или любое другое поле без второго обращения к API.

Перед использованием названий группы и ресурса ниже убедитесь в их актуальности для вашего кластера:

```
$ kubectl api-resources | grep applicationdefinition
applicationdefinitions cozystack.io/v1alpha1 false Applicati...
```

Строка должна содержать `applicationdefinitions` в столбце `NAME`, `cozystack.io/v1alpha1` в столбце `APIVERSION`, `false` в столбце `NAMESPACED` (ресурс имеет кластерный масштаб) и `ApplicationDefinition`

в столбце `KIND`. Если группа отличается в вашем кластере, скорректируйте `GroupVersionResource` в примере соответственно.

```
import (
 "context"
 "fmt"

 metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
 "k8s.io/apimachinery/pkg/apis/meta/v1/unstructured"
 "k8s.io/apimachinery/pkg/runtime/schema"
 "k8s.io/client-go/dynamic"
)

// findByKind возвращает ApplicationDefinition, чей spec.application.kind
// совпадает с запрошенным kind, или ошибку, если совпадение не найдено. Вызывающий
// получает полный объект, поэтому поля вроде spec.application.openAPISchema
// доступны без второго обращения к API.
func findByKind(ctx context.Context, client dynamic.Interface, kind string) (*unstructured.Unstructured, error) {
 if kind == "" {
 return nil, fmt.Errorf("kind must not be empty")
 }

 gvr := schema.GroupVersionResource{
 Group: "cozystack.io",
 Version: "v1alpha1",
 Resource: "applicationdefinitions",
 }

 // Набор ApplicationDefinition на кластере Cozystack невелик
 // (порядка десятков), поэтому одного непагинированного List достаточно.
 // Если вы адаптируете этот хелпер для большего каталога, задайте ListOptions.Limit
 // и переходите по continue-токену, чтобы избежать неявного усечения.
 list, err := client.Resource(gvr).List(ctx, metav1.ListOptions{})
 if err != nil {
 return nil, fmt.Errorf("list %s/%s/%s: %w",
 gvr.Group, gvr.Version, gvr.Resource, err)
 }

 for i := range list.Items {
 specKind, found, err := unstructured.NestedString(
 list.Items[i].Object, "spec", "application", "kind")
 if err != nil || !found {
 // Пропускаем определения с отсутствующим или нестроковым kind, чтобы
 // итерация не совпала с некорректной записью.
 continue
 }

 if specKind == kind {
 return &list.Items[i], nil
 }
 }

 // Включаем GVR в ошибку, чтобы неверная группа (например после переименования
 // CRD) отличалась от настоящего "такого kind нет".
 return nil, fmt.Errorf("no ApplicationDefinition with spec.application.kind %q found under %s/%s/%s",
 kind, gvr.Group, gvr.Version, gvr.Resource)
}
```

Набор `ApplicationDefinition`, предоставляемых через агрегированный API, заморожен на момент запуска `cozystack-api` (см. [Обзор](#)), однако базовые CRD можно редактировать в режиме реального времени: администратор может изменить `spec.application.openAPISchema` или `spec.dashboard` в

---

существующем определении или добавить новый Kind — `applicationdefinition-controller` затем инициирует плавный перезапуск `cozystack-api`, чтобы изменение стало доступным через агрегированный API без ручного вмешательства. Насколько агрессивно клиент должен кэшировать — зависит от его собственного времени жизни:

- Короткоживущие процессы (CLI-инструменты, одноразовые скрипты, `serverless`-функции) могут безопасно кэшировать результат `findByKind` на всё время жизни процесса.
- Долгоживущие процессы (дашборды, контроллеры, операторы) должны повторно получать список `ApplicationDefinition` с периодичностью, соответствующей тому, как часто их операторы редактируют схемы — обычно раз в несколько минут достаточно. Определения меняются редко, поэтому `watch` не оправдывает сложности. Новый `ApplicationDefinition` станет доступен через агрегированный API вскоре после создания, как только инициированный контроллером плавный перезапуск `cozystack-api` завершится.

Plural в нижнем регистре (`httpcaches`, `vmdisks`) является корректным именем для арендатор-ориентированных ресурсов под `apps.cozystack.io/v1alpha1`. Только CRD `applicationdefinitions.cozystack.io` использует форму с дефисами.

## См. также

- [Обзор Cozystack API](#) — использование `kubectl`, `Terraform` и Go-клиента для арендатор-ориентированных ресурсов.
- [Go Types](#) — типизированные Go-клиенты для ресурсов `apps.cozystack.io/v1alpha1`.



# Прочее

## Cozystack Руководство по разработке

<https://cozystack.ru/docs/v1.5/development/>

**Cozystack** — платформа на основе операторов. Начальная загрузка и текущее управление осуществляются набором контроллеров, работающих внутри кластера. Высокоуровневый поток выглядит так:

1. Инсталлятор чарта (`packages/core/installer`) применяется через `helm install`. Он разворачивает Deployment `cozystack-operator` в пространстве имён `cozy-system`.
2. `cozystack-operator` запускается и выполняет однократную начальную загрузку:
  - Устанавливает CRD Cozystack (`Package`, `PackageSource`) из встроенных манифестов (`internal/crdinstall`).
  - Устанавливает компоненты Flux (`source-controller`, `helm-controller`, `source-watcher`) из встроенных манифестов (`internal/fluxinstall`).
  - Создает начальный OCIRepository (`cozystack-platform`) из значений `platformSourceUrl` и `platformSourceRef`, заданных в инсталляторе.
  - Создает `PackageSource`, ссылающийся на начальный OCIRepository.
3. Цикл согласования берёт управление. Оператор следит за CRD `PackageSource` и `Package` и преобразует их в объекты Flux `HelmRelease`. Flux затем устанавливает реальные Helm-чарты и управляет ими.
4. Platform chart (`packages/core/platform`) разворачивается как обычный `Package`. Он читает конфигурацию кластера из ресурса `cozystack.cozystack-platform Package` и создаёт шаблоны манифестов пакетов, определяющих, какие системные компоненты должны быть установлены. Platform chart также создаёт вторичный OCIRepository (`cozystack-packages`), копируя спецификацию из начального OCIRepository. Все `PackageSource` ссылаются на этот вторичный репозиторий. При обновлениях platform chart выполняет миграции как `pre-upgrade` хуки перед созданием или обновлением `HelmRelease` компонентов.
5. FluxCD — движок выполнения, он согласует объекты `HelmRelease`, созданные оператором, загружает артефакты чартов из ресурсов `ExternalArtifact` и применяет их к кластеру.

Полную цепочку согласования (`PackageSource -> ArtifactGenerator -> ExternalArtifact -> Package -> HelmRelease -> Pods`), разрешение зависимостей, потоки обновления и отката, а также CLI `cozyrpk` см. в разделе [Ключевые концепции](#).

## OCIRepository и поток миграции

Cozystack использует два ресурса OCIRepository для управления обновлениями платформы:



| OCIRepository      | Создаётся                        | Ссылается на                                                                     |
|--------------------|----------------------------------|----------------------------------------------------------------------------------|
| cozystack-platform | cozystack-operator               | Настраивается через значения инсталлятора (platformSourceUrl, platformSourceRef) |
| cozystack-packages | Platform chart (repository.yaml) | Копирует спецификацию из cozystack-platform                                      |

Все PackageSource в packages/core/platform/sources/ ссылаются на cozystack-packages.

## Выполнение миграций

Миграции выполняются как Helm pre-upgrade хуки в platform chart:

```
packages/core/platform/templates/migration-hook.yaml
metadata:
 name: cozystack-migration-hook
annotations:
 helm.sh/hook: pre-upgrade,pre-install
 helm.sh/hook-weight: "1"
```

Контейнер миграции считывает текущую версию из ConfigMap cozystack-version и выполняет скрипты миграции последовательно от CURRENT\_VERSION до TARGET\_VERSION - 1. Каждая миграция обновляет ConfigMap при успехе, обеспечивая идиempотентность миграций и возможность возобновления после сбоев.

## Зачем два репозитория?

Разделение гарантирует, что:

1. Начальный OCIRepository управляется оператором (через значения инсталлятора).
2. Все PackageSource имеют согласованную ссылку (cozystack-packages), а не указывают напрямую на источник, управляемый оператором.
3. Platform chart может выполнять миграции перед созданием вторичного OCIRepository, гарантируя выполнение миграций до обновления компонентов.

## Ключевые бинарные файлы

| Бинарный файл      | Источник               | Роль                                                                                                                           |
|--------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| cozystack-operator | cmd/cozystack-operator | Начальная загрузка (CRD, Flux, источник платформы), согласование PackageSource и Package, репликация секрета cozystack-values. |

|                      |                          |                                                                                                                  |
|----------------------|--------------------------|------------------------------------------------------------------------------------------------------------------|
| cozystack-controller | cmd/cozystack-controller | Согласование рабочих нагрузок и ApplicationDefinition, управление дашбордами.                                    |
| cozystack-api        | cmd/cozystack-api        | Уровень агрегации API Kubernetes для групп API <code>apps.cozystack.io</code> и <code>core.cozystack.io</code> . |
| cozypkg              | cmd/cozypkg              | CLI для управления пакетами.                                                                                     |

## Структура репозитория

Основная структура репозитория [cozystack](#):

```

api # Go-типы для CRD Cozystack (Package, PackageSource и т.д.)
cmd # Точки входа для всех бинарных файлов
 cozystack-operator # Основной оператор платформы
 cozystack-controller # Контроллеры рабочих нагрузок и приложений
 cozystack-api # Агрегированный API-сервер
 cozypkg # CLI для управления пакетами
internal # Реализации контроллеров и согласователей
 operator # Согласователи PackageSource и Package
 controller # Контроллеры рабочих нагрузок и ApplicationDefinition
 fluxinstall # Встроенные манифесты Flux и инсталлятор
 crdinstall # Встроенные манифесты CRD и инсталлятор
 cozyvaluesreplicator # Логика репликации секретов
packages # Helm-чарты, организованные по слоям
 core # Начальная загрузка и конфигурация платформы
 system # Инфраструктурные операторы и upstream-чарты
 apps # Пользовательские чарты приложений
 extra # Чарты приложений для конкретных тенантов
pkg # Общие Go-библиотеки
dashboards # Дашборды Grafana
hack # Вспомогательные скрипты для локальной разработки
docs # Журналы изменений и примечания к релизам

```

Разработку можно вести локально, изменяя и обновляя файлы в этом репозитории.

## Пакеты

### core

Core-пакеты отвечают за начальную загрузку и конфигурацию на уровне платформы.

### installer

Helm-чарт, разворачивающий Deployment `cozystack-operator`. Он создаёт пространство имён `cozy-system`, ServiceAccount с правами `cluster-admin` и Deployment оператора с флагами,

---

инициирующими установку CRD и Flux при запуске. Образ оператора и URL источника платформы задаются во время сборки.

## platform

Helm-чарт, разворачиваемый как обычный `Package` (не применяется напрямую). Он читает конфигурацию кластера из ресурса `cozystack.cozystack-platform Package` и создаёт шаблоны манифестов согласно указанному варианту и настройкам компонентов, определяя, какие системные компоненты должны быть установлены.

## flux-ai

Компоненты Flux, упакованные для развёртывания оператором.

## talos

Конфигурационные ресурсы Talos OS.

Core-пакеты не используют Helm для применения манифестов; они предназначены для использования только как `helm template . | kubectl apply -f -`.

## system

System-пакеты настраивают систему для управления и развёртывания пользовательских приложений. Необходимы для работы всей платформы.

System-пакеты включают два вида компонентов:

- Операторы (например, `postgres-operator`, `kafka-operator`, `redis-operator`): Контроллеры, умеющие управлять полным жизненным циклом конкретного приложения, включая операции второго дня.
- Upstream Helm-чарты для приложений без выделенного оператора (например, `nats`, `ingress-nginx`): Эти чарты размещаются в system, чтобы пакеты `apps` и `extra` могли разворачивать их через Flux `HelmRelease` CR, фактически используя FluxCD в качестве оператора.

System-пакеты используют Helm для установки и управляются FluxCD.

## apps

Эти пользовательские приложения отображаются в дашборде и включают манифесты для применения к кластеру.

Чарты `apps` служат высокоуровневым API для пользователей. Они определяют только те параметры, которые должны быть открыты и проверены через `values.schema.json`, сохраняя интерфейс минимальным и безопасным. Чарты `apps` не должны содержать бизнес-логику развёртывания самого приложения — вместо этого они делегируют её оператору или FluxCD.

В зависимости от наличия выделенного оператора приложения `apps` следуют одному из двух паттернов:

---

## Паттерн на основе оператора

Когда для приложения есть выделенный оператор (например, PostgreSQL, MongoDB, Redis, Kafka), чарт `app` создаёт экземпляры CRD, которыми управляет оператор:

Оператор обрабатывает все детали развёртывания и операции второго дня (масштабирование, резервное копирование, восстановление).

```
packages/system/postgres-operator/ # Helm-чарт оператора
packages/apps/postgres/ # App-чарт создаёт postgresql.cnpg.io/v1.Cluster CR
```

## Паттерн на основе HelmRelease

Когда для приложения нет выделенного оператора и стандартным методом развёртывания является Helm-чарт, основной upstream-чарт размещается в `system/`, а app-чарт создаёт Flux HelmRelease CR, указывающий на него:

```
packages/system/nats/ # Upstream Helm-чарт
packages/apps/nats/ # App-чарт создаёт HelmRelease CR
```

В этом случае FluxCD выступает оператором, управляя жизненным циклом Helm-релиза. App-чарт контролирует `values.yaml` для релиза.

Другие примеры этого паттерна: `extra/ingress`, `extra/seaweedfs`, `extra/monitoring`.

## extra

Аналогично `apps`, но не отображается в каталоге приложений. Могут устанавливаться только в составе тенанта. Разрешения на установку этих приложений настраиваются для каждого тенанта индивидуально.

Подробнее о [Системе тенантов](#) читайте на странице основных концепций.

В одном пространстве имён тенанта можно использовать только один тип приложения.

Extra-пакеты следуют тем же двум архитектурным паттернам, что и `apps` (на основе оператора или HelmRelease).

Пакеты `apps` и `extra` используют Helm для установки приложения и управляются FluxCD через дашборд.

---

## Структура пакета

Каждый пакет — это типичный Helm-чарт, содержащий все необходимые образы и манифесты для платформы. Мы рекомендуем упаковывать upstream-чарты в директорию `./charts` и переопределять `values.yaml` в корне приложения. Такая структура упрощает обновление upstream-чартов и внесение изменений в манифесты.

|                                                   |                                                                                |
|---------------------------------------------------|--------------------------------------------------------------------------------|
| <code>Chart.yaml</code>                           | # Определение Helm-чарта и описание параметров                                 |
| <code>Makefile</code>                             | # Общие цели для упрощения локальной разработки                                |
| <code>charts</code>                               | # Директория для upstream-чартов                                               |
| <code>images</code>                               | # Директория для Docker-образов                                                |
| <code>patches</code>                              | # Опциональная директория для патчей upstream-чартов                           |
| <code>templates</code>                            | # Дополнительные манифесты для upstream Helm-чарта                             |
| <code>templates/dashboard-resourcemap.yaml</code> | # Роль для отображения ресурсов k8s в дашборде                                 |
| <code>values.yaml</code>                          | # Значения переопределения для upstream Helm-чарта                             |
| <code>values.schema.json</code>                   | # JSON-схема для валидации входных значений и отрисовки элементов UI в дашб... |

Для генерации файлов `README.md` и `values.schema.json` можно использовать [readme-generator](#) от Bitnami.

Просто установите его как бинарный файл `readme-generator` в своей системе и запустите генерацию командой `make generate`.

## Принципы разработки Helm-чартов

Структура пакетов и рабочий процесс разработки в Cozystack основаны на следующих принципах:

### Простое обновление upstream-чартов

Оригинальный upstream-чарт должен быть легко обновляемым, переопределяемым и изменяемым. Мы используем подход, при котором оригинальные чарты загружаются в `./charts` и хранятся в оригинальном виде. Кастомизации вносятся через переопределения `values.yaml` и дополнительные `templates/`, а структурные изменения в upstream-чарте применяются через `patches/`. Такое разделение гарантирует простое обновление до новой upstream-версии: выполните `make update`, просмотрите diff и при необходимости повторно примените патчи.

### Локальные артефакты

Патчи и образы контейнеров хранятся локально и являются частью пакета. Директория `patches/` содержит любые изменения upstream-чарта, а директория `images/` — Dockerfile для сборки всех необходимых образов. Это обеспечивает полную воспроизводимость — всё необходимое для сборки и запуска пакета находится внутри его директории.

В настоящее время не все пакеты собирают свои образы локально — некоторые всё ещё ссылаются на образы из внешних реестров.

### Рабочий процесс локальной разработки и тестирования

Каждый пакет должен быть легко обновляемым и тестируемым локально на реальном кластере без использования CI-пайплайнов. Стандартные цели `make` (`make image`, `make diff`, `make apply`) обеспечивают быструю обратную связь: сборка образов, сравнение отрендеренных манифестов с живым кластером и их применение.

### Без внешних зависимостей

Пакеты не должны зависеть от внешних ресурсов во время выполнения. Все чарты, образы и патчи включены в пакет и поставляются вместе с платформой. Это гарантирует стабильность и независимость от внешних сбоев.

Как отмечалось выше, полная самодостаточность образов находится в процессе реализации. Некоторые пакеты всё ещё зависят от внешних образов.

## Разработка

### Настройка Buildx

Для сборки образов необходимо установить и настроить плагин [docker buildx](#).

Вместо встроенного сборщика можно [настроить дополнительные](#), которые могут быть удалёнными или поддерживать несколько архитектур. В этом примере показано, как настроить драйвер [kubernetes](#), позволяющим собирать образы непосредственно в кластере Kubernetes:

```
docker buildx create \
 --bootstrap \
 --name=buildkit \
 --driver=kubernetes \
 --driver-opt=namespace=tenant-kvaps,replicas=2 \
 --platform=linux/amd64 \
 --platform=linux/arm64
```

Либо опустите параметры `--driver*`, чтобы настроить среду сборки в локальном Docker-окружении.

### Управление пакетами

Каждое приложение включает Makefile для упрощения процесса разработки. Для каждого пакета мы следуем стандартному набору команд:

```
make update # Обновить Helm-чарт и версии из upstream-источника
make image # Собрать Docker-образы, используемые в пакете
make show # Показать вывод отрендеренных шаблонов
make diff # Сравнить Helm-релиз с объектами в кластере Kubernetes
make apply # Применить Helm-релиз к кластеру Kubernetes
```

Например, для обновления cilium:

```
cd packages/system/cilium # Перейти в директорию приложения
make update # Загрузить новую версию из upstream
make image # Собрать образ cilium
git diff . # Показать diff с изменёнными манифестами
make diff # Показать diff с применёнными манифестами кластера
make apply # Применить изменённые манифесты к кластеру
kubectl get pod -n cozy-cilium # Проверить корректность работы
git commit -m "Update cilium to v1.15.5" # Зафиксировать изменения в ветке
```

Для сборки контейнера cozystack с обновлённым чартом:

```
cd packages/core/installer # Перейти в директорию пакета cozystack
make image-packages # Собрать образ пакетов
make apply # Применить к кластеру
kubectl get pod -n cozy-system # Проверить корректность работы
kubectl get hr -A # Проверить объекты HelmRelease
```

При пересборке образов указывайте переменную окружения `REGISTRY`, указывающую на ваш Docker-реестр.

Не стесняйтесь заглядывать в каждый Makefile, чтобы лучше понять логику.

## Тестирование

Платформа включает скрипт `e2e.sh`, выполняющий следующие задачи:

- Запускает три виртуальные машины QEMU
- Настраивает Talos Linux
- Устанавливает Cozystack
- Ожидает установки всех HelmRelease
- Выполняет дополнительные проверки работоспособности компонентов

Скрипт `e2e.sh` можно запускать как локально, так и непосредственно в контейнере Kubernetes.

Для запуска тестов в кластере Kubernetes перейдите в директорию `packages/core/testing` и выполните следующие команды:

```
make apply # Создать тестовую песочницу в кластере Kubernetes
make test # Запустить end-to-end тесты в существующей песочнице
make delete # Удалить тестовую песочницу из кластера Kubernetes
```

[!] Для запуска e2e-тестов в кластере Kubernetes узлы должны иметь достаточно свободных ресурсов для запуска вложенных виртуальных машин (QEMU) с поддержкой KVM.

Рекомендуется использовать bare-metal узлы родительского кластера Cozystack.

## Динамическая среда разработки

Если вы предпочитаете разрабатывать Cozystack в виртуальных машинах вместо изменения существующего кластера, пакет `packages/core/testing` включает дополнительные опции:

```
make exec # Открывает интерактивную оболочку в контейнере песочницы.
make login # Загружает kubecfg во временную директорию и запускает оболочку с окружением песочницы; треб...
make proxy # Включает SOCKS5 прокси-сервер; требуются mirrord и gost.
```

Прокси Socks5 можно настроить в браузере для доступа к сервисам кластера, работающего в песочнице. Для удобства переключения можно использовать расширение [Proxy Toggle](#).

---

## Cozystack Рoadmap

<https://cozystack.ru/docs/v1.5/roadmap/>

Вы можете ознакомиться с нашей дорожной картой по адресу: [Ссылка](#).

Если у вас есть вопросы и комментарии по поводу плана развития или вам необходима определенная функциональность, пожалуйста, сообщите нам об этом по адресу [info@katamarina.ru](mailto:info@katamarina.ru)

Присоединяйтесь к нашим [community-встречам](#).

---



---

# Контрибуторы

<https://cozystack.ru/docs/v1.5/contributors/>

Cozystack развивается благодаря людям, которые не только пишут код, но и помогают делать проект понятнее, доступнее и удобнее для новых пользователей.

## Артем Старик

---

Артем помог с русской версией документации Cozystack.

Эти материалы важны для первого знакомства с проектом: они проводят пользователя от подготовки инфраструктуры до установки Talos Linux, Kubernetes, Cozystack и первых tenant'ов. Хорошая документация здесь снижает порог входа и помогает быстрее перейти от интереса к работающему кластеру.

Спасибо за вклад в документацию и за внимание к тому, чтобы Cozystack был понятен русскоязычному сообществу.

## Леонид Филиппов

---

Леонид перевёл на русский язык разделы документации Cozystack о подсистеме хранения (storage) и сетевых возможностях (networking) для версий v1.4 и v1.5.

Эти разделы охватывают ключевые темы эксплуатации платформы: настройку LINSTOR и DRBD, подготовку дисков, шифрование хранилища, сетевую архитектуру на основе Cilium и Kube-OVN, Gateway API, балансировку нагрузки и публикацию сервисов. Их перевод делает эксплуатацию Cozystack доступнее для русскоязычных инженеров.

## Команда Katamarina

---

Сотрудники компании содействовали как переводу документации Cozystack на русский, так и обеспечению работы сайта <https://cozystack.ru> и развитию экосистемы проекта в целом.

Компания Katamarina первой добилась включения в реестр российского ПО проекта на основе Cozystack и работает над сертификацией решения по требованиям ФСТЭК.

Благодарим компанию Katamarina за вклад в развитие проекта Cozystack на территории РФ и за ее пределы.

---